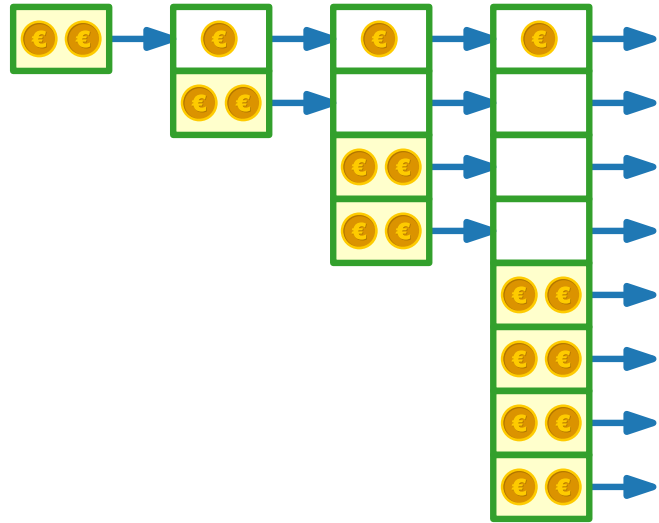


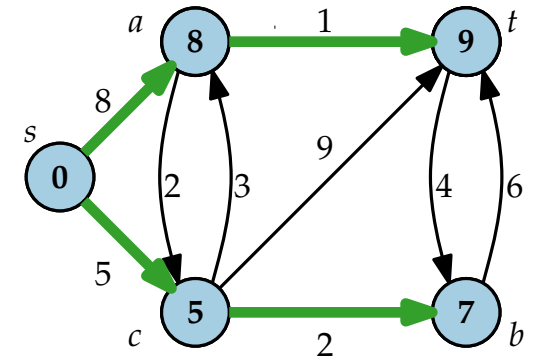


Fortgeschrittene Algorithmen

17. Vorlesung Datenstrukturen III: Amortisierte Analyse



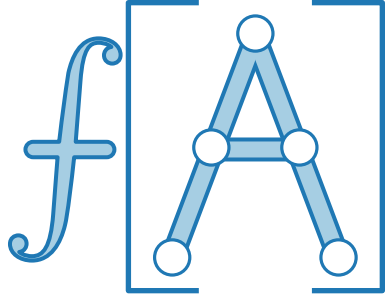
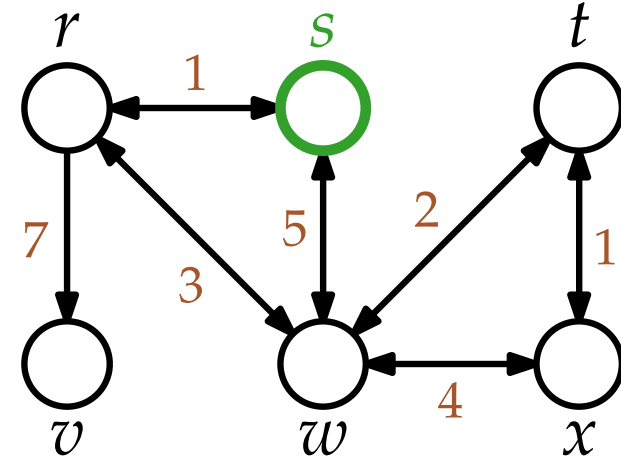
Teil I: DIJKSTRA



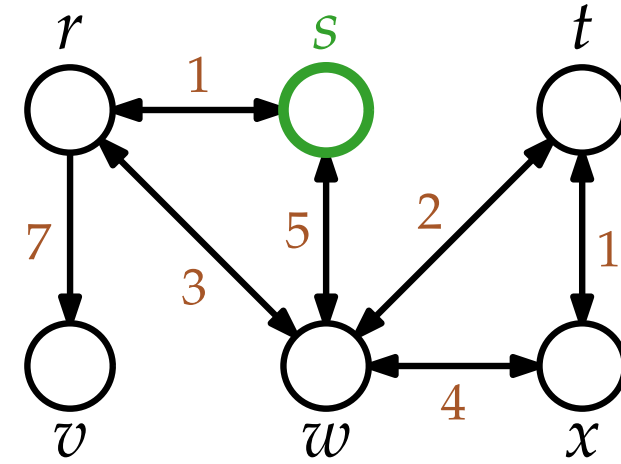
Wiederholung DIJKSTRA



Edsger W. Dijkstra
(1930 – 2002)



Wiederholung DIJKSTRA



DIJKSTRA(WeightedGraph G , Vertex s)

INITIALIZE(G, s)

$Q = \text{new PriorityQueue}(V, d)$

while not $Q.\text{EMPTY}()$ **do**

$u = Q.\text{EXTRACTMIN}()$

foreach $v \in \text{Adj}[u]$ **do**

if $v.d > u.d + w(u, v)$ **then**

$v.\text{color} = \text{red}$

$v.d = u.d + w(u, v)$

$v.\pi = u$

$Q.\text{DECREASEKEY}(v, v.d)$

$u.\text{color} = \text{blue}$

INITIALIZE(Graph G , Vertex s)

foreach $u \in V$ **do**

$u.\text{color} = \text{white}$

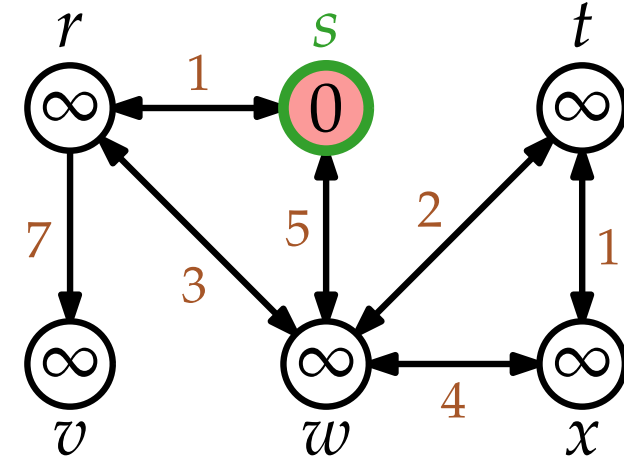
$u.d = \infty$

$u.\pi = \text{nil}$

$s.\text{color} = \text{red}$

$s.d = 0$

Wiederholung DIJKSTRA



DIJKSTRA(WeightedGraph G , Vertex s)

INITIALIZE(G, s)

$Q = \text{new PriorityQueue}(V, d)$

while not $Q.\text{EMPTY}()$ **do**

$u = Q.\text{EXTRACTMIN}()$

foreach $v \in \text{Adj}[u]$ **do**

if $v.d > u.d + w(u, v)$ **then**

$v.\text{color} = \text{red}$

$v.d = u.d + w(u, v)$

$v.\pi = u$

$Q.\text{DECREASEKEY}(v, v.d)$

$u.\text{color} = \text{blue}$

INITIALIZE(Graph G , Vertex s)

foreach $u \in V$ **do**

$u.\text{color} = \text{white}$

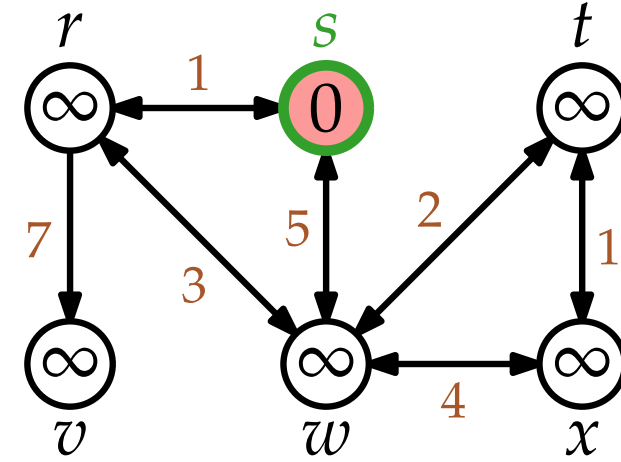
$u.d = \infty$

$u.\pi = \text{nil}$

$s.\text{color} = \text{red}$

$s.d = 0$

Wiederholung DIJKSTRA



DIJKSTRA(WeightedGraph G , Vertex s)

INITIALIZE(G, s)

$Q = \text{new PriorityQueue}(V, d)$

while not $Q.\text{EMPTY}()$ **do**

$u = Q.\text{EXTRACTMIN}()$

foreach $v \in \text{Adj}[u]$ **do**

if $v.d > u.d + w(u, v)$ **then**

$v.\text{color} = \text{red}$

$v.d = u.d + w(u, v)$

$v.\pi = u$

$Q.\text{DECREASEKEY}(v, v.d)$

$u.\text{color} = \text{blue}$

INITIALIZE(Graph G , Vertex s)

foreach $u \in V$ **do**

$u.\text{color} = \text{white}$

$u.d = \infty$

$u.\pi = \text{nil}$

$s.\text{color} = \text{red}$

$s.d = 0$

Wiederholung DIJKSTRA

DIJKSTRA(WeightedGraph G , Vertex s)

INITIALIZE(G, s)

$Q = \text{new PriorityQueue}(V, d)$

while not $Q.\text{EMPTY}()$ **do**

$u = Q.\text{EXTRACTMIN}()$

foreach $v \in \text{Adj}[u]$ **do**

if $v.d > u.d + w(u, v)$ **then**

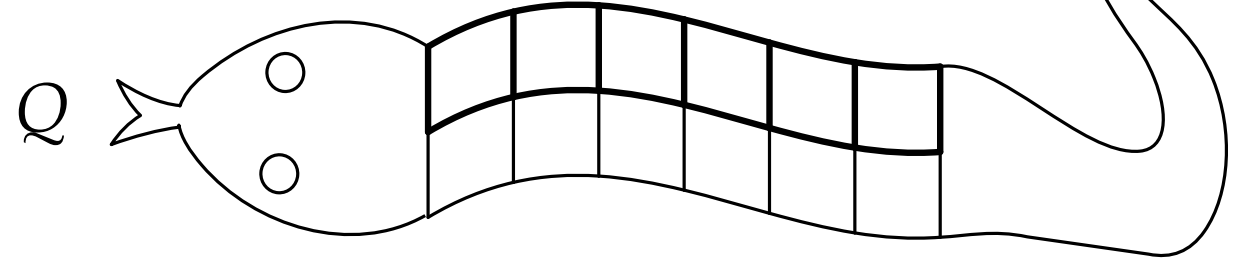
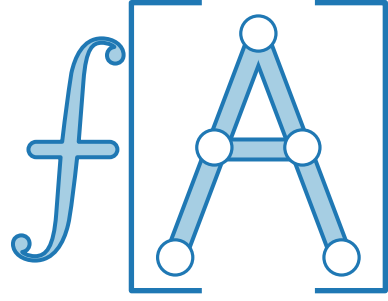
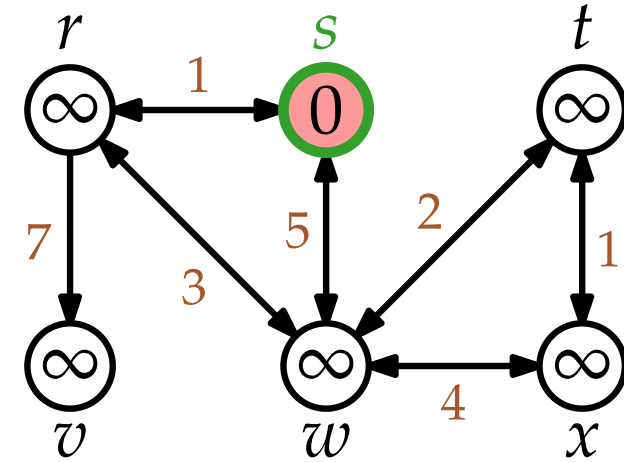
$v.\text{color} = \text{red}$

$v.d = u.d + w(u, v)$

$v.\pi = u$

$Q.\text{DECREASEKEY}(v, v.d)$

$u.\text{color} = \text{blue}$



INITIALIZE(Graph G , Vertex s)

foreach $u \in V$ **do**

$u.\text{color} = \text{white}$

$u.d = \infty$

$u.\pi = \text{nil}$

$s.\text{color} = \text{red}$

$s.d = 0$

Wiederholung DIJKSTRA



DIJKSTRA(WeightedGraph G , Vertex s)

INITIALIZE(G, s)

$Q = \text{new PriorityQueue}(V, d)$

while not $Q.\text{EMPTY}()$ **do**

$u = Q.\text{EXTRACTMIN}()$

foreach $v \in \text{Adj}[u]$ **do**

if $v.d > u.d + w(u, v)$ **then**

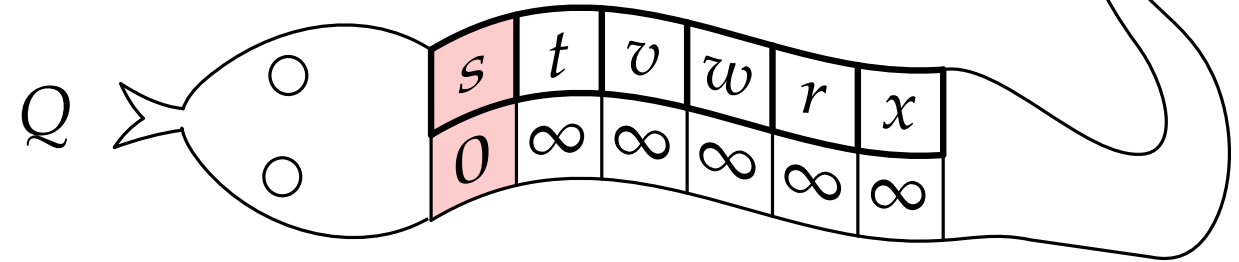
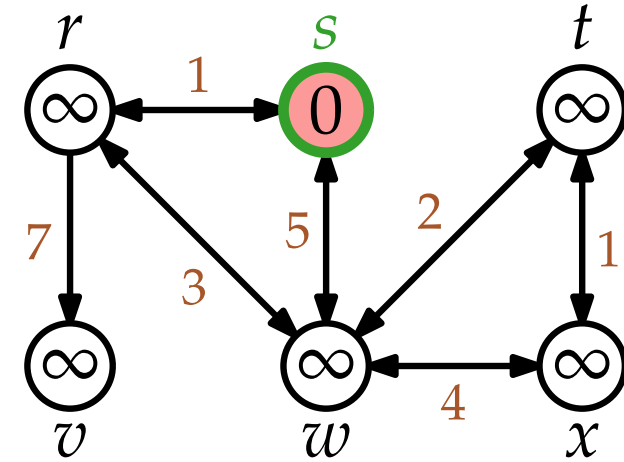
$v.\text{color} = \text{red}$

$v.d = u.d + w(u, v)$

$v.\pi = u$

$Q.\text{DECREASEKEY}(v, v.d)$

$u.\text{color} = \text{blue}$



INITIALIZE(Graph G , Vertex s)

foreach $u \in V$ **do**

$u.\text{color} = \text{white}$

$u.d = \infty$

$u.\pi = \text{nil}$

$s.\text{color} = \text{red}$

$s.d = 0$

Wiederholung DIJKSTRA



DIJKSTRA(WeightedGraph G , Vertex s)

INITIALIZE(G, s)

$Q = \text{new PriorityQueue}(V, d)$

while not $Q.\text{EMPTY}()$ **do**

$u = Q.\text{EXTRACTMIN}()$

foreach $v \in \text{Adj}[u]$ **do**

if $v.d > u.d + w(u, v)$ **then**

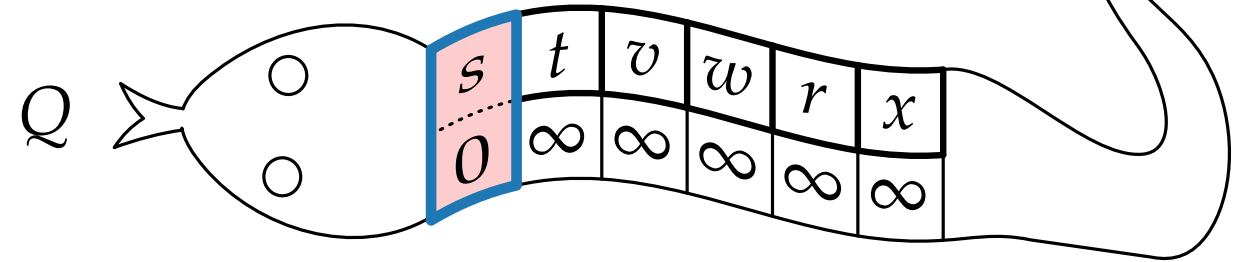
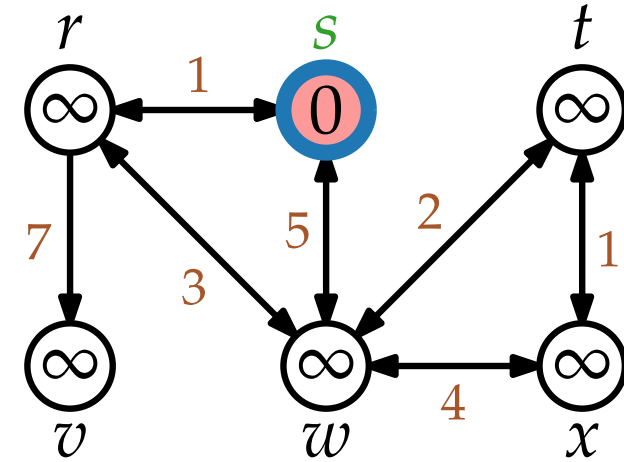
$v.\text{color} = \text{red}$

$v.d = u.d + w(u, v)$

$v.\pi = u$

$Q.\text{DECREASEKEY}(v, v.d)$

$u.\text{color} = \text{blue}$



INITIALIZE(Graph G , Vertex s)

foreach $u \in V$ **do**

$u.\text{color} = \text{white}$

$u.d = \infty$

$u.\pi = \text{nil}$

$s.\text{color} = \text{red}$

$s.d = 0$

Wiederholung DIJKSTRA

DIJKSTRA(WeightedGraph G , Vertex s)

INITIALIZE(G, s)

$Q = \text{new PriorityQueue}(V, d)$

while not $Q.\text{EMPTY}()$ **do**

$u = Q.\text{EXTRACTMIN}()$

foreach $v \in \text{Adj}[u]$ **do**

if $v.d > u.d + w(u, v)$ **then**

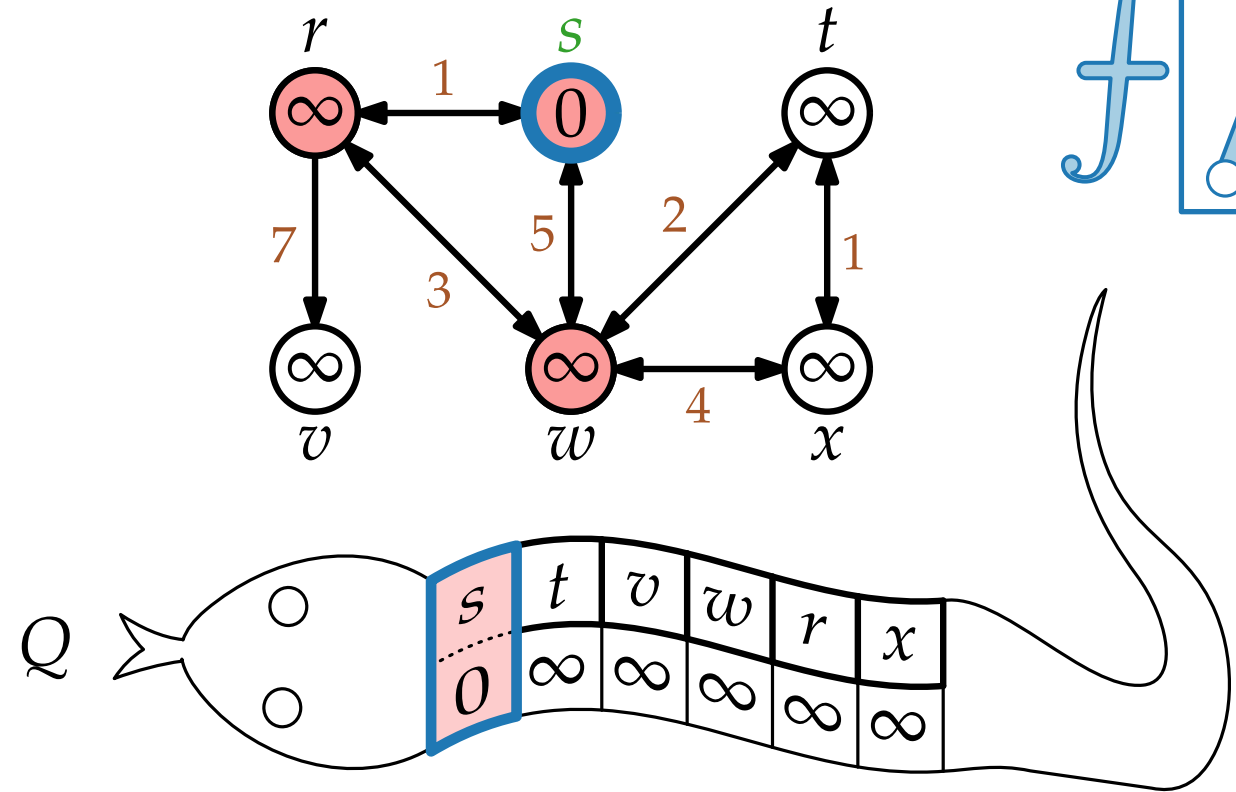
$v.\text{color} = \text{red}$

$v.d = u.d + w(u, v)$

$v.\pi = u$

$Q.\text{DECREASEKEY}(v, v.d)$

$u.\text{color} = \text{blue}$



INITIALIZE(Graph G , Vertex s)

foreach $u \in V$ **do**

$u.\text{color} = \text{white}$

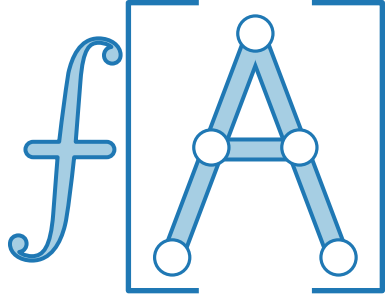
$u.d = \infty$

$u.\pi = \text{nil}$

$s.\text{color} = \text{red}$

$s.d = 0$

Wiederholung DIJKSTRA



DIJKSTRA(WeightedGraph G , Vertex s)

INITIALIZE(G, s)

$Q = \text{new PriorityQueue}(V, d)$

while not $Q.\text{EMPTY}()$ **do**

$u = Q.\text{EXTRACTMIN}()$

foreach $v \in \text{Adj}[u]$ **do**

if $v.d > u.d + w(u, v)$ **then**

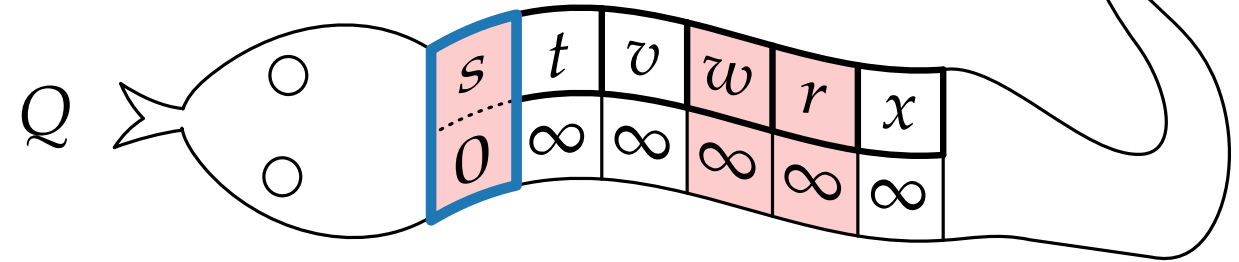
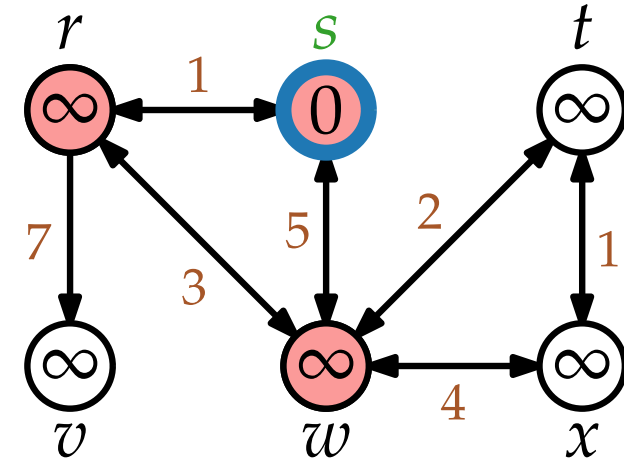
$v.\text{color} = \text{red}$

$v.d = u.d + w(u, v)$

$v.\pi = u$

$Q.\text{DECREASEKEY}(v, v.d)$

$u.\text{color} = \text{blue}$



INITIALIZE(Graph G , Vertex s)

foreach $u \in V$ **do**

$u.\text{color} = \text{white}$

$u.d = \infty$

$u.\pi = \text{nil}$

$s.\text{color} = \text{red}$

$s.d = 0$

Wiederholung DIJKSTRA

DIJKSTRA(WeightedGraph G , Vertex s)

INITIALIZE(G, s)

$Q = \text{new PriorityQueue}(V, d)$

while not $Q.\text{EMPTY}()$ **do**

$u = Q.\text{EXTRACTMIN}()$

foreach $v \in \text{Adj}[u]$ **do**

if $v.d > u.d + w(u, v)$ **then**

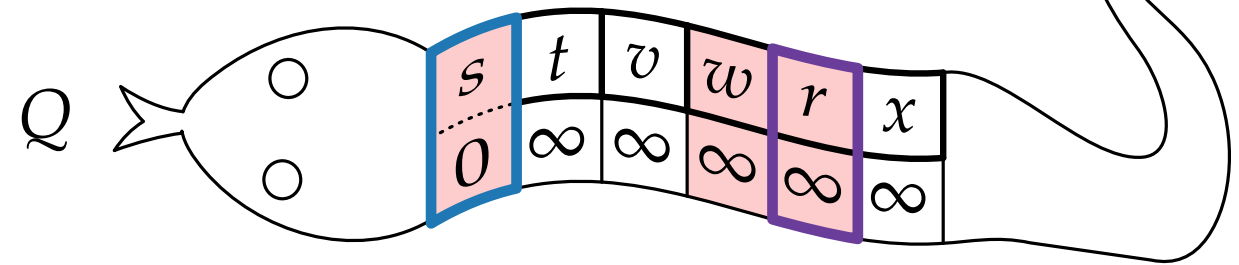
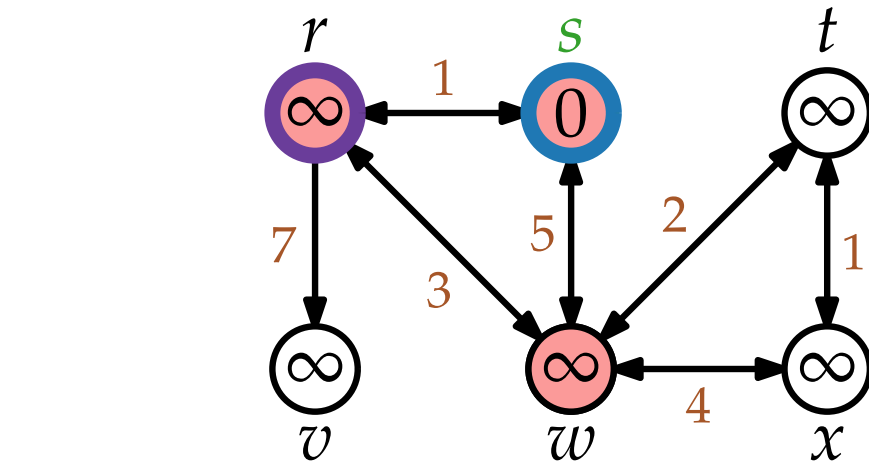
$v.\text{color} = \text{red}$

$v.d = u.d + w(u, v)$

$v.\pi = u$

$Q.\text{DECREASEKEY}(v, v.d)$

$u.\text{color} = \text{blue}$



INITIALIZE(Graph G , Vertex s)

foreach $u \in V$ **do**

$u.\text{color} = \text{white}$

$u.d = \infty$

$u.\pi = \text{nil}$

$s.\text{color} = \text{red}$

$s.d = 0$

Wiederholung DIJKSTRA

DIJKSTRA(WeightedGraph G , Vertex s)

INITIALIZE(G, s)

$Q = \text{new PriorityQueue}(V, d)$

while not $Q.\text{EMPTY}()$ **do**

$u = Q.\text{EXTRACTMIN}()$

foreach $v \in \text{Adj}[u]$ **do**

if $v.d > u.d + w(u, v)$ **then**

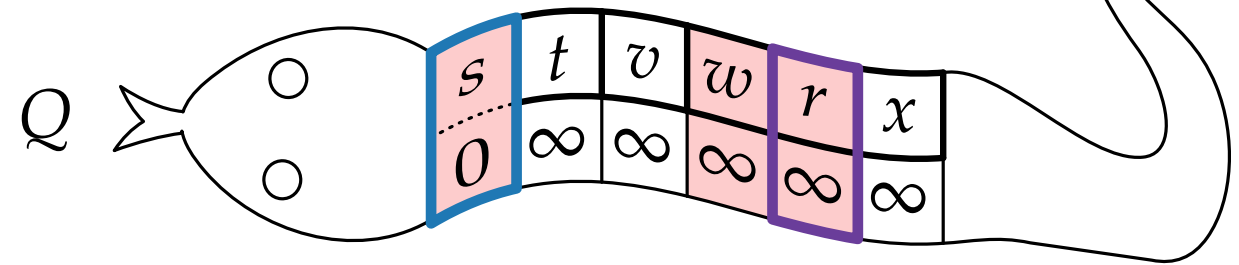
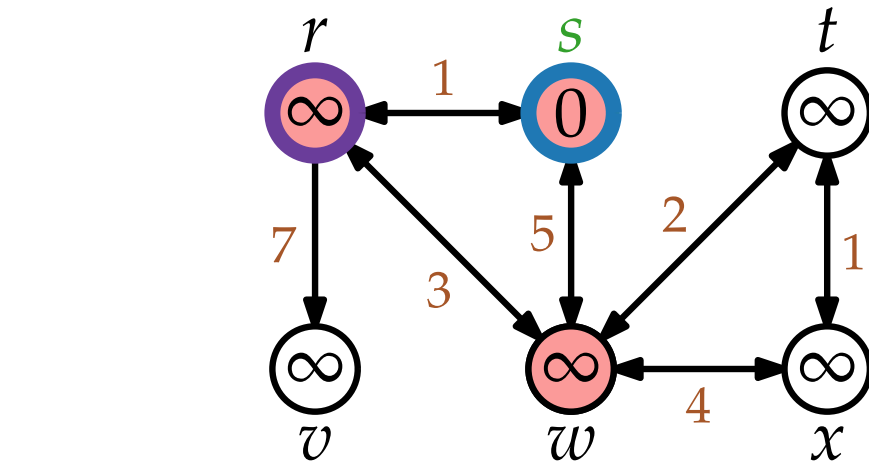
$v.\text{color} = \text{red}$

$v.d = u.d + w(u, v)$

$v.\pi = u$

$Q.\text{DECREASEKEY}(v, v.d)$

$u.\text{color} = \text{blue}$



INITIALIZE(Graph G , Vertex s)

foreach $u \in V$ **do**

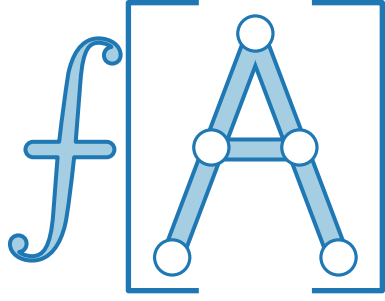
$u.\text{color} = \text{white}$

$u.d = \infty$

$u.\pi = \text{nil}$

$s.\text{color} = \text{red}$

$s.d = 0$



Wiederholung DIJKSTRA

DIJKSTRA(WeightedGraph G , Vertex s)

INITIALIZE(G, s)

$Q = \text{new PriorityQueue}(V, d)$

while not $Q.\text{EMPTY}()$ **do**

$u = Q.\text{EXTRACTMIN}()$

foreach $v \in \text{Adj}[u]$ **do**

if $v.d > u.d + w(u, v)$ **then**

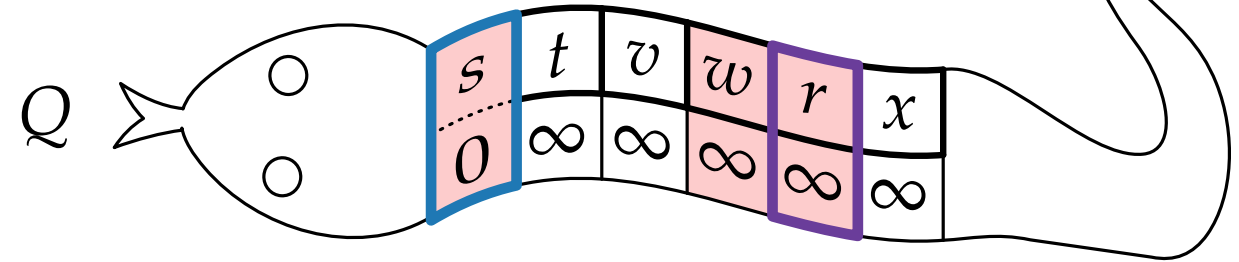
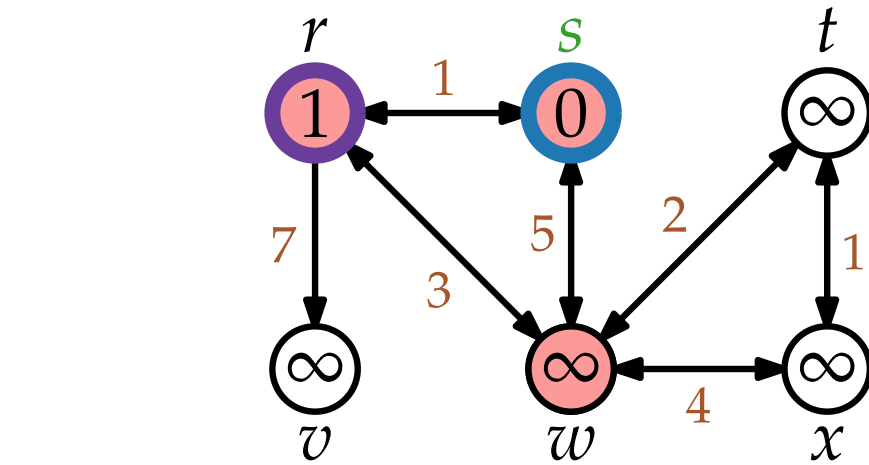
$v.\text{color} = \text{red}$

$v.d = u.d + w(u, v)$

$v.\pi = u$

$Q.\text{DECREASEKEY}(v, v.d)$

$u.\text{color} = \text{blue}$



INITIALIZE(Graph G , Vertex s)

foreach $u \in V$ **do**

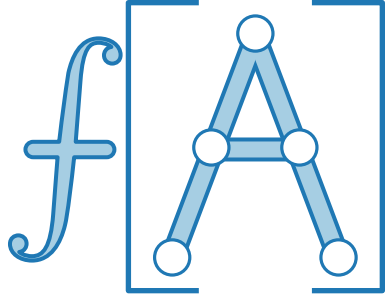
$u.\text{color} = \text{white}$

$u.d = \infty$

$u.\pi = \text{nil}$

$s.\text{color} = \text{red}$

$s.d = 0$



Wiederholung DIJKSTRA



DIJKSTRA(WeightedGraph G , Vertex s)

INITIALIZE(G, s)

$Q = \text{new PriorityQueue}(V, d)$

while not $Q.\text{EMPTY}()$ **do**

$u = Q.\text{EXTRACTMIN}()$

foreach $v \in \text{Adj}[u]$ **do**

if $v.d > u.d + w(u, v)$ **then**

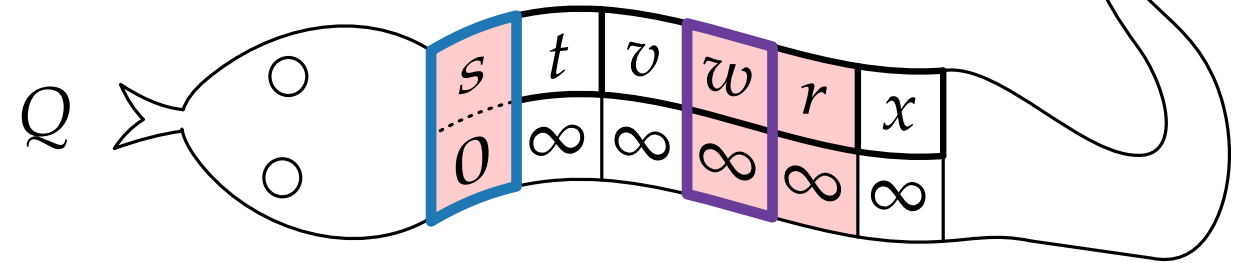
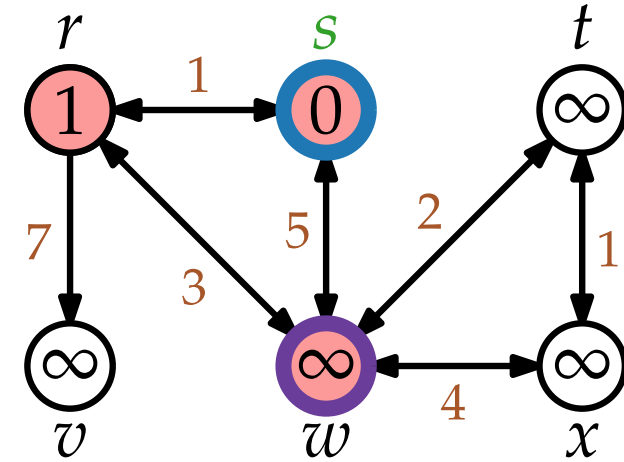
$v.\text{color} = \text{red}$

$v.d = u.d + w(u, v)$

$v.\pi = u$

$Q.\text{DECREASEKEY}(v, v.d)$

$u.\text{color} = \text{blue}$



INITIALIZE(Graph G , Vertex s)

foreach $u \in V$ **do**

$u.\text{color} = \text{white}$

$u.d = \infty$

$u.\pi = \text{nil}$

$s.\text{color} = \text{red}$

$s.d = 0$

Wiederholung DIJKSTRA

DIJKSTRA(WeightedGraph G , Vertex s)

INITIALIZE(G, s)

$Q = \text{new PriorityQueue}(V, d)$

while not $Q.\text{EMPTY}()$ **do**

$u = Q.\text{EXTRACTMIN}()$

foreach $v \in \text{Adj}[u]$ **do**

if $v.d > u.d + w(u, v)$ **then**

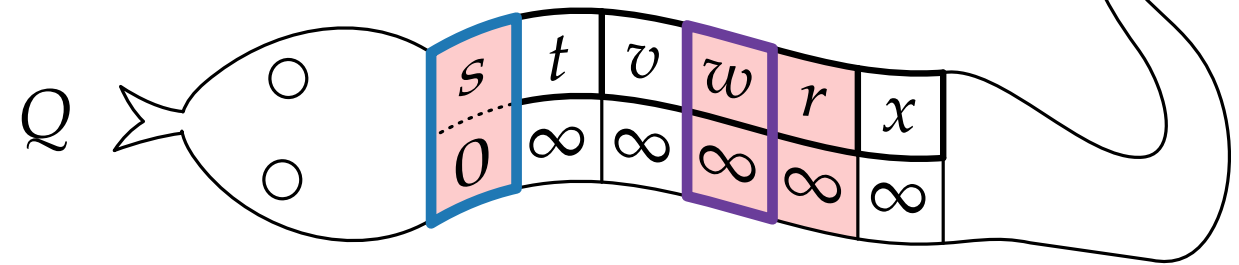
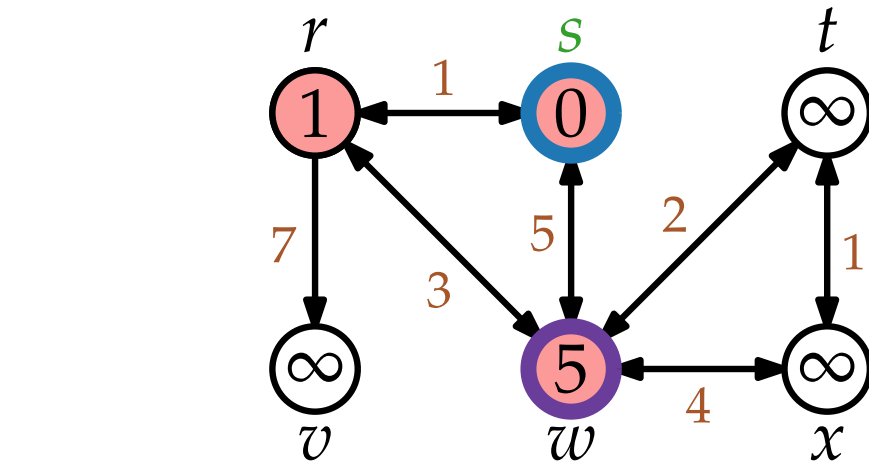
$v.\text{color} = \text{red}$

$v.d = u.d + w(u, v)$

$v.\pi = u$

$Q.\text{DECREASEKEY}(v, v.d)$

$u.\text{color} = \text{blue}$



INITIALIZE(Graph G , Vertex s)

foreach $u \in V$ **do**

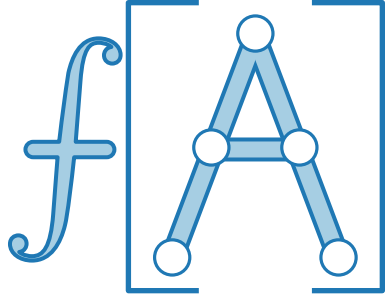
$u.\text{color} = \text{white}$

$u.d = \infty$

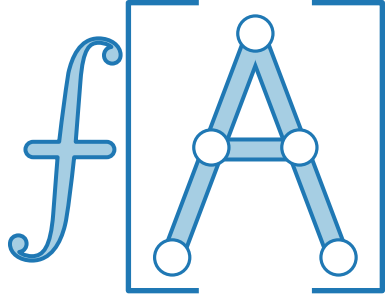
$u.\pi = \text{nil}$

$s.\text{color} = \text{red}$

$s.d = 0$



Wiederholung DIJKSTRA



DIJKSTRA(WeightedGraph G , Vertex s)

INITIALIZE(G, s)

$Q = \text{new PriorityQueue}(V, d)$

while not $Q.\text{EMPTY}()$ **do**

$u = Q.\text{EXTRACTMIN}()$

foreach $v \in \text{Adj}[u]$ **do**

if $v.d > u.d + w(u, v)$ **then**

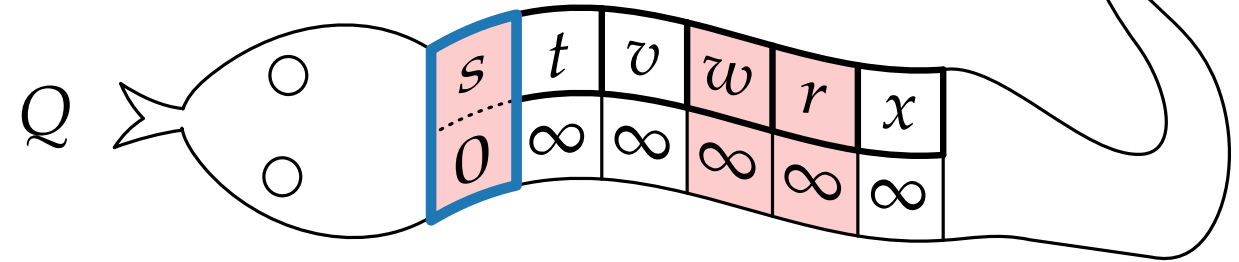
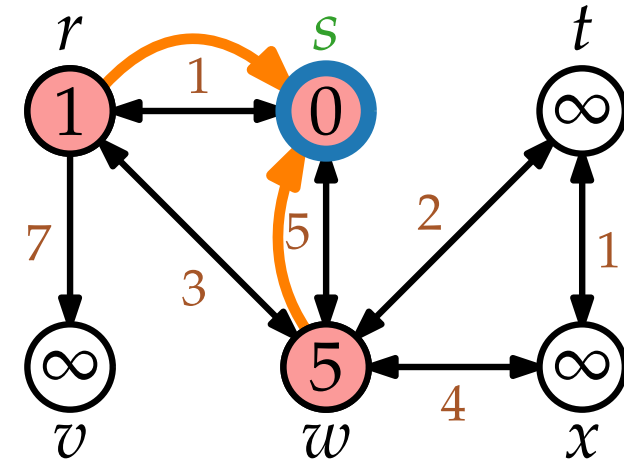
$v.\text{color} = \text{red}$

$v.d = u.d + w(u, v)$

$v.\pi = u$

$Q.\text{DECREASEKEY}(v, v.d)$

$u.\text{color} = \text{blue}$



INITIALIZE(Graph G , Vertex s)

foreach $u \in V$ **do**

$u.\text{color} = \text{white}$

$u.d = \infty$

$u.\pi = \text{nil}$

$s.\text{color} = \text{red}$

$s.d = 0$

Wiederholung DIJKSTRA

DIJKSTRA(WeightedGraph G , Vertex s)

INITIALIZE(G, s)

$Q = \text{new PriorityQueue}(V, d)$

while not $Q.\text{EMPTY}()$ **do**

$u = Q.\text{EXTRACTMIN}()$

foreach $v \in \text{Adj}[u]$ **do**

if $v.d > u.d + w(u, v)$ **then**

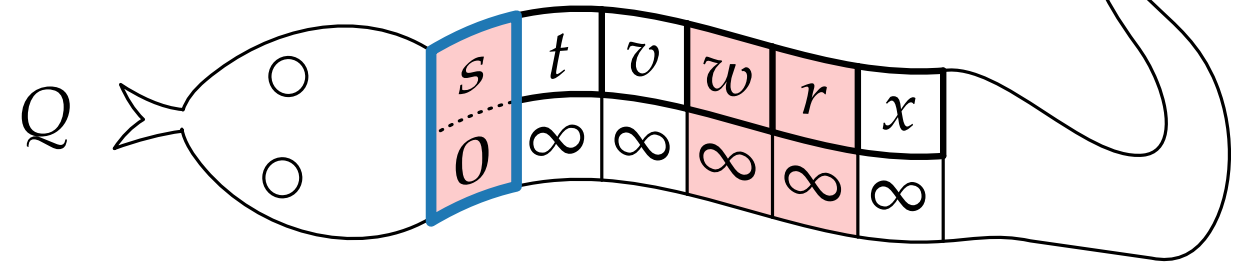
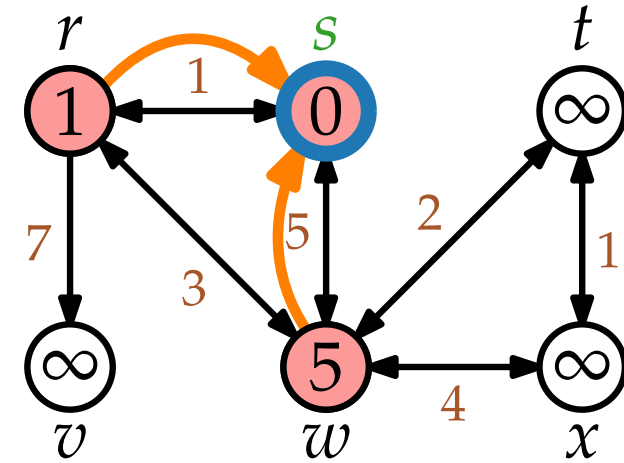
$v.\text{color} = \text{red}$

$v.d = u.d + w(u, v)$

$v.\pi = u$

$Q.\text{DECREASEKEY}(v, v.d)$

$u.\text{color} = \text{blue}$



INITIALIZE(Graph G , Vertex s)

foreach $u \in V$ **do**

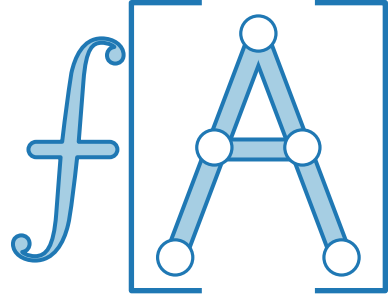
$u.\text{color} = \text{white}$

$u.d = \infty$

$u.\pi = \text{nil}$

$s.\text{color} = \text{red}$

$s.d = 0$



Wiederholung DIJKSTRA



DIJKSTRA(WeightedGraph G , Vertex s)

INITIALIZE(G, s)

$Q = \text{new PriorityQueue}(V, d)$

while not $Q.\text{EMPTY}()$ **do**

$u = Q.\text{EXTRACTMIN}()$

foreach $v \in \text{Adj}[u]$ **do**

if $v.d > u.d + w(u, v)$ **then**

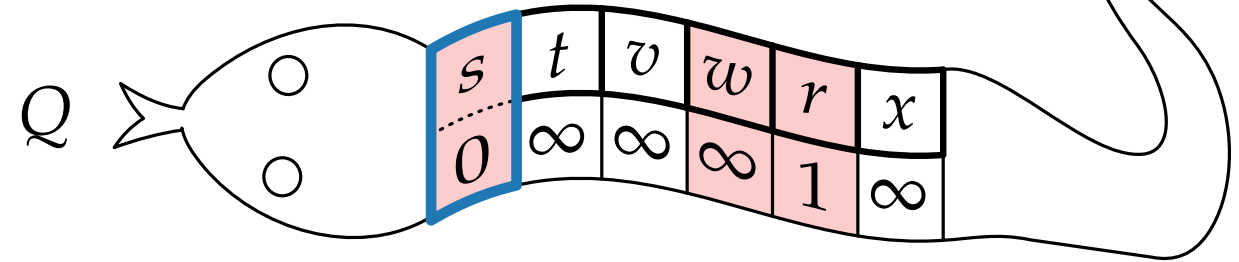
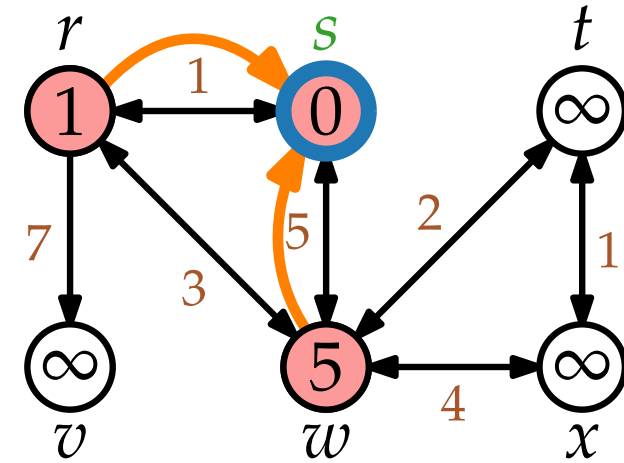
$v.\text{color} = \text{red}$

$v.d = u.d + w(u, v)$

$v.\pi = u$

$Q.\text{DECREASEKEY}(v, v.d)$

$u.\text{color} = \text{blue}$



INITIALIZE(Graph G , Vertex s)

foreach $u \in V$ **do**

$u.\text{color} = \text{white}$

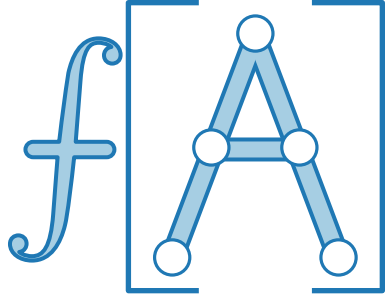
$u.d = \infty$

$u.\pi = \text{nil}$

$s.\text{color} = \text{red}$

$s.d = 0$

Wiederholung DIJKSTRA



DIJKSTRA(WeightedGraph G , Vertex s)

INITIALIZE(G, s)

$Q = \text{new PriorityQueue}(V, d)$

while not $Q.\text{EMPTY}()$ **do**

$u = Q.\text{EXTRACTMIN}()$

foreach $v \in \text{Adj}[u]$ **do**

if $v.d > u.d + w(u, v)$ **then**

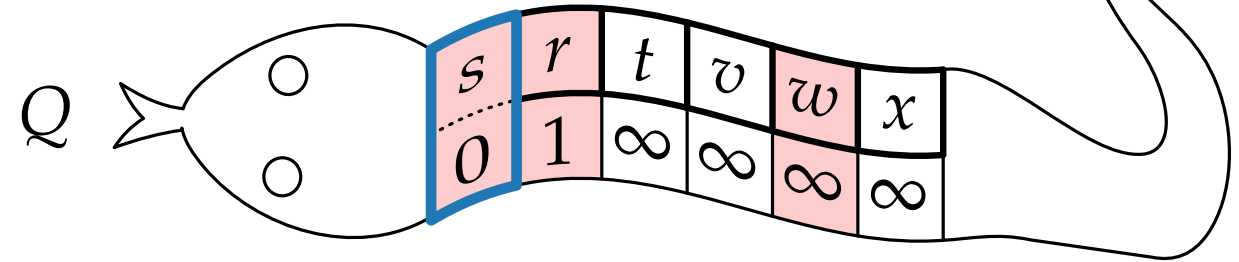
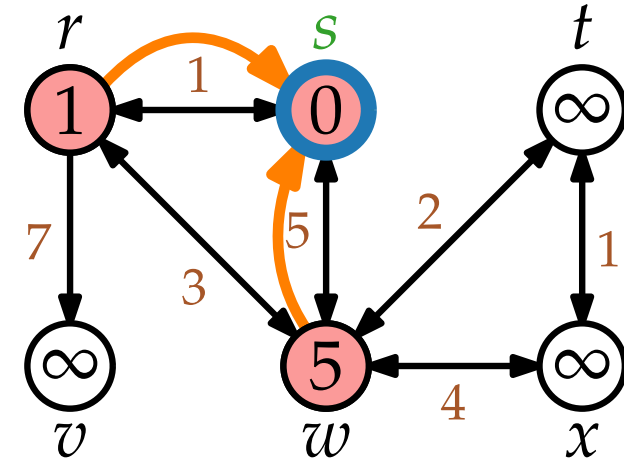
$v.\text{color} = \text{red}$

$v.d = u.d + w(u, v)$

$v.\pi = u$

$Q.\text{DECREASEKEY}(v, v.d)$

$u.\text{color} = \text{blue}$



INITIALIZE(Graph G , Vertex s)

foreach $u \in V$ **do**

$u.\text{color} = \text{white}$

$u.d = \infty$

$u.\pi = \text{nil}$

$s.\text{color} = \text{red}$

$s.d = 0$

Wiederholung DIJKSTRA



DIJKSTRA(WeightedGraph G , Vertex s)

INITIALIZE(G, s)

$Q = \text{new PriorityQueue}(V, d)$

while not $Q.\text{EMPTY}()$ **do**

$u = Q.\text{EXTRACTMIN}()$

foreach $v \in \text{Adj}[u]$ **do**

if $v.d > u.d + w(u, v)$ **then**

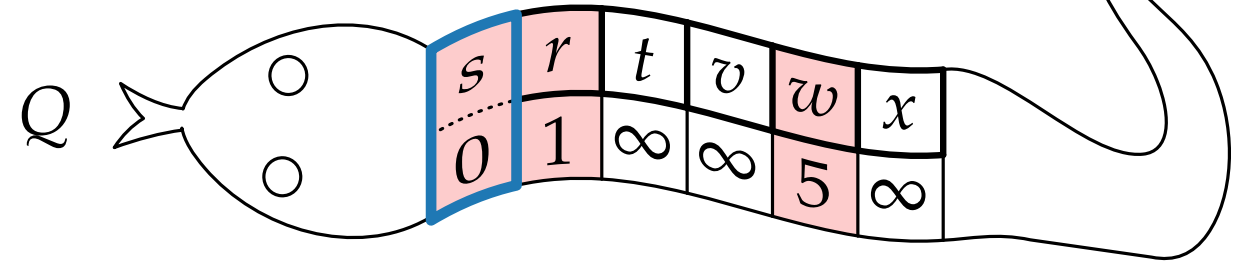
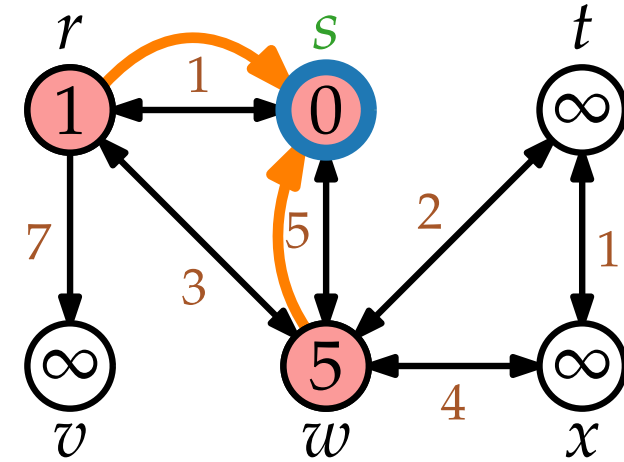
$v.\text{color} = \text{red}$

$v.d = u.d + w(u, v)$

$v.\pi = u$

$Q.\text{DECREASEKEY}(v, v.d)$

$u.\text{color} = \text{blue}$



INITIALIZE(Graph G , Vertex s)

foreach $u \in V$ **do**

$u.\text{color} = \text{white}$

$u.d = \infty$

$u.\pi = \text{nil}$

$s.\text{color} = \text{red}$

$s.d = 0$

Wiederholung DIJKSTRA

DIJKSTRA(WeightedGraph G , Vertex s)

INITIALIZE(G, s)

$Q = \text{new PriorityQueue}(V, d)$

while not $Q.\text{EMPTY}()$ **do**

$u = Q.\text{EXTRACTMIN}()$

foreach $v \in \text{Adj}[u]$ **do**

if $v.d > u.d + w(u, v)$ **then**

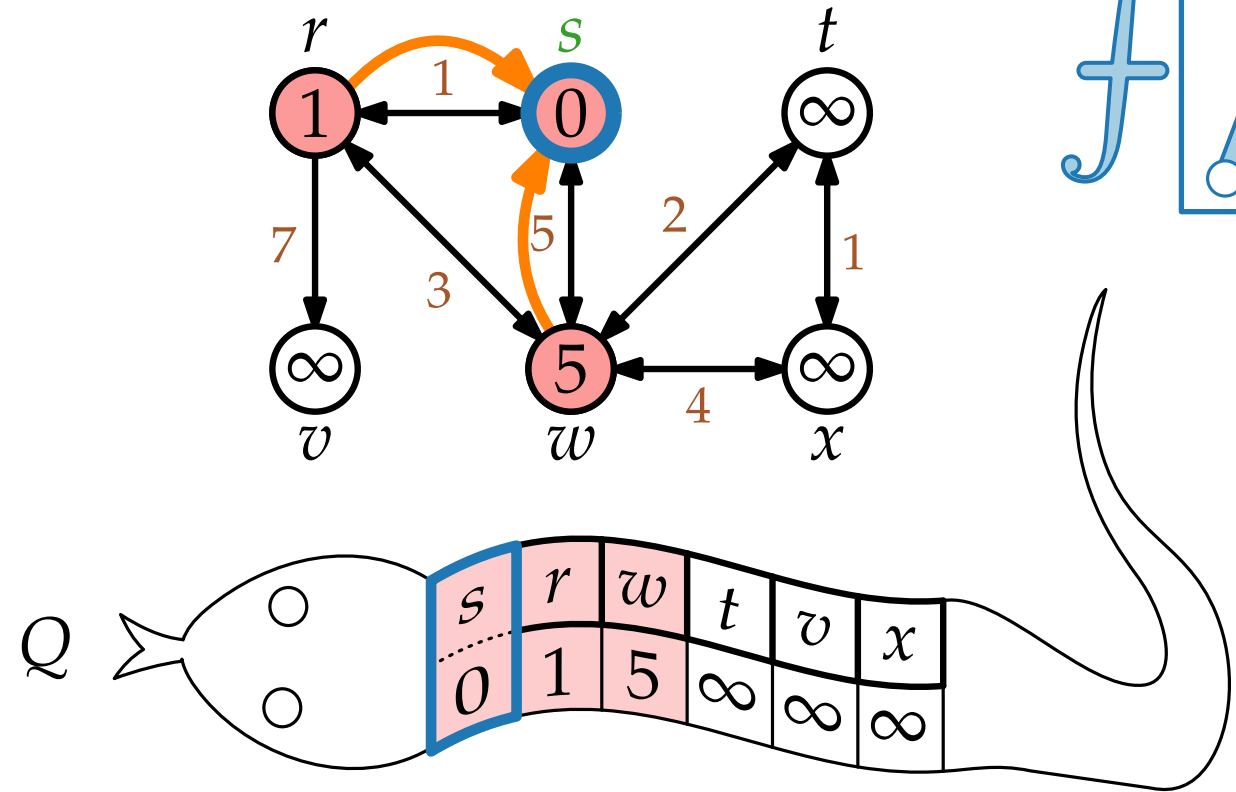
$v.\text{color} = \text{red}$

$v.d = u.d + w(u, v)$

$v.\pi = u$

$Q.\text{DECREASEKEY}(v, v.d)$

$u.\text{color} = \text{blue}$



INITIALIZE(Graph G , Vertex s)

foreach $u \in V$ **do**

$u.\text{color} = \text{white}$

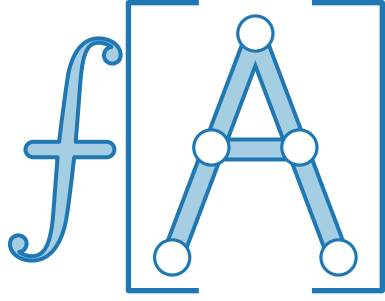
$u.d = \infty$

$u.\pi = \text{nil}$

$s.\text{color} = \text{red}$

$s.d = 0$

Wiederholung DIJKSTRA



DIJKSTRA(WeightedGraph G , Vertex s)

INITIALIZE(G, s)

$Q = \text{new PriorityQueue}(V, d)$

while not $Q.\text{EMPTY}()$ **do**

$u = Q.\text{EXTRACTMIN}()$

foreach $v \in \text{Adj}[u]$ **do**

if $v.d > u.d + w(u, v)$ **then**

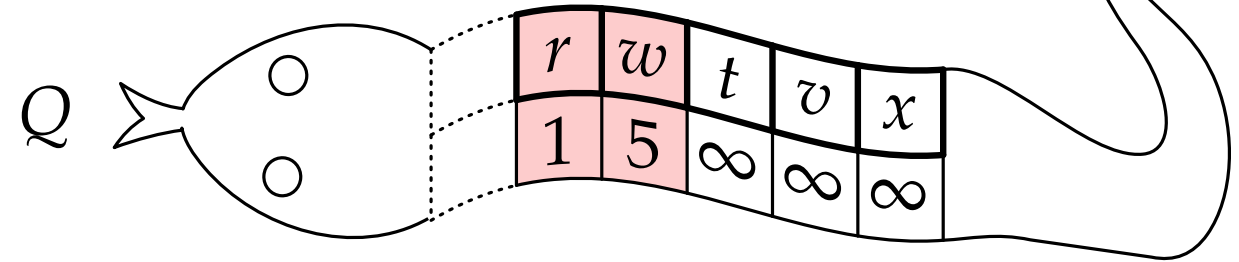
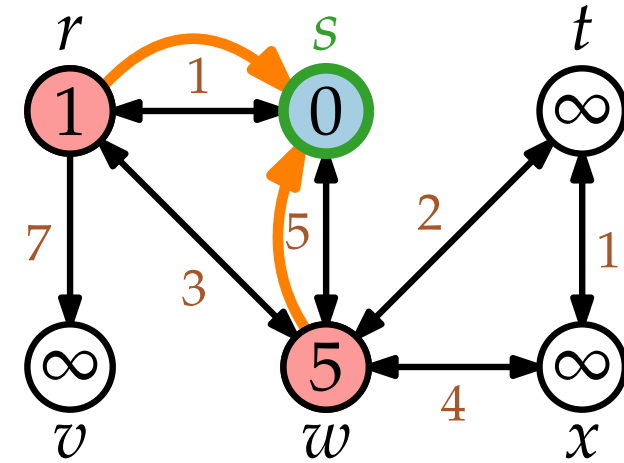
$v.\text{color} = \text{red}$

$v.d = u.d + w(u, v)$

$v.\pi = u$

$Q.\text{DECREASEKEY}(v, v.d)$

$u.\text{color} = \text{blue}$



INITIALIZE(Graph G , Vertex s)

foreach $u \in V$ **do**

$u.\text{color} = \text{white}$

$u.d = \infty$

$u.\pi = \text{nil}$

$s.\text{color} = \text{red}$

$s.d = 0$

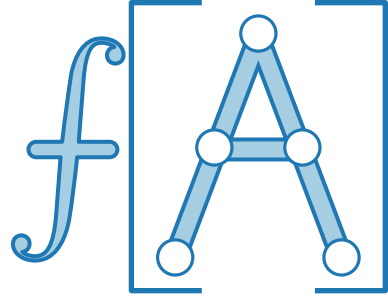
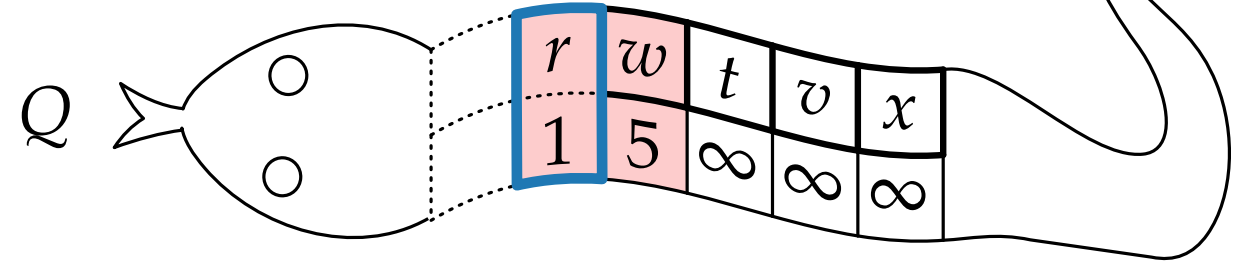
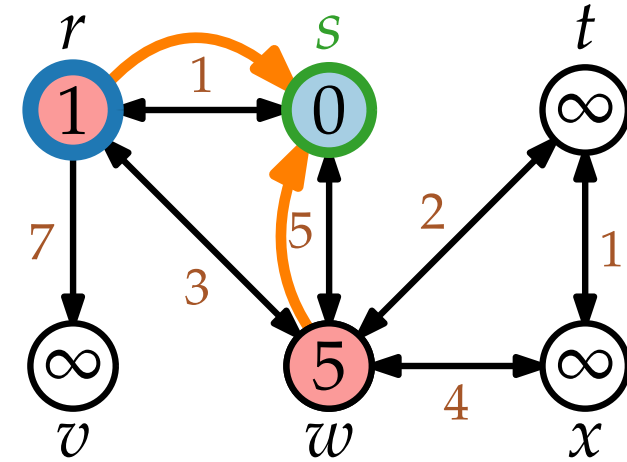
Wiederholung DIJKSTRA

```

DIJKSTRA(WeightedGraph G, Vertex  $s$ )
  INITIALIZE( $G, s$ )
   $Q = \text{new PriorityQueue}(V, d)$ 
  while not  $Q.\text{EMPTY}()$  do
     $u = Q.\text{EXTRACTMIN}()$ 
    foreach  $v \in \text{Adj}[u]$  do
      if  $v.d > u.d + w(u, v)$  then
         $v.\text{color} = \text{red}$ 
         $v.d = u.d + w(u, v)$ 
         $v.\pi = u$ 
         $Q.\text{DECREASEKEY}(v, v.d)$ 
     $u.\text{color} = \text{blue}$ 
  
```

```

INITIALIZE(Graph G, Vertex  $s$ )
  foreach  $u \in V$  do
     $u.\text{color} = \text{white}$ 
     $u.d = \infty$ 
     $u.\pi = \text{nil}$ 
   $s.\text{color} = \text{red}$ 
   $s.d = 0$ 
  
```



Wiederholung DIJKSTRA

DIJKSTRA(WeightedGraph G , Vertex s)

INITIALIZE(G, s)

$Q = \text{new PriorityQueue}(V, d)$

while not $Q.\text{EMPTY}()$ **do**

$u = Q.\text{EXTRACTMIN}()$

foreach $v \in \text{Adj}[u]$ **do**

if $v.d > u.d + w(u, v)$ **then**

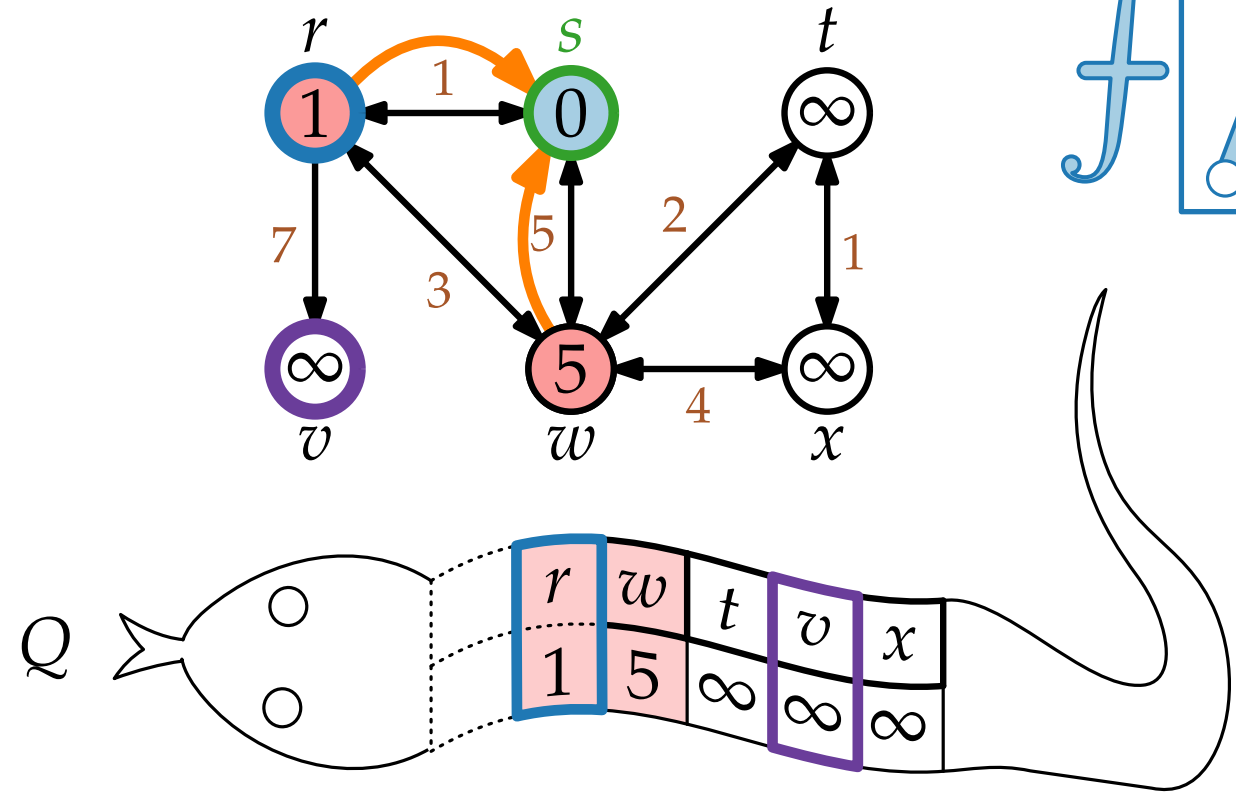
$v.\text{color} = \text{red}$

$v.d = u.d + w(u, v)$

$v.\pi = u$

$Q.\text{DECREASEKEY}(v, v.d)$

$u.\text{color} = \text{blue}$



INITIALIZE(Graph G , Vertex s)

foreach $u \in V$ **do**

$u.\text{color} = \text{white}$

$u.d = \infty$

$u.\pi = \text{nil}$

$s.\text{color} = \text{red}$

$s.d = 0$

Wiederholung DIJKSTRA

DIJKSTRA(WeightedGraph G , Vertex s)

INITIALIZE(G, s)

$Q = \text{new PriorityQueue}(V, d)$

while not $Q.\text{EMPTY}()$ **do**

$u = Q.\text{EXTRACTMIN}()$

foreach $v \in \text{Adj}[u]$ **do**

if $v.d > u.d + w(u, v)$ **then**

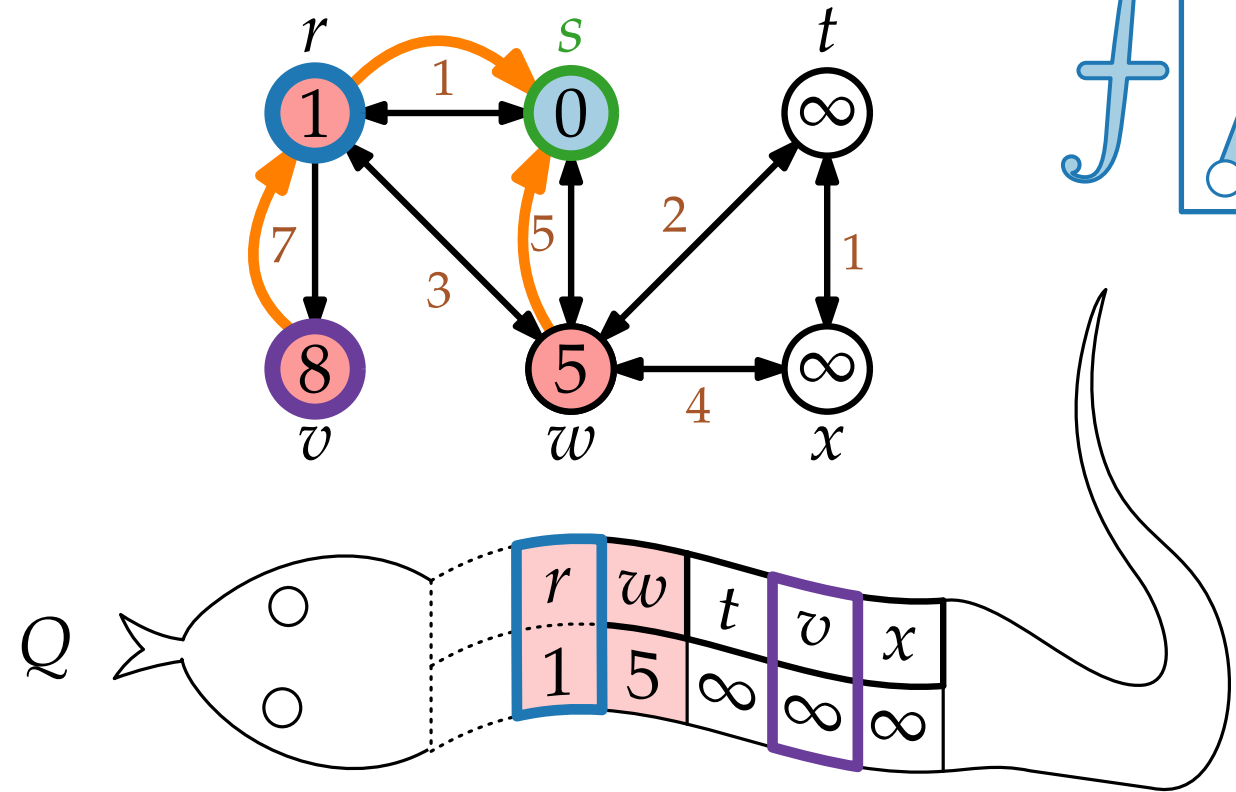
$v.\text{color} = \text{red}$

$v.d = u.d + w(u, v)$

$v.\pi = u$

$Q.\text{DECREASEKEY}(v, v.d)$

$u.\text{color} = \text{blue}$



INITIALIZE(Graph G , Vertex s)

foreach $u \in V$ **do**

$u.\text{color} = \text{white}$

$u.d = \infty$

$u.\pi = \text{nil}$

$s.\text{color} = \text{red}$

$s.d = 0$

Wiederholung DIJKSTRA

DIJKSTRA(WeightedGraph G , Vertex s)

INITIALIZE(G, s)

$Q = \text{new PriorityQueue}(V, d)$

while not $Q.\text{EMPTY}()$ do

$u = Q.\text{EXTRACTMIN}()$

 foreach $v \in \text{Adj}[u]$ do

 if $v.d > u.d + w(u, v)$ then

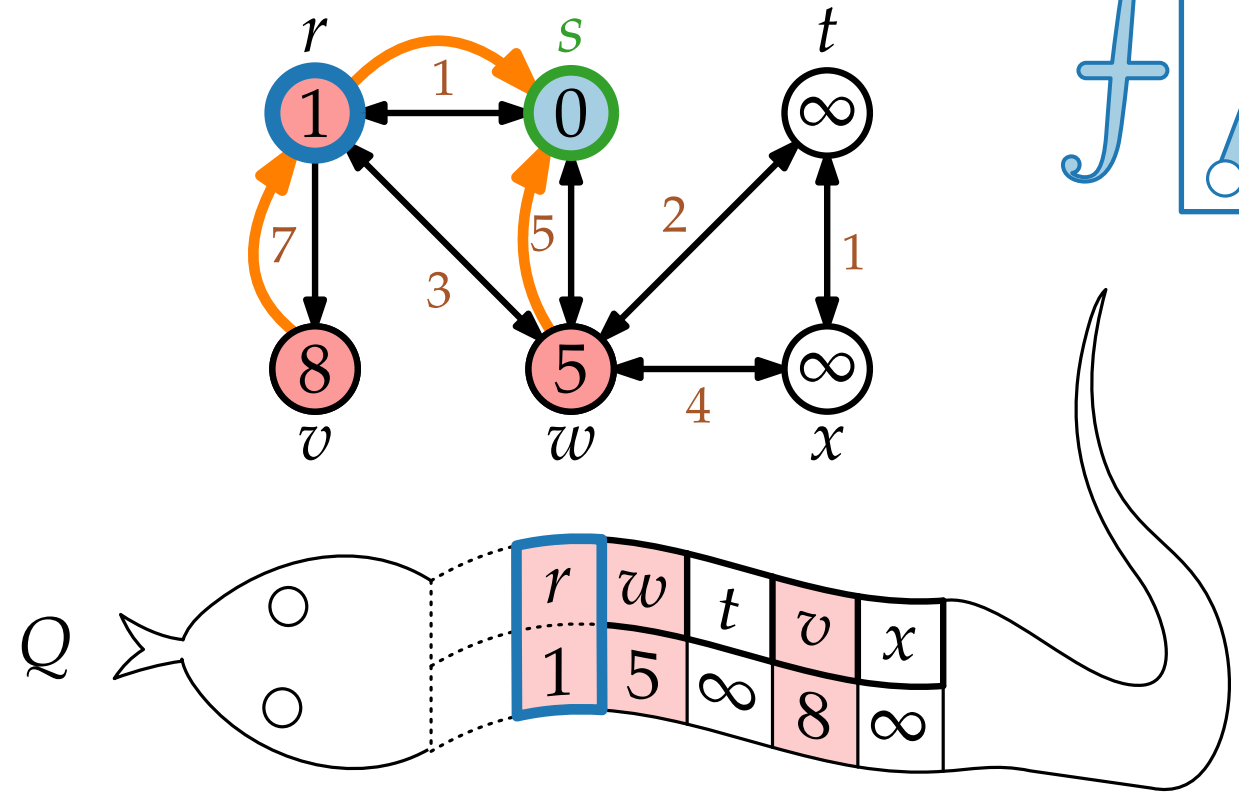
$v.\text{color} = \text{red}$

$v.d = u.d + w(u, v)$

$v.\pi = u$

$Q.\text{DECREASEKEY}(v, v.d)$

$u.\text{color} = \text{blue}$



INITIALIZE(Graph G , Vertex s)

foreach $u \in V$ do

$u.\text{color} = \text{white}$

$u.d = \infty$

$u.\pi = \text{nil}$

$s.\text{color} = \text{red}$

$s.d = 0$

Wiederholung DIJKSTRA

DIJKSTRA(WeightedGraph G , Vertex s)

INITIALIZE(G, s)

$Q = \text{new PriorityQueue}(V, d)$

while not $Q.\text{EMPTY}()$ do

$u = Q.\text{EXTRACTMIN}()$

 foreach $v \in \text{Adj}[u]$ do

 if $v.d > u.d + w(u, v)$ then

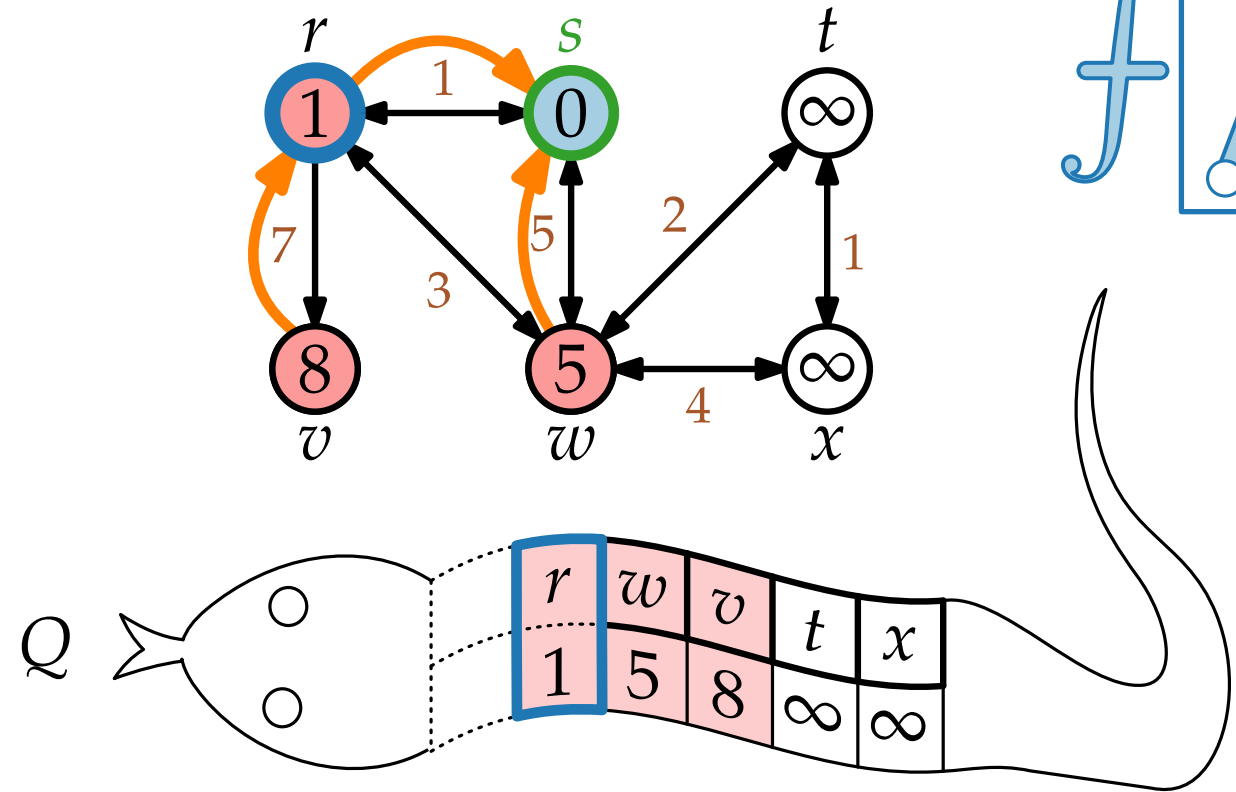
$v.\text{color} = \text{red}$

$v.d = u.d + w(u, v)$

$v.\pi = u$

$Q.\text{DECREASEKEY}(v, v.d)$

$u.\text{color} = \text{blue}$



INITIALIZE(Graph G , Vertex s)

foreach $u \in V$ do

$u.\text{color} = \text{white}$

$u.d = \infty$

$u.\pi = \text{nil}$

$s.\text{color} = \text{red}$

$s.d = 0$

Wiederholung DIJKSTRA

DIJKSTRA(WeightedGraph G , Vertex s)

INITIALIZE(G, s)

$Q = \text{new PriorityQueue}(V, d)$

while not $Q.\text{EMPTY}()$ **do**

$u = Q.\text{EXTRACTMIN}()$

foreach $v \in \text{Adj}[u]$ **do**

if $v.d > u.d + w(u, v)$ **then**

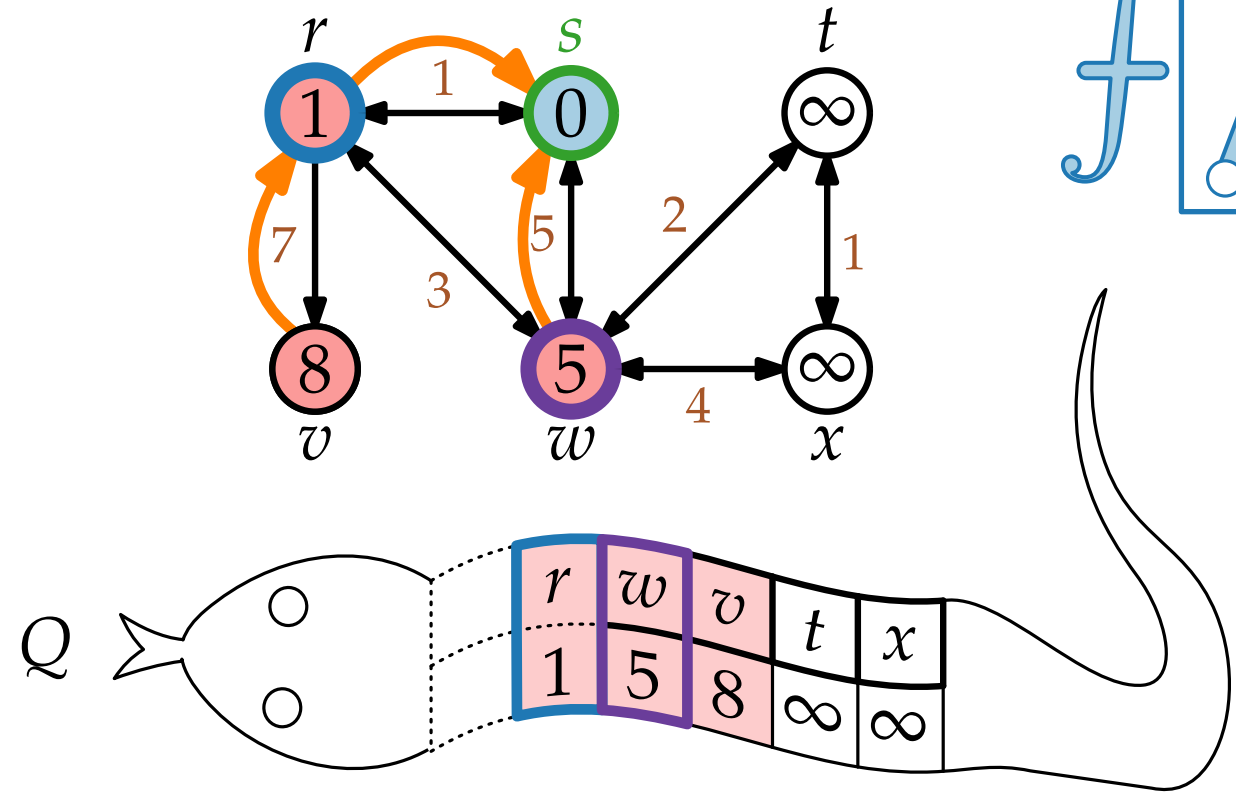
$v.\text{color} = \text{red}$

$v.d = u.d + w(u, v)$

$v.\pi = u$

$Q.\text{DECREASEKEY}(v, v.d)$

$u.\text{color} = \text{blue}$



INITIALIZE(Graph G , Vertex s)

foreach $u \in V$ **do**

$u.\text{color} = \text{white}$

$u.d = \infty$

$u.\pi = \text{nil}$

$s.\text{color} = \text{red}$

$s.d = 0$

Wiederholung DIJKSTRA

DIJKSTRA(WeightedGraph G , Vertex s)

INITIALIZE(G, s)

$Q = \text{new PriorityQueue}(V, d)$

while not $Q.\text{EMPTY}()$ do

$u = Q.\text{EXTRACTMIN}()$

 foreach $v \in \text{Adj}[u]$ do

 if $v.d > u.d + w(u, v)$ then

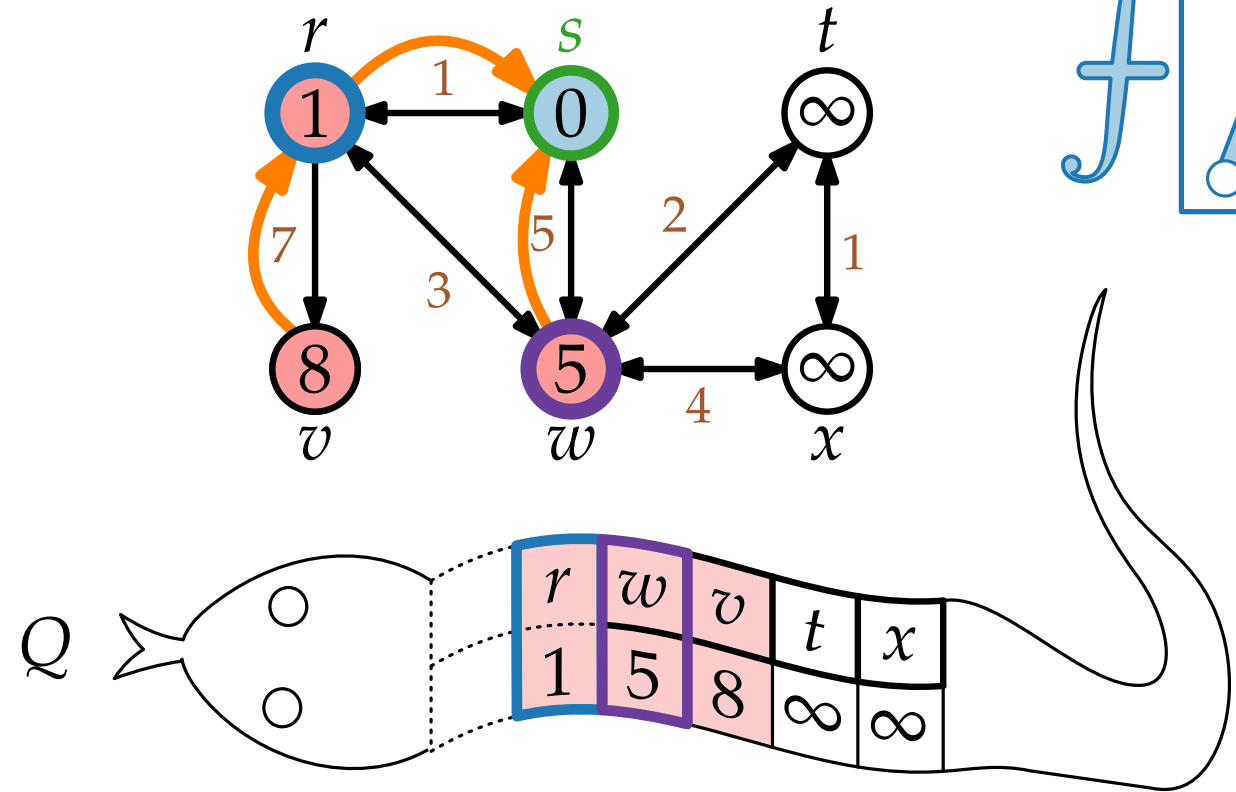
$v.\text{color} = \text{red}$

$v.d = u.d + w(u, v)$

$v.\pi = u$

$Q.\text{DECREASEKEY}(v, v.d)$

$u.\text{color} = \text{blue}$



INITIALIZE(Graph G , Vertex s)

foreach $u \in V$ do

$u.\text{color} = \text{white}$

$u.d = \infty$

$u.\pi = \text{nil}$

$s.\text{color} = \text{red}$

$s.d = 0$

Wiederholung DIJKSTRA

DIJKSTRA(WeightedGraph G , Vertex s)

INITIALIZE(G, s)

$Q = \text{new PriorityQueue}(V, d)$

while not $Q.\text{EMPTY}()$ **do**

$u = Q.\text{EXTRACTMIN}()$

foreach $v \in \text{Adj}[u]$ **do**

if $v.d > u.d + w(u, v)$ **then**

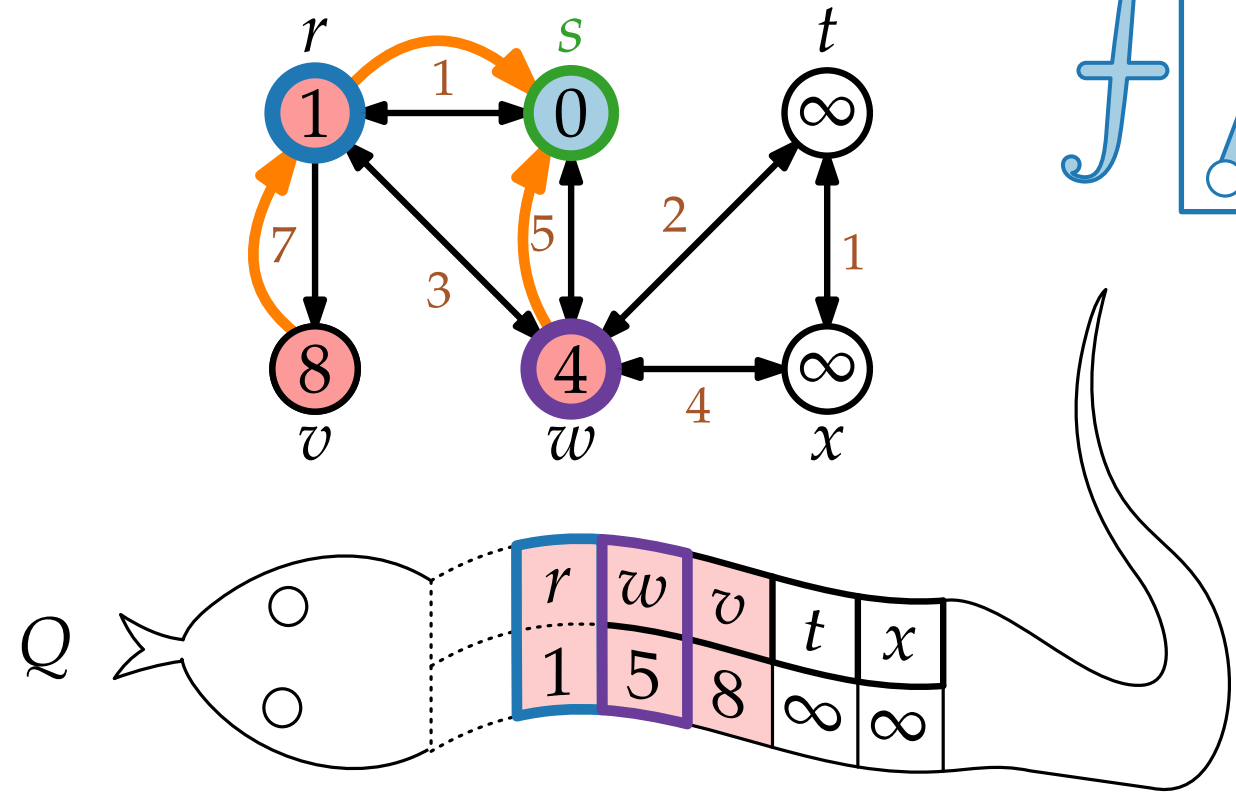
$v.\text{color} = \text{red}$

$v.d = u.d + w(u, v)$

$v.\pi = u$

$Q.\text{DECREASEKEY}(v, v.d)$

$u.\text{color} = \text{blue}$



INITIALIZE(Graph G , Vertex s)

foreach $u \in V$ **do**

$u.\text{color} = \text{white}$

$u.d = \infty$

$u.\pi = \text{nil}$

$s.\text{color} = \text{red}$

$s.d = 0$

Wiederholung DIJKSTRA

DIJKSTRA(WeightedGraph G , Vertex s)

INITIALIZE(G, s)

$Q = \text{new PriorityQueue}(V, d)$

while not $Q.\text{EMPTY}()$ do

$u = Q.\text{EXTRACTMIN}()$

 foreach $v \in \text{Adj}[u]$ do

 if $v.d > u.d + w(u, v)$ then

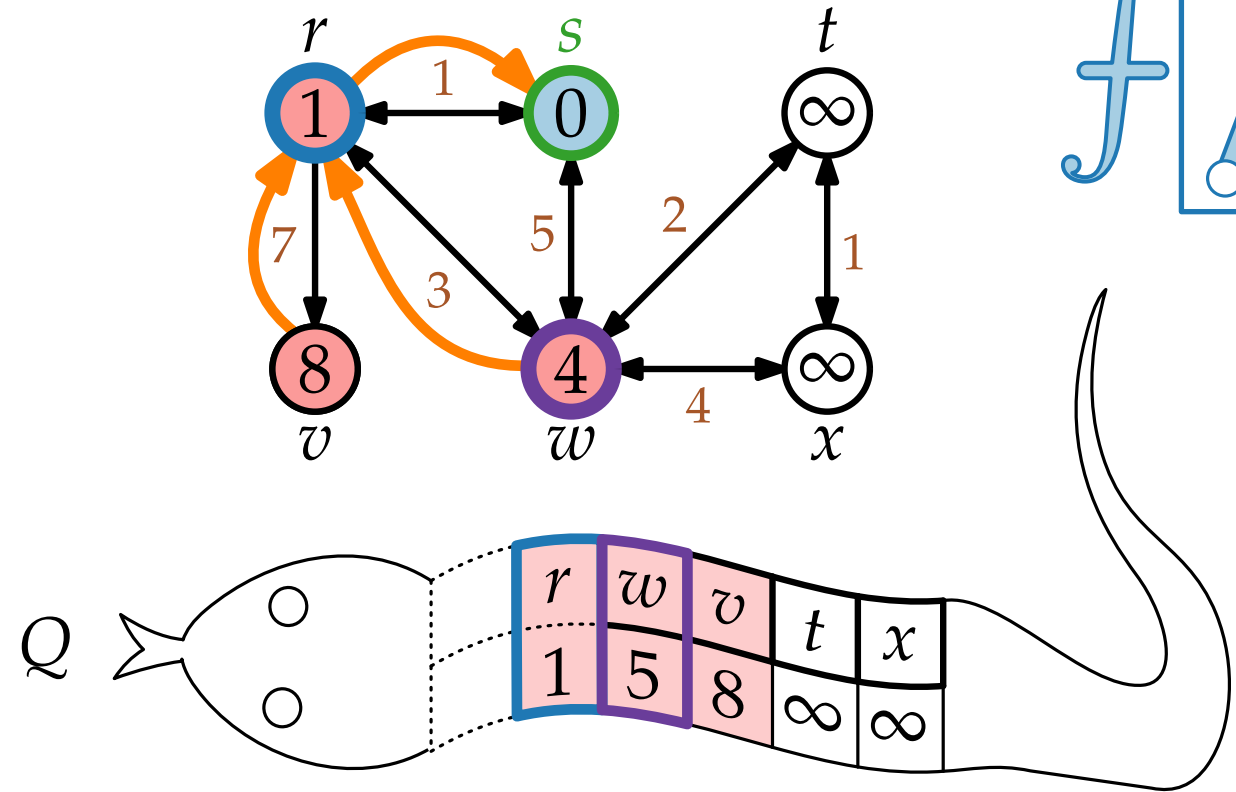
$v.\text{color} = \text{red}$

$v.d = u.d + w(u, v)$

$v.\pi = u$

$Q.\text{DECREASEKEY}(v, v.d)$

$u.\text{color} = \text{blue}$



INITIALIZE(Graph G , Vertex s)

foreach $u \in V$ do

$u.\text{color} = \text{white}$

$u.d = \infty$

$u.\pi = \text{nil}$

$s.\text{color} = \text{red}$

$s.d = 0$

Wiederholung DIJKSTRA

DIJKSTRA(WeightedGraph G , Vertex s)

INITIALIZE(G, s)

$Q = \text{new PriorityQueue}(V, d)$

while not $Q.\text{EMPTY}()$ **do**

$u = Q.\text{EXTRACTMIN}()$

foreach $v \in \text{Adj}[u]$ **do**

if $v.d > u.d + w(u, v)$ **then**

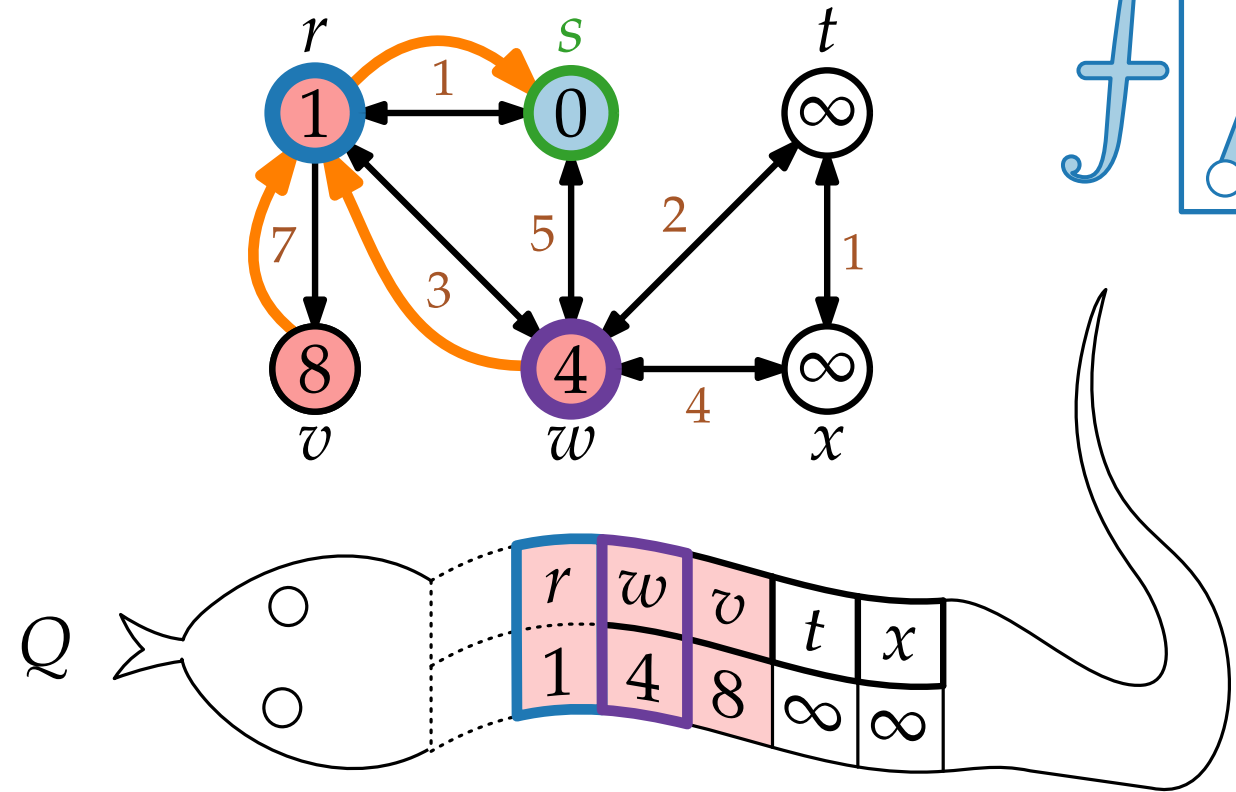
$v.\text{color} = \text{red}$

$v.d = u.d + w(u, v)$

$v.\pi = u$

$Q.\text{DECREASEKEY}(v, v.d)$

$u.\text{color} = \text{blue}$



INITIALIZE(Graph G , Vertex s)

foreach $u \in V$ **do**

$u.\text{color} = \text{white}$

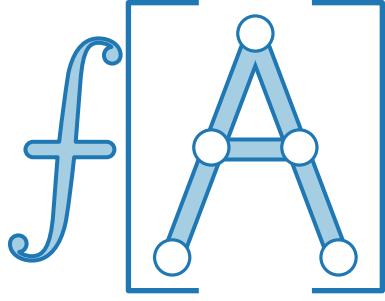
$u.d = \infty$

$u.\pi = \text{nil}$

$s.\text{color} = \text{red}$

$s.d = 0$

Wiederholung DIJKSTRA



DIJKSTRA(WeightedGraph G , Vertex s)

INITIALIZE(G, s)

$Q = \text{new PriorityQueue}(V, d)$

while not $Q.\text{EMPTY}()$ **do**

$u = Q.\text{EXTRACTMIN}()$

foreach $v \in \text{Adj}[u]$ **do**

if $v.d > u.d + w(u, v)$ **then**

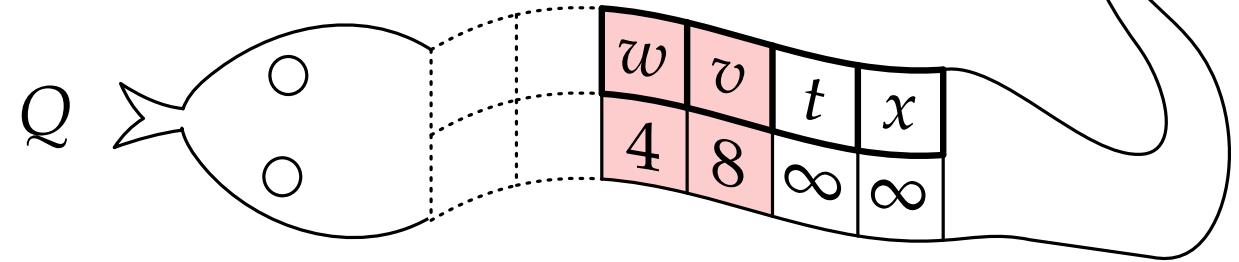
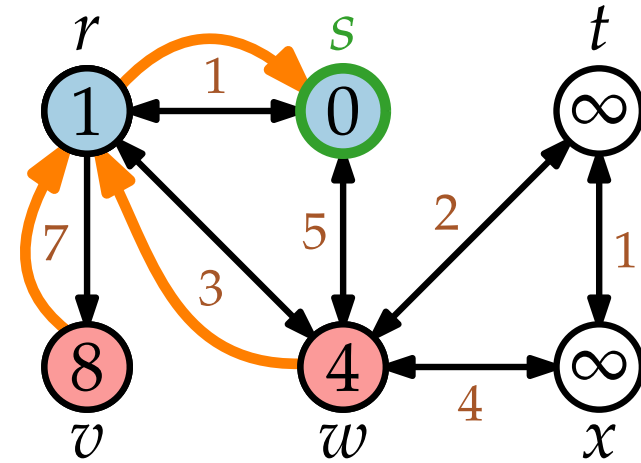
$v.\text{color} = \text{red}$

$v.d = u.d + w(u, v)$

$v.\pi = u$

$Q.\text{DECREASEKEY}(v, v.d)$

$u.\text{color} = \text{blue}$



INITIALIZE(Graph G , Vertex s)

foreach $u \in V$ **do**

$u.\text{color} = \text{white}$

$u.d = \infty$

$u.\pi = \text{nil}$

$s.\text{color} = \text{red}$

$s.d = 0$

Wiederholung DIJKSTRA

DIJKSTRA(WeightedGraph G , Vertex s)

INITIALIZE(G, s)

$Q = \text{new PriorityQueue}(V, d)$

while not $Q.\text{EMPTY}()$ do

$u = Q.\text{EXTRACTMIN}()$

 foreach $v \in \text{Adj}[u]$ do

 if $v.d > u.d + w(u, v)$ then

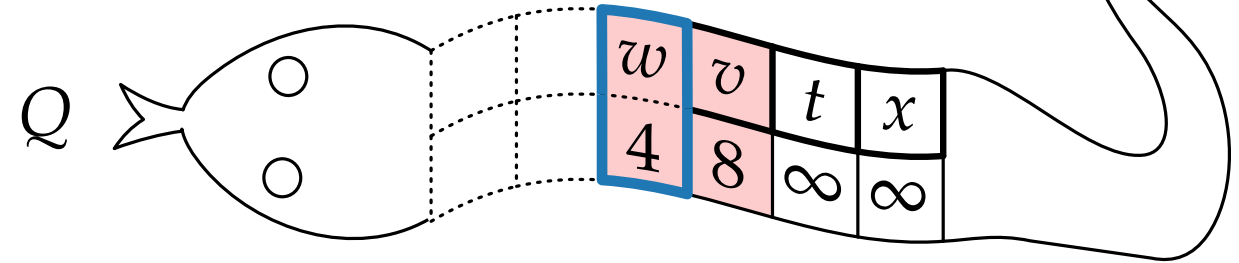
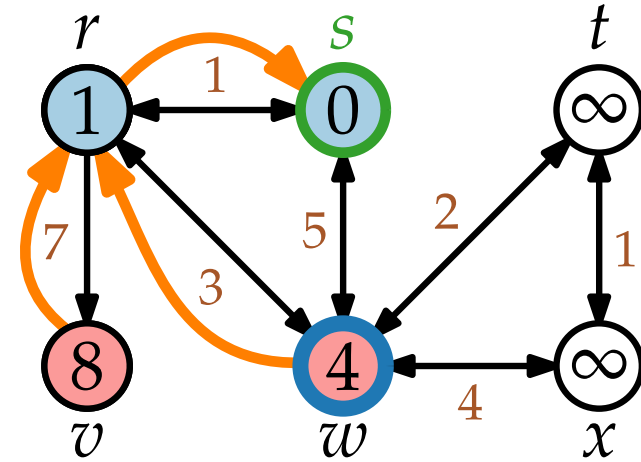
$v.\text{color} = \text{red}$

$v.d = u.d + w(u, v)$

$v.\pi = u$

$Q.\text{DECREASEKEY}(v, v.d)$

$u.\text{color} = \text{blue}$



INITIALIZE(Graph G , Vertex s)

foreach $u \in V$ do

$u.\text{color} = \text{white}$

$u.d = \infty$

$u.\pi = \text{nil}$

$s.\text{color} = \text{red}$

$s.d = 0$

Wiederholung DIJKSTRA

DIJKSTRA(WeightedGraph G , Vertex s)

INITIALIZE(G, s)

$Q = \text{new PriorityQueue}(V, d)$

while not $Q.\text{EMPTY}()$ do

$u = Q.\text{EXTRACTMIN}()$

 foreach $v \in \text{Adj}[u]$ do

 if $v.d > u.d + w(u, v)$ then

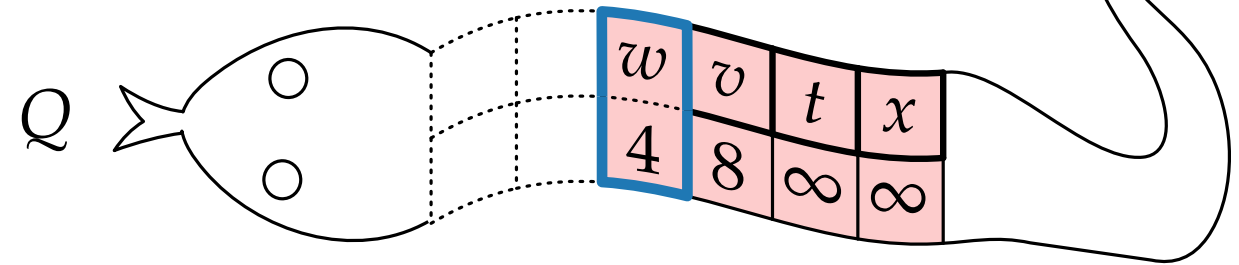
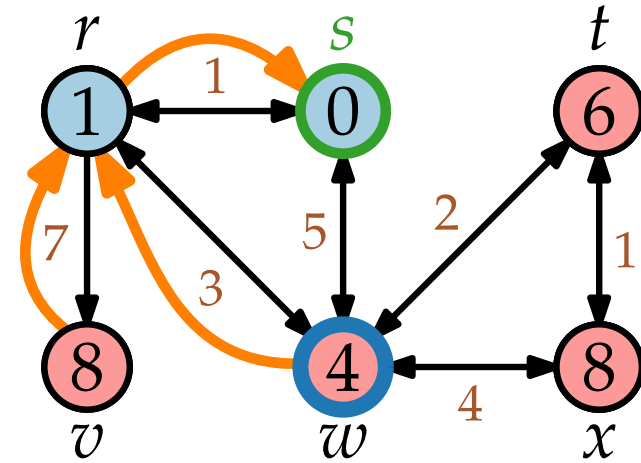
$v.\text{color} = \text{red}$

$v.d = u.d + w(u, v)$

$v.\pi = u$

$Q.\text{DECREASEKEY}(v, v.d)$

$u.\text{color} = \text{blue}$



INITIALIZE(Graph G , Vertex s)

foreach $u \in V$ do

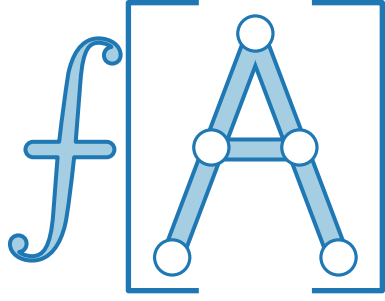
$u.\text{color} = \text{white}$

$u.d = \infty$

$u.\pi = \text{nil}$

$s.\text{color} = \text{red}$

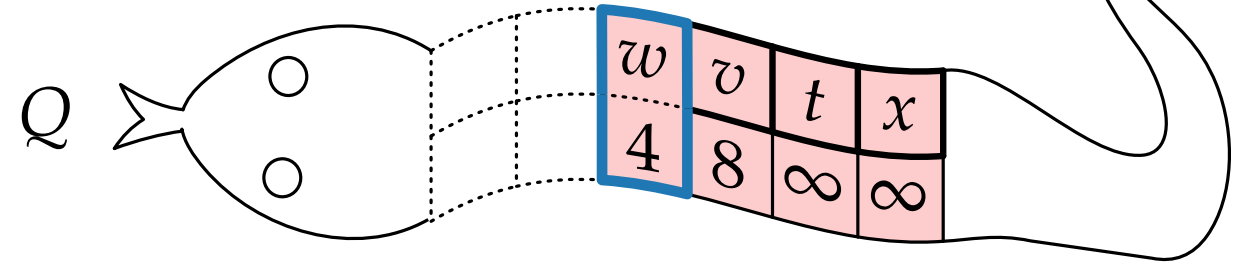
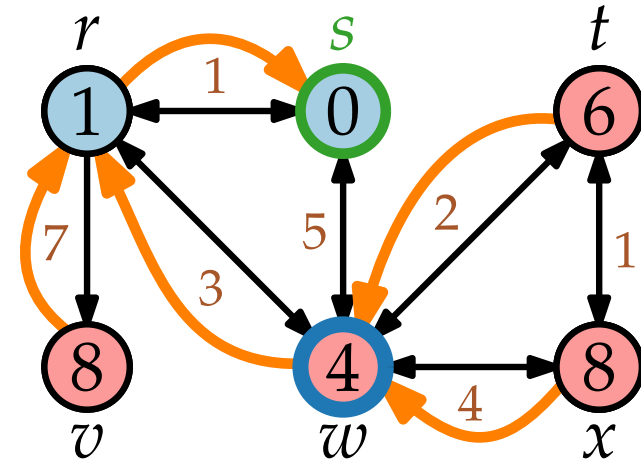
$s.d = 0$



Wiederholung DIJKSTRA

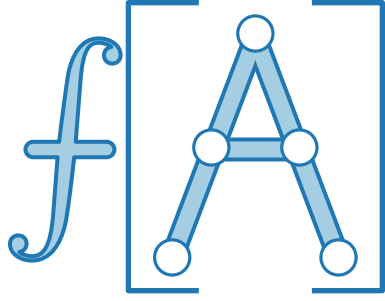
```

DIJKSTRA(WeightedGraph G, Vertex  $s$ )
  INITIALIZE( $G, s$ )
   $Q = \text{new PriorityQueue}(V, d)$ 
  while not  $Q.\text{EMPTY}()$  do
     $u = Q.\text{EXTRACTMIN}()$ 
    foreach  $v \in \text{Adj}[u]$  do
      if  $v.d > u.d + w(u, v)$  then
         $v.\text{color} = \text{red}$ 
         $v.d = u.d + w(u, v)$ 
         $v.\pi = u$ 
         $Q.\text{DECREASEKEY}(v, v.d)$ 
     $u.\text{color} = \text{blue}$ 
  
```



```

INITIALIZE(Graph G, Vertex  $s$ )
  foreach  $u \in V$  do
     $u.\text{color} = \text{white}$ 
     $u.d = \infty$ 
     $u.\pi = \text{nil}$ 
   $s.\text{color} = \text{red}$ 
   $s.d = 0$ 
  
```



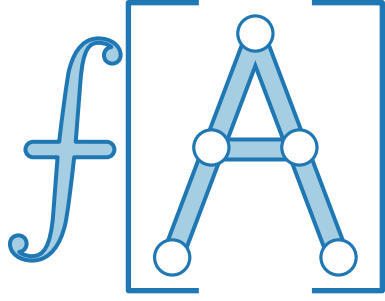
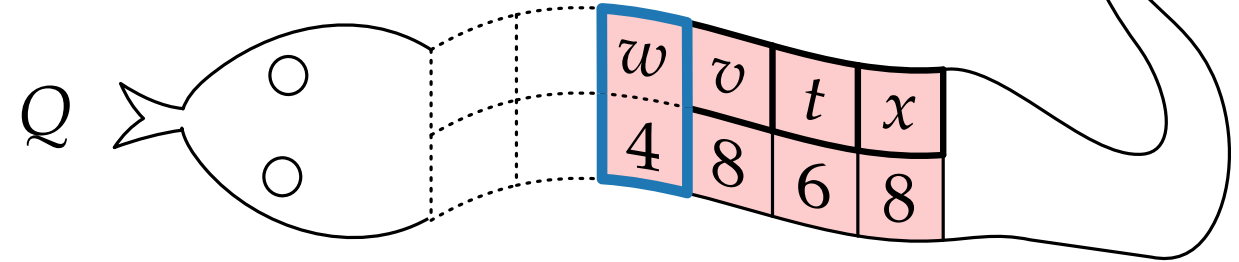
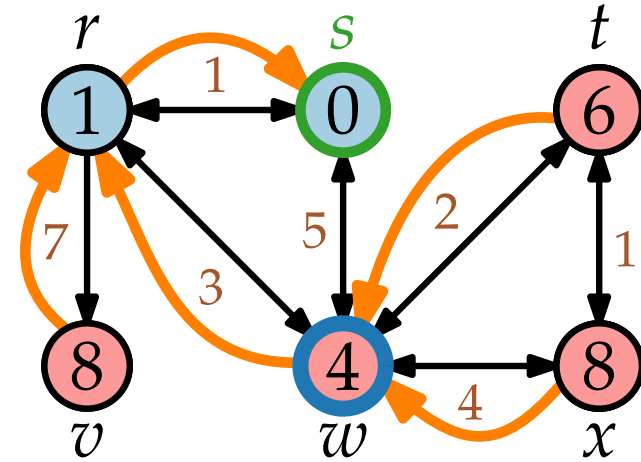
Wiederholung DIJKSTRA

```

DIJKSTRA(WeightedGraph G, Vertex  $s$ )
  INITIALIZE( $G, s$ )
   $Q = \text{new PriorityQueue}(V, d)$ 
  while not  $Q.\text{EMPTY}()$  do
     $u = Q.\text{EXTRACTMIN}()$ 
    foreach  $v \in \text{Adj}[u]$  do
      if  $v.d > u.d + w(u, v)$  then
         $v.\text{color} = \text{red}$ 
         $v.d = u.d + w(u, v)$ 
         $v.\pi = u$ 
         $Q.\text{DECREASEKEY}(v, v.d)$ 
     $u.\text{color} = \text{blue}$ 
  
```

```

INITIALIZE(Graph G, Vertex  $s$ )
  foreach  $u \in V$  do
     $u.\text{color} = \text{white}$ 
     $u.d = \infty$ 
     $u.\pi = \text{nil}$ 
   $s.\text{color} = \text{red}$ 
   $s.d = 0$ 
  
```



Wiederholung DIJKSTRA

DIJKSTRA(WeightedGraph G , Vertex s)

INITIALIZE(G, s)

$Q = \text{new PriorityQueue}(V, d)$

while not $Q.\text{EMPTY}()$ **do**

$u = Q.\text{EXTRACTMIN}()$

foreach $v \in \text{Adj}[u]$ **do**

if $v.d > u.d + w(u, v)$ **then**

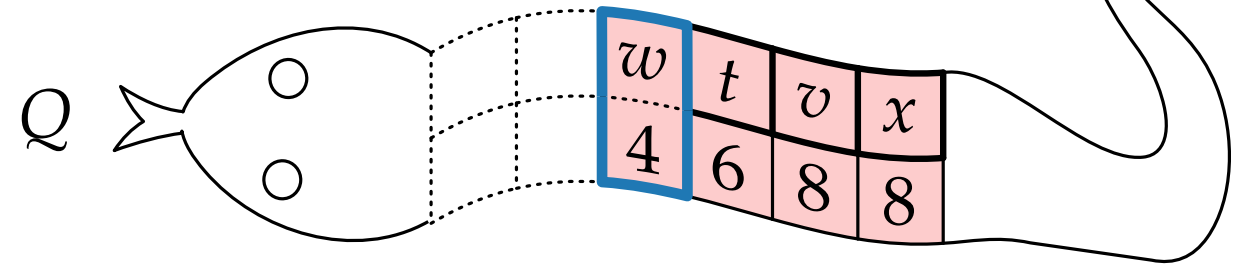
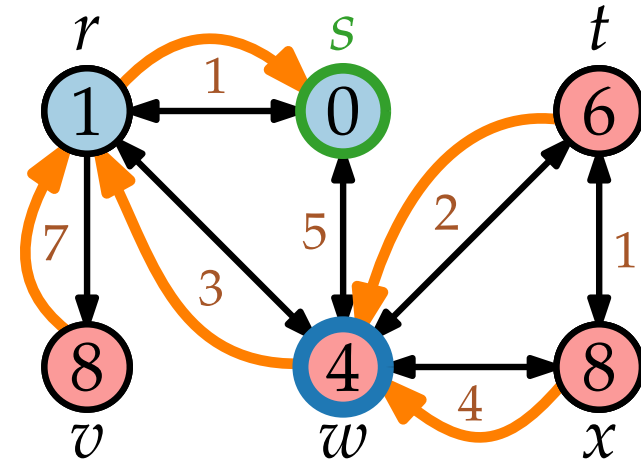
$v.\text{color} = \text{red}$

$v.d = u.d + w(u, v)$

$v.\pi = u$

$Q.\text{DECREASEKEY}(v, v.d)$

$u.\text{color} = \text{blue}$



INITIALIZE(Graph G , Vertex s)

foreach $u \in V$ **do**

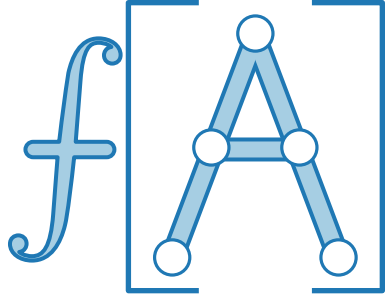
$u.\text{color} = \text{white}$

$u.d = \infty$

$u.\pi = \text{nil}$

$s.\text{color} = \text{red}$

$s.d = 0$



Wiederholung DIJKSTRA

DIJKSTRA(WeightedGraph G , Vertex s)

INITIALIZE(G, s)

$Q = \text{new PriorityQueue}(V, d)$

while not $Q.\text{EMPTY}()$ **do**

$u = Q.\text{EXTRACTMIN}()$

foreach $v \in \text{Adj}[u]$ **do**

if $v.d > u.d + w(u, v)$ **then**

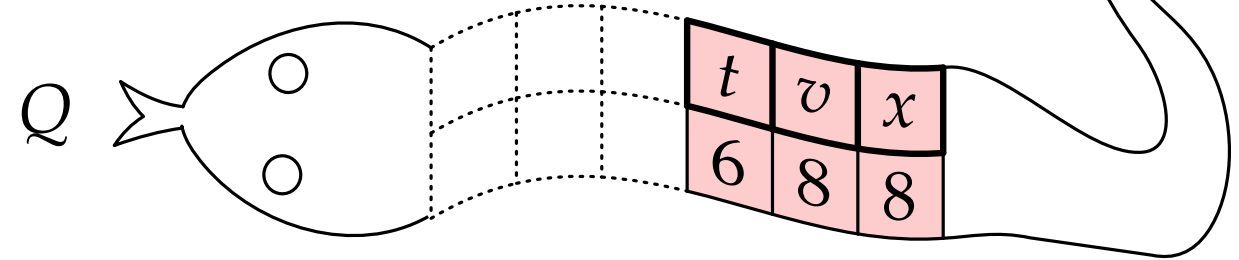
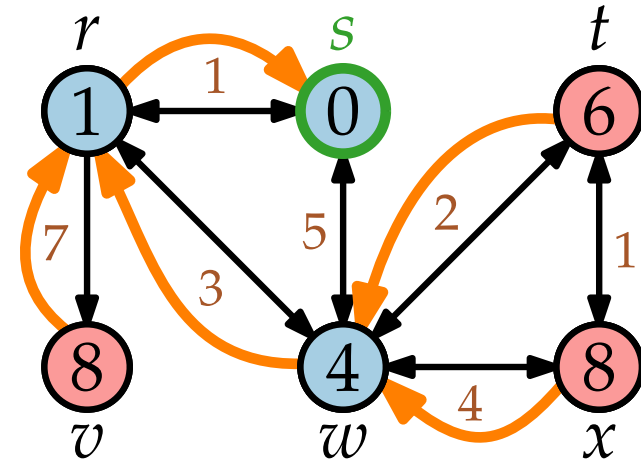
$v.\text{color} = \text{red}$

$v.d = u.d + w(u, v)$

$v.\pi = u$

$Q.\text{DECREASEKEY}(v, v.d)$

$u.\text{color} = \text{blue}$



INITIALIZE(Graph G , Vertex s)

foreach $u \in V$ **do**

$u.\text{color} = \text{white}$

$u.d = \infty$

$u.\pi = \text{nil}$

$s.\text{color} = \text{red}$

$s.d = 0$

Wiederholung DIJKSTRA

DIJKSTRA(WeightedGraph G , Vertex s)

INITIALIZE(G, s)

$Q = \text{new PriorityQueue}(V, d)$

while not $Q.\text{EMPTY}()$ do

$u = Q.\text{EXTRACTMIN}()$

 foreach $v \in \text{Adj}[u]$ do

 if $v.d > u.d + w(u, v)$ then

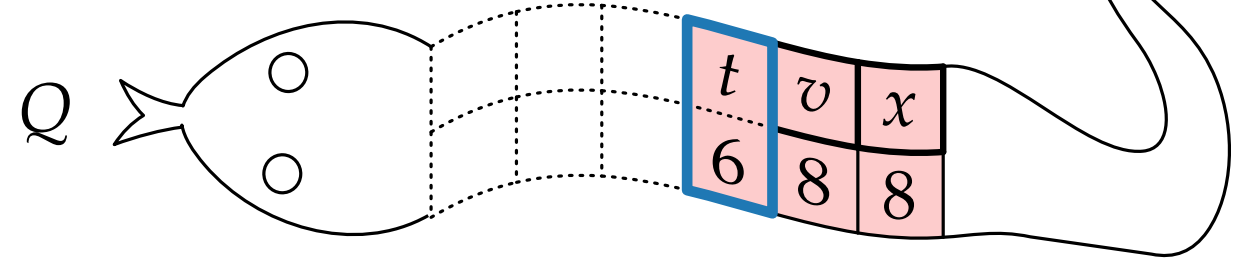
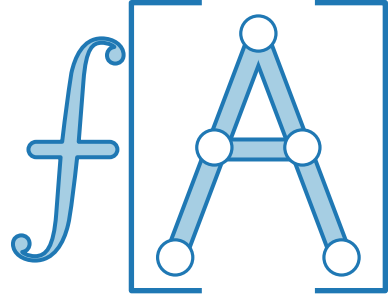
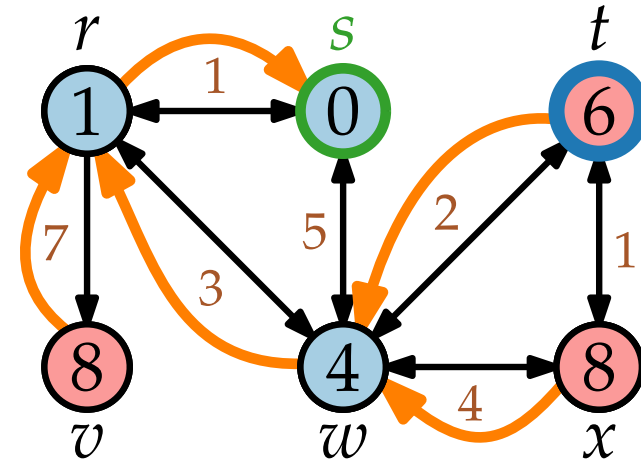
$v.\text{color} = \text{red}$

$v.d = u.d + w(u, v)$

$v.\pi = u$

$Q.\text{DECREASEKEY}(v, v.d)$

$u.\text{color} = \text{blue}$



INITIALIZE(Graph G , Vertex s)

foreach $u \in V$ do

$u.\text{color} = \text{white}$

$u.d = \infty$

$u.\pi = \text{nil}$

$s.\text{color} = \text{red}$

$s.d = 0$

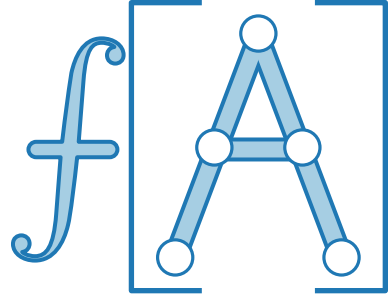
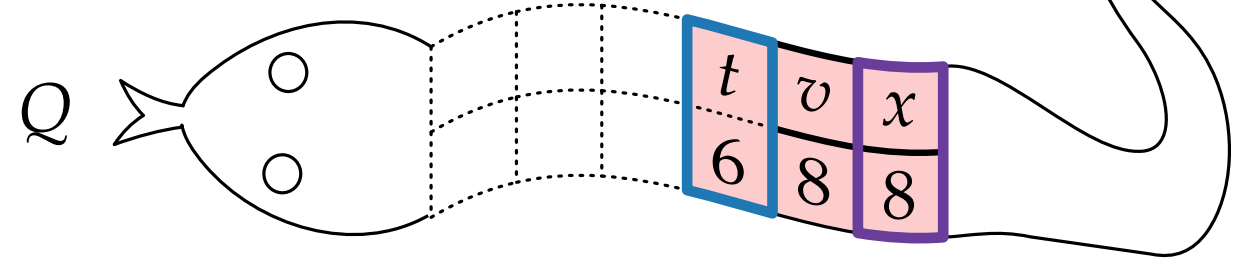
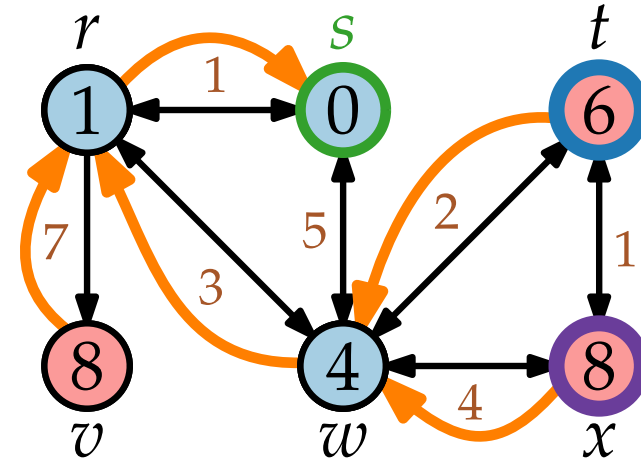
Wiederholung DIJKSTRA

```

DIJKSTRA(WeightedGraph G, Vertex  $s$ )
  INITIALIZE( $G, s$ )
   $Q = \text{new PriorityQueue}(V, d)$ 
  while not  $Q.\text{EMPTY}()$  do
     $u = Q.\text{EXTRACTMIN}()$ 
    foreach  $v \in \text{Adj}[u]$  do
      if  $v.d > u.d + w(u, v)$  then
         $v.\text{color} = \text{red}$ 
         $v.d = u.d + w(u, v)$ 
         $v.\pi = u$ 
         $Q.\text{DECREASEKEY}(v, v.d)$ 
     $u.\text{color} = \text{blue}$ 
  
```

```

INITIALIZE(Graph G, Vertex  $s$ )
  foreach  $u \in V$  do
     $u.\text{color} = \text{white}$ 
     $u.d = \infty$ 
     $u.\pi = \text{nil}$ 
   $s.\text{color} = \text{red}$ 
   $s.d = 0$ 
  
```



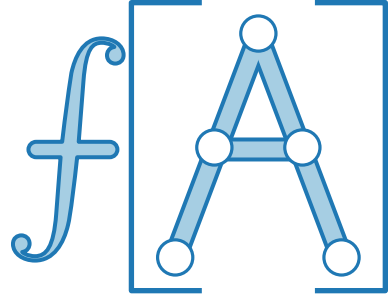
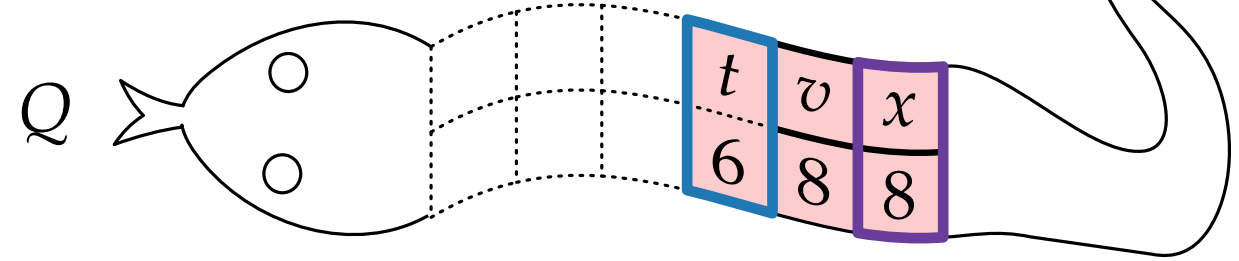
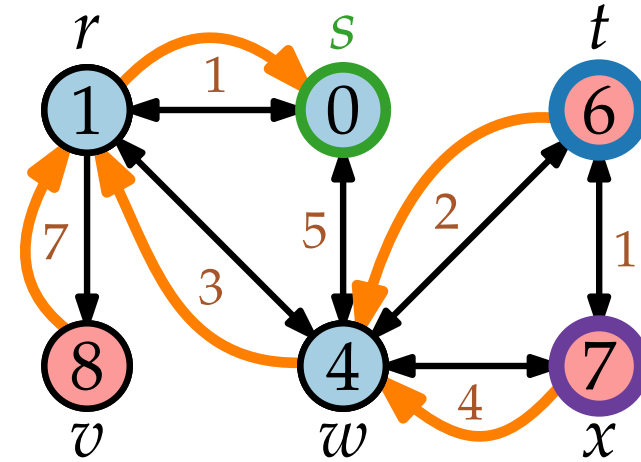
Wiederholung DIJKSTRA

```

DIJKSTRA(WeightedGraph G, Vertex  $s$ )
  INITIALIZE( $G, s$ )
   $Q = \text{new PriorityQueue}(V, d)$ 
  while not  $Q.\text{EMPTY}()$  do
     $u = Q.\text{EXTRACTMIN}()$ 
    foreach  $v \in \text{Adj}[u]$  do
      if  $v.d > u.d + w(u, v)$  then
         $v.\text{color} = \text{red}$ 
         $v.d = u.d + w(u, v)$ 
         $v.\pi = u$ 
         $Q.\text{DECREASEKEY}(v, v.d)$ 
     $u.\text{color} = \text{blue}$ 
  
```

```

INITIALIZE(Graph G, Vertex  $s$ )
  foreach  $u \in V$  do
     $u.\text{color} = \text{white}$ 
     $u.d = \infty$ 
     $u.\pi = \text{nil}$ 
   $s.\text{color} = \text{red}$ 
   $s.d = 0$ 
  
```



Wiederholung DIJKSTRA

DIJKSTRA(WeightedGraph G , Vertex s)

INITIALIZE(G, s)

$Q = \text{new PriorityQueue}(V, d)$

while not $Q.\text{EMPTY}()$ **do**

$u = Q.\text{EXTRACTMIN}()$

foreach $v \in \text{Adj}[u]$ **do**

if $v.d > u.d + w(u, v)$ **then**

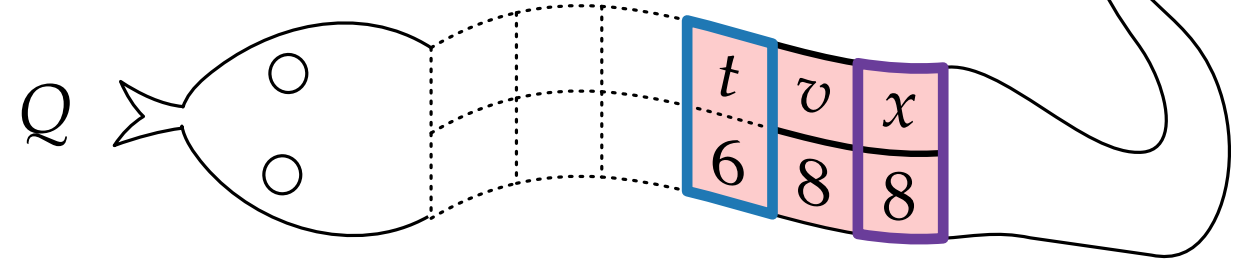
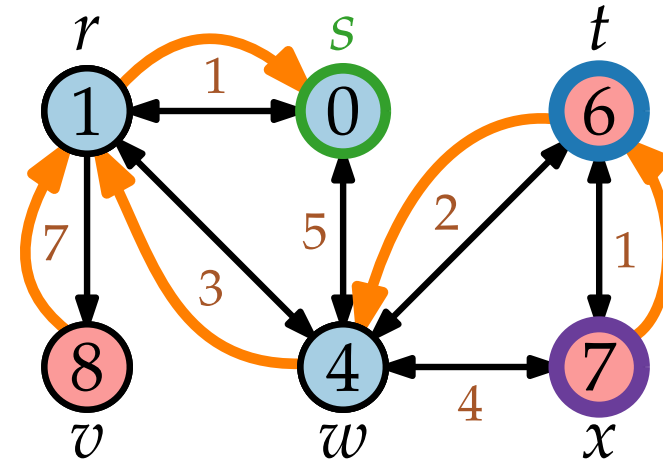
$v.\text{color} = \text{red}$

$v.d = u.d + w(u, v)$

$v.\pi = u$

$Q.\text{DECREASEKEY}(v, v.d)$

$u.\text{color} = \text{blue}$



INITIALIZE(Graph G , Vertex s)

foreach $u \in V$ **do**

$u.\text{color} = \text{white}$

$u.d = \infty$

$u.\pi = \text{nil}$

$s.\text{color} = \text{red}$

$s.d = 0$

Wiederholung DIJKSTRA

DIJKSTRA(WeightedGraph G , Vertex s)

INITIALIZE(G, s)

$Q = \text{new PriorityQueue}(V, d)$

while not $Q.\text{EMPTY}()$ do

$u = Q.\text{EXTRACTMIN}()$

 foreach $v \in \text{Adj}[u]$ do

 if $v.d > u.d + w(u, v)$ then

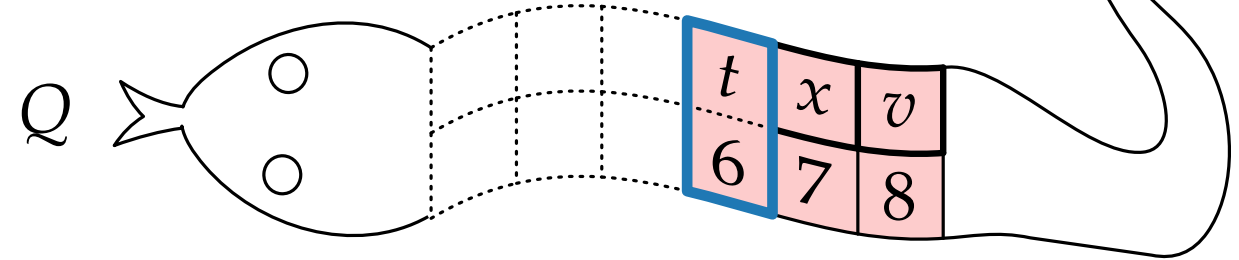
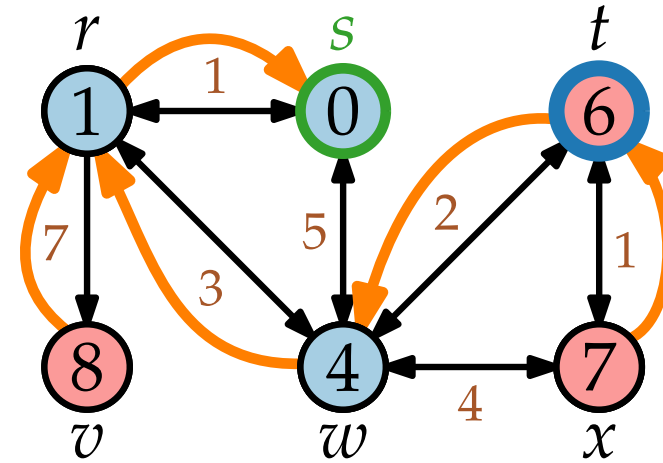
$v.\text{color} = \text{red}$

$v.d = u.d + w(u, v)$

$v.\pi = u$

$Q.\text{DECREASEKEY}(v, v.d)$

$u.\text{color} = \text{blue}$



INITIALIZE(Graph G , Vertex s)

foreach $u \in V$ do

$u.\text{color} = \text{white}$

$u.d = \infty$

$u.\pi = \text{nil}$

$s.\text{color} = \text{red}$

$s.d = 0$

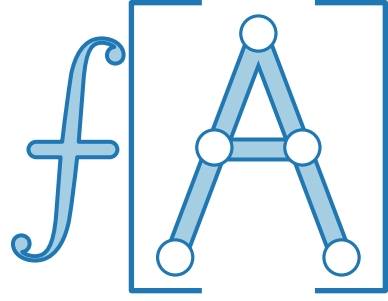
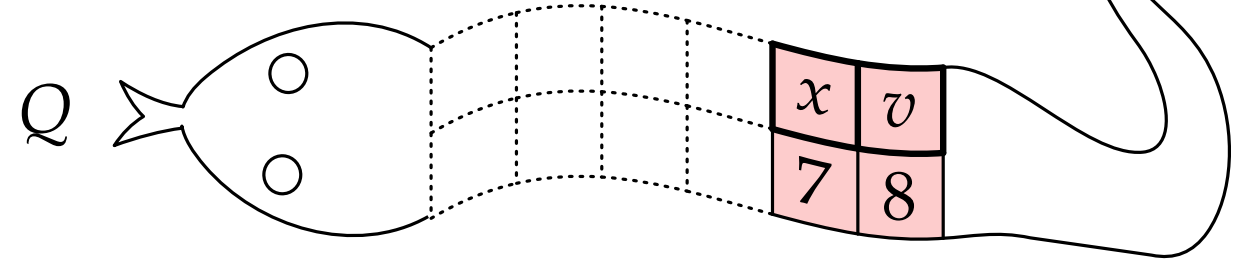
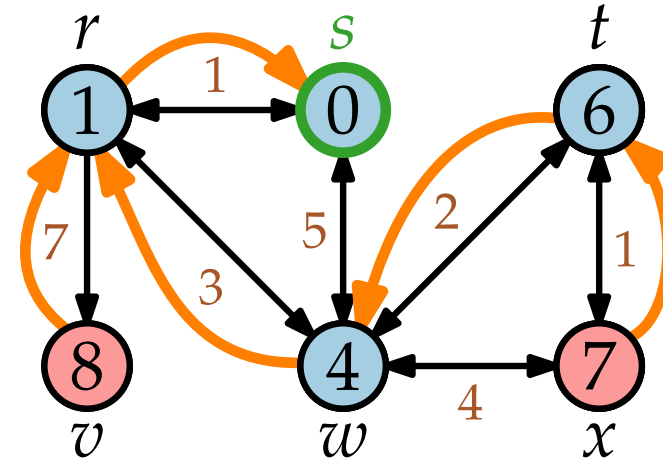
Wiederholung DIJKSTRA

```

DIJKSTRA(WeightedGraph G, Vertex  $s$ )
  INITIALIZE( $G, s$ )
   $Q = \text{new PriorityQueue}(V, d)$ 
  while not  $Q.\text{EMPTY}()$  do
     $u = Q.\text{EXTRACTMIN}()$ 
    foreach  $v \in \text{Adj}[u]$  do
      if  $v.d > u.d + w(u, v)$  then
         $v.\text{color} = \text{red}$ 
         $v.d = u.d + w(u, v)$ 
         $v.\pi = u$ 
         $Q.\text{DECREASEKEY}(v, v.d)$ 
     $u.\text{color} = \text{blue}$ 
  
```

```

INITIALIZE(Graph G, Vertex  $s$ )
  foreach  $u \in V$  do
     $u.\text{color} = \text{white}$ 
     $u.d = \infty$ 
     $u.\pi = \text{nil}$ 
   $s.\text{color} = \text{red}$ 
   $s.d = 0$ 
  
```



Wiederholung DIJKSTRA

DIJKSTRA(WeightedGraph G , Vertex s)

INITIALIZE(G, s)

$Q = \text{new PriorityQueue}(V, d)$

while not $Q.\text{EMPTY}()$ **do**

$u = Q.\text{EXTRACTMIN}()$

foreach $v \in \text{Adj}[u]$ **do**

if $v.d > u.d + w(u, v)$ **then**

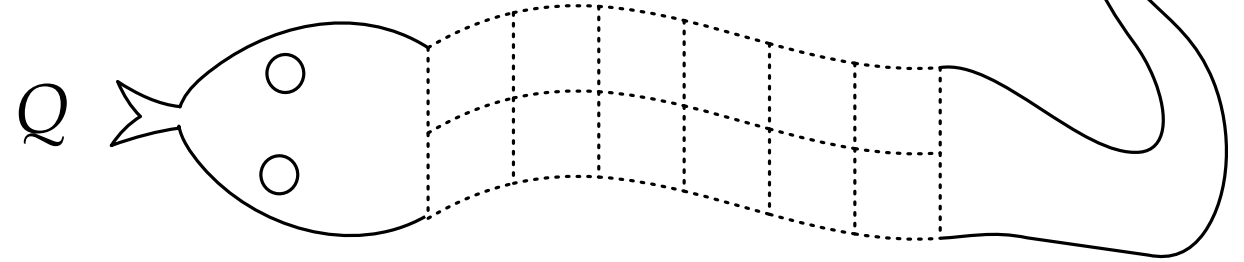
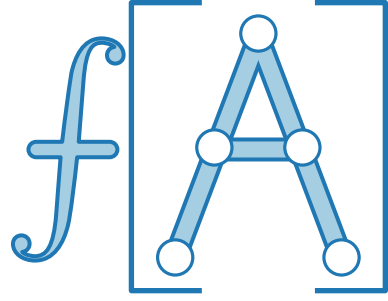
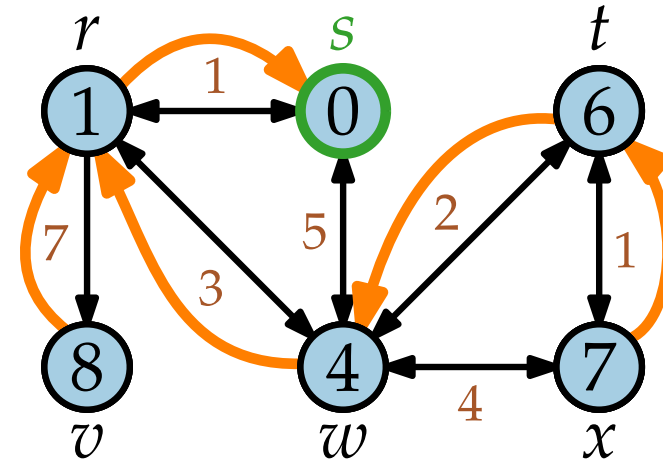
$v.\text{color} = \text{red}$

$v.d = u.d + w(u, v)$

$v.\pi = u$

$Q.\text{DECREASEKEY}(v, v.d)$

$u.\text{color} = \text{blue}$



INITIALIZE(Graph G , Vertex s)

foreach $u \in V$ **do**

$u.\text{color} = \text{white}$

$u.d = \infty$

$u.\pi = \text{nil}$

$s.\text{color} = \text{red}$

$s.d = 0$

Wiederholung DIJKSTRA

DIJKSTRA(WeightedGraph G , Vertex s)

INITIALIZE(G, s)

$Q = \text{new PriorityQueue}(V, d)$

while not $Q.\text{EMPTY}()$ do

$u = Q.\text{EXTRACTMIN}()$

 foreach $v \in \text{Adj}[u]$ do

 if $v.d > u.d + w(u, v)$ then

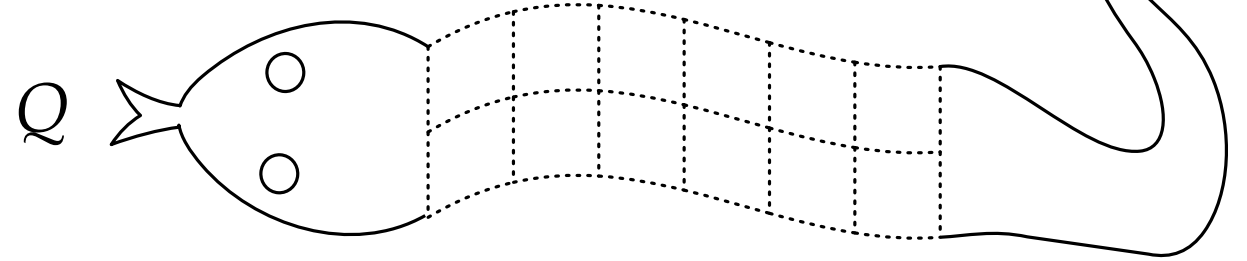
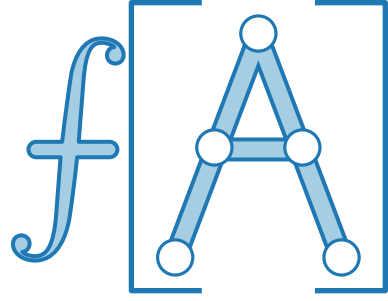
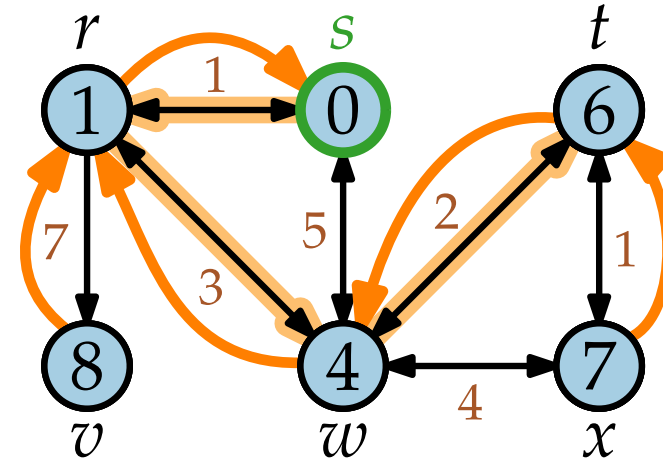
$v.\text{color} = \text{red}$

$v.d = u.d + w(u, v)$

$v.\pi = u$

$Q.\text{DECREASEKEY}(v, v.d)$

$u.\text{color} = \text{blue}$



INITIALIZE(Graph G , Vertex s)

foreach $u \in V$ do

$u.\text{color} = \text{white}$

$u.d = \infty$

$u.\pi = \text{nil}$

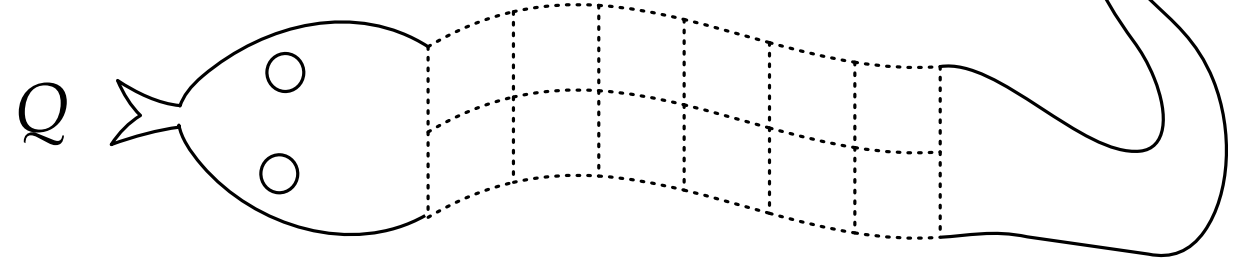
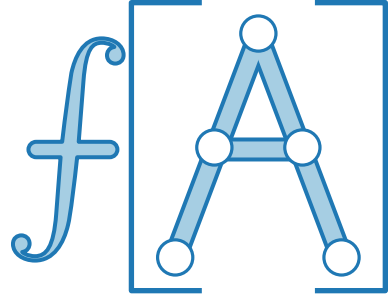
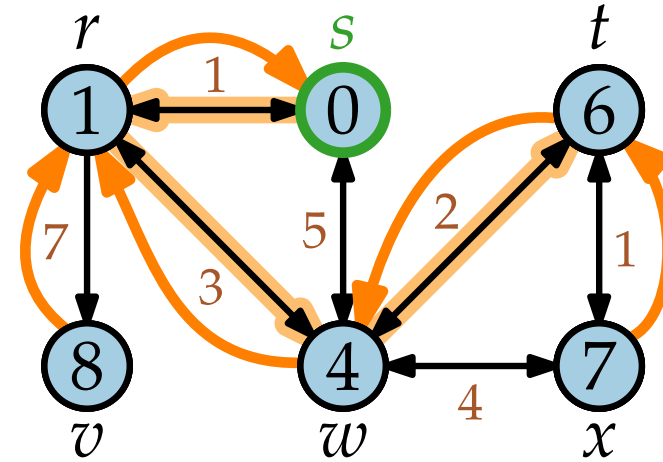
$s.\text{color} = \text{red}$

$s.d = 0$

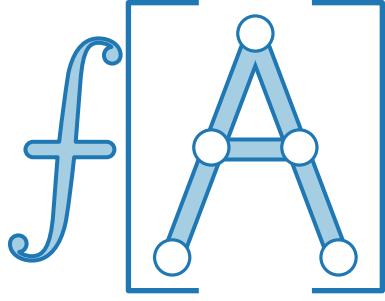
Wiederholung DIJKSTRA

```
DIJKSTRA(WeightedGraph G, Vertex s)
  INITIALIZE(G, s)
  Q = new PriorityQueue(V, d)
  while not Q.EMPTY() do
    u = Q.EXTRACTMIN()
    foreach v ∈ Adj[u] do
      if v.d > u.d + w(u, v) then
        v.color = red
        v.d = u.d + w(u, v)
        v.π = u
        Q.DECREASEKEY(v, v.d)
    u.color = blue
```

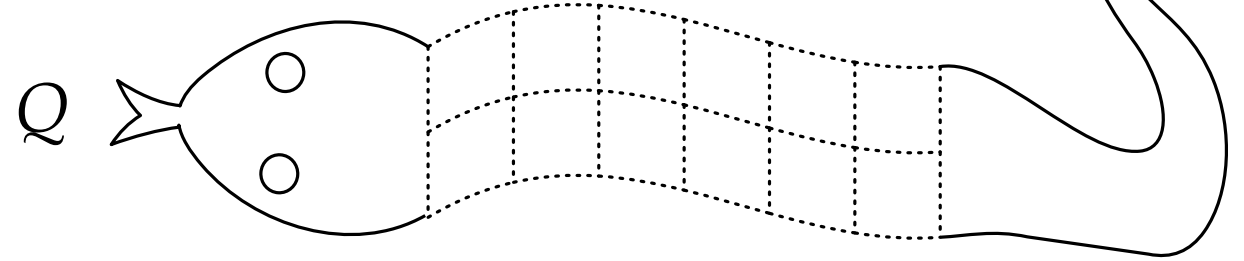
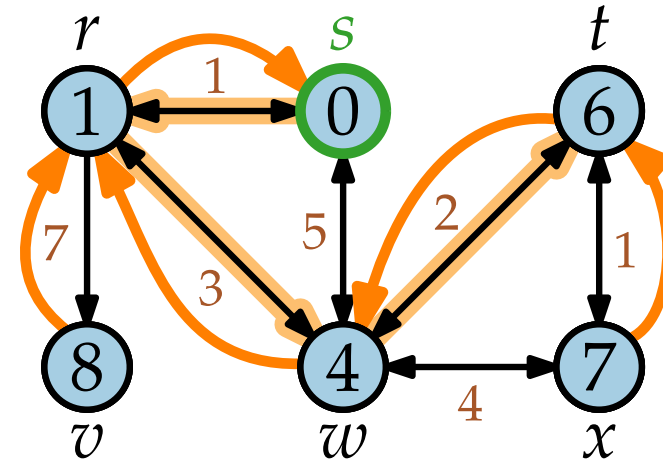
```
INITIALIZE(Graph G, Vertex s)
  foreach u ∈ V do
    u.color = white
    u.d = ∞
    u.π = nil
  s.color = red
  s.d = 0
```



Wiederholung DIJKSTRA



```
DIJKSTRA(WeightedGraph G, Vertex s)
  INITIALIZE(G, s)
  Q = new PriorityQueue(V, d)
  while not Q.EMPTY() do
    u = Q.EXTRACTMIN()
    foreach v ∈ Adj[u] do
      if v.d > u.d + w(u, v) then
        v.color = red
        v.d = u.d + w(u, v)
        v.π = u
        Q.DECREASEKEY(v, v.d)
    u.color = blue
```



```
INITIALIZE(Graph G, Vertex s)
  foreach u ∈ V do
    u.color = white
    u.d = ∞
    u.π = nil
  s.color = red
  s.d = 0
```

Demo.

<https://algo.uni-trier.de/demos/graphtraversal.html>

DIJKSTRA – Laufzeit



DIJKSTRA(WeightedGraph G , Vertex s)

INITIALIZE(G, s)

$Q = \text{new PriorityQueue}(V, d)$

while not $Q.\text{EMPTY}()$ **do**

$u = Q.\text{EXTRACTMIN}()$

foreach $v \in \text{Adj}[u]$ **do**

if $v.d > u.d + w(u, v)$ **then**

$v.\text{color} = \text{red}$

$v.d = u.d + w(u, v)$

$v.\pi = u$

$Q.\text{DECREASEKEY}(v, v.d)$

$u.\text{color} = \text{blue}$

DIJKSTRA – Laufzeit



DIJKSTRA(WeightedGraph G , Vertex s)

INITIALIZE(G, s)

$Q = \text{new PriorityQueue}(V, d)$

while not $Q.\text{EMPTY}()$ **do**

$u = Q.\text{EXTRACTMIN}()$

foreach $v \in \text{Adj}[u]$ **do**

if $v.d > u.d + w(u, v)$ **then**

$v.\text{color} = \text{red}$

$v.d = u.d + w(u, v)$

$v.\pi = u$

$Q.\text{DECREASEKEY}(v, v.d)$

$u.\text{color} = \text{blue}$

DIJKSTRA – Laufzeit



DIJKSTRA(WeightedGraph G , Vertex s)

INITIALIZE(G, s)

$Q = \text{new PriorityQueue}(V, d)$

$\mathcal{O}(n)$ Zeit

while not $Q.\text{EMPTY}()$ **do**

$u = Q.\text{EXTRACTMIN}()$

foreach $v \in \text{Adj}[u]$ **do**

if $v.d > u.d + w(u, v)$ **then**

$v.\text{color} = \text{red}$

$v.d = u.d + w(u, v)$

$v.\pi = u$

$Q.\text{DECREASEKEY}(v, v.d)$

$u.\text{color} = \text{blue}$

DIJKSTRA – Laufzeit



DIJKSTRA(WeightedGraph G , Vertex s)

INITIALIZE(G, s)

$Q = \text{new PriorityQueue}(V, d)$

$\mathcal{O}(n)$ Zeit

while not $Q.\text{EMPTY}()$ **do**

$u = Q.\text{EXTRACTMIN}()$

foreach $v \in \text{Adj}[u]$ **do**

if $v.d > u.d + w(u, v)$ **then**

$v.\text{color} = \text{red}$

$v.d = u.d + w(u, v)$

$v.\pi = u$

$Q.\text{DECREASEKEY}(v, v.d)$

$u.\text{color} = \text{blue}$

DIJKSTRA – Laufzeit



DIJKSTRA(WeightedGraph G , Vertex s)

INITIALIZE(G, s)

$Q = \text{new PriorityQueue}(V, d)$

$\mathcal{O}(n)$ Zeit

while not $Q.\text{EMPTY}()$ **do**

$u = Q.\text{EXTRACTMIN}()$

genau n mal

foreach $v \in \text{Adj}[u]$ **do**

if $v.d > u.d + w(u, v)$ **then**

$v.\text{color} = \text{red}$

$v.d = u.d + w(u, v)$

$v.\pi = u$

$Q.\text{DECREASEKEY}(v, v.d)$

$u.\text{color} = \text{blue}$

DIJKSTRA – Laufzeit



DIJKSTRA(WeightedGraph G , Vertex s)

INITIALIZE(G, s)

$Q = \text{new PriorityQueue}(V, d)$

$\mathcal{O}(n)$ Zeit

while not $Q.\text{EMPTY}()$ do

$u = Q.\text{EXTRACTMIN}()$

genau n mal

foreach $v \in \text{Adj}[u]$ do

if $v.d > u.d + w(u, v)$ then

Wie oft wird der
Schleifeninhalt aufgerufen?

$v.\text{color} = \text{red}$

$v.d = u.d + w(u, v)$

$v.\pi = u$

$Q.\text{DECREASEKEY}(v, v.d)$

$u.\text{color} = \text{blue}$

DIJKSTRA – Laufzeit



DIJKSTRA(WeightedGraph G , Vertex s)

INITIALIZE(G, s)

$Q = \text{new PriorityQueue}(V, d)$

$\mathcal{O}(n)$ Zeit

while not $Q.\text{EMPTY}()$ **do**

$u = Q.\text{EXTRACTMIN}()$

genau n mal

foreach $v \in \text{Adj}[u]$ **do**

if $v.d > u.d + w(u, v)$ **then**

Wie oft wird der
Schleifeninhalt aufgerufen?

$v.\text{color} = \text{red}$

$v.d = u.d + w(u, v)$

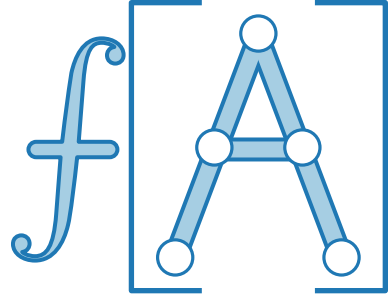
$v.\pi = u$

$Q.\text{DECREASEKEY}(v, v.d)$

Für jeden Knoten $u \in V$
genau $|\text{Adj}[u]|$ mal,

$u.\text{color} = \text{blue}$

DIJKSTRA – Laufzeit



DIJKSTRA(WeightedGraph G , Vertex s)

INITIALIZE(G, s)

$Q = \text{new PriorityQueue}(V, d)$

$\mathcal{O}(n)$ Zeit

while not $Q.\text{EMPTY}()$ do

$u = Q.\text{EXTRACTMIN}()$

genau n mal

foreach $v \in \text{Adj}[u]$ do

if $v.d > u.d + w(u, v)$ then

Wie oft wird der
Schleifeninhalt aufgerufen?

$v.\text{color} = \text{red}$

$v.d = u.d + w(u, v)$

$v.\pi = u$

$Q.\text{DECREASEKEY}(v, v.d)$

Für jeden Knoten $u \in V$
genau $|\text{Adj}[u]| = \deg u$ mal,

$u.\text{color} = \text{blue}$

DIJKSTRA – Laufzeit



DIJKSTRA(WeightedGraph G , Vertex s)

INITIALIZE(G, s)

$Q = \text{new PriorityQueue}(V, d)$

$\mathcal{O}(n)$ Zeit

while not $Q.\text{EMPTY}()$ do

$u = Q.\text{EXTRACTMIN}()$

genau n mal

foreach $v \in \text{Adj}[u]$ do

if $v.d > u.d + w(u, v)$ then

Wie oft wird der
Schleifeninhalt aufgerufen?

$v.\text{color} = \text{red}$

$v.d = u.d + w(u, v)$

$v.\pi = u$

$Q.\text{DECREASEKEY}(v, v.d)$

Für jeden Knoten $u \in V$
genau $|\text{Adj}[u]| = \deg u$ mal,

$u.\text{color} = \text{blue}$

also insg. $\Theta(m)$ mal.

DIJKSTRA – Laufzeit



DIJKSTRA(WeightedGraph G , Vertex s)

INITIALIZE(G, s)

$Q = \text{new PriorityQueue}(V, d)$

$\mathcal{O}(n)$ Zeit

while not $Q.\text{EMPTY}()$ **do**

$u = Q.\text{EXTRACTMIN}()$

genau n mal

foreach $v \in \text{Adj}[u]$ **do**

if $v.d > u.d + w(u, v)$ **then**

Wie oft wird der
Schleifeninhalt aufgerufen?

$v.\text{color} = \text{red}$

$v.d = u.d + w(u, v)$

$v.\pi = u$

$Q.\text{DECREASEKEY}(v, v.d)$

Für jeden Knoten $u \in V$
genau $|\text{Adj}[u]| = \deg u$ mal,
also insg. $\Theta(m)$ mal.

$u.\text{color} = \text{blue}$

Also wird DECREASEKEY
 $\mathcal{O}(m)$ mal aufgerufen.

DIJKSTRA – Laufzeit



DIJKSTRA(WeightedGraph G , Vertex s)

INITIALIZE(G, s)

$Q = \text{new PriorityQueue}(V, d)$

$\mathcal{O}(n)$ Zeit

while not $Q.\text{EMPTY}()$ **do**

$u = Q.\text{EXTRACTMIN}()$

genau n mal

foreach $v \in \text{Adj}[u]$ **do**

if $v.d > u.d + w(u, v)$ **then**

Wie oft wird der
Schleifeninhalt aufgerufen?

$v.\text{color} = \text{red}$

$v.d = u.d + w(u, v)$

$v.\pi = u$

$Q.\text{DECREASEKEY}(v, v.d)$

Für jeden Knoten $u \in V$
genau $|\text{Adj}[u]| = \deg u$ mal,

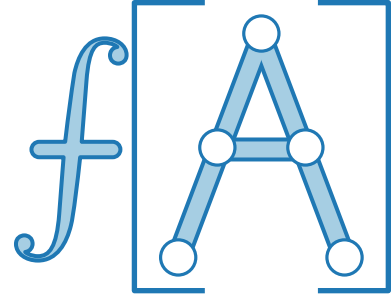
also insg. $\Theta(m)$ mal.

$u.\text{color} = \text{blue}$

Also wird DECREASEKEY
 $\mathcal{O}(m)$ mal aufgerufen.

Satz.

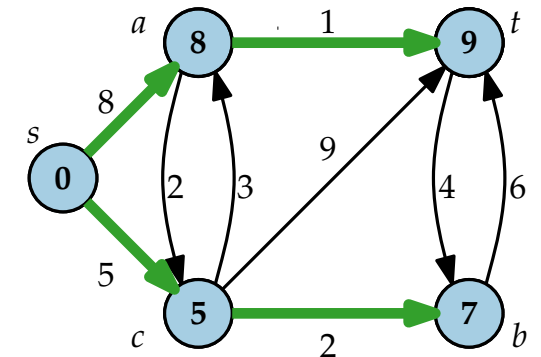
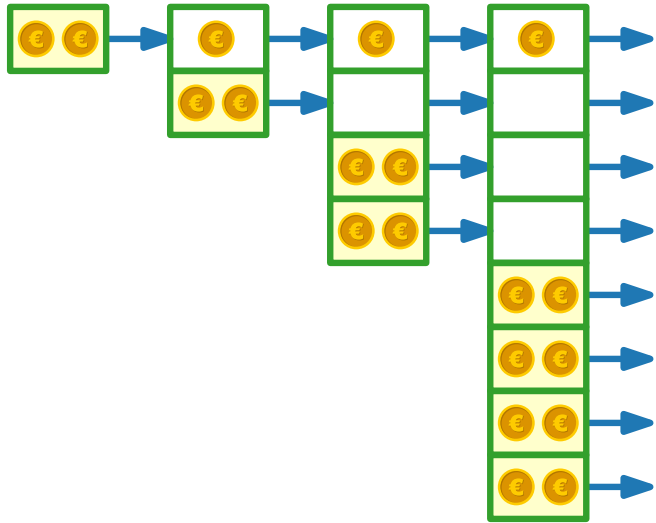
Gegeben ein Graph $G = (V, E)$, läuft Dijkstras Alg. in
 $\Theta(n \cdot T_{\text{EXTRACTMIN}}(n) + m \cdot T_{\text{DECREASEKEY}}(n))$ Zeit.



Fortgeschrittene Algorithmen

17. Vorlesung Datenstrukturen III: Amortisierte Analyse

Teil II: Prioritätsschlangen



Prioritätsschlange



Abstrakter Datentyp. PRIORITÄTSSCHLANGE

verwaltet Elemente einer Menge Q ,
wobei jedes Element $x \in Q$ eine Priorität $x.\text{key}$ hat.

Operation	Funktionalität

Prioritätsschlange



Abstrakter Datentyp. PRIORITÄTSSCHLANGE

verwaltet Elemente einer Menge Q ,
wobei jedes Element $x \in Q$ eine Priorität $x.\text{key}$ hat.

(Wir benutzen Q , um sowohl
die Menge, als auch die
Prioritätsschlange selber zu
bezeichnen)

Operation	Funktionalität

Prioritätsschlange



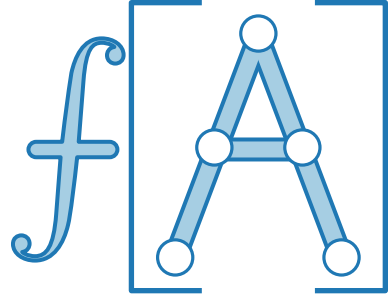
Abstrakter Datentyp. PRIORITÄTSSCHLANGE

verwaltet Elemente einer Menge Q ,
wobei jedes Element $x \in Q$ eine Priorität $x.\text{key}$ hat.

(Wir benutzen Q , um sowohl
die Menge, als auch die
Prioritätsschlange selber zu
bezeichnen)

Operation	Funktionalität
$Q.\text{insert}(x)$	

Prioritätsschlange



Abstrakter Datentyp. PRIORITÄTSSCHLANGE

verwaltet Elemente einer Menge Q ,
wobei jedes Element $x \in Q$ eine Priorität $x.\text{key}$ hat.

(Wir benutzen Q , um sowohl die Menge, als auch die Prioritätsschlange selber zu bezeichnen)

Operation	Funktionalität
$Q.\text{insert}(x)$	$Q = Q \cup \{x\}$

Prioritätsschlange



Abstrakter Datentyp. PRIORITÄTSSCHLANGE

verwaltet Elemente einer Menge Q ,
wobei jedes Element $x \in Q$ eine Priorität $x.\text{key}$ hat.

(Wir benutzen Q , um sowohl
die Menge, als auch die
Prioritätsschlange selber zu
bezeichnen)

Operation	Funktionalität
$Q.\text{insert}(x)$	$Q = Q \cup \{x\}$
$Q.\text{min}()$	

Prioritätsschlange



Abstrakter Datentyp. PRIORITÄTSSCHLANGE

verwaltet Elemente einer Menge Q ,
wobei jedes Element $x \in Q$ eine Priorität $x.\text{key}$ hat.

(Wir benutzen Q , um sowohl
die Menge, als auch die
Prioritätsschlange selber zu
bezeichnen)

Operation	Funktionalität
$Q.\text{insert}(x)$	$Q = Q \cup \{x\}$
$Q.\text{min}()$	liefere $x \in Q$ mit $x.\text{key} = \min\{y.\text{key} \mid y \in Q\}$

Prioritätsschlange



Abstrakter Datentyp. PRIORITÄTSSCHLANGE

verwaltet Elemente einer Menge Q ,
wobei jedes Element $x \in Q$ eine Priorität $x.\text{key}$ hat.

(Wir benutzen Q , um sowohl
die Menge, als auch die
Prioritätsschlange selber zu
bezeichnen)

Operation	Funktionalität
$Q.\text{insert}(x)$	$Q = Q \cup \{x\}$
$Q.\text{min}()$	liefere $x \in Q$ mit $x.\text{key} = \min\{y.\text{key} \mid y \in Q\}$
$Q.\text{extractMin}()$	

Prioritätsschlange



Abstrakter Datentyp. PRIORITÄTSSCHLANGE

verwaltet Elemente einer Menge Q ,
wobei jedes Element $x \in Q$ eine Priorität $x.key$ hat.

(Wir benutzen Q , um sowohl
die Menge, als auch die
Prioritätsschlange selber zu
bezeichnen)

Operation	Funktionalität
$Q.insert(x)$	$Q = Q \cup \{x\}$
$Q.min()$	liefere $x \in Q$ mit $x.key = \min\{y.key \mid y \in Q\}$
$Q.extractMin()$	liefere und lösche $x \in Q$ mit $x.key = \min\{y.key \mid y \in Q\}$

Prioritätsschlange



Abstrakter Datentyp. PRIORITÄTSSCHLANGE

verwaltet Elemente einer Menge Q ,
wobei jedes Element $x \in Q$ eine Priorität $x.key$ hat.

(Wir benutzen Q , um sowohl
die Menge, als auch die
Prioritätsschlange selber zu
bezeichnen)

Operation	Funktionalität
$Q.insert(x)$	$Q = Q \cup \{x\}$
$Q.min()$	liefere $x \in Q$ mit $x.key = \min\{y.key \mid y \in Q\}$
$Q.extractMin()$	liefere und lösche $x \in Q$ mit $x.key = \min\{y.key \mid y \in Q\}$
$Q.decreaseKey(x, p)$	

Prioritätsschlange



Abstrakter Datentyp. PRIORITÄTSSCHLANGE

verwaltet Elemente einer Menge Q ,
wobei jedes Element $x \in Q$ eine Priorität $x.\text{key}$ hat.

(Wir benutzen Q , um sowohl
die Menge, als auch die
Prioritätsschlange selber zu
bezeichnen)

Operation	Funktionalität
$Q.\text{insert}(x)$	$Q = Q \cup \{x\}$
$Q.\text{min}()$	liefere $x \in Q$ mit $x.\text{key} = \min\{y.\text{key} \mid y \in Q\}$
$Q.\text{extractMin}()$	liefere und lösche $x \in Q$ mit $x.\text{key} = \min\{y.\text{key} \mid y \in Q\}$
$Q.\text{decreaseKey}(x, p)$	Setze $x.\text{key} = p$, wobei $p < x.\text{key}$

Prioritätsschlange



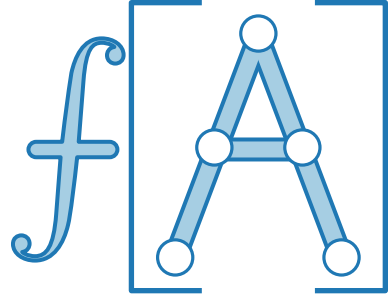
Abstrakter Datentyp. PRIORITÄTSSCHLANGE

verwaltet Elemente einer Menge Q ,
wobei jedes Element $x \in Q$ eine Priorität $x.\text{key}$ hat.

(Wir benutzen Q , um sowohl
die Menge, als auch die
Prioritätsschlange selber zu
bezeichnen)

Operation	Funktionalität
$Q.\text{insert}(x)$	$Q = Q \cup \{x\}$
$Q.\text{min}()$	liefere $x \in Q$ mit $x.\text{key} = \min\{y.\text{key} \mid y \in Q\}$
$Q.\text{extractMin}()$	liefere und lösche $x \in Q$ mit $x.\text{key} = \min\{y.\text{key} \mid y \in Q\}$
$Q.\text{decreaseKey}(x, p)$	Setze $x.\text{key} = p$, wobei $p < x.\text{key}$
$Q.\text{union}(Q')$	

Prioritätsschlange



Abstrakter Datentyp. PRIORITÄTSSCHLANGE

verwaltet Elemente einer Menge Q ,
wobei jedes Element $x \in Q$ eine Priorität $x.\text{key}$ hat.

(Wir benutzen Q , um sowohl
die Menge, als auch die
Prioritätsschlange selber zu
bezeichnen)

Operation	Funktionalität
$Q.\text{insert}(x)$	$Q = Q \cup \{x\}$
$Q.\text{min}()$	liefere $x \in Q$ mit $x.\text{key} = \min\{y.\text{key} \mid y \in Q\}$
$Q.\text{extractMin}()$	liefere und lösche $x \in Q$ mit $x.\text{key} = \min\{y.\text{key} \mid y \in Q\}$
$Q.\text{decreaseKey}(x, p)$	Setze $x.\text{key} = p$, wobei $p < x.\text{key}$
$Q.\text{union}(Q')$	$Q = Q \cup Q'$

Prioritätsschlange



Abstrakter Datentyp. PRIORITÄTSSCHLANGE

verwaltet Elemente einer Menge Q ,
wobei jedes Element $x \in Q$ eine Priorität $x.\text{key}$ hat.

(Wir benutzen Q , um sowohl
die Menge, als auch die
Prioritätsschlange selber zu
bezeichnen)

Operation	Funktionalität
$Q.\text{insert}(x)$	$Q = Q \cup \{x\}$
$Q.\text{min}()$	liefere $x \in Q$ mit $x.\text{key} = \min\{y.\text{key} \mid y \in Q\}$
$Q.\text{extractMin}()$	liefere und lösche $x \in Q$ mit $x.\text{key} = \min\{y.\text{key} \mid y \in Q\}$
$Q.\text{decreaseKey}(x, p)$	Setze $x.\text{key} = p$, wobei $p < x.\text{key}$
$Q.\text{union}(Q')$	$Q = Q \cup Q'$
$Q.\text{delete}(x)$	

Prioritätsschlange



Abstrakter Datentyp. PRIORITÄTSSCHLANGE

verwaltet Elemente einer Menge Q ,
wobei jedes Element $x \in Q$ eine Priorität $x.\text{key}$ hat.

(Wir benutzen Q , um sowohl
die Menge, als auch die
Prioritätsschlange selber zu
bezeichnen)

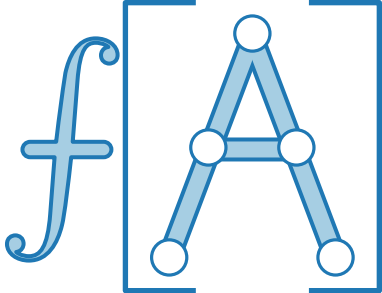
Operation	Funktionalität
$Q.\text{insert}(x)$	$Q = Q \cup \{x\}$
$Q.\text{min}()$	liefere $x \in Q$ mit $x.\text{key} = \min\{y.\text{key} \mid y \in Q\}$
$Q.\text{extractMin}()$	liefere und lösche $x \in Q$ mit $x.\text{key} = \min\{y.\text{key} \mid y \in Q\}$
$Q.\text{decreaseKey}(x, p)$	Setze $x.\text{key} = p$, wobei $p < x.\text{key}$
$Q.\text{union}(Q')$	$Q = Q \cup Q'$
$Q.\text{delete}(x)$	$Q.\text{decreaseKey}(x, -\infty); Q.\text{extractMin}()$

Implementierung I: Arrays



Operation	Implementierung	Laufzeit
$Q.\text{insert}(x)$		
$Q.\text{min}()$		
$Q.\text{extractMin}()$		
$Q.\text{decreaseKey}(x, p)$		
$Q.\text{union}(Q')$		

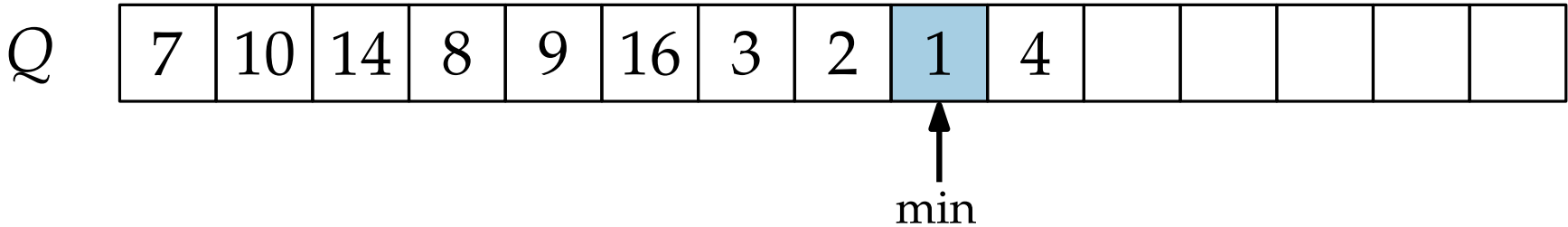
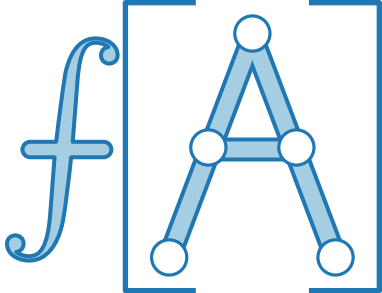
Implementierung I: Arrays



Q	7	10	14	8	9	16	3	2	1	4					
-----	---	----	----	---	---	----	---	---	---	---	--	--	--	--	--

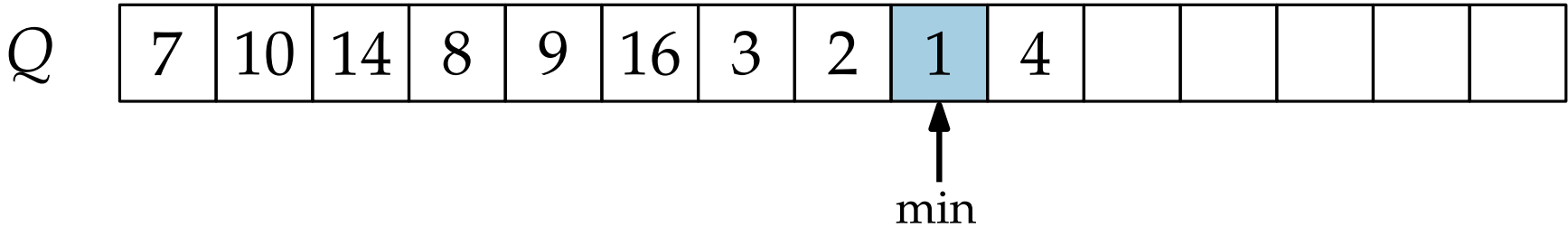
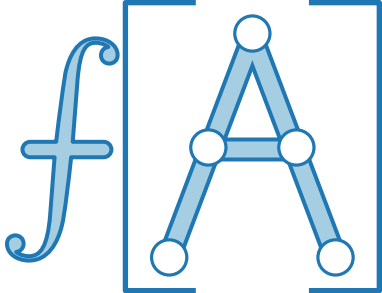
Operation	Implementierung	Laufzeit
$Q.insert(x)$		
$Q.min()$		
$Q.extractMin()$		
$Q.decreaseKey(x, p)$		
$Q.union(Q')$		

Implementierung I: Arrays



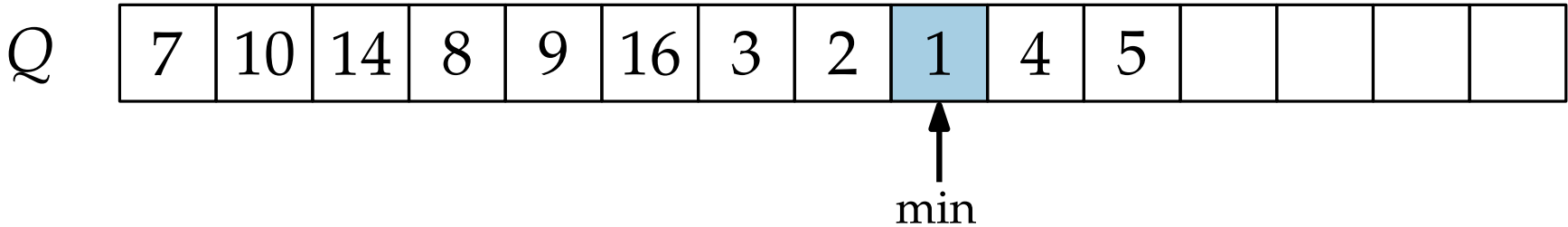
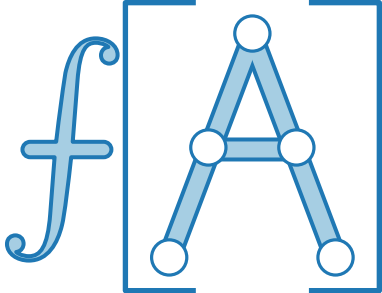
Operation	Implementierung	Laufzeit
$Q.insert(x)$		
$Q.min()$		
$Q.extractMin()$		
$Q.decreaseKey(x, p)$		
$Q.union(Q')$		

Implementierung I: Arrays



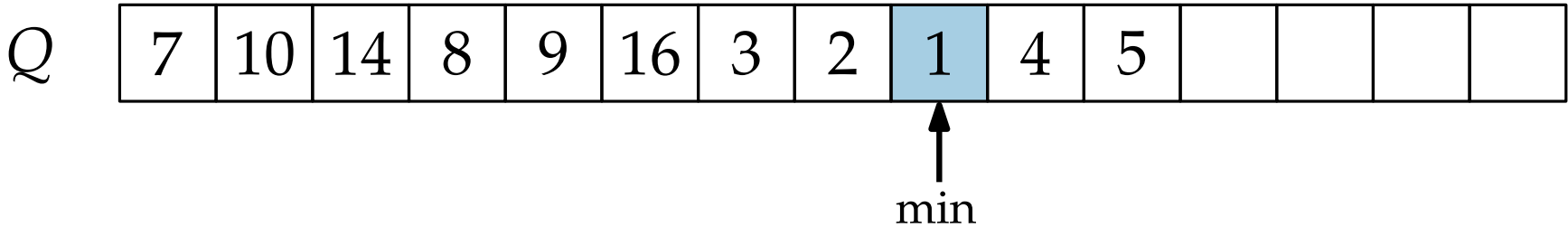
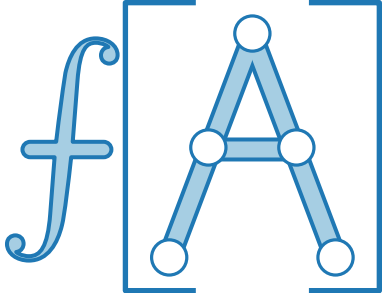
Operation	Implementierung	Laufzeit
$Q.insert(x)$	Füge x hinten ein, aktualisiere min Pointer	
$Q.min()$		
$Q.extractMin()$		
$Q.decreaseKey(x, p)$		
$Q.union(Q')$		

Implementierung I: Arrays



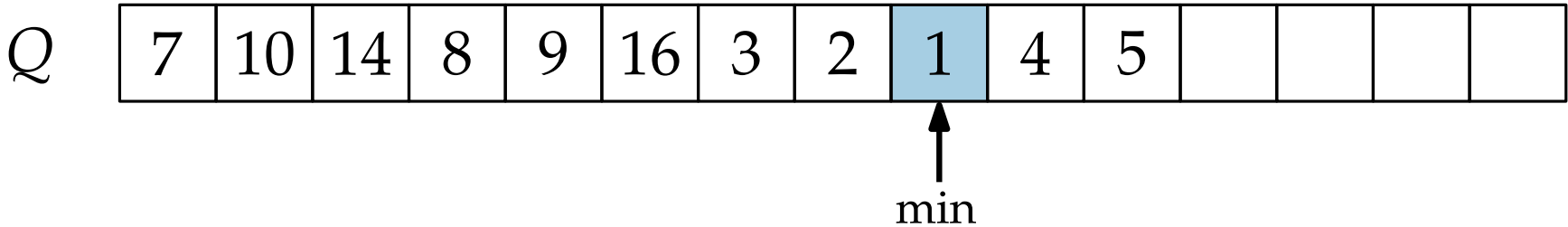
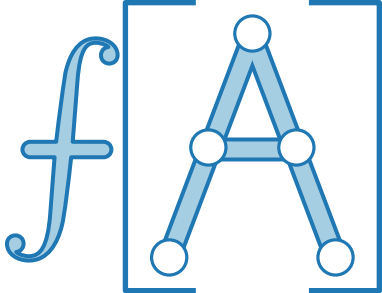
Operation	Implementierung	Laufzeit
$Q.insert(x)$	Füge x hinten ein, aktualisiere min Pointer	
$Q.min()$		
$Q.extractMin()$		
$Q.decreaseKey(x, p)$		
$Q.union(Q')$		

Implementierung I: Arrays



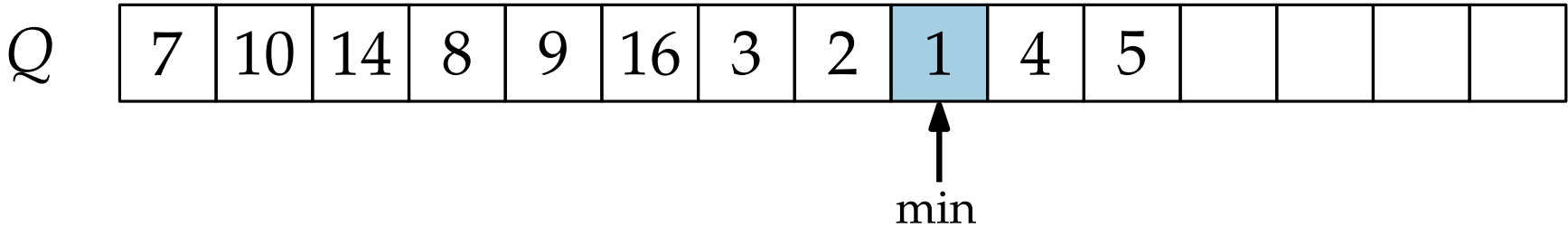
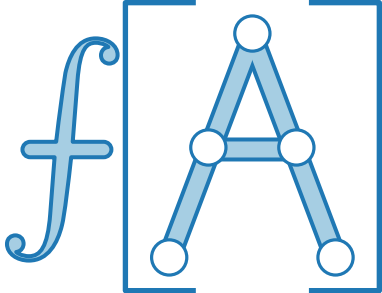
Operation	Implementierung	Laufzeit
$Q.insert(x)$	Füge x hinten ein, aktualisiere min Pointer	$\Theta(1)$
$Q.min()$		
$Q.extractMin()$		
$Q.decreaseKey(x, p)$		
$Q.union(Q')$		

Implementierung I: Arrays



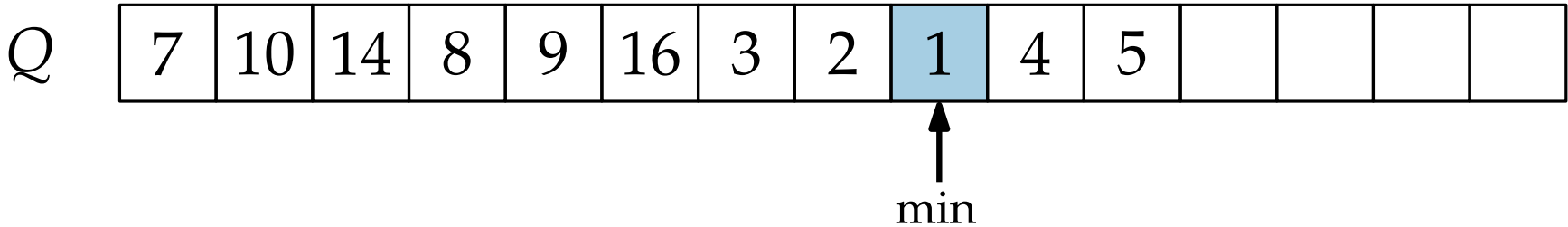
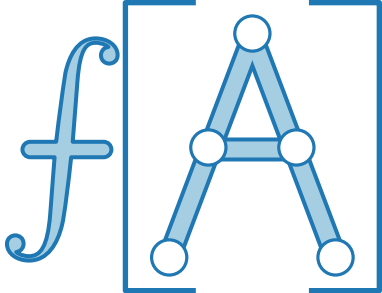
Operation	Implementierung	Laufzeit
$Q.insert(x)$	Füge x hinten ein, aktualisiere min Pointer	$\Theta(1)$
$Q.min()$	Benutze Pointer	
$Q.extractMin()$		
$Q.decreaseKey(x, p)$		
$Q.union(Q')$		

Implementierung I: Arrays



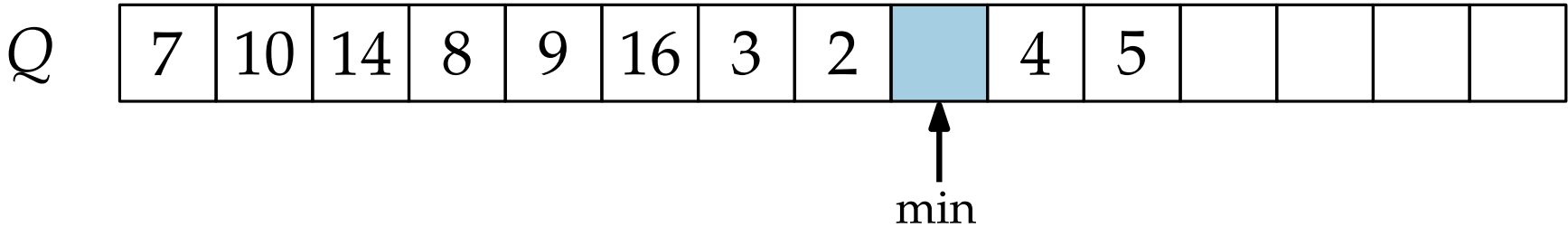
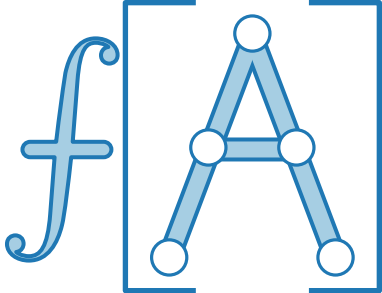
Operation	Implementierung	Laufzeit
$Q.insert(x)$	Füge x hinten ein, aktualisiere min Pointer	$\Theta(1)$
$Q.min()$	Benutze Pointer	$\Theta(1)$
$Q.extractMin()$		
$Q.decreaseKey(x, p)$		
$Q.union(Q')$		

Implementierung I: Arrays



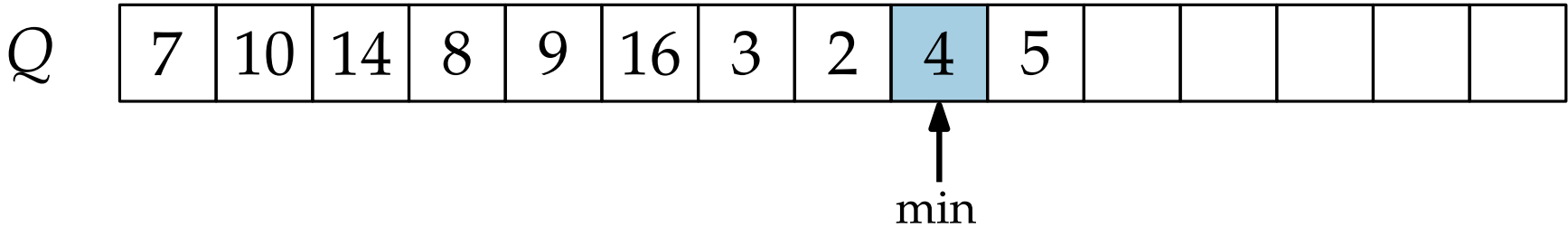
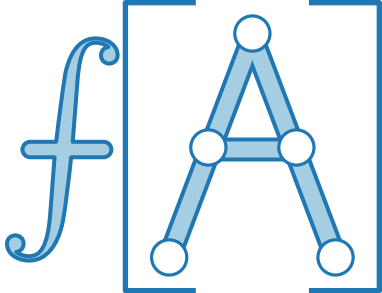
Operation	Implementierung	Laufzeit
$Q.insert(x)$	Füge x hinten ein, aktualisiere min Pointer	$\Theta(1)$
$Q.min()$	Benutze Pointer	$\Theta(1)$
$Q.extractMin()$	Lösche min, verschiebe Elemente dahinter, suche neues min	
$Q.decreaseKey(x, p)$		
$Q.union(Q')$		

Implementierung I: Arrays



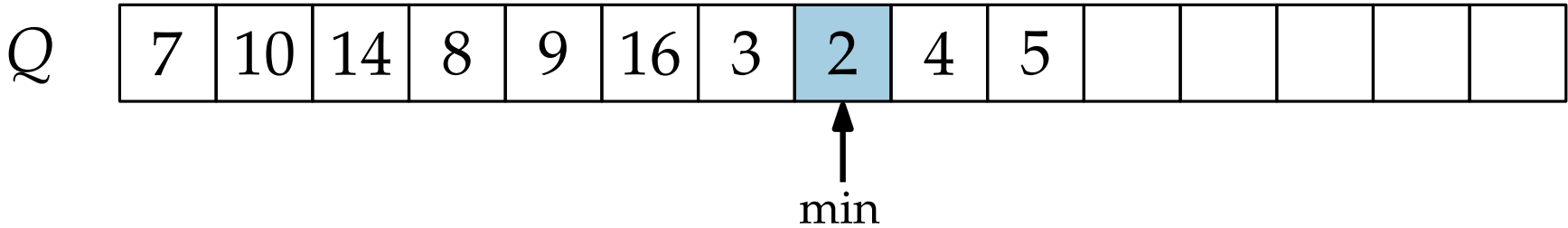
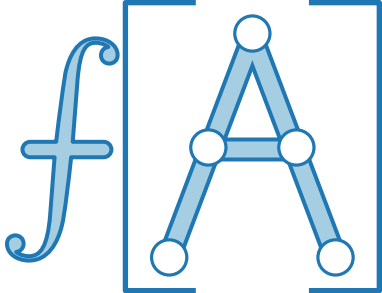
Operation	Implementierung	Laufzeit
$Q.insert(x)$	Füge x hinten ein, aktualisiere min Pointer	$\Theta(1)$
$Q.min()$	Benutze Pointer	$\Theta(1)$
$Q.extractMin()$	Lösche min, verschiebe Elemente dahinter, suche neues min	
$Q.decreaseKey(x, p)$		
$Q.union(Q')$		

Implementierung I: Arrays



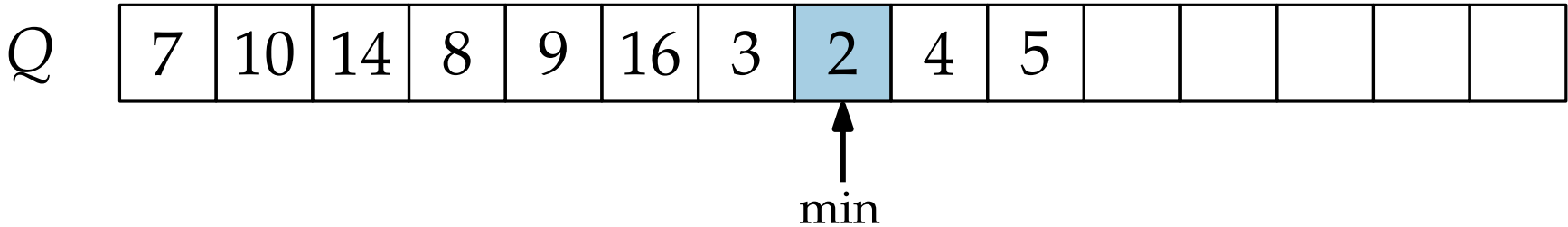
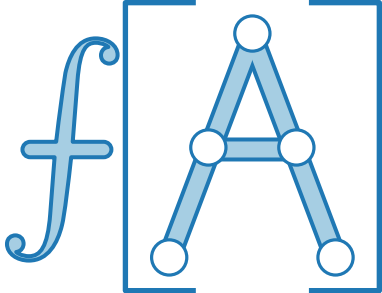
Operation	Implementierung	Laufzeit
$Q.insert(x)$	Füge x hinten ein, aktualisiere min Pointer	$\Theta(1)$
$Q.min()$	Benutze Pointer	$\Theta(1)$
$Q.extractMin()$	Lösche min, verschiebe Elemente dahinter, suche neues min	
$Q.decreaseKey(x, p)$		
$Q.union(Q')$		

Implementierung I: Arrays



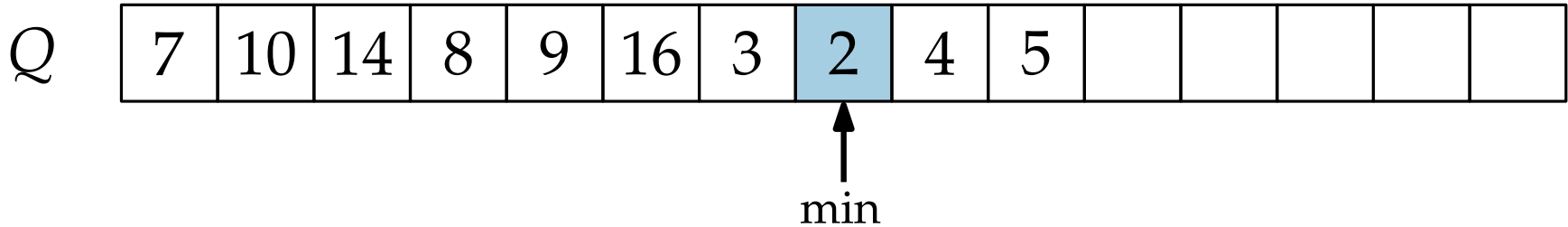
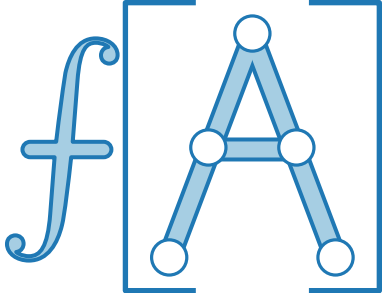
Operation	Implementierung	Laufzeit
$Q.insert(x)$	Füge x hinten ein, aktualisiere min Pointer	$\Theta(1)$
$Q.min()$	Benutze Pointer	$\Theta(1)$
$Q.extractMin()$	Lösche min, verschiebe Elemente dahinter, suche neues min	
$Q.decreaseKey(x, p)$		
$Q.union(Q')$		

Implementierung I: Arrays



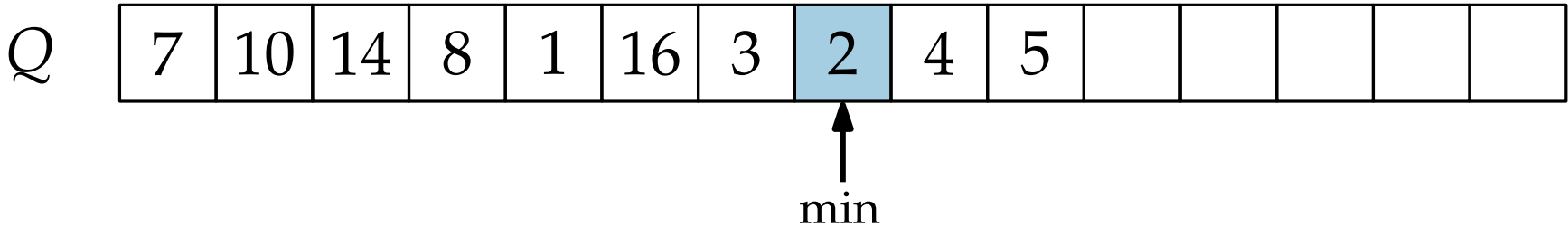
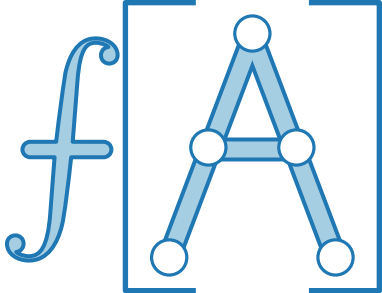
Operation	Implementierung	Laufzeit
$Q.insert(x)$	Füge x hinten ein, aktualisiere min Pointer	$\Theta(1)$
$Q.min()$	Benutze Pointer	$\Theta(1)$
$Q.extractMin()$	Lösche min, verschiebe Elemente dahinter, suche neues min	$\Theta(n)$
$Q.decreaseKey(x, p)$		
$Q.union(Q')$		

Implementierung I: Arrays



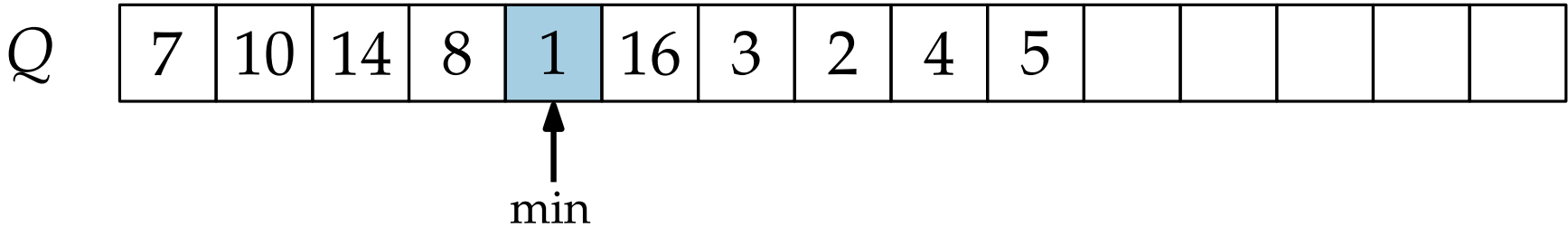
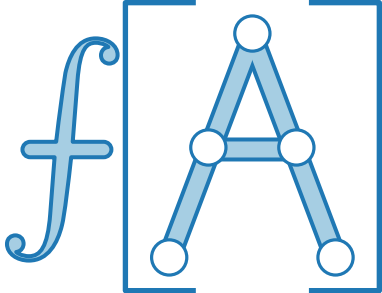
Operation	Implementierung	Laufzeit
$Q.insert(x)$	Füge x hinten ein, aktualisiere min Pointer	$\Theta(1)$
$Q.min()$	Benutze Pointer	$\Theta(1)$
$Q.extractMin()$	Lösche min, verschiebe Elemente dahinter, suche neues min	$\Theta(n)$
$Q.decreaseKey(x, p)$	Update x , aktualisiere min Pointer	
$Q.union(Q')$		

Implementierung I: Arrays



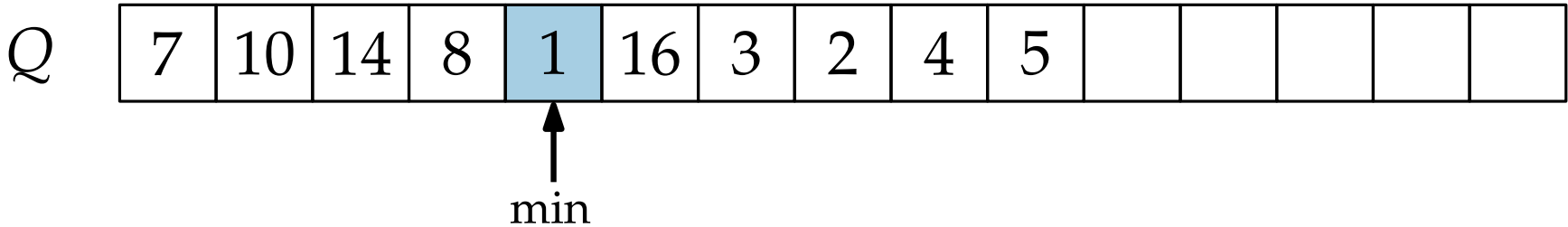
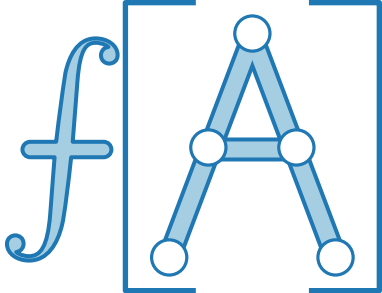
Operation	Implementierung	Laufzeit
$Q.insert(x)$	Füge x hinten ein, aktualisiere min Pointer	$\Theta(1)$
$Q.min()$	Benutze Pointer	$\Theta(1)$
$Q.extractMin()$	Lösche min, verschiebe Elemente dahinter, suche neues min	$\Theta(n)$
$Q.decreaseKey(x, p)$	Update x , aktualisiere min Pointer	
$Q.union(Q')$		

Implementierung I: Arrays



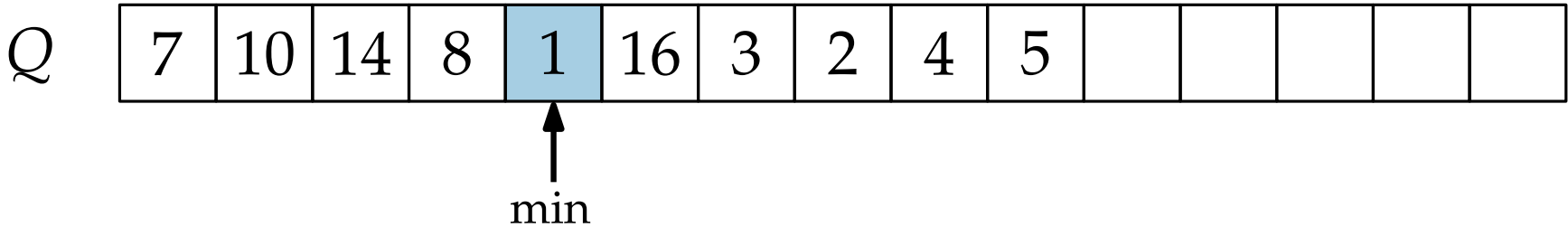
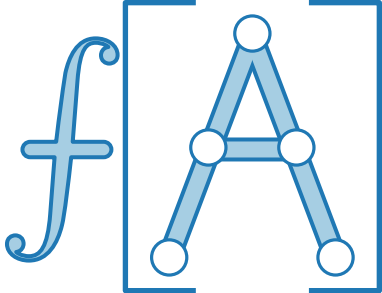
Operation	Implementierung	Laufzeit
$Q.insert(x)$	Füge x hinten ein, aktualisiere min Pointer	$\Theta(1)$
$Q.min()$	Benutze Pointer	$\Theta(1)$
$Q.extractMin()$	Lösche min, verschiebe Elemente dahinter, suche neues min	$\Theta(n)$
$Q.decreaseKey(x, p)$	Update x , aktualisiere min Pointer	
$Q.union(Q')$		

Implementierung I: Arrays



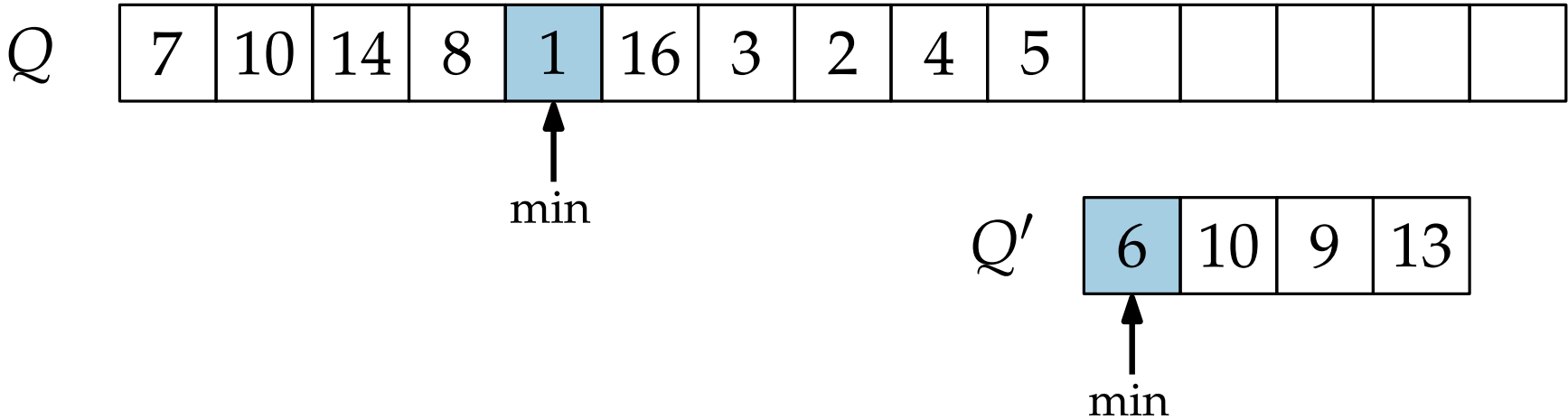
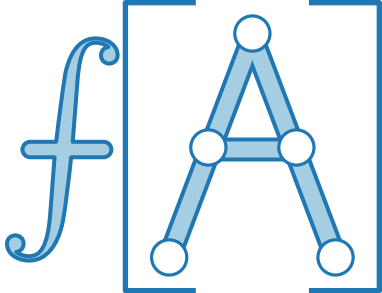
Operation	Implementierung	Laufzeit
$Q.insert(x)$	Füge x hinten ein, aktualisiere min Pointer	$\Theta(1)$
$Q.min()$	Benutze Pointer	$\Theta(1)$
$Q.extractMin()$	Lösche min, verschiebe Elemente dahinter, suche neues min	$\Theta(n)$
$Q.decreaseKey(x, p)$	Update x , aktualisiere min Pointer	$\Theta(1)$
$Q.union(Q')$		

Implementierung I: Arrays



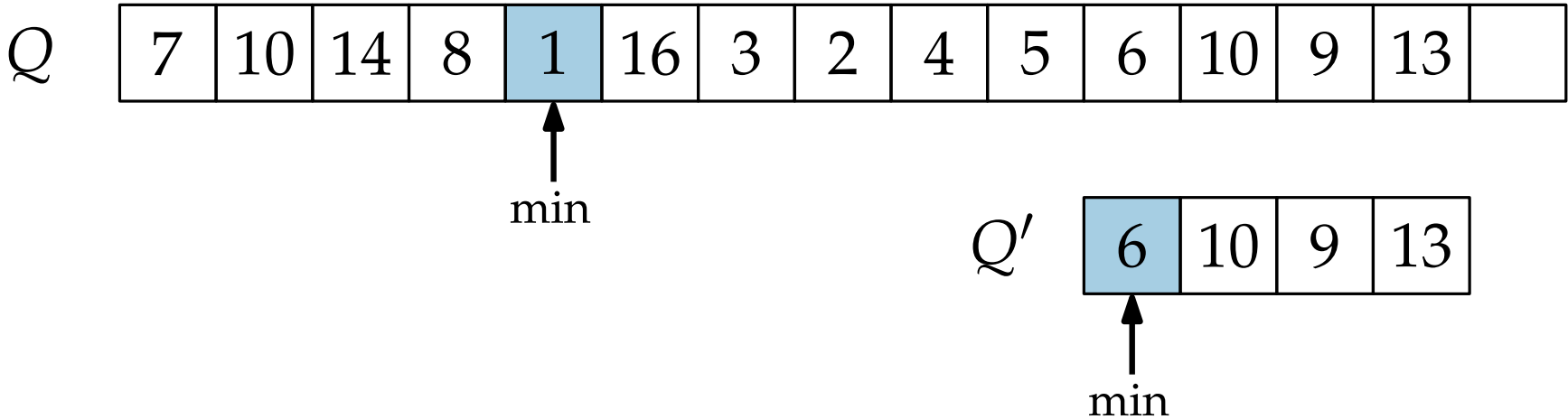
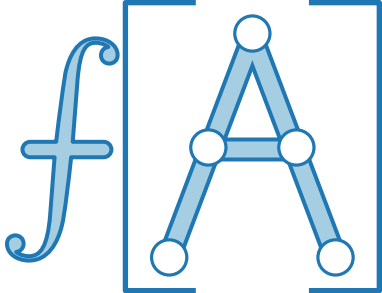
Operation	Implementierung	Laufzeit
$Q.insert(x)$	Füge x hinten ein, aktualisiere min Pointer	$\Theta(1)$
$Q.min()$	Benutze Pointer	$\Theta(1)$
$Q.extractMin()$	Lösche min, verschiebe Elemente dahinter, suche neues min	$\Theta(n)$
$Q.decreaseKey(x, p)$	Update x , aktualisiere min Pointer	$\Theta(1)$
$Q.union(Q')$	Hänge Q' hinten an, aktualisiere min Pointer	

Implementierung I: Arrays



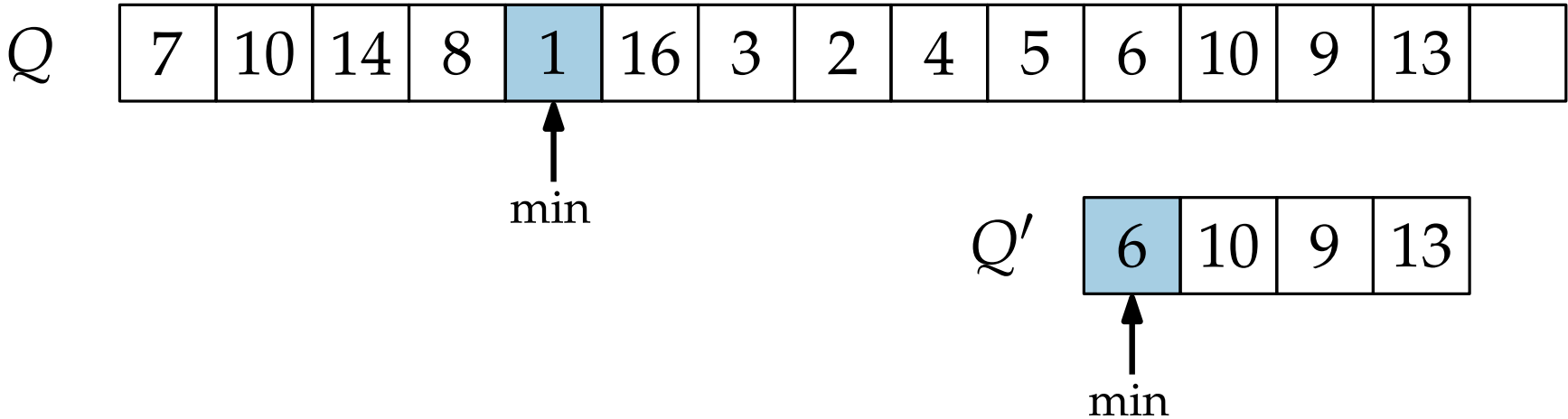
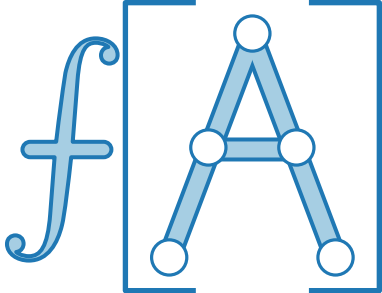
Operation	Implementierung	Laufzeit
$Q.\text{insert}(x)$	Füge x hinten ein, aktualisiere min Pointer	$\Theta(1)$
$Q.\text{min}()$	Benutze Pointer	$\Theta(1)$
$Q.\text{extractMin}()$	Lösche min, verschiebe Elemente dahinter, suche neues min	$\Theta(n)$
$Q.\text{decreaseKey}(x, p)$	Update x , aktualisiere min Pointer	$\Theta(1)$
$Q.\text{union}(Q')$	Hänge Q' hinten an, aktualisiere min Pointer	

Implementierung I: Arrays



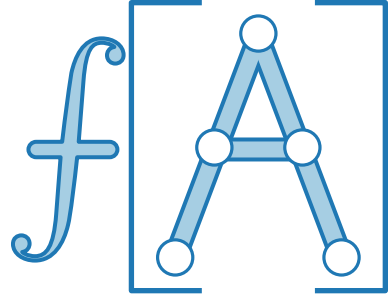
Operation	Implementierung	Laufzeit
$Q.insert(x)$	Füge x hinten ein, aktualisiere min Pointer	$\Theta(1)$
$Q.min()$	Benutze Pointer	$\Theta(1)$
$Q.extractMin()$	Lösche min, verschiebe Elemente dahinter, suche neues min	$\Theta(n)$
$Q.decreaseKey(x, p)$	Update x , aktualisiere min Pointer	$\Theta(1)$
$Q.union(Q')$	Hänge Q' hinten an, aktualisiere min Pointer	

Implementierung I: Arrays



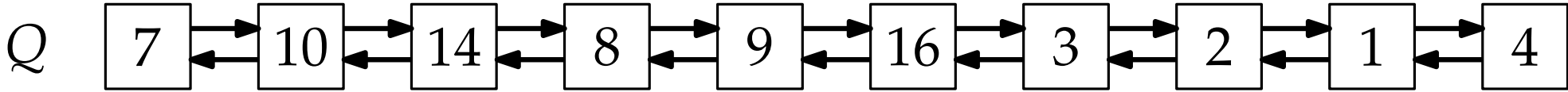
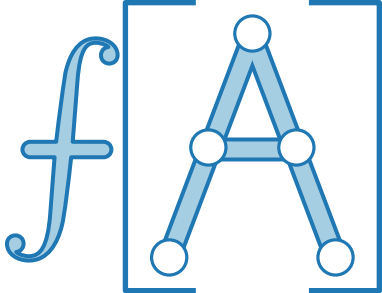
Operation	Implementierung	Laufzeit
$Q.insert(x)$	Füge x hinten ein, aktualisiere min Pointer	$\Theta(1)$
$Q.min()$	Benutze Pointer	$\Theta(1)$
$Q.extractMin()$	Lösche min, verschiebe Elemente dahinter, suche neues min	$\Theta(n)$
$Q.decreaseKey(x, p)$	Update x , aktualisiere min Pointer	$\Theta(1)$
$Q.union(Q')$	Hänge Q' hinten an, aktualisiere min Pointer	$\Theta(Q')$

Implementierung II: Liste



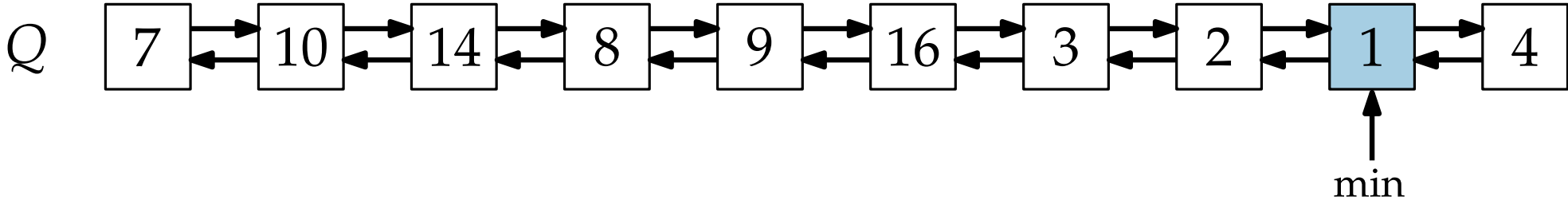
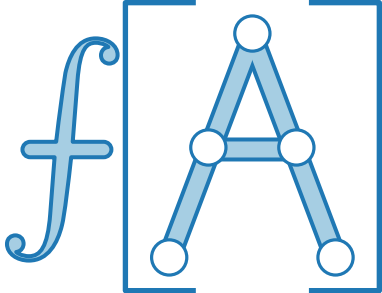
Operation	Implementierung	Laufzeit
$Q.\text{insert}(x)$		
$Q.\text{min}()$		
$Q.\text{extractMin}()$		
$Q.\text{decreaseKey}(x, p)$		
$Q.\text{union}(Q')$		

Implementierung II: Liste



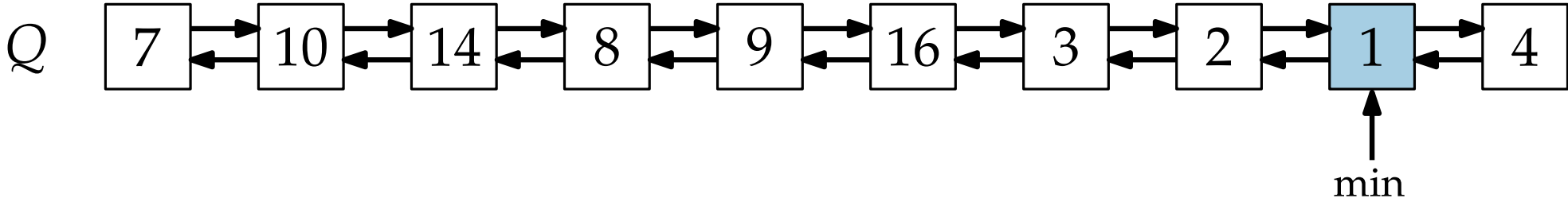
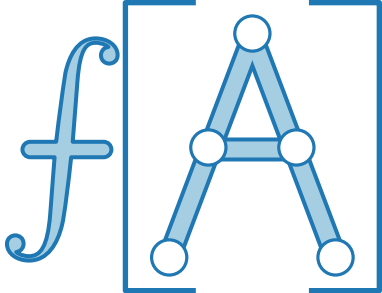
Operation	Implementierung	Laufzeit
$Q.insert(x)$		
$Q.min()$		
$Q.extractMin()$		
$Q.decreaseKey(x, p)$		
$Q.union(Q')$		

Implementierung II: Liste



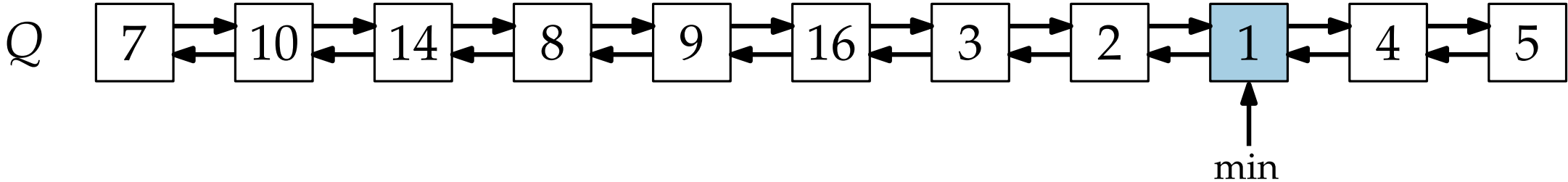
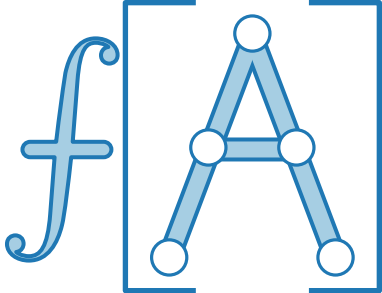
Operation	Implementierung	Laufzeit
$Q.\text{insert}(x)$		
$Q.\text{min}()$		
$Q.\text{extractMin}()$		
$Q.\text{decreaseKey}(x, p)$		
$Q.\text{union}(Q')$		

Implementierung II: Liste



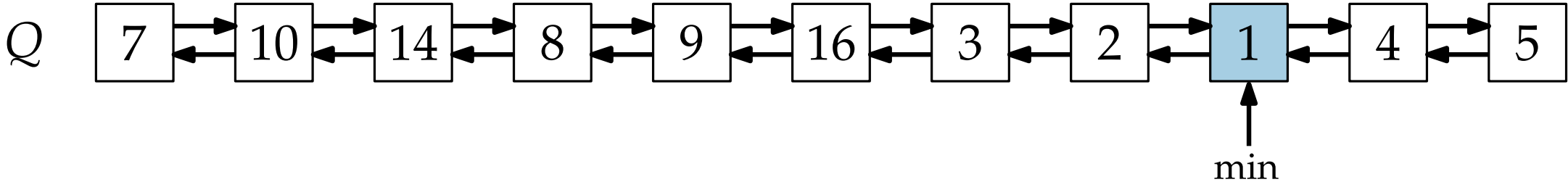
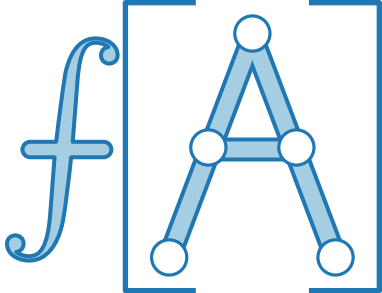
Operation	Implementierung	Laufzeit
$Q.insert(x)$	Füge x hinten ein, aktualisiere min Pointer	
$Q.min()$		
$Q.extractMin()$		
$Q.decreaseKey(x, p)$		
$Q.union(Q')$		

Implementierung II: Liste



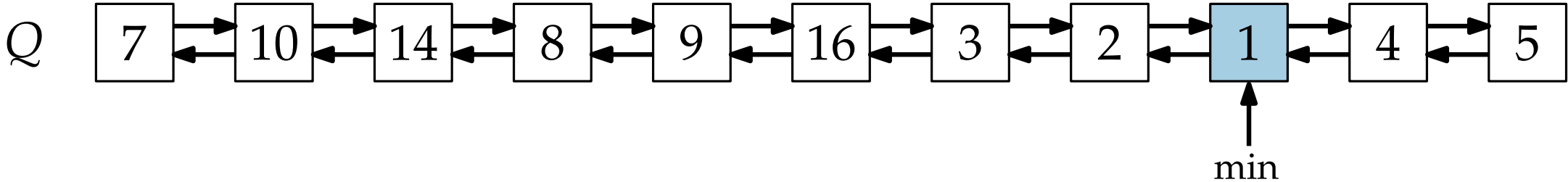
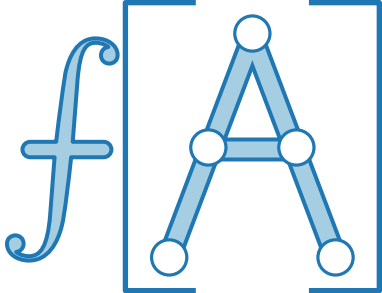
Operation	Implementierung	Laufzeit
$Q.insert(x)$	Füge x hinten ein, aktualisiere min Pointer	
$Q.min()$		
$Q.extractMin()$		
$Q.decreaseKey(x, p)$		
$Q.union(Q')$		

Implementierung II: Liste



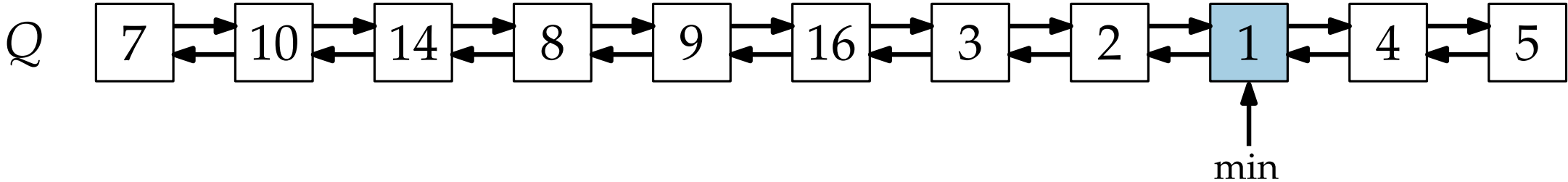
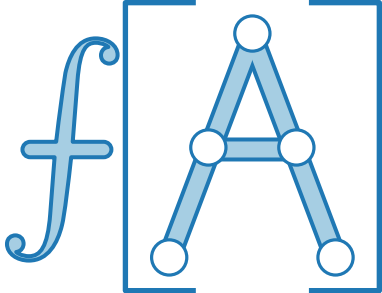
Operation	Implementierung	Laufzeit
$Q.insert(x)$	Füge x hinten ein, aktualisiere min Pointer	$\Theta(1)$
$Q.min()$		
$Q.extractMin()$		
$Q.decreaseKey(x, p)$		
$Q.union(Q')$		

Implementierung II: Liste



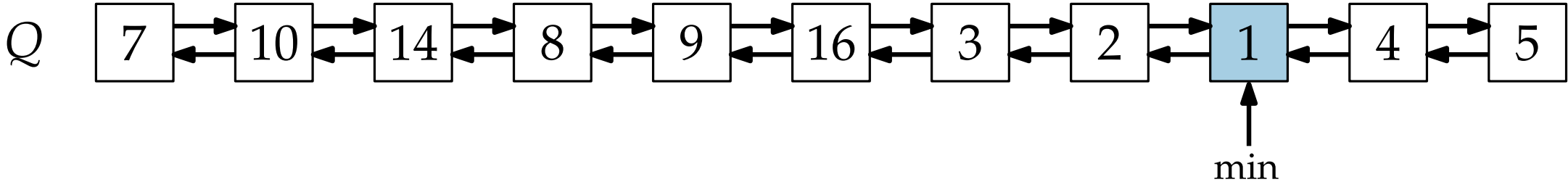
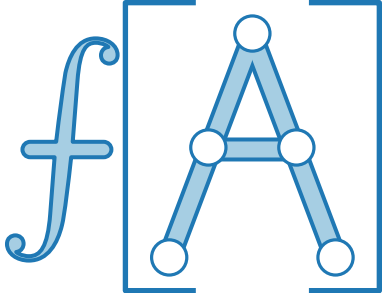
Operation	Implementierung	Laufzeit
$Q.insert(x)$	Füge x hinten ein, aktualisiere min Pointer	$\Theta(1)$
$Q.min()$	Benutze Pointer	
$Q.extractMin()$		
$Q.decreaseKey(x, p)$		
$Q.union(Q')$		

Implementierung II: Liste



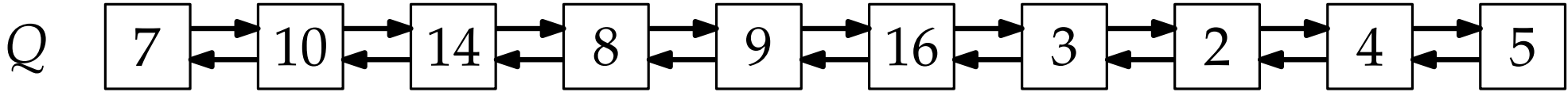
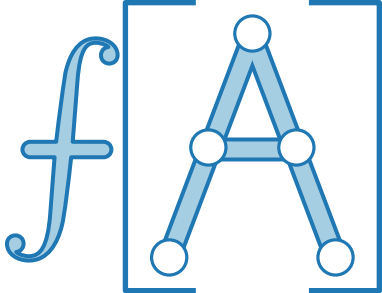
Operation	Implementierung	Laufzeit
$Q.insert(x)$	Füge x hinten ein, aktualisiere min Pointer	$\Theta(1)$
$Q.min()$	Benutze Pointer	$\Theta(1)$
$Q.extractMin()$		
$Q.decreaseKey(x, p)$		
$Q.union(Q')$		

Implementierung II: Liste



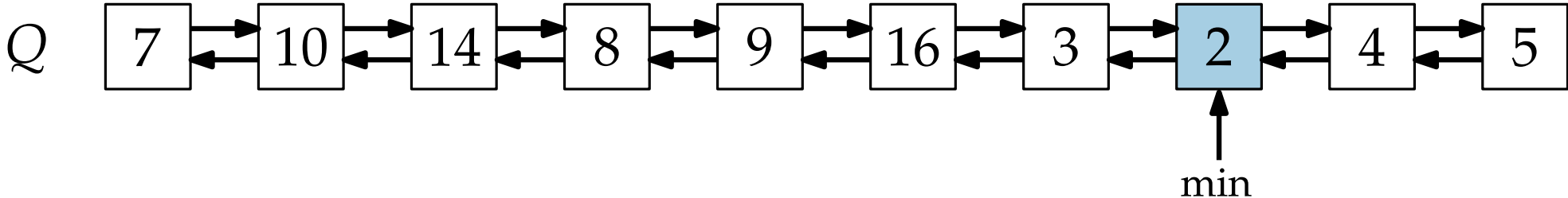
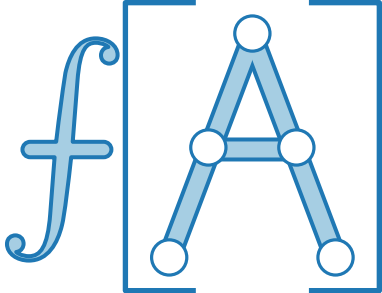
Operation	Implementierung	Laufzeit
$Q.insert(x)$	Füge x hinten ein, aktualisiere min Pointer	$\Theta(1)$
$Q.min()$	Benutze Pointer	$\Theta(1)$
$Q.extractMin()$	Lösche min, verknüpfe Vorgänger und Nachfolger, suche neues min	
$Q.decreaseKey(x, p)$		
$Q.union(Q')$		

Implementierung II: Liste



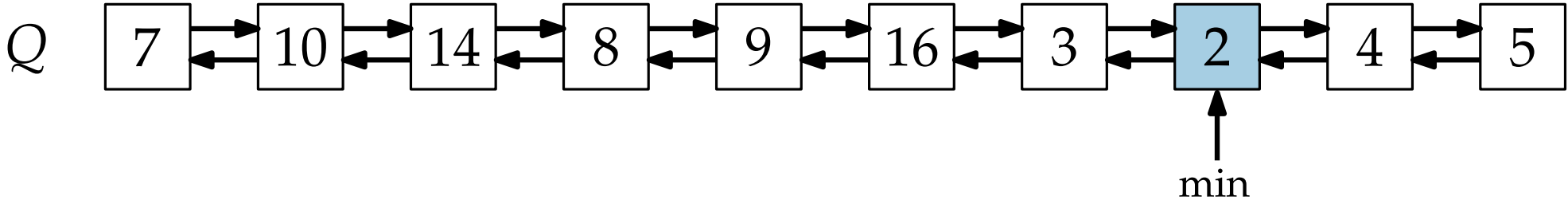
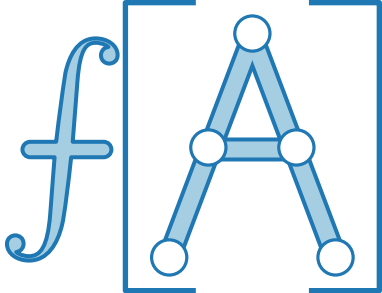
Operation	Implementierung	Laufzeit
$Q.insert(x)$	Füge x hinten ein, aktualisiere min Pointer	$\Theta(1)$
$Q.min()$	Benutze Pointer	$\Theta(1)$
$Q.extractMin()$	Lösche min, verknüpfe Vorgänger und Nachfolger, suche neues min	
$Q.decreaseKey(x, p)$		
$Q.union(Q')$		

Implementierung II: Liste



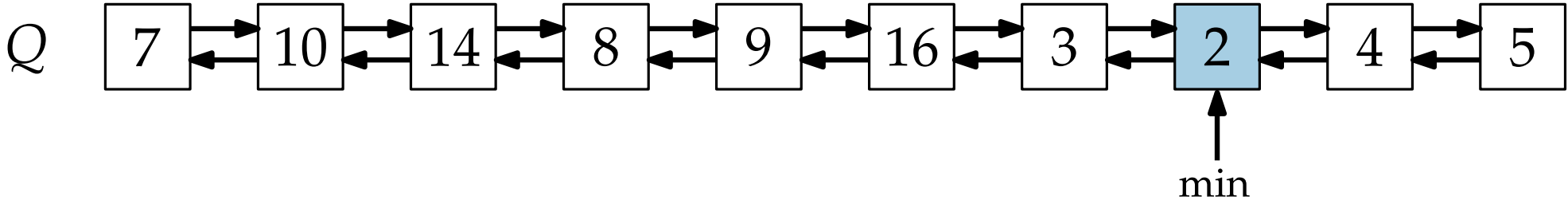
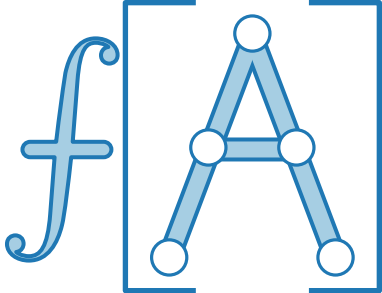
Operation	Implementierung	Laufzeit
$Q.insert(x)$	Füge x hinten ein, aktualisiere min Pointer	$\Theta(1)$
$Q.min()$	Benutze Pointer	$\Theta(1)$
$Q.extractMin()$	Lösche min, verknüpfe Vorgänger und Nachfolger, suche neues min	
$Q.decreaseKey(x, p)$		
$Q.union(Q')$		

Implementierung II: Liste



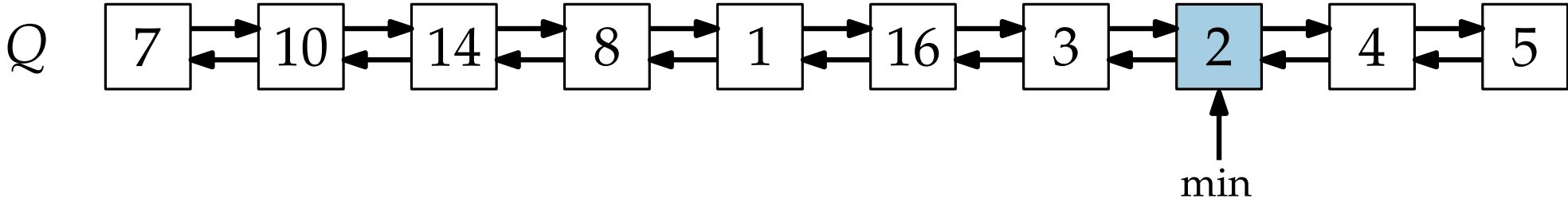
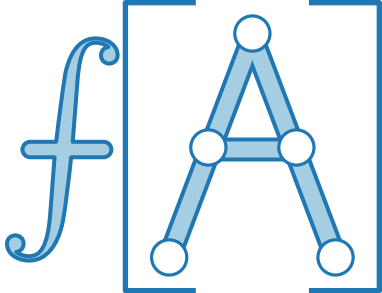
Operation	Implementierung	Laufzeit
$Q.insert(x)$	Füge x hinten ein, aktualisiere min Pointer	$\Theta(1)$
$Q.min()$	Benutze Pointer	$\Theta(1)$
$Q.extractMin()$	Lösche min, verknüpfe Vorgänger und Nachfolger, suche neues min	$\Theta(n)$
$Q.decreaseKey(x, p)$		
$Q.union(Q')$		

Implementierung II: Liste



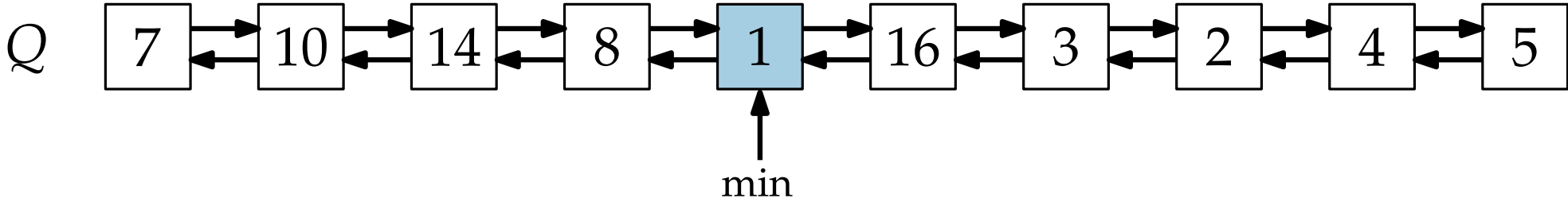
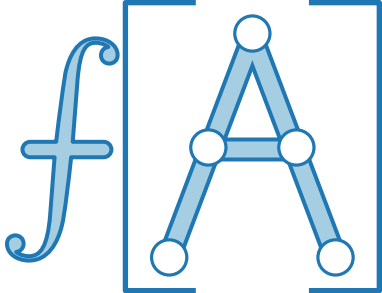
Operation	Implementierung	Laufzeit
$Q.insert(x)$	Füge x hinten ein, aktualisiere min Pointer	$\Theta(1)$
$Q.min()$	Benutze Pointer	$\Theta(1)$
$Q.extractMin()$	Lösche min, verknüpfe Vorgänger und Nachfolger, suche neues min	$\Theta(n)$
$Q.decreaseKey(x, p)$	Update x , aktualisiere min Pointer	
$Q.union(Q')$		

Implementierung II: Liste



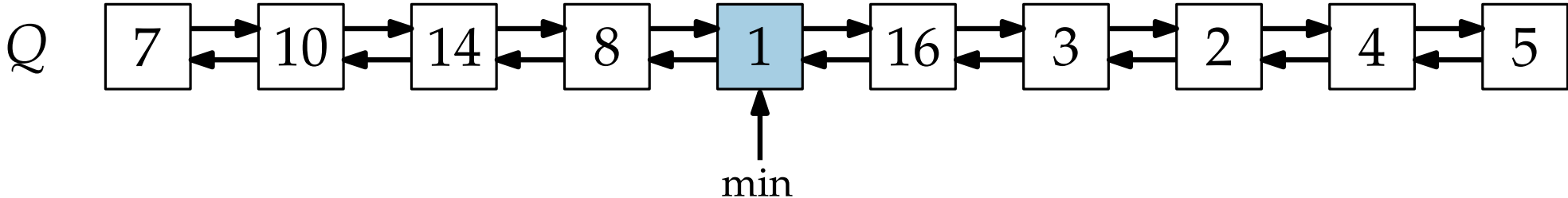
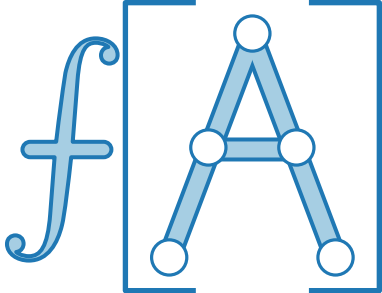
Operation	Implementierung	Laufzeit
$Q.insert(x)$	Füge x hinten ein, aktualisiere min Pointer	$\Theta(1)$
$Q.min()$	Benutze Pointer	$\Theta(1)$
$Q.extractMin()$	Lösche min, verknüpfe Vorgänger und Nachfolger, suche neues min	$\Theta(n)$
$Q.decreaseKey(x, p)$	Update x , aktualisiere min Pointer	
$Q.union(Q')$		

Implementierung II: Liste



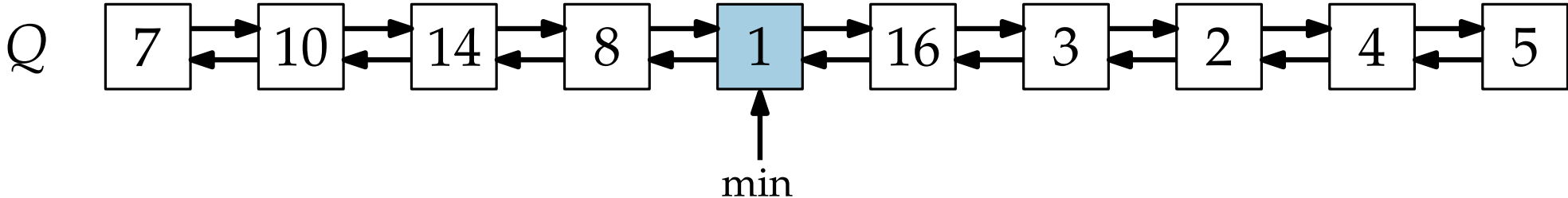
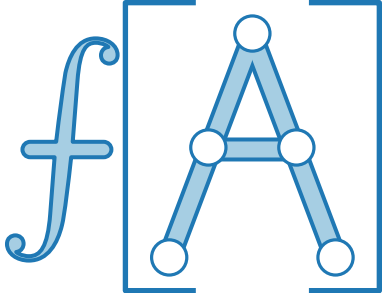
Operation	Implementierung	Laufzeit
$Q.insert(x)$	Füge x hinten ein, aktualisiere min Pointer	$\Theta(1)$
$Q.min()$	Benutze Pointer	$\Theta(1)$
$Q.extractMin()$	Lösche min, verknüpfe Vorgänger und Nachfolger, suche neues min	$\Theta(n)$
$Q.decreaseKey(x, p)$	Update x , aktualisiere min Pointer	
$Q.union(Q')$		

Implementierung II: Liste



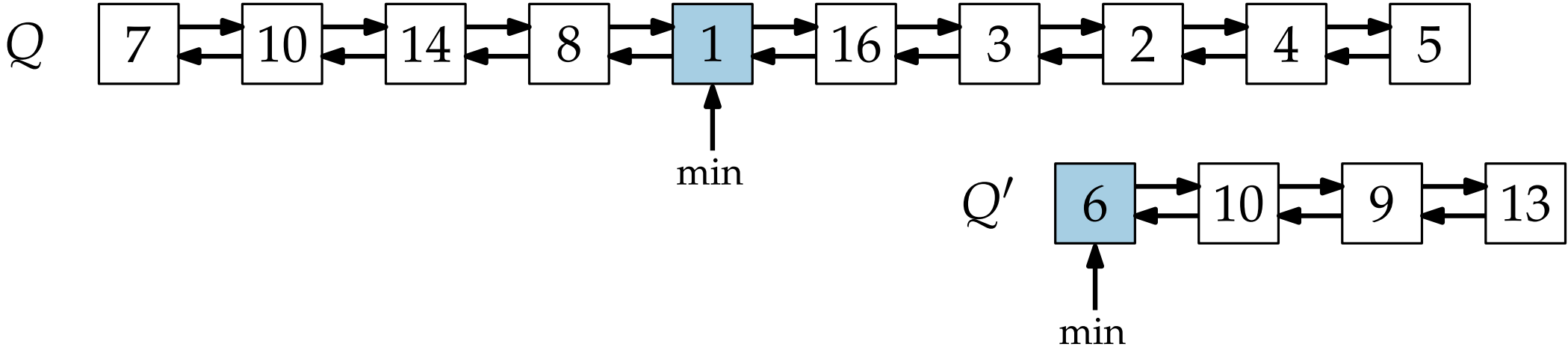
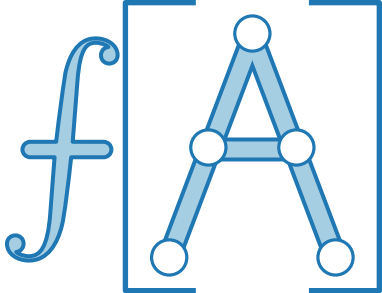
Operation	Implementierung	Laufzeit
$Q.insert(x)$	Füge x hinten ein, aktualisiere min Pointer	$\Theta(1)$
$Q.min()$	Benutze Pointer	$\Theta(1)$
$Q.extractMin()$	Lösche min, verknüpfe Vorgänger und Nachfolger, suche neues min	$\Theta(n)$
$Q.decreaseKey(x, p)$	Update x , aktualisiere min Pointer	$\Theta(1)$
$Q.union(Q')$		

Implementierung II: Liste



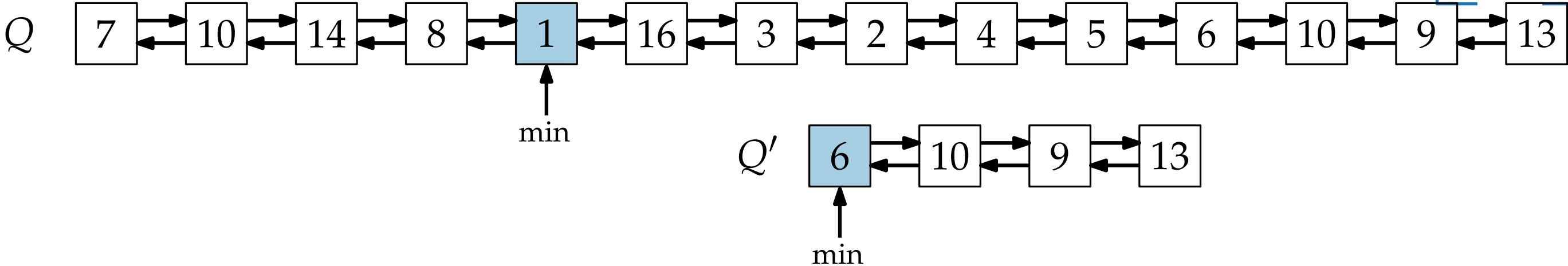
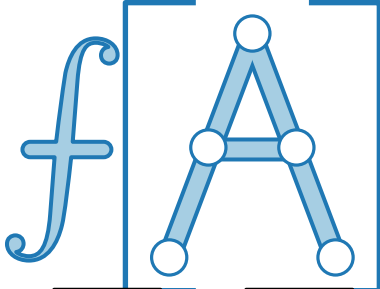
Operation	Implementierung	Laufzeit
$Q.insert(x)$	Füge x hinten ein, aktualisiere min Pointer	$\Theta(1)$
$Q.min()$	Benutze Pointer	$\Theta(1)$
$Q.extractMin()$	Lösche min, verknüpfe Vorgänger und Nachfolger, suche neues min	$\Theta(n)$
$Q.decreaseKey(x, p)$	Update x , aktualisiere min Pointer	$\Theta(1)$
$Q.union(Q')$	Hänge Q' hinten an, aktualisiere min Pointer	

Implementierung II: Liste



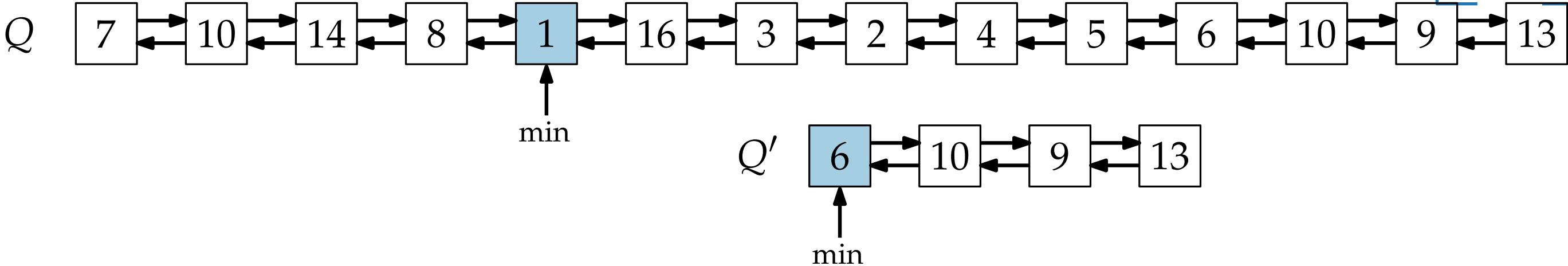
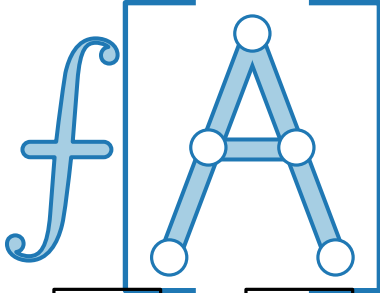
Operation	Implementierung	Laufzeit
$Q.\text{insert}(x)$	Füge x hinten ein, aktualisiere min Pointer	$\Theta(1)$
$Q.\text{min}()$	Benutze Pointer	$\Theta(1)$
$Q.\text{extractMin}()$	Lösche min , verknüpfe Vorgänger und Nachfolger, suche neues min	$\Theta(n)$
$Q.\text{decreaseKey}(x, p)$	Update x , aktualisiere min Pointer	$\Theta(1)$
$Q.\text{union}(Q')$	Hänge Q' hinten an, aktualisiere min Pointer	

Implementierung II: Liste



Operation	Implementierung	Laufzeit
$Q.\text{insert}(x)$	Füge x hinten ein, aktualisiere min Pointer	$\Theta(1)$
$Q.\text{min}()$	Benutze Pointer	$\Theta(1)$
$Q.\text{extractMin}()$	Lösche min , verknüpfe Vorgänger und Nachfolger, suche neues min	$\Theta(n)$
$Q.\text{decreaseKey}(x, p)$	Update x , aktualisiere min Pointer	$\Theta(1)$
$Q.\text{union}(Q')$	Hänge Q' hinten an, aktualisiere min Pointer	

Implementierung II: Liste



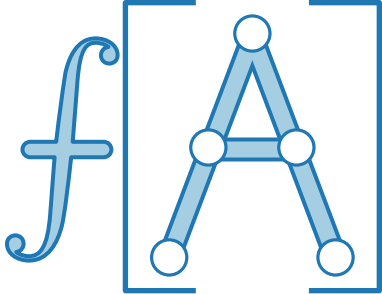
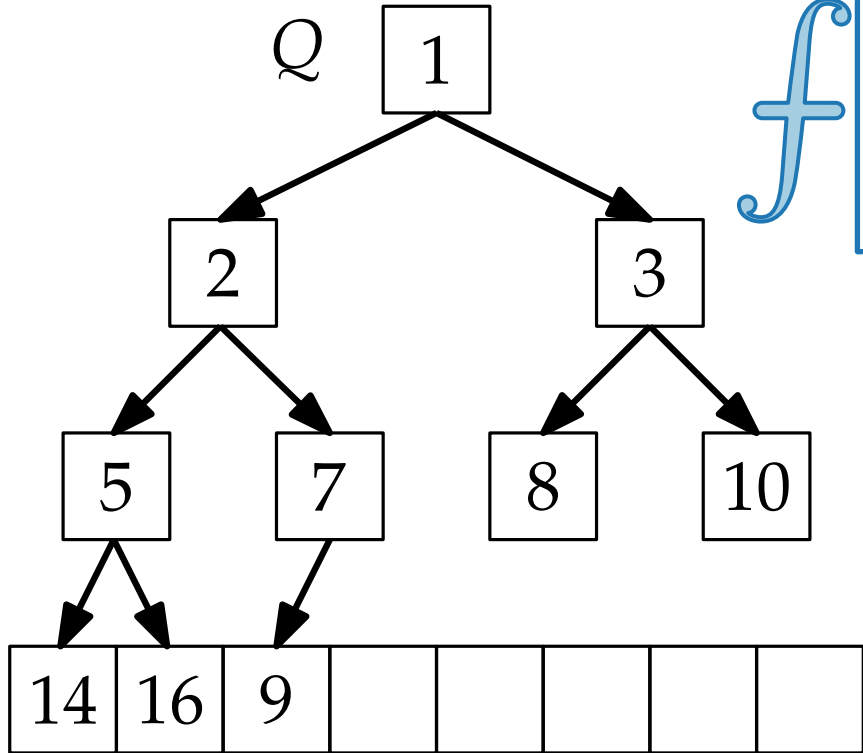
Operation	Implementierung	Laufzeit
$Q.\text{insert}(x)$	Füge x hinten ein, aktualisiere min Pointer	$\Theta(1)$
$Q.\text{min}()$	Benutze Pointer	$\Theta(1)$
$Q.\text{extractMin}()$	Lösche min , verknüpfe Vorgänger und Nachfolger, suche neues min	$\Theta(n)$
$Q.\text{decreaseKey}(x, p)$	Update x , aktualisiere min Pointer	$\Theta(1)$
$Q.\text{union}(Q')$	Hänge Q' hinten an, aktualisiere min Pointer	$\Theta(1)$

Implementierung III: MinHeap



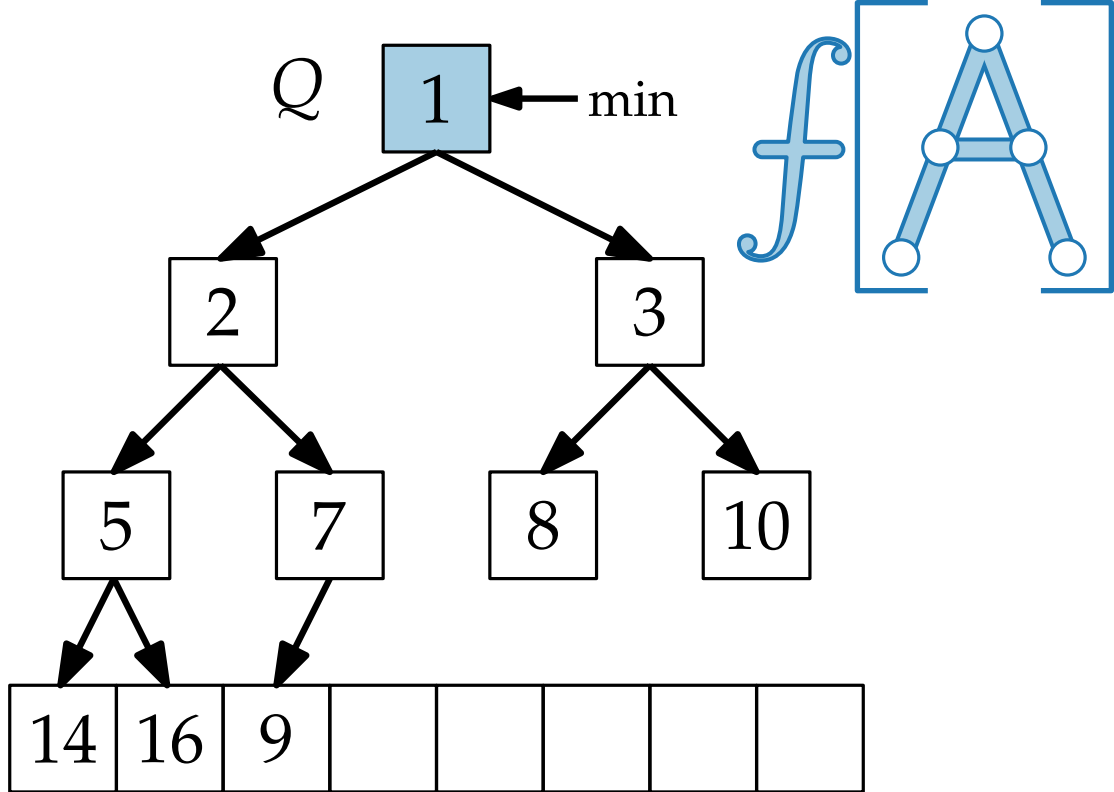
Operation	Implementierung	Laufzeit
<code>Q.insert(x)</code>		
<code>Q.min()</code>		
<code>Q.extractMin()</code>		
<code>Q.decreaseKey(x, p)</code>		
<code>Q.union(Q')</code>		

Implementierung III: MinHeap



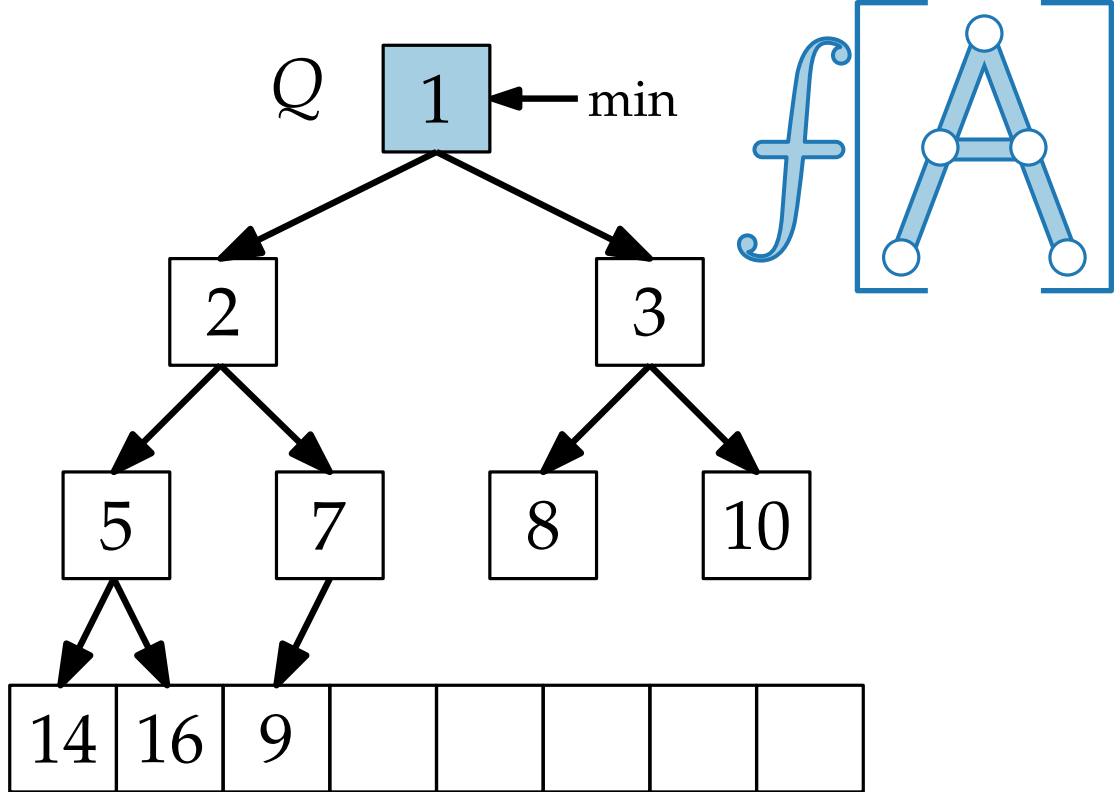
Operation	Implementierung	Laufzeit
$Q.insert(x)$		
$Q.min()$		
$Q.extractMin()$		
$Q.decreaseKey(x, p)$		
$Q.union(Q')$		

Implementierung III: MinHeap



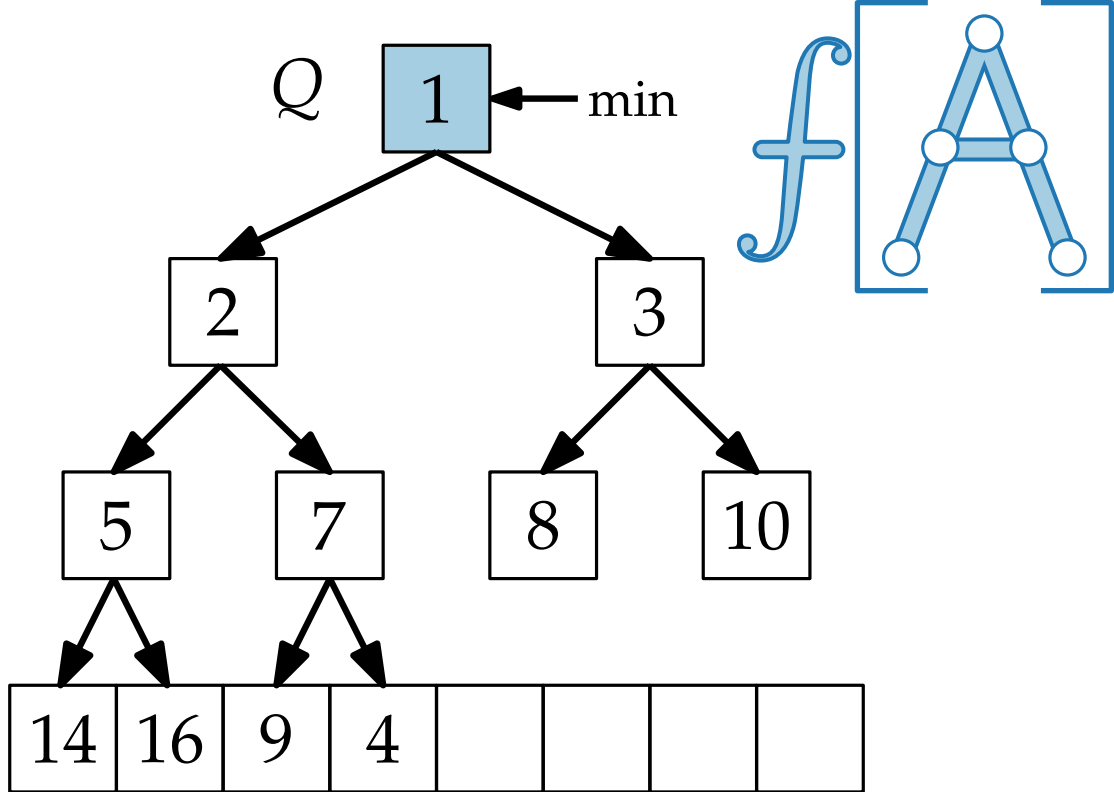
Operation	Implementierung	Laufzeit
$Q.\text{insert}(x)$		
$Q.\text{min}()$		
$Q.\text{extractMin}()$		
$Q.\text{decreaseKey}(x, p)$		
$Q.\text{union}(Q')$		

Implementierung III: MinHeap



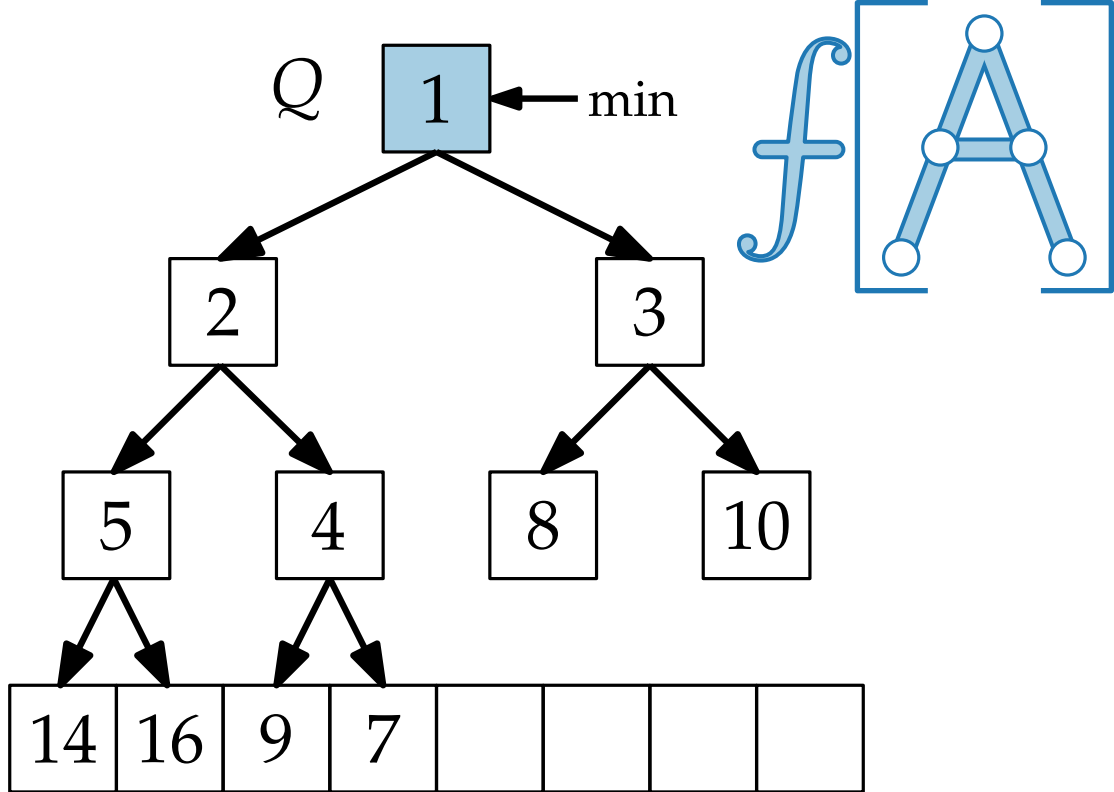
Operation	Implementierung	Laufzeit
$Q.\text{insert}(x)$	Füge x hinten ein, „rotiere“ x nach oben	
$Q.\text{min}()$		
$Q.\text{extractMin}()$		
$Q.\text{decreaseKey}(x, p)$		
$Q.\text{union}(Q')$		

Implementierung III: MinHeap



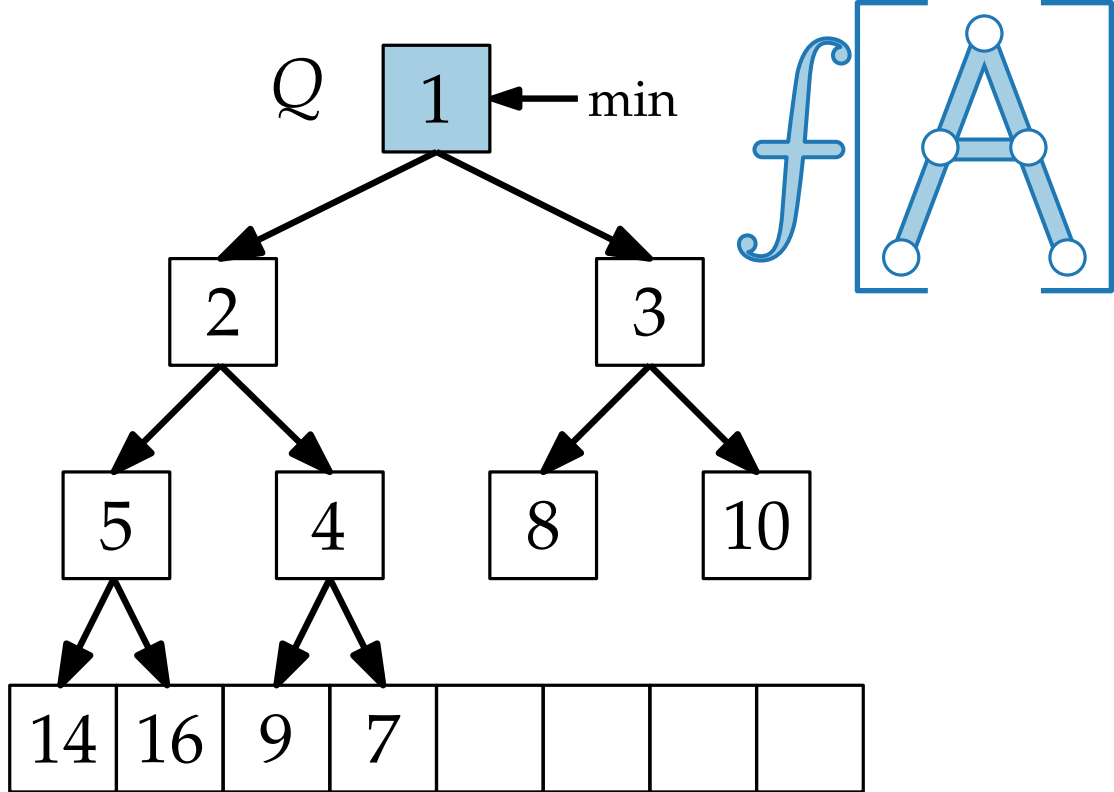
Operation	Implementierung	Laufzeit
$Q.\text{insert}(x)$	Füge x hinten ein, „rotiere“ x nach oben	
$Q.\text{min}()$		
$Q.\text{extractMin}()$		
$Q.\text{decreaseKey}(x, p)$		
$Q.\text{union}(Q')$		

Implementierung III: MinHeap



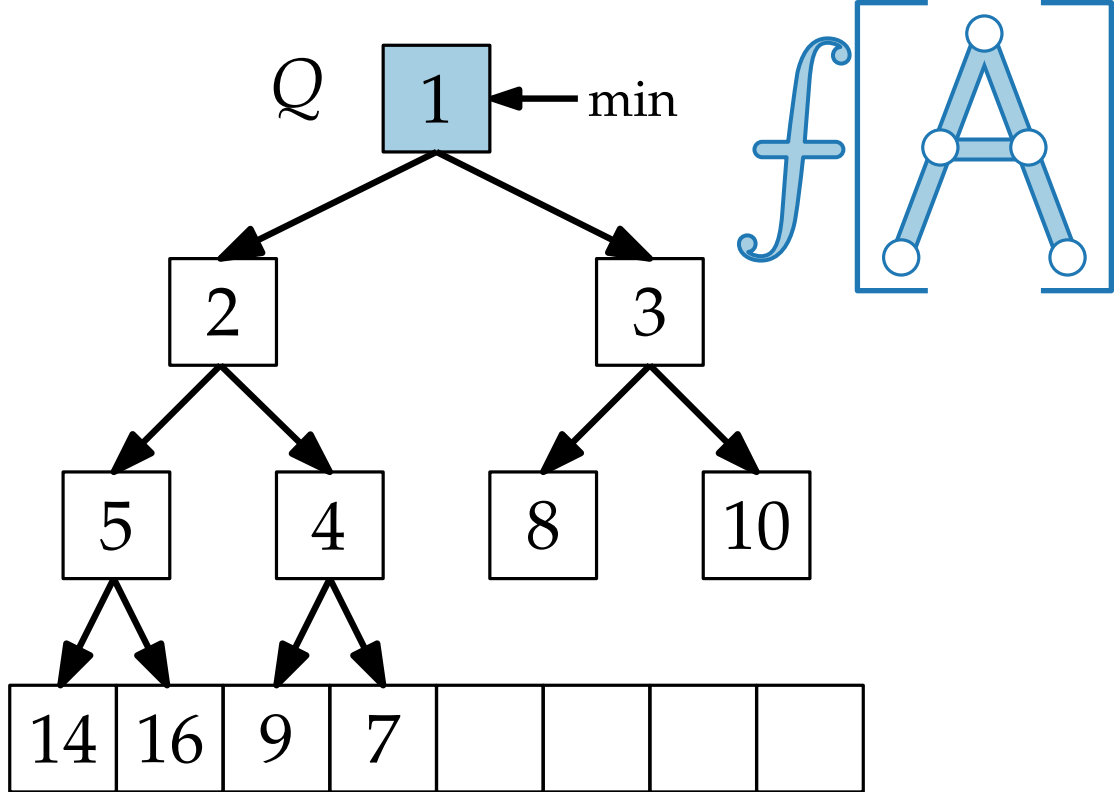
Operation	Implementierung	Laufzeit
<code>Q.insert(x)</code>	Füge x hinten ein, „rotiere“ x nach oben	
<code>Q.min()</code>		
<code>Q.extractMin()</code>		
<code>Q.decreaseKey(x, p)</code>		
<code>Q.union(Q')</code>		

Implementierung III: MinHeap



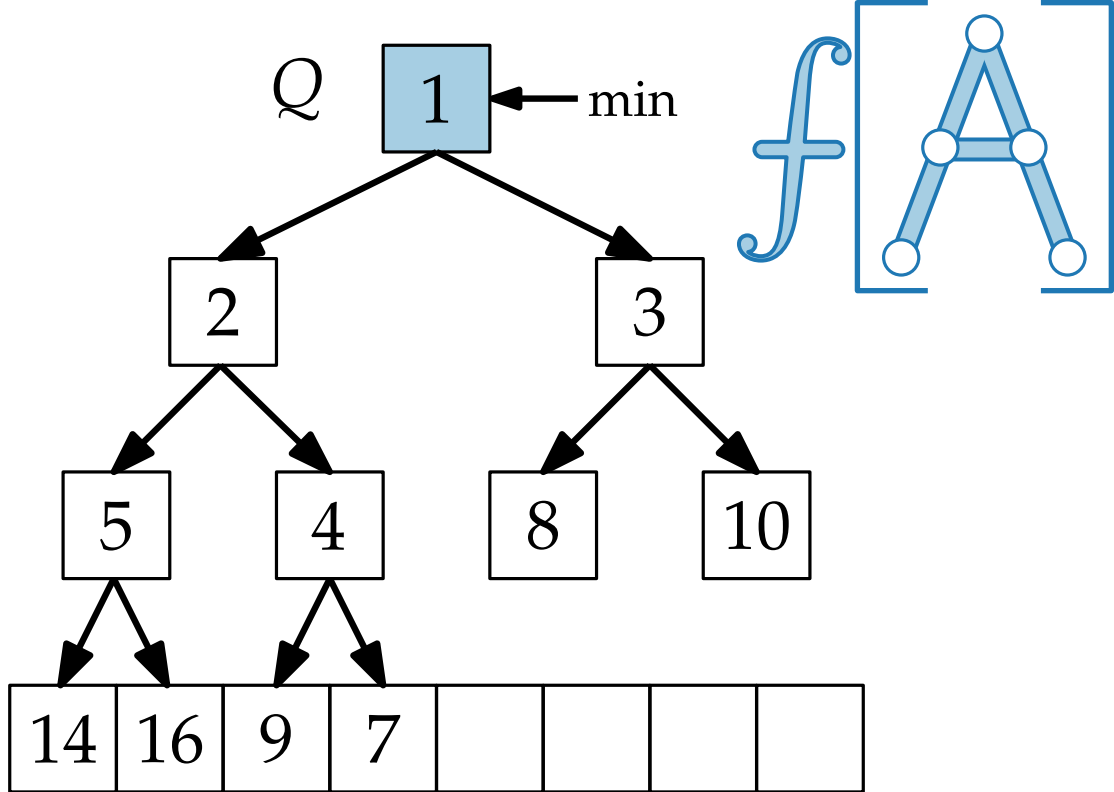
Operation	Implementierung	Laufzeit
$Q.\text{insert}(x)$	Füge x hinten ein, „rotiere“ x nach oben	$\Theta(\log n)$
$Q.\text{min}()$		
$Q.\text{extractMin}()$		
$Q.\text{decreaseKey}(x, p)$		
$Q.\text{union}(Q')$		

Implementierung III: MinHeap



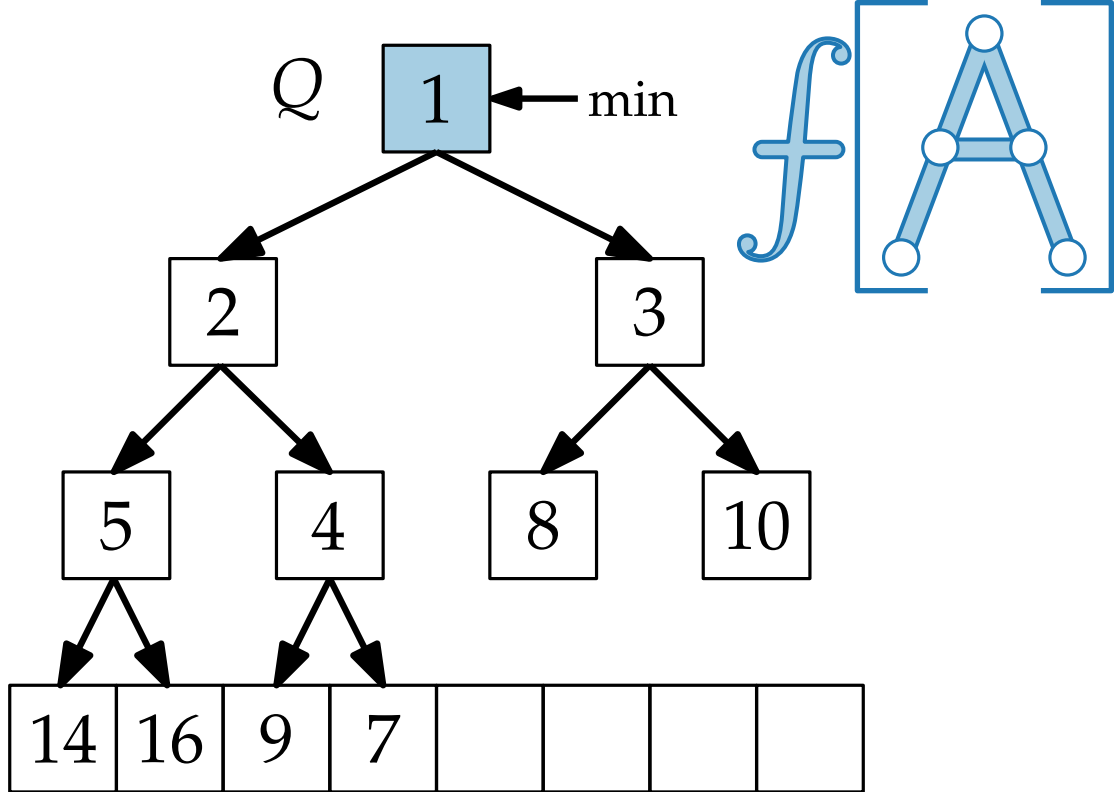
Operation	Implementierung	Laufzeit
$Q.\text{insert}(x)$	Füge x hinten ein, „rotiere“ x nach oben	$\Theta(\log n)$
$Q.\text{min}()$	Benutze Pointer	
$Q.\text{extractMin}()$		
$Q.\text{decreaseKey}(x, p)$		
$Q.\text{union}(Q')$		

Implementierung III: MinHeap



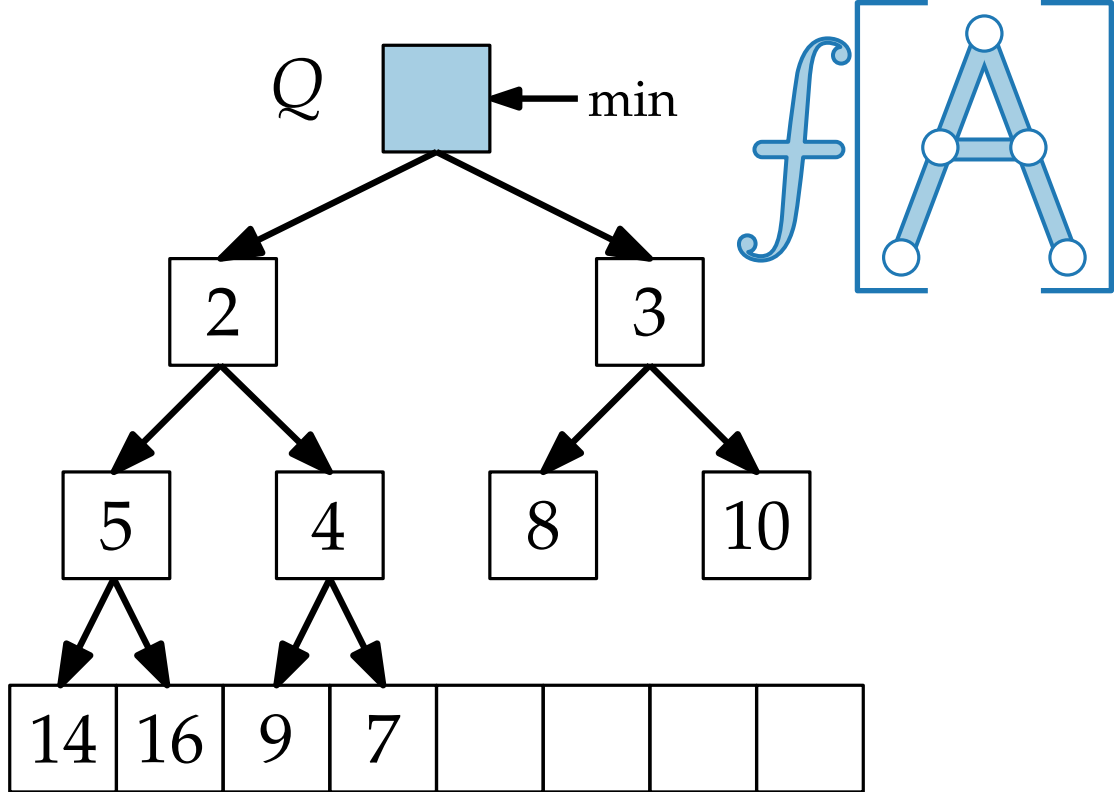
Operation	Implementierung	Laufzeit
$Q.insert(x)$	Füge x hinten ein, „rotiere“ x nach oben	$\Theta(\log n)$
$Q.min()$	Benutze Pointer	$\Theta(1)$
$Q.extractMin()$		
$Q.decreaseKey(x, p)$		
$Q.union(Q')$		

Implementierung III: MinHeap



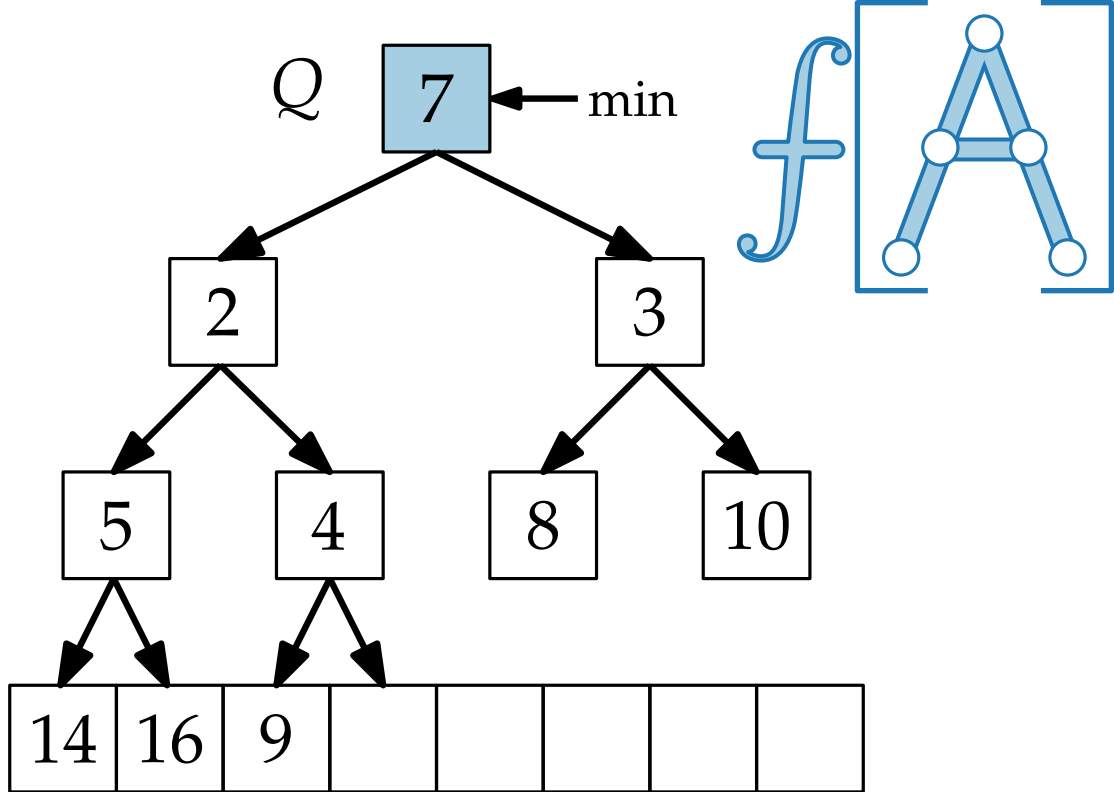
Operation	Implementierung	Laufzeit
$Q.insert(x)$	Füge x hinten ein, „rotiere“ x nach oben	$\Theta(\log n)$
$Q.min()$	Benutze Pointer	$\Theta(1)$
$Q.extractMin()$	Lösche min, schiebe letztes Element vor, rotiere es nach unten	
$Q.decreaseKey(x, p)$		
$Q.union(Q')$		

Implementierung III: MinHeap



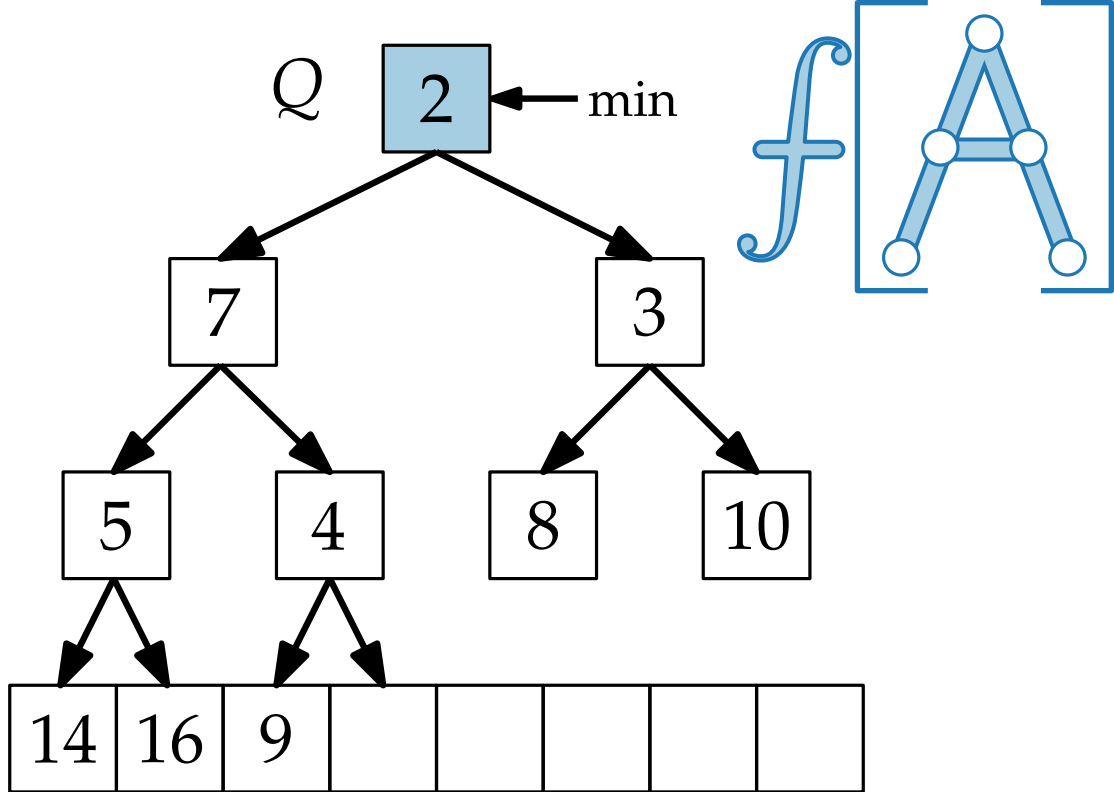
Operation	Implementierung	Laufzeit
$Q.insert(x)$	Füge x hinten ein, „rotiere“ x nach oben	$\Theta(\log n)$
$Q.min()$	Benutze Pointer	$\Theta(1)$
$Q.extractMin()$	Lösche min, schiebe letztes Element vor, rotiere es nach unten	
$Q.decreaseKey(x, p)$		
$Q.union(Q')$		

Implementierung III: MinHeap



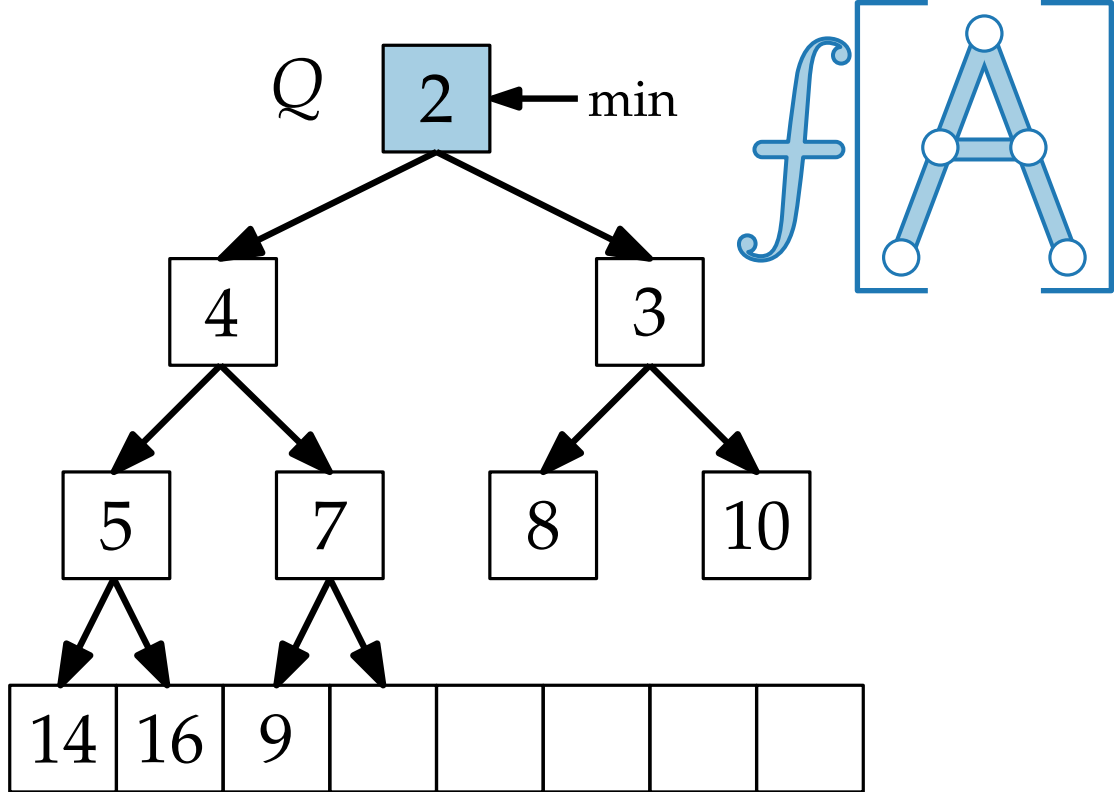
Operation	Implementierung	Laufzeit
$Q.\text{insert}(x)$	Füge x hinten ein, „rotiere“ x nach oben	$\Theta(\log n)$
$Q.\text{min}()$	Benutze Pointer	$\Theta(1)$
$Q.\text{extractMin}()$	Lösche $\min$, schiebe letztes Element vor, rotiere es nach unten	
$Q.\text{decreaseKey}(x, p)$		
$Q.\text{union}(Q')$		

Implementierung III: MinHeap



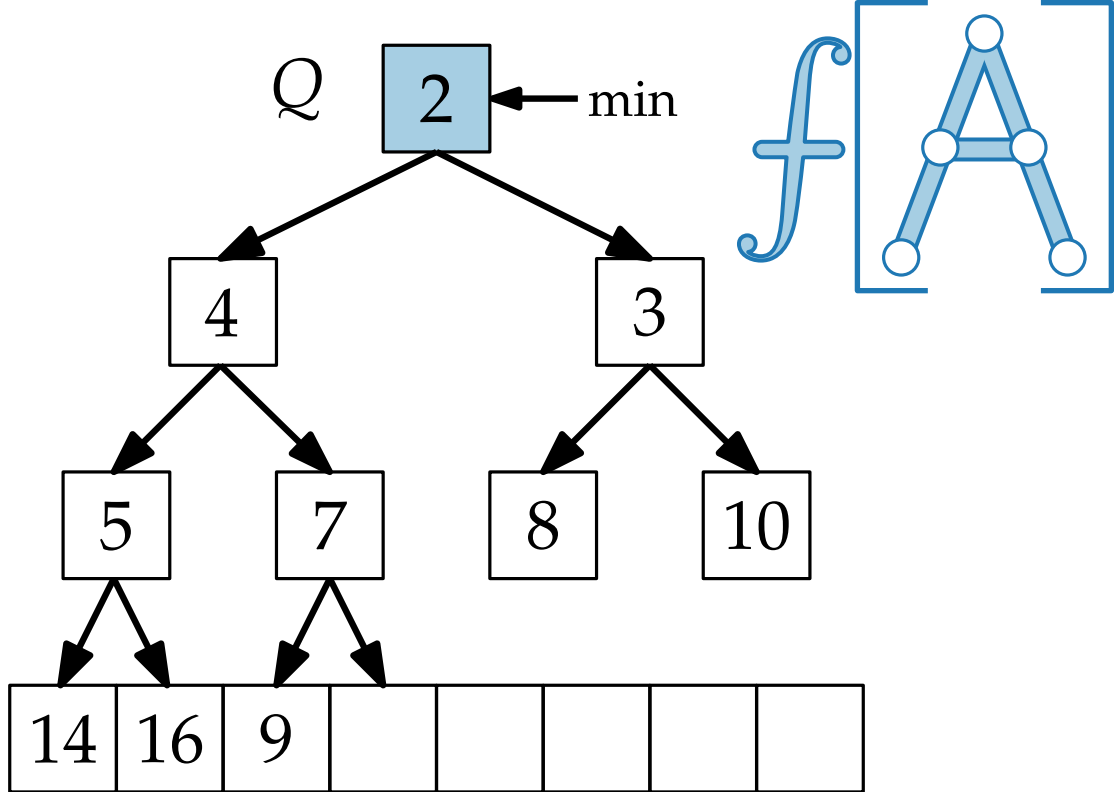
Operation	Implementierung	Laufzeit
$Q.insert(x)$	Füge x hinten ein, „rotiere“ x nach oben	$\Theta(\log n)$
$Q.min()$	Benutze Pointer	$\Theta(1)$
$Q.extractMin()$	Lösche min, schiebe letztes Element vor, rotiere es nach unten	
$Q.decreaseKey(x, p)$		
$Q.union(Q')$		

Implementierung III: MinHeap



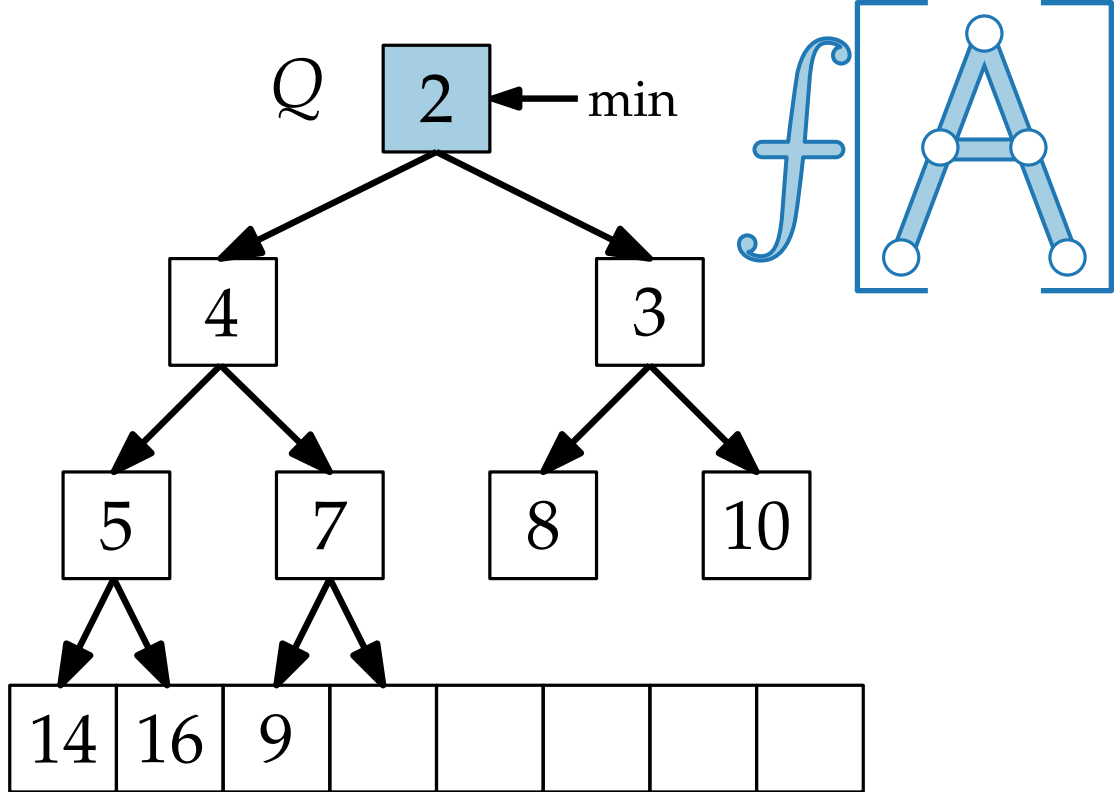
Operation	Implementierung	Laufzeit
$Q.insert(x)$	Füge x hinten ein, „rotiere“ x nach oben	$\Theta(\log n)$
$Q.min()$	Benutze Pointer	$\Theta(1)$
$Q.extractMin()$	Lösche min, schiebe letztes Element vor, rotiere es nach unten	
$Q.decreaseKey(x, p)$		
$Q.union(Q')$		

Implementierung III: MinHeap



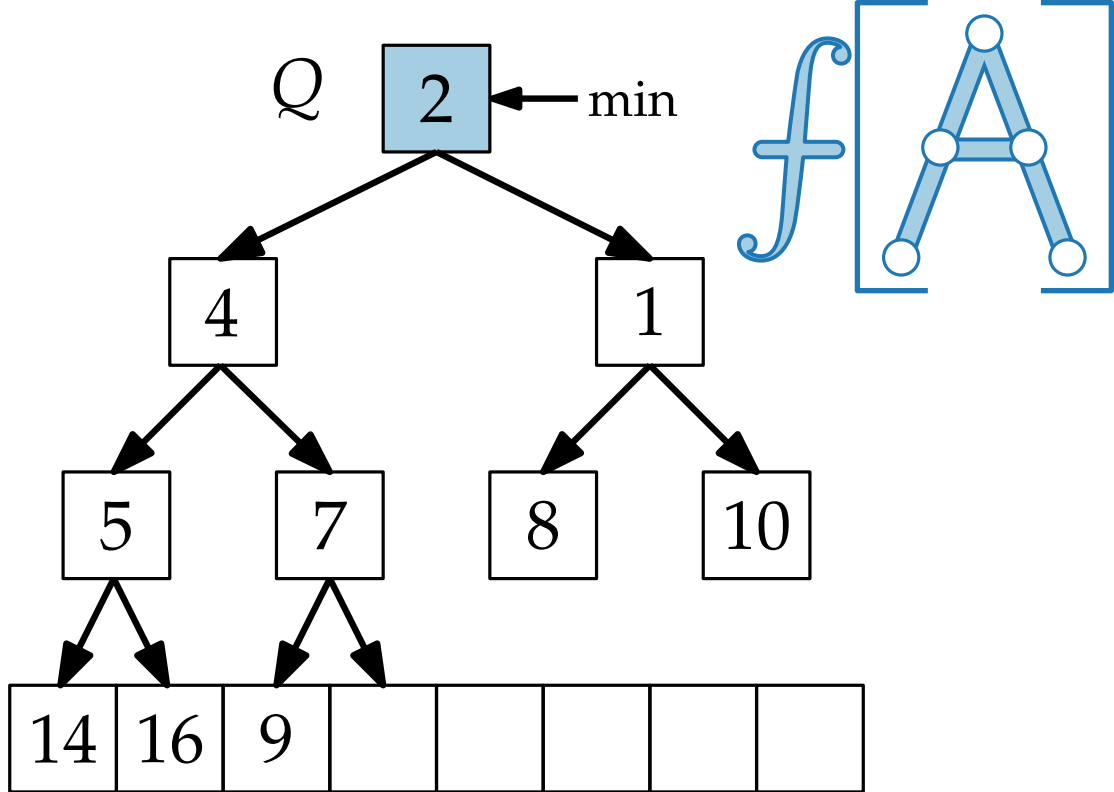
Operation	Implementierung	Laufzeit
$Q.insert(x)$	Füge x hinten ein, „rotiere“ x nach oben	$\Theta(\log n)$
$Q.min()$	Benutze Pointer	$\Theta(1)$
$Q.extractMin()$	Lösche min, schiebe letztes Element vor, rotiere es nach unten	$\Theta(\log n)$
$Q.decreaseKey(x, p)$		
$Q.union(Q')$		

Implementierung III: MinHeap



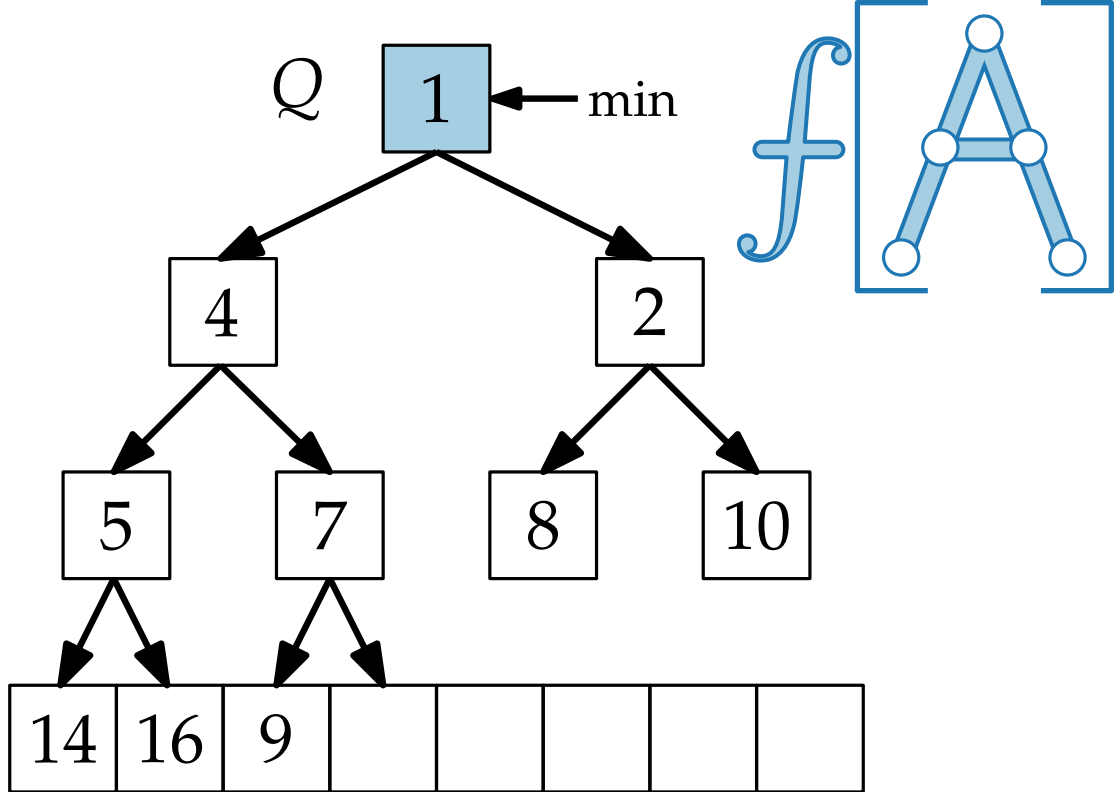
Operation	Implementierung	Laufzeit
$Q.\text{insert}(x)$	Füge x hinten ein, „rotiere“ x nach oben	$\Theta(\log n)$
$Q.\text{min}()$	Benutze Pointer	$\Theta(1)$
$Q.\text{extractMin}()$	Lösche $\min$, schiebe letztes Element vor, rotiere es nach unten	$\Theta(\log n)$
$Q.\text{decreaseKey}(x, p)$	Update x , rotiere x nach oben	
$Q.\text{union}(Q')$		

Implementierung III: MinHeap



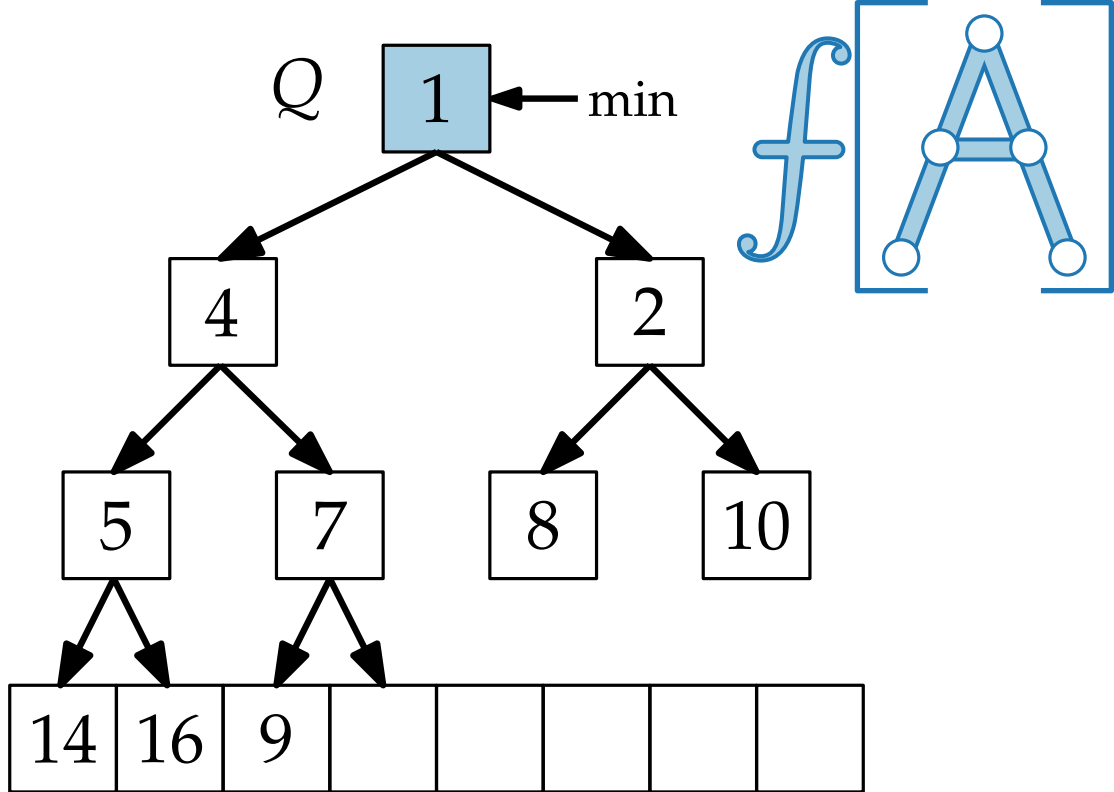
Operation	Implementierung	Laufzeit
$Q.insert(x)$	Füge x hinten ein, „rotiere“ x nach oben	$\Theta(\log n)$
$Q.min()$	Benutze Pointer	$\Theta(1)$
$Q.extractMin()$	Lösche min, schiebe letztes Element vor, rotiere es nach unten	$\Theta(\log n)$
$Q.decreaseKey(x, p)$	Update x , rotiere x nach oben	
$Q.union(Q')$		

Implementierung III: MinHeap



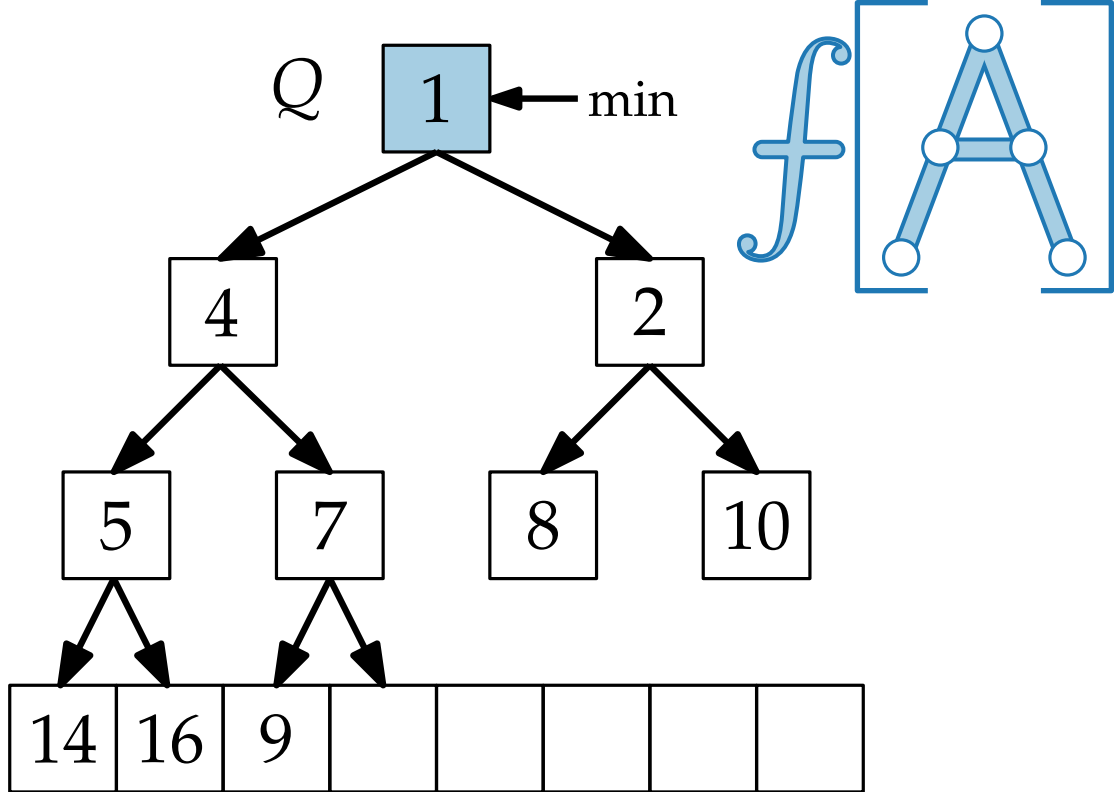
Operation	Implementierung	Laufzeit
$Q.insert(x)$	Füge x hinten ein, „rotiere“ x nach oben	$\Theta(\log n)$
$Q.min()$	Benutze Pointer	$\Theta(1)$
$Q.extractMin()$	Lösche min, schiebe letztes Element vor, rotiere es nach unten	$\Theta(\log n)$
$Q.decreaseKey(x, p)$	Update x , rotiere x nach oben	
$Q.union(Q')$		

Implementierung III: MinHeap



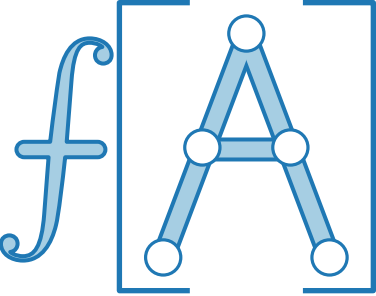
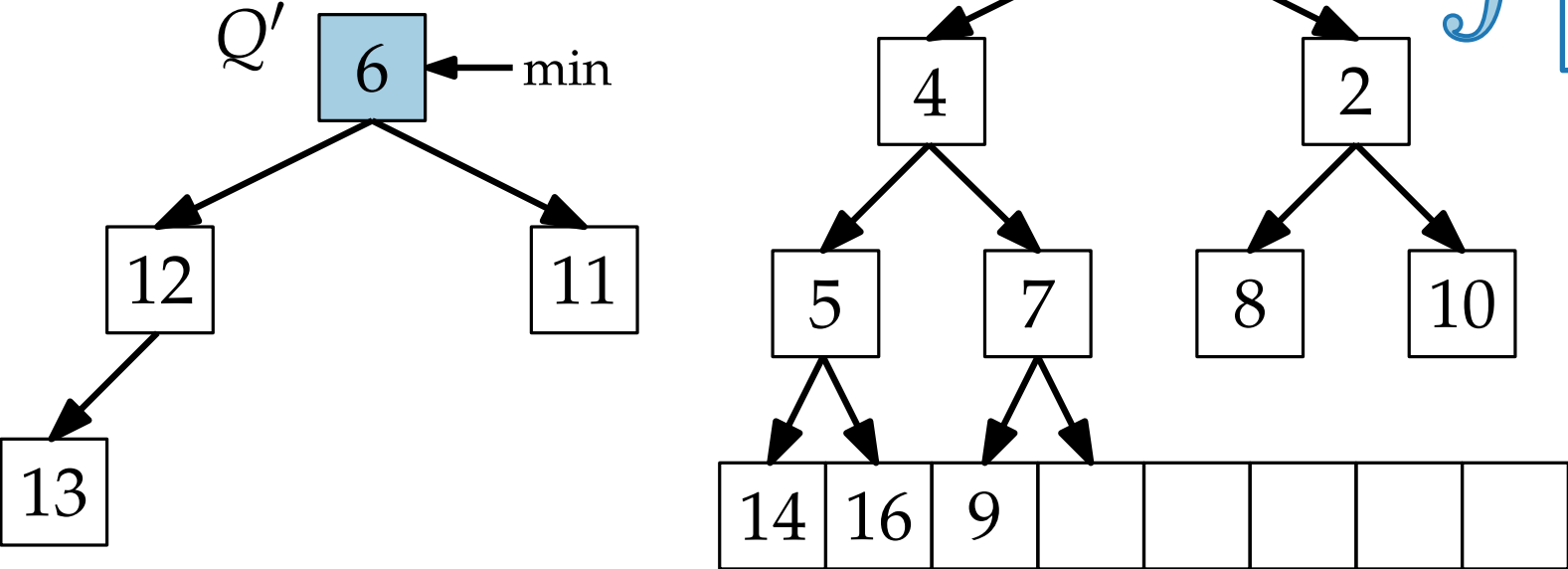
Operation	Implementierung	Laufzeit
$Q.insert(x)$	Füge x hinten ein, „rotiere“ x nach oben	$\Theta(\log n)$
$Q.min()$	Benutze Pointer	$\Theta(1)$
$Q.extractMin()$	Lösche min, schiebe letztes Element vor, rotiere es nach unten	$\Theta(\log n)$
$Q.decreaseKey(x, p)$	Update x , rotiere x nach oben	$\Theta(\log n)$
$Q.union(Q')$		

Implementierung III: MinHeap



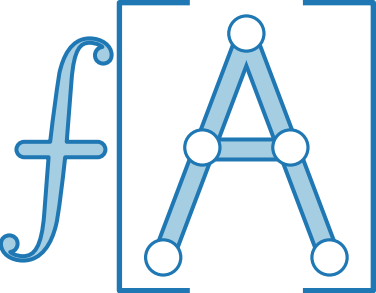
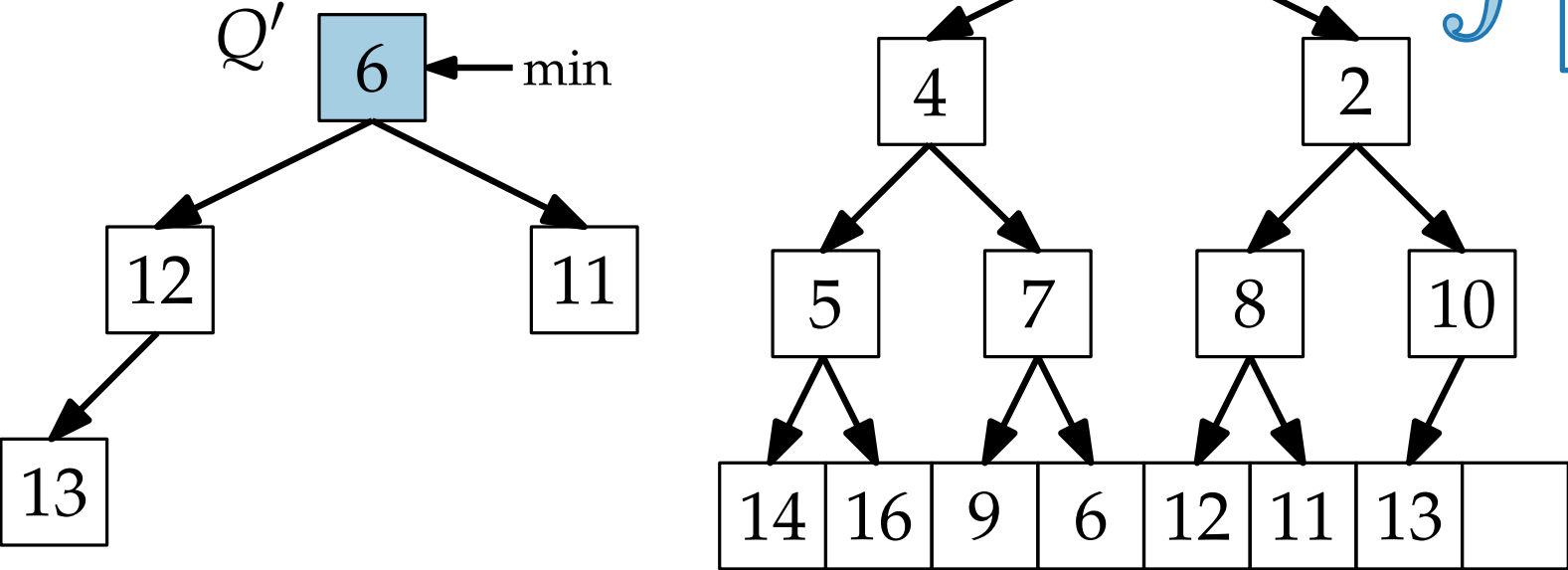
Operation	Implementierung	Laufzeit
$Q.insert(x)$	Füge x hinten ein, „rotiere“ x nach oben	$\Theta(\log n)$
$Q.min()$	Benutze Pointer	$\Theta(1)$
$Q.extractMin()$	Lösche min, schiebe letztes Element vor, rotiere es nach unten	$\Theta(\log n)$
$Q.decreaseKey(x, p)$	Update x , rotiere x nach oben	$\Theta(\log n)$
$Q.union(Q')$	Hänge Q' hinten an, rotiere alle Elemente nach oben	

Implementierung III: MinHeap



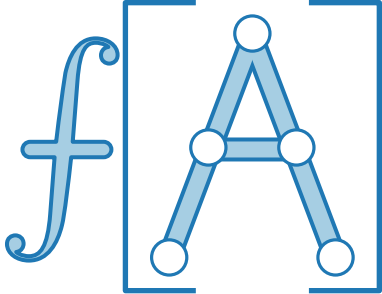
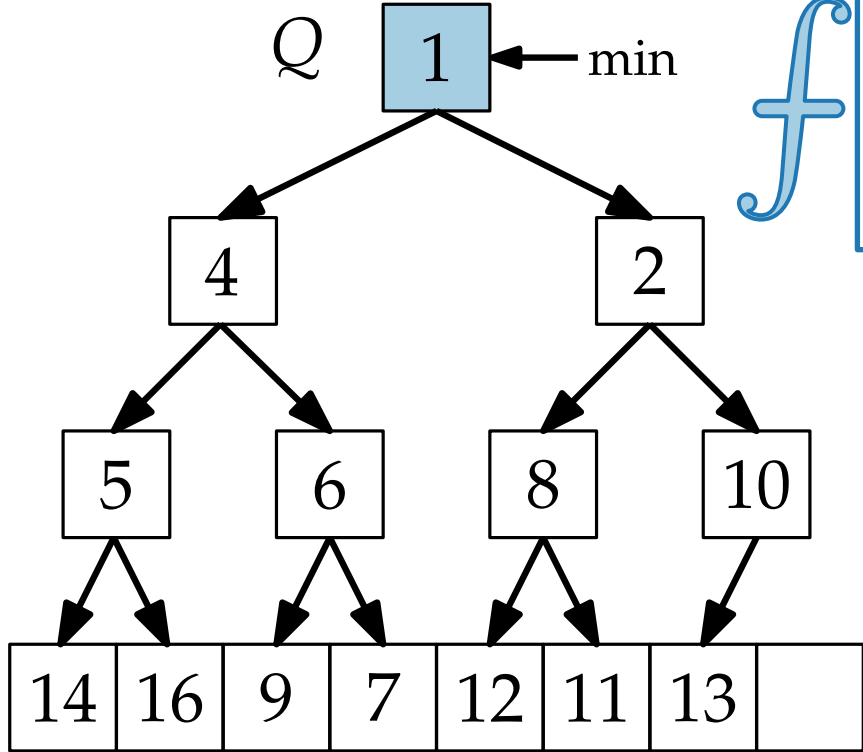
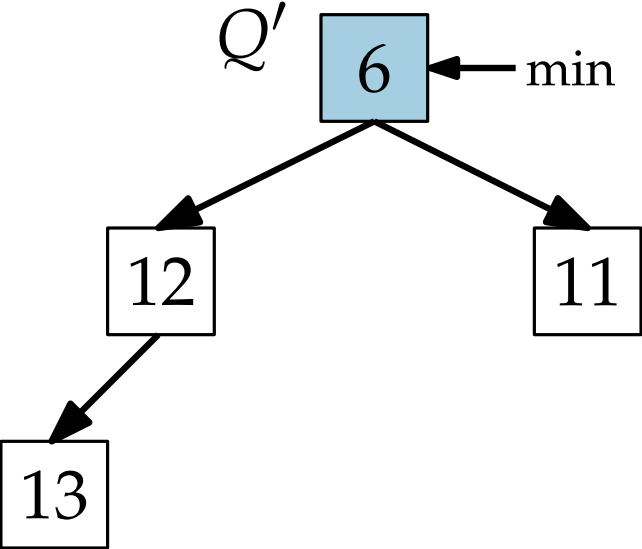
Operation	Implementierung	Laufzeit
$Q.insert(x)$	Füge x hinten ein, „rotiere“ x nach oben	$\Theta(\log n)$
$Q.min()$	Benutze Pointer	$\Theta(1)$
$Q.extractMin()$	Lösche $\min$, schiebe letztes Element vor, rotiere es nach unten	$\Theta(\log n)$
$Q.decreaseKey(x, p)$	Update x , rotiere x nach oben	$\Theta(\log n)$
$Q.union(Q')$	Hänge Q' hinten an, rotiere alle Elemente nach oben	

Implementierung III: MinHeap



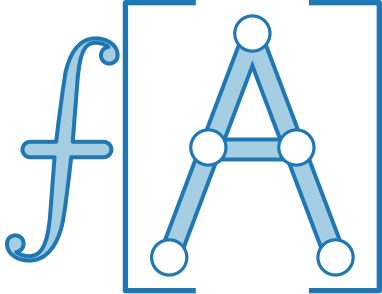
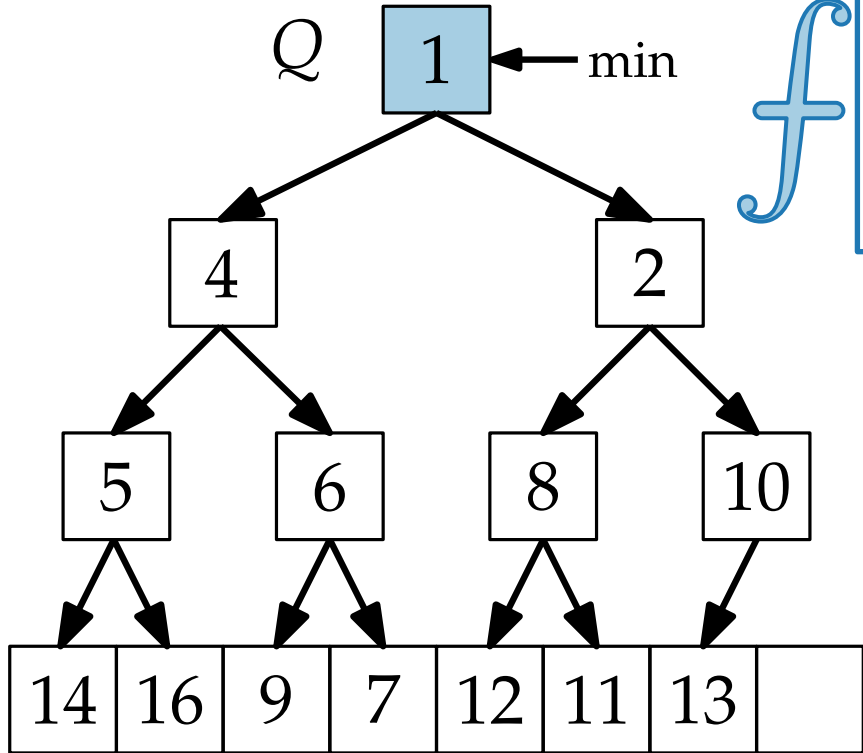
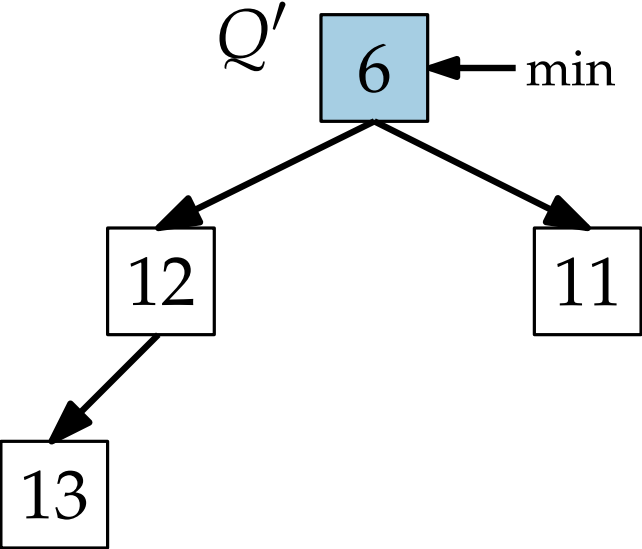
Operation	Implementierung	Laufzeit
$Q.\text{insert}(x)$	Füge x hinten ein, „rotiere“ x nach oben	$\Theta(\log n)$
$Q.\text{min}()$	Benutze Pointer	$\Theta(1)$
$Q.\text{extractMin}()$	Lösche $\min$, schiebe letztes Element vor, rotiere es nach unten	$\Theta(\log n)$
$Q.\text{decreaseKey}(x, p)$	Update x , rotiere x nach oben	$\Theta(\log n)$
$Q.\text{union}(Q')$	Hänge Q' hinten an, rotiere alle Elemente nach oben	

Implementierung III: MinHeap



Operation	Implementierung	Laufzeit
$Q.insert(x)$	Füge x hinten ein, „rotiere“ x nach oben	$\Theta(\log n)$
$Q.min()$	Benutze Pointer	$\Theta(1)$
$Q.extractMin()$	Lösche min, schiebe letztes Element vor, rotiere es nach unten	$\Theta(\log n)$
$Q.decreaseKey(x, p)$	Update x , rotiere x nach oben	$\Theta(\log n)$
$Q.union(Q')$	Hänge Q' hinten an, rotiere alle Elemente nach oben	

Implementierung III: MinHeap



Operation	Implementierung	Laufzeit
$Q.insert(x)$	Füge x hinten ein, „rotiere“ x nach oben	$\Theta(\log n)$
$Q.min()$	Benutze Pointer	$\Theta(1)$
$Q.extractMin()$	Lösche min, schiebe letztes Element vor, rotiere es nach unten	$\Theta(\log n)$
$Q.decreaseKey(x, p)$	Update x , rotiere x nach oben	$\Theta(\log n)$
$Q.union(Q')$	Hänge Q' hinten an, rotiere alle Elemente nach oben	$\Theta(Q' \log n)$

Überblick



Operation	ARRAY	LISTE	HEAP
$Q.\text{insert}(x)$	$\Theta(1)$	$\Theta(1)$	$\Theta(\log n)$
$Q.\text{min}()$	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$
$Q.\text{extractMin}()$	$\Theta(n)$	$\Theta(n)$	$\Theta(\log n)$
$Q.\text{decreaseKey}(x, p)$	$\Theta(1)$	$\Theta(1)$	$\Theta(\log n)$
$Q.\text{union}(Q')$	$\Theta(Q')$	$\Theta(1)$	$\Theta(Q' \log n)$

Überblick



Operation	ARRAY	LISTE	HEAP
$Q.\text{insert}(x)$	$\Theta(1)$	$\Theta(1)$	$\Theta(\log n)$
$Q.\text{min}()$	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$
$Q.\text{extractMin}()$	$\Theta(n)$	$\Theta(n)$	$\Theta(\log n)$
$Q.\text{decreaseKey}(x, p)$	$\Theta(1)$	$\Theta(1)$	$\Theta(\log n)$
$Q.\text{union}(Q')$	$\Theta(Q')$	$\Theta(1)$	$\Theta(Q' \log n)$

Überblick



Satz. Gegeben ein Graph $G = (V, E)$, läuft Dijkstras Alg. in $\Theta(n \cdot T_{\text{EXTRACTMIN}}(n) + m \cdot T_{\text{DECREASEKEY}}(n))$ Zeit.

Operation	ARRAY	LISTE	HEAP
$Q.\text{insert}(x)$	$\Theta(1)$	$\Theta(1)$	$\Theta(\log n)$
$Q.\text{min}()$	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$
$Q.\text{extractMin}()$	$\Theta(n)$	$\Theta(n)$	$\Theta(\log n)$
$Q.\text{decreaseKey}(x, p)$	$\Theta(1)$	$\Theta(1)$	$\Theta(\log n)$
$Q.\text{union}(Q')$	$\Theta(Q')$	$\Theta(1)$	$\Theta(Q' \log n)$
DIJKSTRA			

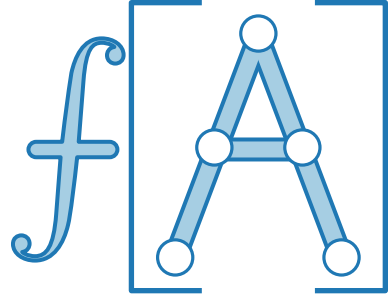
Überblick



Satz. Gegeben ein Graph $G = (V, E)$, läuft Dijkstras Alg. in $\Theta(n \cdot T_{\text{EXTRACTMIN}}(n) + m \cdot T_{\text{DECREASEKEY}}(n))$ Zeit.

Operation	ARRAY	LISTE	HEAP
$Q.\text{insert}(x)$	$\Theta(1)$	$\Theta(1)$	$\Theta(\log n)$
$Q.\text{min}()$	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$
$Q.\text{extractMin}()$	$\Theta(n)$	$\Theta(n)$	$\Theta(\log n)$
$Q.\text{decreaseKey}(x, p)$	$\Theta(1)$	$\Theta(1)$	$\Theta(\log n)$
$Q.\text{union}(Q')$	$\Theta(Q')$	$\Theta(1)$	$\Theta(Q' \log n)$
DIJKSTRA	$\Theta(n^2 + m)$		

Überblick



Satz. Gegeben ein Graph $G = (V, E)$, läuft Dijkstras Alg. in $\Theta(n \cdot T_{\text{EXTRACTMIN}}(n) + m \cdot T_{\text{DECREASEKEY}}(n))$ Zeit.

Operation	ARRAY	LISTE	HEAP
$Q.\text{insert}(x)$	$\Theta(1)$	$\Theta(1)$	$\Theta(\log n)$
$Q.\text{min}()$	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$
$Q.\text{extractMin}()$	$\Theta(n)$	$\Theta(n)$	$\Theta(\log n)$
$Q.\text{decreaseKey}(x, p)$	$\Theta(1)$	$\Theta(1)$	$\Theta(\log n)$
$Q.\text{union}(Q')$	$\Theta(Q')$	$\Theta(1)$	$\Theta(Q' \log n)$
DIJKSTRA	$\Theta(n^2 + m)$	$\Theta(n^2 + m)$	

Überblick



Satz. Gegeben ein Graph $G = (V, E)$, läuft Dijkstras Alg. in $\Theta(n \cdot T_{\text{EXTRACTMIN}}(n) + m \cdot T_{\text{DECREASEKEY}}(n))$ Zeit.

Operation	ARRAY	LISTE	HEAP
$Q.\text{insert}(x)$	$\Theta(1)$	$\Theta(1)$	$\Theta(\log n)$
$Q.\text{min}()$	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$
$Q.\text{extractMin}()$	$\Theta(n)$	$\Theta(n)$	$\Theta(\log n)$
$Q.\text{decreaseKey}(x, p)$	$\Theta(1)$	$\Theta(1)$	$\Theta(\log n)$
$Q.\text{union}(Q')$	$\Theta(Q')$	$\Theta(1)$	$\Theta(Q' \log n)$
DIJKSTRA	$\Theta(n^2 + m)$	$\Theta(n^2 + m)$	$\Theta((n + m) \log n)$

Überblick



Satz. Gegeben ein Graph $G = (V, E)$, läuft Dijkstras Alg. in $\Theta(n \cdot T_{\text{EXTRACTMIN}}(n) + m \cdot T_{\text{DECREASEKEY}}(n))$ Zeit.

Operation	ARRAY	LISTE	HEAP	FIBONACCIHEAP
$Q.\text{insert}(x)$	$\Theta(1)$	$\Theta(1)$	$\Theta(\log n)$	$\Theta(1)$
$Q.\text{min}()$	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$
$Q.\text{extractMin}()$	$\Theta(n)$	$\Theta(n)$	$\Theta(\log n)$	$\Theta(\log n)$
$Q.\text{decreaseKey}(x, p)$	$\Theta(1)$	$\Theta(1)$	$\Theta(\log n)$	$\Theta(1)$
$Q.\text{union}(Q')$	$\Theta(Q')$	$\Theta(1)$	$\Theta(Q' \log n)$	$\Theta(1)$
DIJKSTRA	$\Theta(n^2 + m)$	$\Theta(n^2 + m)$	$\Theta((n + m) \log n)$	

Überblick



Satz. Gegeben ein Graph $G = (V, E)$, läuft Dijkstras Alg. in $\Theta(n \cdot T_{\text{EXTRACTMIN}}(n) + m \cdot T_{\text{DECREASEKEY}}(n))$ Zeit.

Operation	ARRAY	LISTE	HEAP	FIBONACCIHEAP
$Q.\text{insert}(x)$	$\Theta(1)$	$\Theta(1)$	$\Theta(\log n)$	$\Theta(1)$
$Q.\text{min}()$	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$
$Q.\text{extractMin}()$	$\Theta(n)$	$\Theta(n)$	$\Theta(\log n)$	$\Theta(\log n)$
$Q.\text{decreaseKey}(x, p)$	$\Theta(1)$	$\Theta(1)$	$\Theta(\log n)$	$\Theta(1)$
$Q.\text{union}(Q')$	$\Theta(Q')$	$\Theta(1)$	$\Theta(Q' \log n)$	$\Theta(1)$
DIJKSTRA	$\Theta(n^2 + m)$	$\Theta(n^2 + m)$	$\Theta((n + m) \log n)$	$\Theta(n \log n + m)$

Überblick



Satz. Gegeben ein Graph $G = (V, E)$, läuft Dijkstras Alg. in $\Theta(n \cdot T_{\text{EXTRACTMIN}}(n) + m \cdot T_{\text{DECREASEKEY}}(n))$ Zeit.

* amortisiert

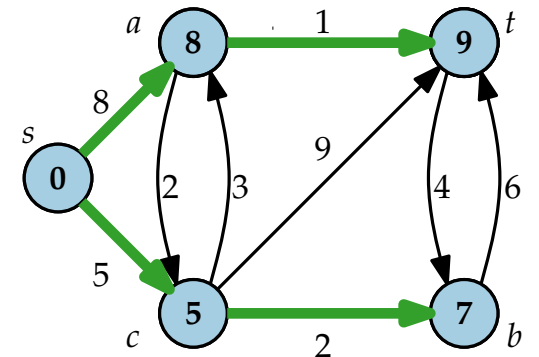
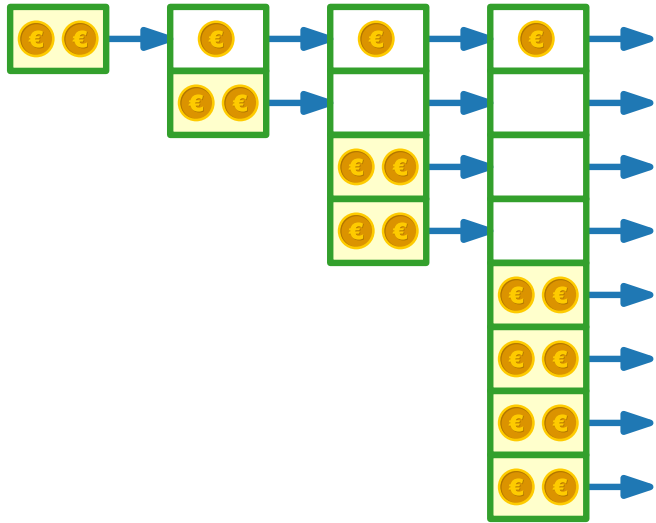
Operation	ARRAY	LISTE	HEAP	FIBONACCIHEAP
$Q.\text{insert}(x)$	$\Theta(1)$	$\Theta(1)$	$\Theta(\log n)$	$\Theta(1)$
$Q.\text{min}()$	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$
$Q.\text{extractMin}()$	$\Theta(n)$	$\Theta(n)$	$\Theta(\log n)$	$\Theta(\log n)^*$
$Q.\text{decreaseKey}(x, p)$	$\Theta(1)$	$\Theta(1)$	$\Theta(\log n)$	$\Theta(1)^*$
$Q.\text{union}(Q')$	$\Theta(Q')$	$\Theta(1)$	$\Theta(Q' \log n)$	$\Theta(1)$
DIJKSTRA	$\Theta(n^2 + m)$	$\Theta(n^2 + m)$	$\Theta((n + m) \log n)$	$\Theta(n \log n + m)$



Fortgeschrittene Algorithmen

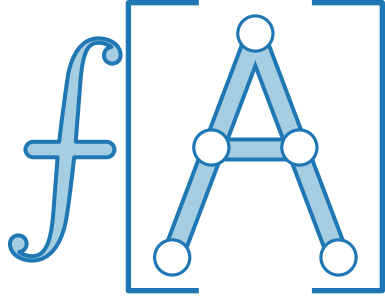
17. Vorlesung Datenstrukturen III: Amortisierte Analyse

Teil III: Dynamische Tabellen



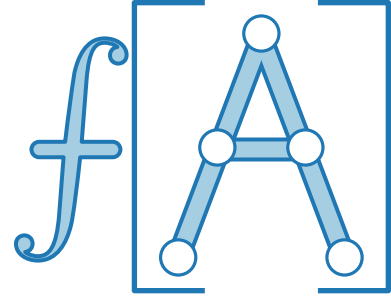
Dynamische Tabellen

Idee.



Dynamische Tabellen

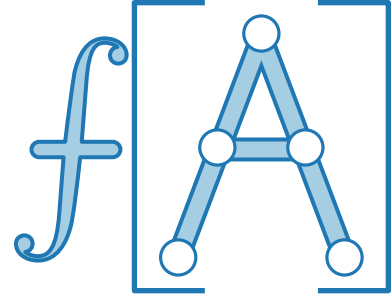
Idee.



Dynamische Tabellen

Idee.

INSERT(1)

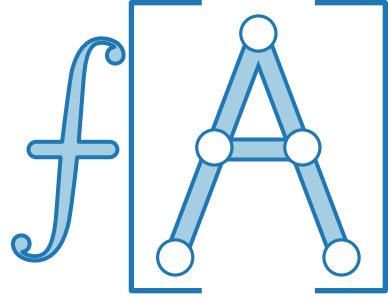


Dynamische Tabellen

Idee.

INSERT(1)

1



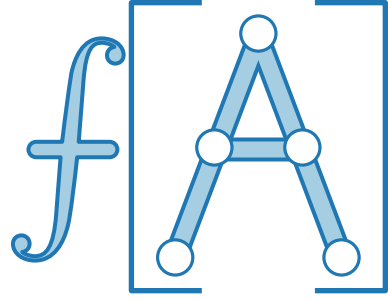
Dynamische Tabellen

Idee.

INSERT(1)

INSERT(2)

1



Dynamische Tabellen

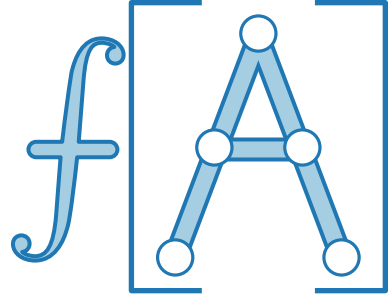
Idee.

- Wenn Tabelle voll,
fordere doppelt so große Tabelle an

INSERT(1)

INSERT(2)

1



Dynamische Tabellen

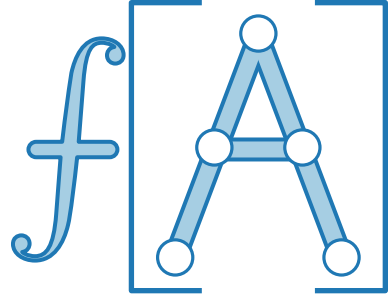
Idee.

- Wenn Tabelle voll,
fordere doppelt so große Tabelle an

INSERT(1)

1

INSERT(2)



Dynamische Tabellen

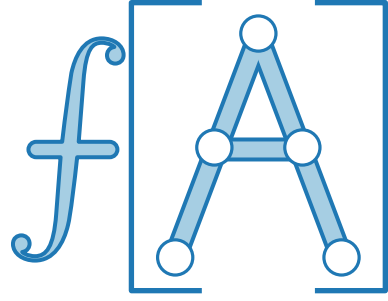
Idee.

- Wenn Tabelle voll,
fordere doppelt so große Tabelle an
- Kopiere alle Einträge
von alter in neue Tabelle.

INSERT(1)

1

INSERT(2)



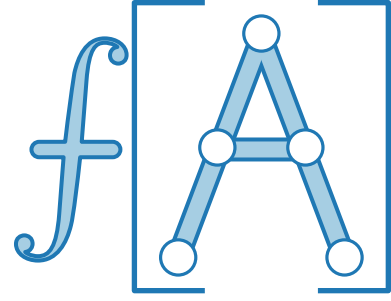
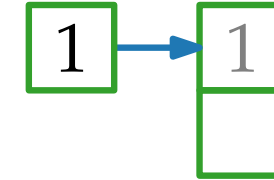
Dynamische Tabellen

Idee.

- Wenn Tabelle voll,
fordere doppelt so große Tabelle an
- Kopiere alle Einträge
von alter in neue Tabelle.

INSERT(1)

INSERT(2)



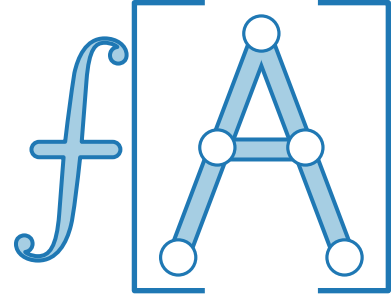
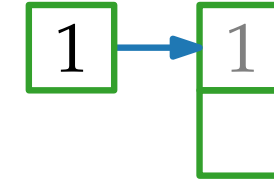
Dynamische Tabellen

Idee.

- Wenn Tabelle voll,
fordere doppelt so große Tabelle an
- Kopiere alle Einträge
von alter in neue Tabelle.
- Gib Speicher für alte Tabelle frei.

INSERT(1)

INSERT(2)



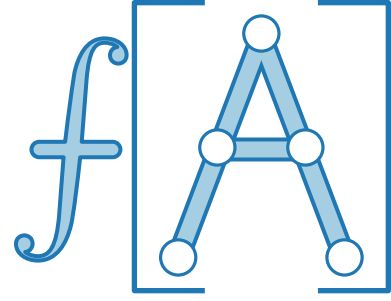
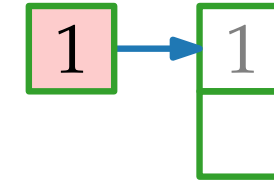
Dynamische Tabellen

Idee.

- Wenn Tabelle voll,
fordere doppelt so große Tabelle an
- Kopiere alle Einträge
von alter in neue Tabelle.
- Gib Speicher für alte Tabelle frei.

INSERT(1)

INSERT(2)



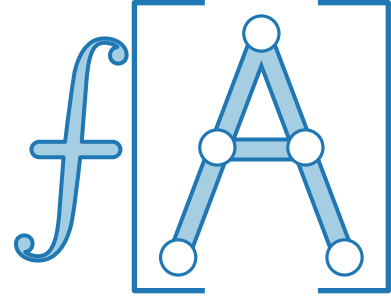
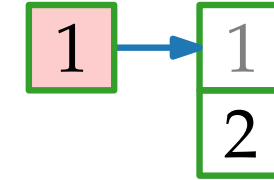
Dynamische Tabellen

Idee.

- Wenn Tabelle voll,
fordere doppelt so große Tabelle an
- Kopiere alle Einträge
von alter in neue Tabelle.
- Gib Speicher für alte Tabelle frei.

INSERT(1)

INSERT(2)



Dynamische Tabellen

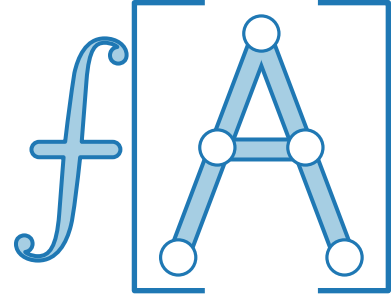
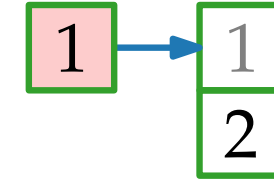
Idee.

- Wenn Tabelle voll,
fordere doppelt so große Tabelle an
- Kopiere alle Einträge
von alter in neue Tabelle.
- Gib Speicher für alte Tabelle frei.

INSERT(1)

INSERT(2)

INSERT(3)



Dynamische Tabellen

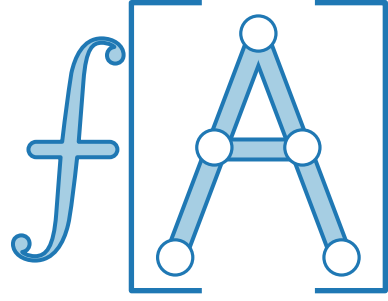
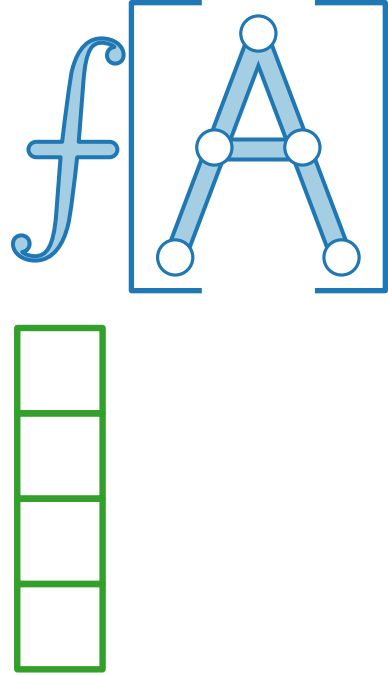
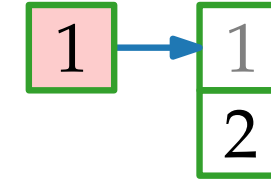
Idee.

- Wenn Tabelle voll, fordere doppelt so große Tabelle an
- Kopiere alle Einträge von alter in neue Tabelle.
- Gib Speicher für alte Tabelle frei.

INSERT(1)

INSERT(2)

INSERT(3)



Dynamische Tabellen

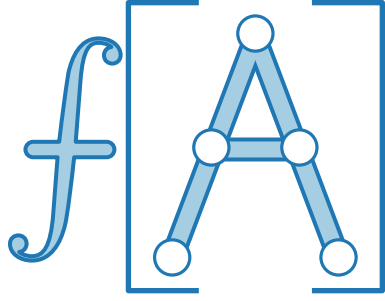
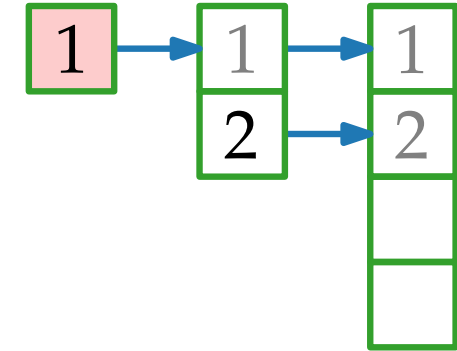
Idee.

- Wenn Tabelle voll, fordere doppelt so große Tabelle an
- Kopiere alle Einträge von alter in neue Tabelle.
- Gib Speicher für alte Tabelle frei.

INSERT(1)

INSERT(2)

INSERT(3)



Dynamische Tabellen

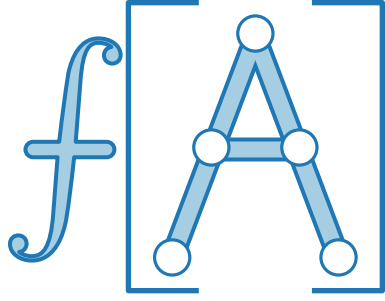
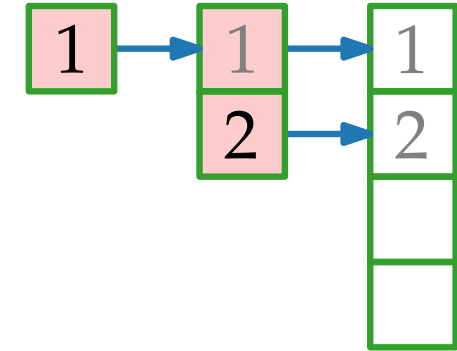
Idee.

- Wenn Tabelle voll, fordere doppelt so große Tabelle an
- Kopiere alle Einträge von alter in neue Tabelle.
- Gib Speicher für alte Tabelle frei.

INSERT(1)

INSERT(2)

INSERT(3)



Dynamische Tabellen

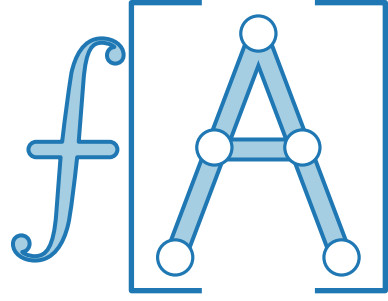
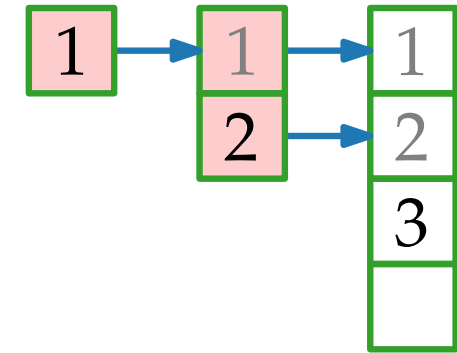
Idee.

- Wenn Tabelle voll, fordere doppelt so große Tabelle an
- Kopiere alle Einträge von alter in neue Tabelle.
- Gib Speicher für alte Tabelle frei.

INSERT(1)

INSERT(2)

INSERT(3)

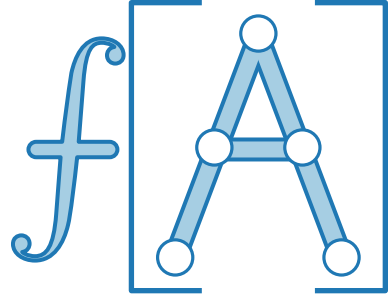
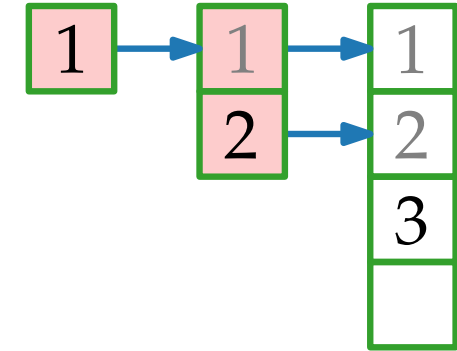


Dynamische Tabellen

Idee.

- Wenn Tabelle voll, fordere doppelt so große Tabelle an
- Kopiere alle Einträge von alter in neue Tabelle.
- Gib Speicher für alte Tabelle frei.

INSERT(1)
INSERT(2)
INSERT(3)
INSERT(4)

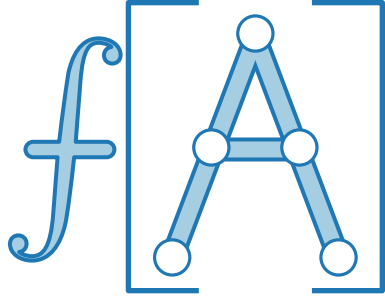
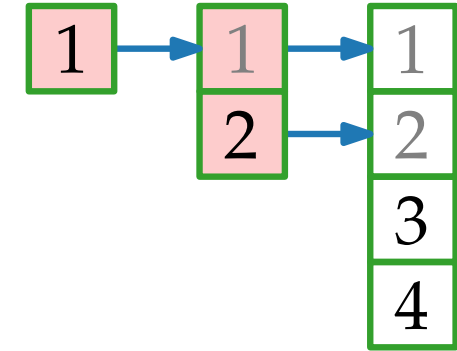


Dynamische Tabellen

Idee.

- Wenn Tabelle voll, fordere doppelt so große Tabelle an
- Kopiere alle Einträge von alter in neue Tabelle.
- Gib Speicher für alte Tabelle frei.

INSERT(1)
INSERT(2)
INSERT(3)
INSERT(4)

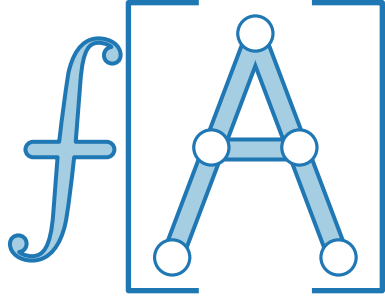
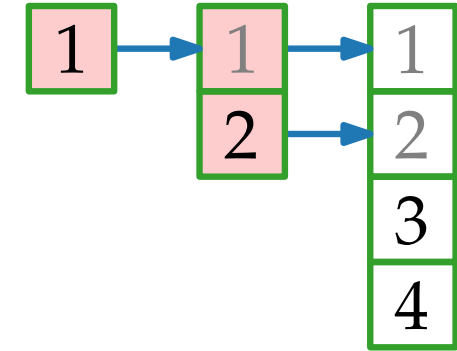


Dynamische Tabellen

Idee.

- Wenn Tabelle voll, fordere doppelt so große Tabelle an
- Kopiere alle Einträge von alter in neue Tabelle.
- Gib Speicher für alte Tabelle frei.

INSERT(1)
INSERT(2)
INSERT(3)
INSERT(4)
INSERT(5)

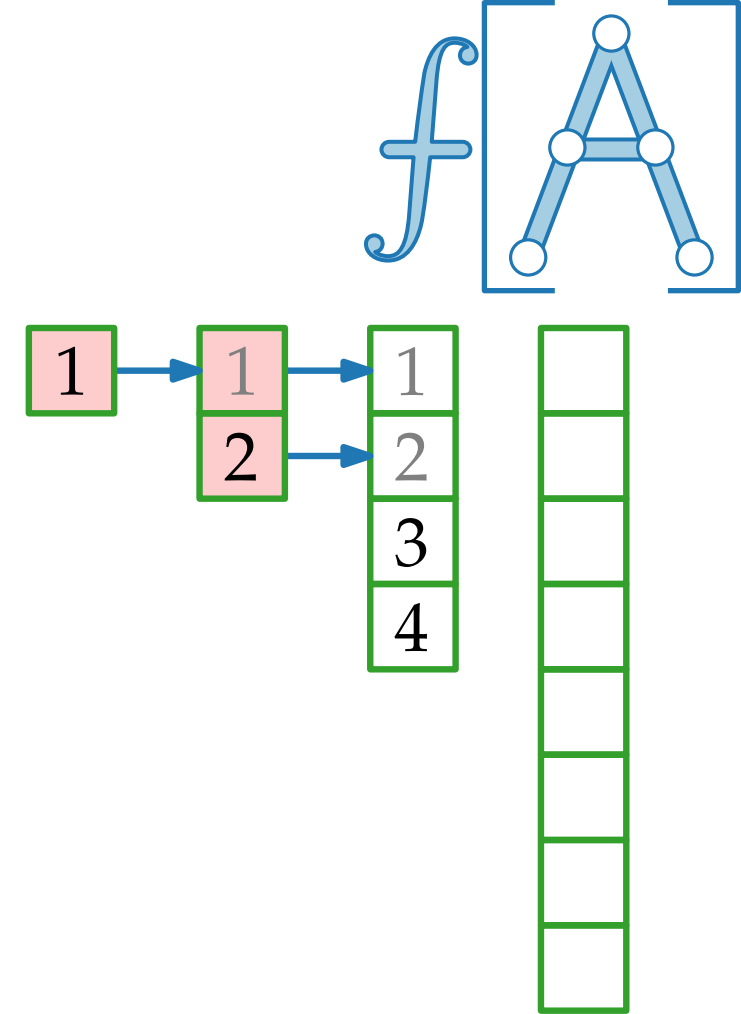


Dynamische Tabellen

Idee.

- Wenn Tabelle voll, fordere doppelt so große Tabelle an
- Kopiere alle Einträge von alter in neue Tabelle.
- Gib Speicher für alte Tabelle frei.

INSERT(1)
INSERT(2)
INSERT(3)
INSERT(4)
INSERT(5)

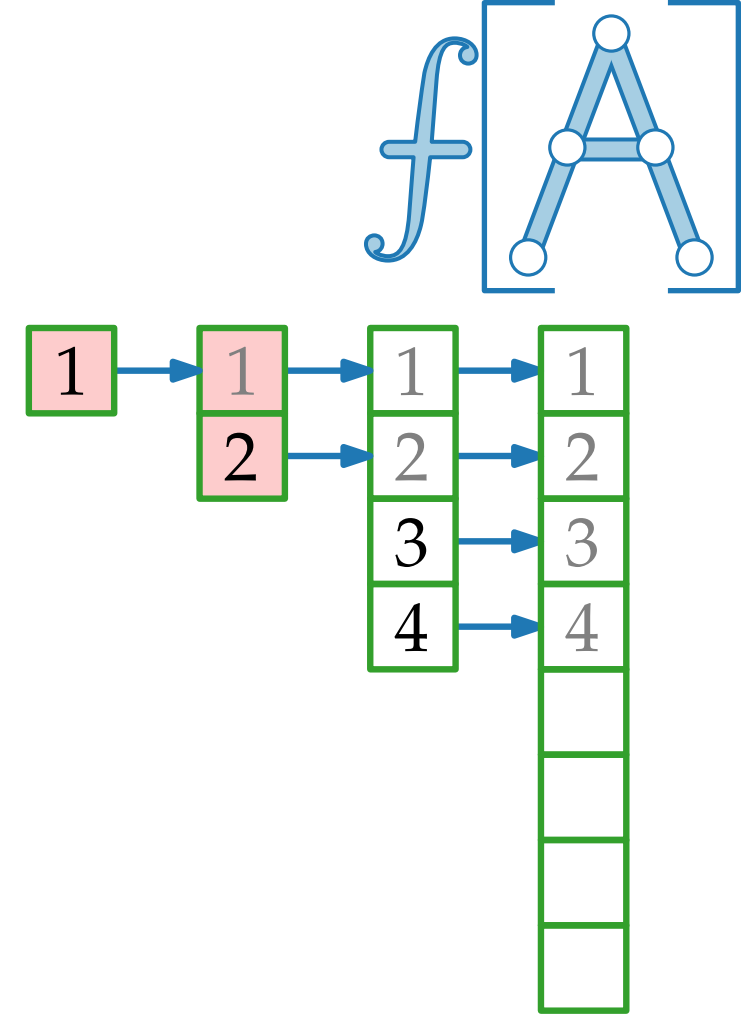


Dynamische Tabellen

Idee.

- Wenn Tabelle voll, fordere doppelt so große Tabelle an
- Kopiere alle Einträge von alter in neue Tabelle.
- Gib Speicher für alte Tabelle frei.

INSERT(1)
INSERT(2)
INSERT(3)
INSERT(4)
INSERT(5)

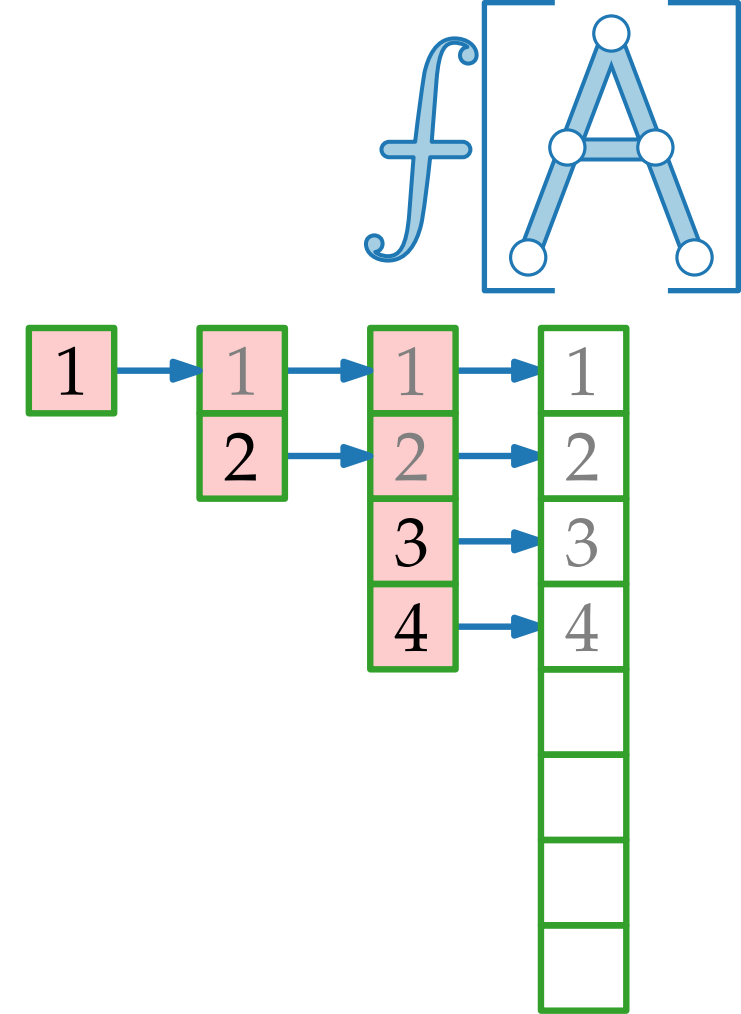


Dynamische Tabellen

Idee.

- Wenn Tabelle voll, fordere doppelt so große Tabelle an
- Kopiere alle Einträge von alter in neue Tabelle.
- Gib Speicher für alte Tabelle frei.

INSERT(1)
INSERT(2)
INSERT(3)
INSERT(4)
INSERT(5)

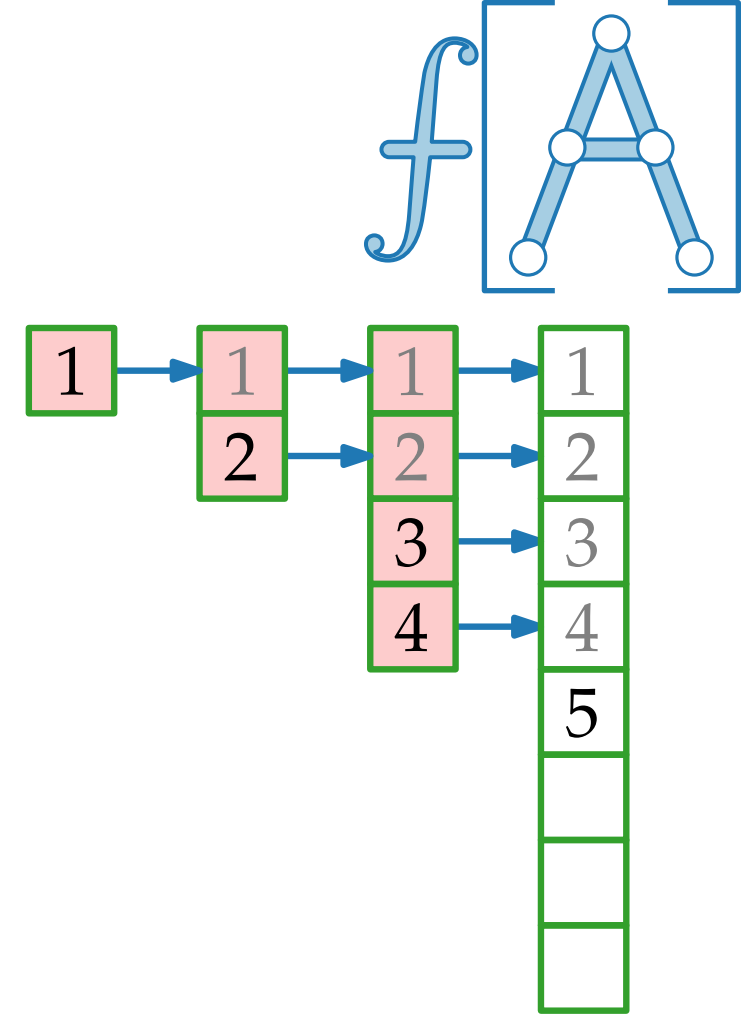


Dynamische Tabellen

Idee.

- Wenn Tabelle voll, fordere doppelt so große Tabelle an
- Kopiere alle Einträge von alter in neue Tabelle.
- Gib Speicher für alte Tabelle frei.

INSERT(1)
INSERT(2)
INSERT(3)
INSERT(4)
INSERT(5)

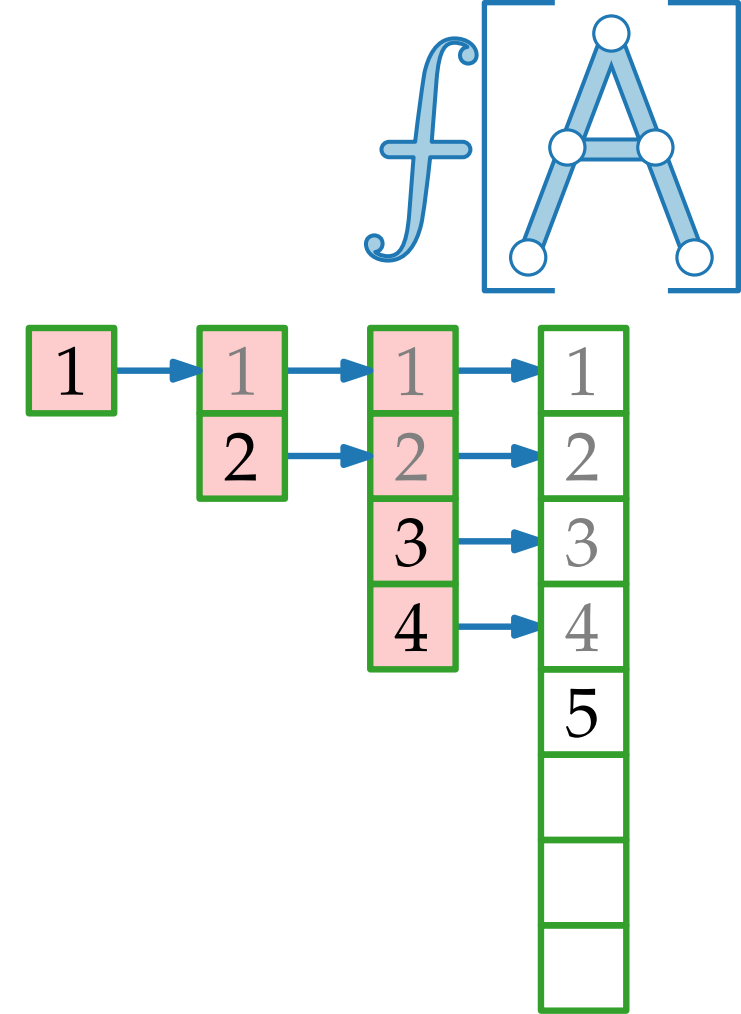


Dynamische Tabellen

Idee.

- Wenn Tabelle voll, fordere doppelt so große Tabelle an
- Kopiere alle Einträge von alter in neue Tabelle.
- Gib Speicher für alte Tabelle frei.

INSERT(1)
INSERT(2)
INSERT(3)
INSERT(4)
INSERT(5)
INSERT(6)

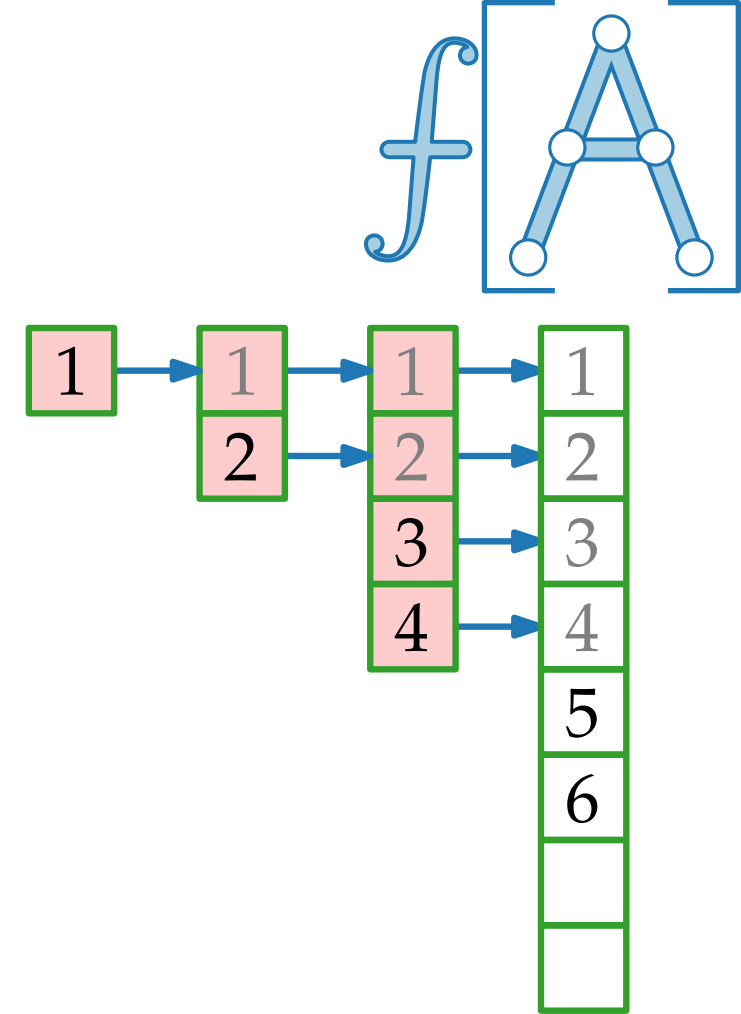


Dynamische Tabellen

Idee.

- Wenn Tabelle voll, fordere doppelt so große Tabelle an
- Kopiere alle Einträge von alter in neue Tabelle.
- Gib Speicher für alte Tabelle frei.

INSERT(1)
INSERT(2)
INSERT(3)
INSERT(4)
INSERT(5)
INSERT(6)

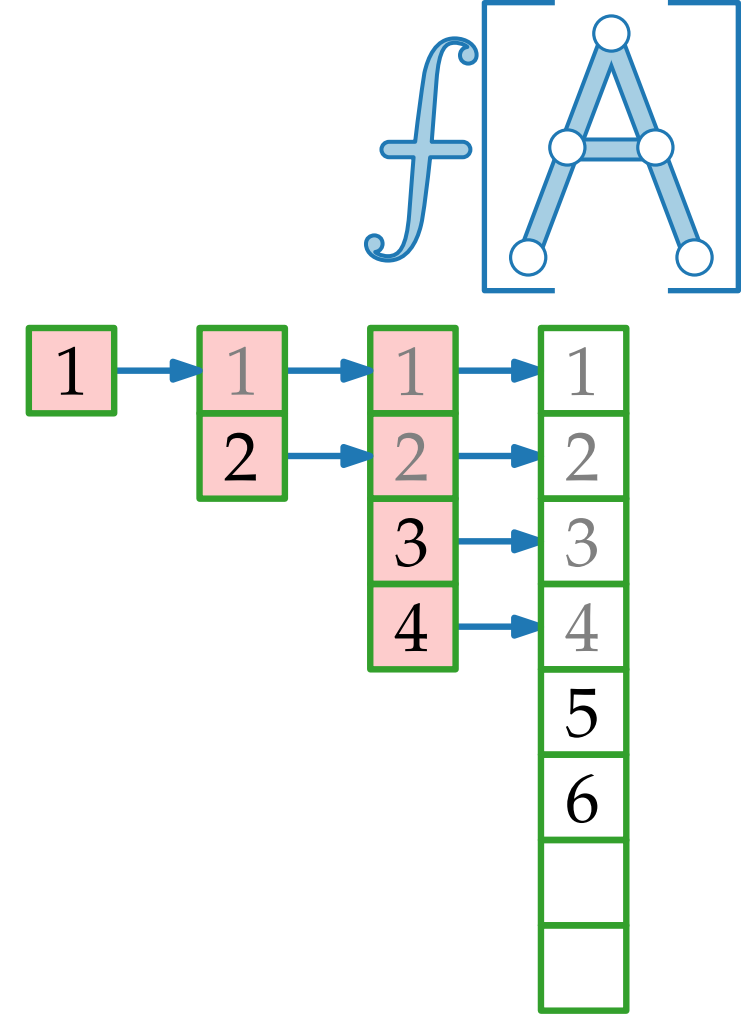


Dynamische Tabellen

Idee.

- Wenn Tabelle voll, fordere doppelt so große Tabelle an
- Kopiere alle Einträge von alter in neue Tabelle.
- Gib Speicher für alte Tabelle frei.

INSERT(1)
INSERT(2)
INSERT(3)
INSERT(4)
INSERT(5)
INSERT(6)
INSERT(7)

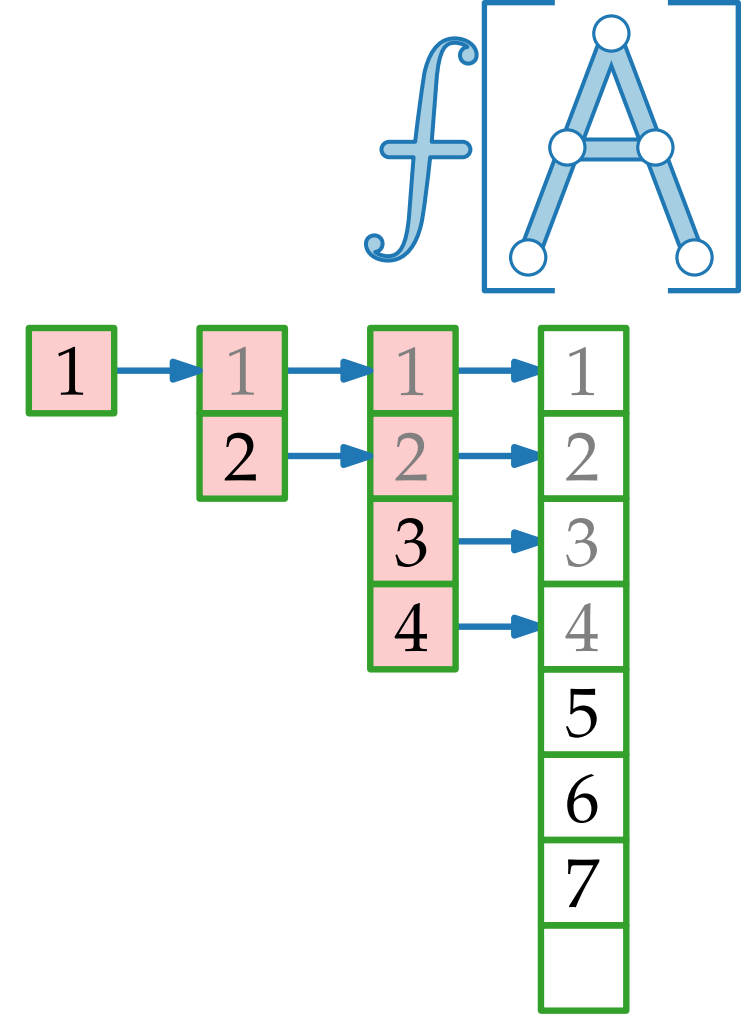


Dynamische Tabellen

Idee.

- Wenn Tabelle voll, fordere doppelt so große Tabelle an
- Kopiere alle Einträge von alter in neue Tabelle.
- Gib Speicher für alte Tabelle frei.

INSERT(1)
INSERT(2)
INSERT(3)
INSERT(4)
INSERT(5)
INSERT(6)
INSERT(7)

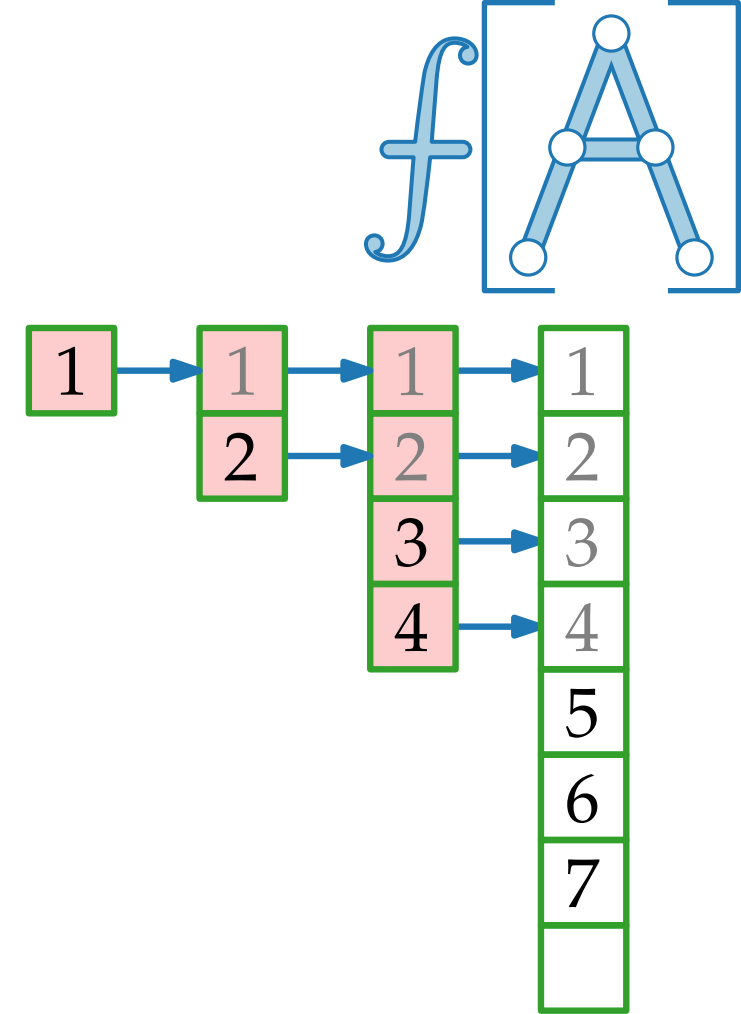


Dynamische Tabellen

Idee.

- Wenn Tabelle voll, fordere doppelt so große Tabelle an
- Kopiere alle Einträge von alter in neue Tabelle.
- Gib Speicher für alte Tabelle frei.

INSERT(1)
INSERT(2)
INSERT(3)
INSERT(4)
INSERT(5)
INSERT(6)
INSERT(7)
...



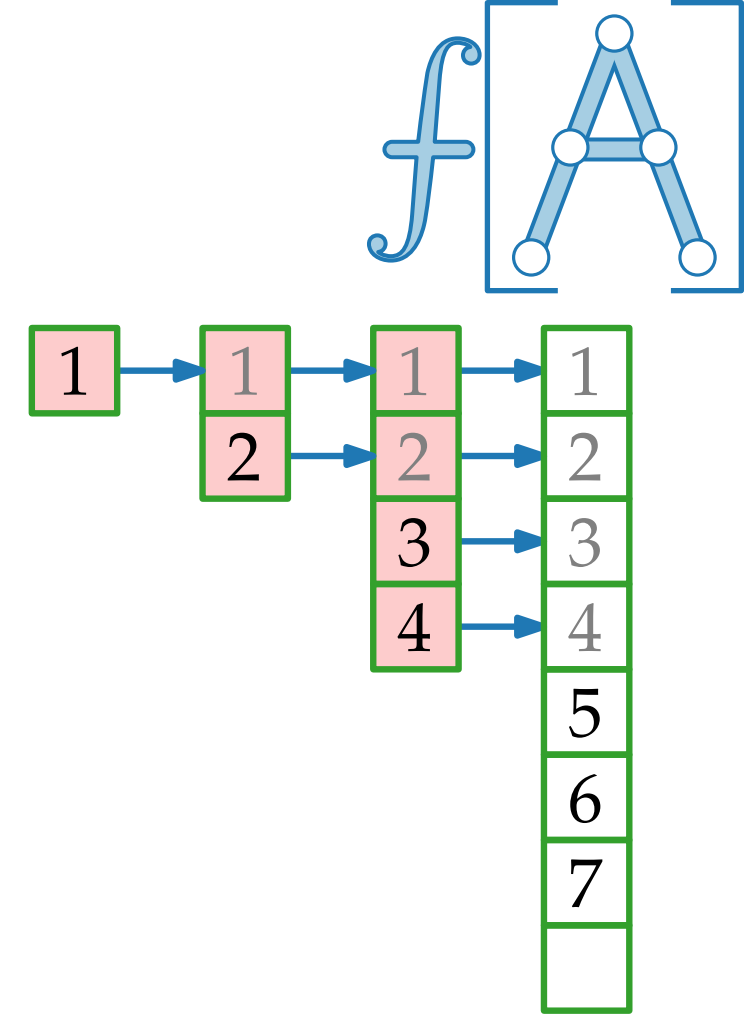
Dynamische Tabellen

Idee.

- Wenn Tabelle voll, fordere doppelt so große Tabelle an
- Kopiere alle Einträge von alter in neue Tabelle.
- Gib Speicher für alte Tabelle frei.

Analyse. Welche Laufzeit benötigen n Einfügeoperationen im schlimmsten Fall?

INSERT(1)
INSERT(2)
INSERT(3)
INSERT(4)
INSERT(5)
INSERT(6)
INSERT(7)
...

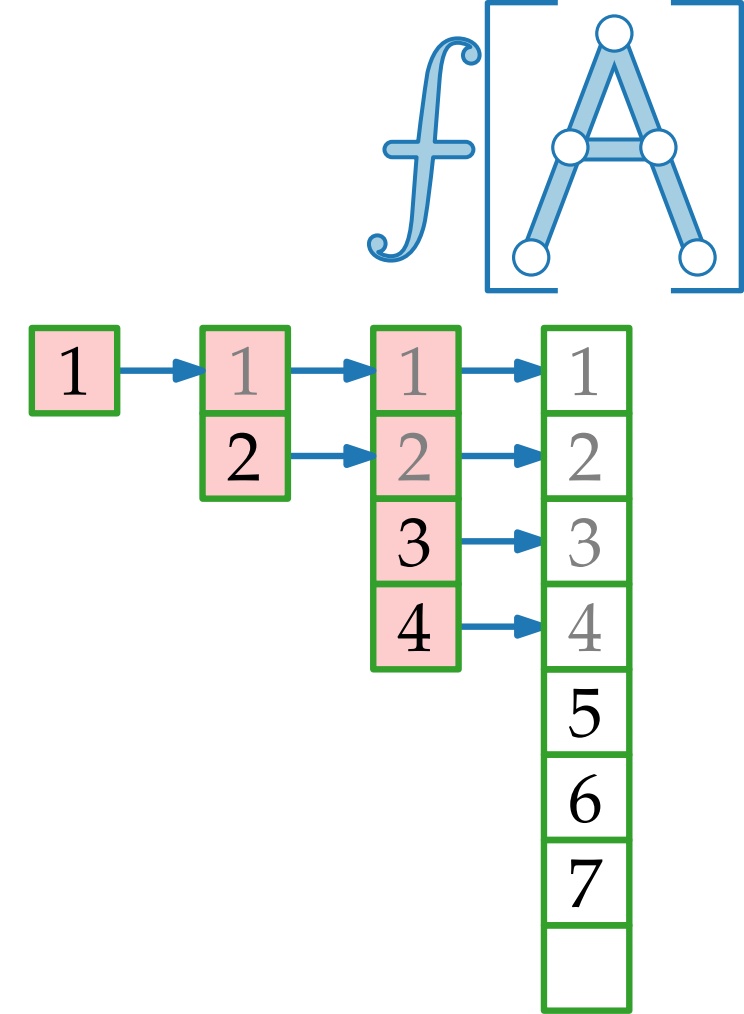


Dynamische Tabellen

Idee.

- Wenn Tabelle voll, fordere doppelt so große Tabelle an
- Kopiere alle Einträge von alter in neue Tabelle.
- Gib Speicher für alte Tabelle frei.

INSERT(1)
INSERT(2)
INSERT(3)
INSERT(4)
INSERT(5)
INSERT(6)
INSERT(7)
...



Analyse. Welche Laufzeit benötigen n Einfügeoperationen im schlimmsten Fall?

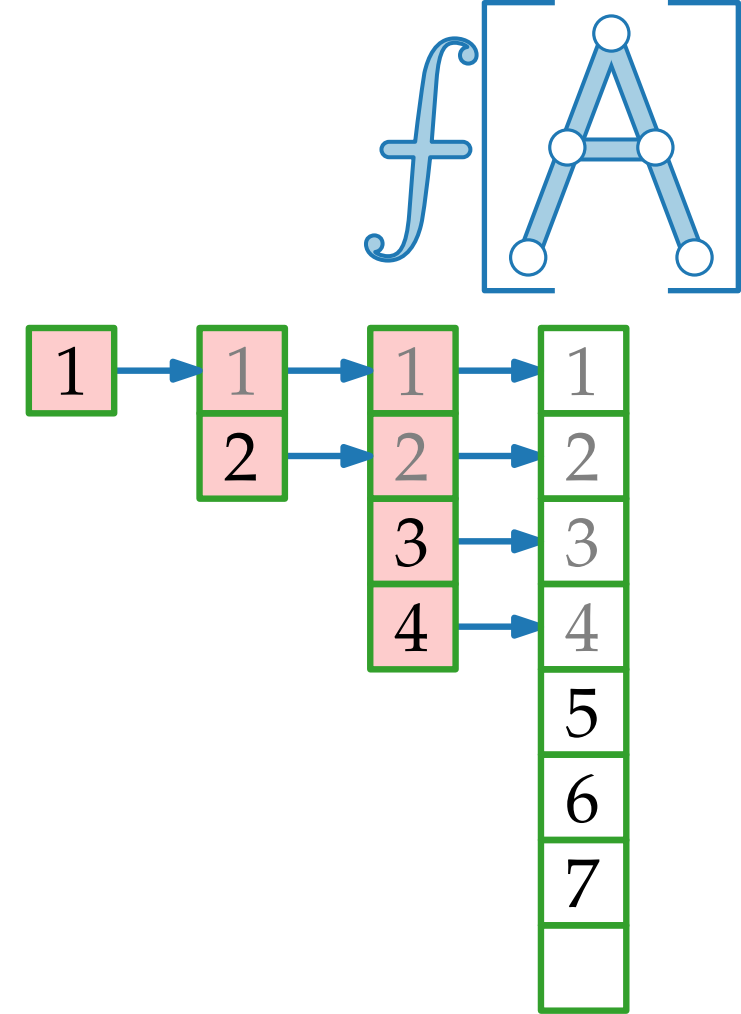
Antwort.

Dynamische Tabellen

Idee.

- Wenn Tabelle voll, fordere doppelt so große Tabelle an
- Kopiere alle Einträge von alter in neue Tabelle.
- Gib Speicher für alte Tabelle frei.

INSERT(1)
INSERT(2)
INSERT(3)
INSERT(4)
INSERT(5)
INSERT(6)
INSERT(7)
...



Analyse. Welche Laufzeit benötigen n Einfügeoperationen im schlimmsten Fall?

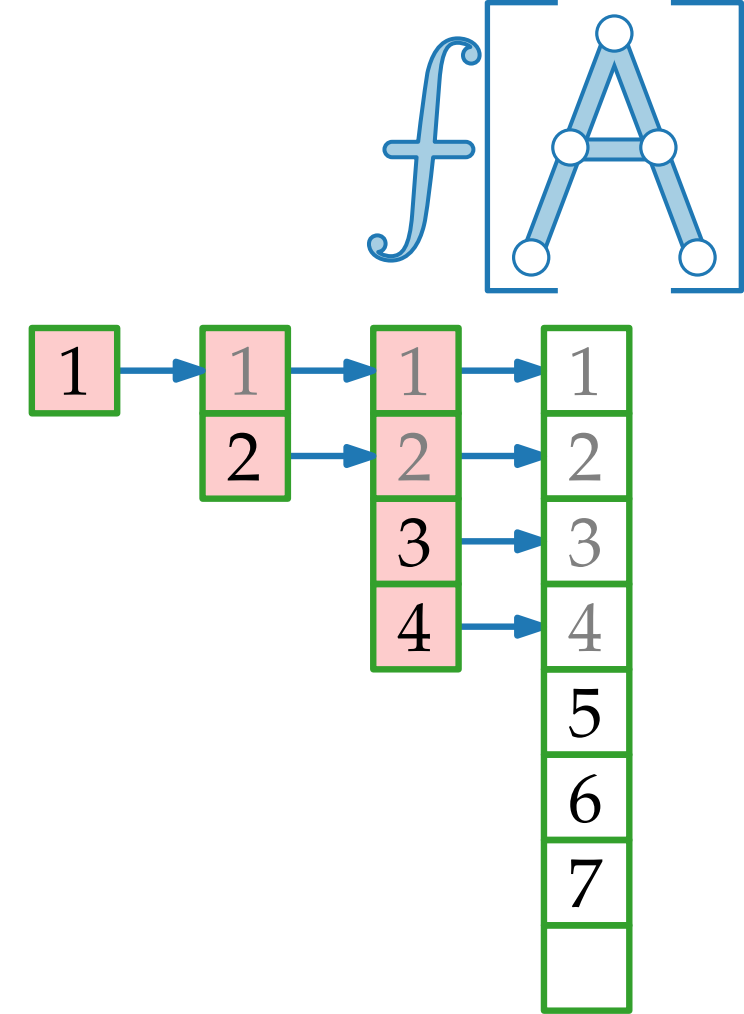
Antwort. ■ Tabelle wird genau mal kopiert.

Dynamische Tabellen

Idee.

- Wenn Tabelle voll, fordere doppelt so große Tabelle an
- Kopiere alle Einträge von alter in neue Tabelle.
- Gib Speicher für alte Tabelle frei.

INSERT(1)
INSERT(2)
INSERT(3)
INSERT(4)
INSERT(5)
INSERT(6)
INSERT(7)
...



Analyse. Welche Laufzeit benötigen n Einfügeoperationen im schlimmsten Fall?

Antwort. ■ Tabelle wird genau mal kopiert.

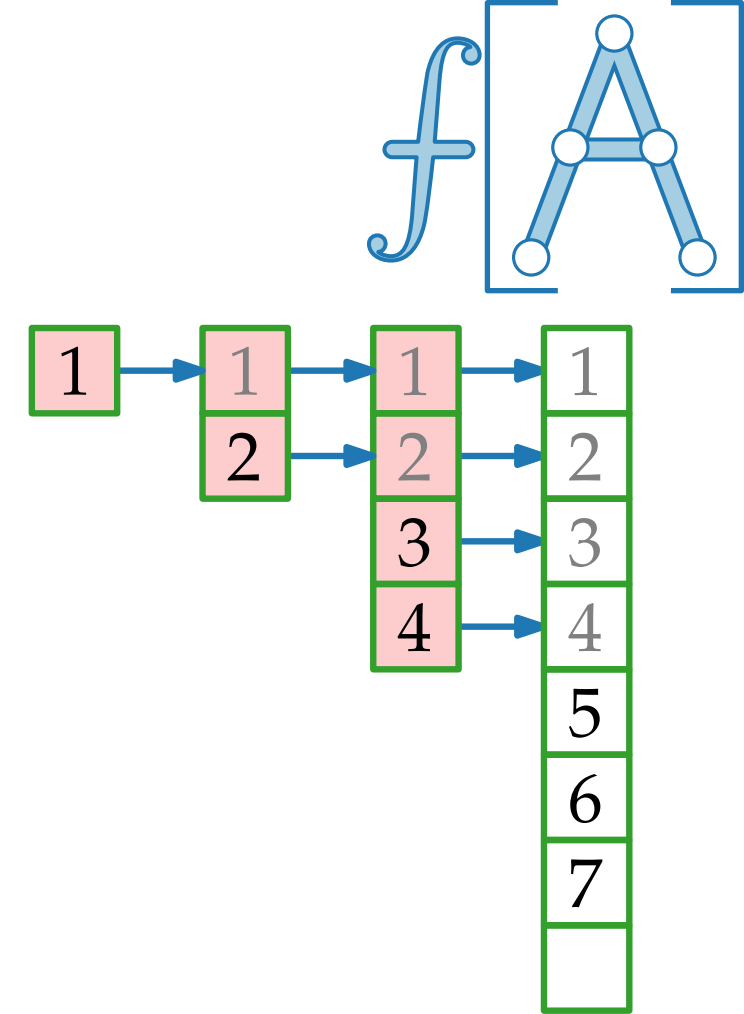
■ Im schlimmsten (letzten!) Fall ist der Aufwand

Dynamische Tabellen

Idee.

- Wenn Tabelle voll, fordere doppelt so große Tabelle an
- Kopiere alle Einträge von alter in neue Tabelle.
- Gib Speicher für alte Tabelle frei.

INSERT(1)
INSERT(2)
INSERT(3)
INSERT(4)
INSERT(5)
INSERT(6)
INSERT(7)
...



Analyse. Welche Laufzeit benötigen n Einfügeoperationen im schlimmsten Fall?

Antwort. ■ Tabelle wird genau mal kopiert.

■ Im schlimmsten (letzten!) Fall ist der Aufwand

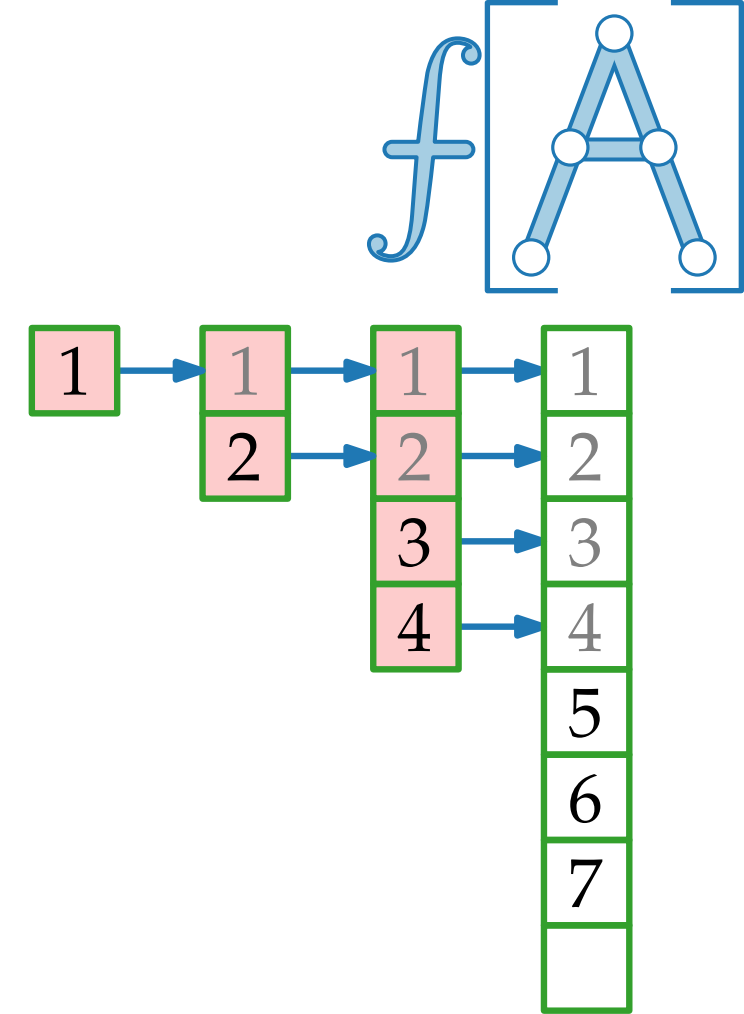
Also ist der Gesamtaufwand

Dynamische Tabellen

Idee.

- Wenn Tabelle voll, fordere doppelt so große Tabelle an
- Kopiere alle Einträge von alter in neue Tabelle.
- Gib Speicher für alte Tabelle frei.

INSERT(1)
INSERT(2)
INSERT(3)
INSERT(4)
INSERT(5)
INSERT(6)
INSERT(7)
...



Analyse. Welche Laufzeit benötigen n Einfügeoperationen im schlimmsten Fall?

Antwort. ■ Tabelle wird genau $\lceil \log_2 n \rceil$ mal kopiert.

■ Im schlimmsten (letzten!) Fall ist der Aufwand

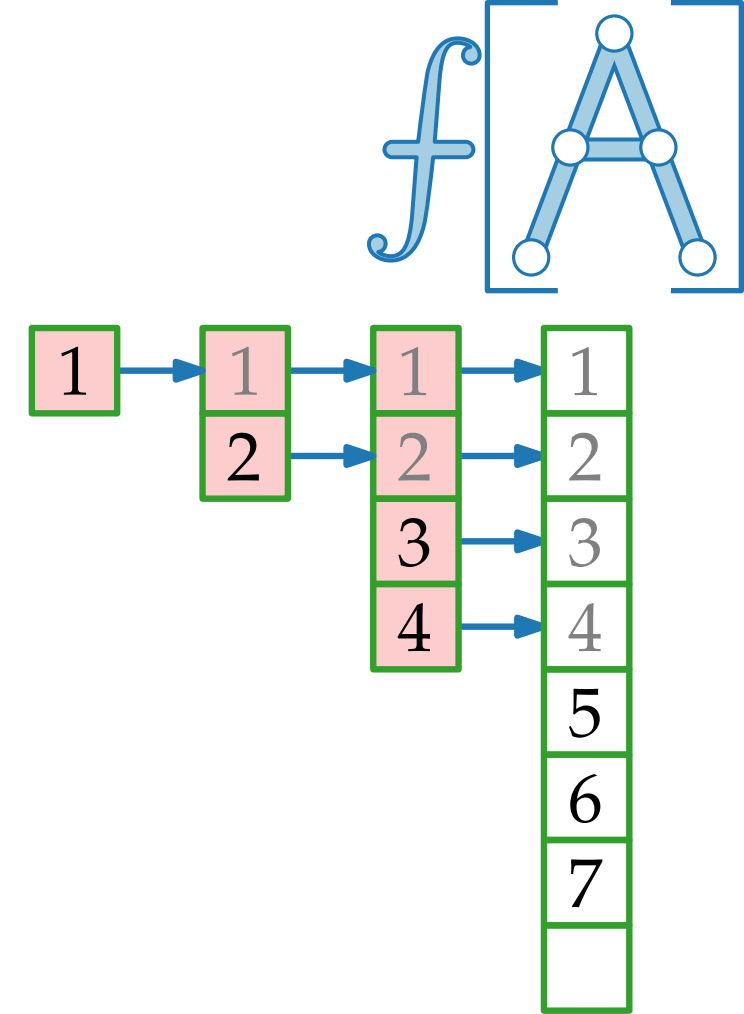
Also ist der Gesamtaufwand

Dynamische Tabellen

Idee.

- Wenn Tabelle voll, fordere doppelt so große Tabelle an
- Kopiere alle Einträge von alter in neue Tabelle.
- Gib Speicher für alte Tabelle frei.

INSERT(1)
INSERT(2)
INSERT(3)
INSERT(4)
INSERT(5)
INSERT(6)
INSERT(7)
...



Analyse. Welche Laufzeit benötigen n Einfügeoperationen im schlimmsten Fall?

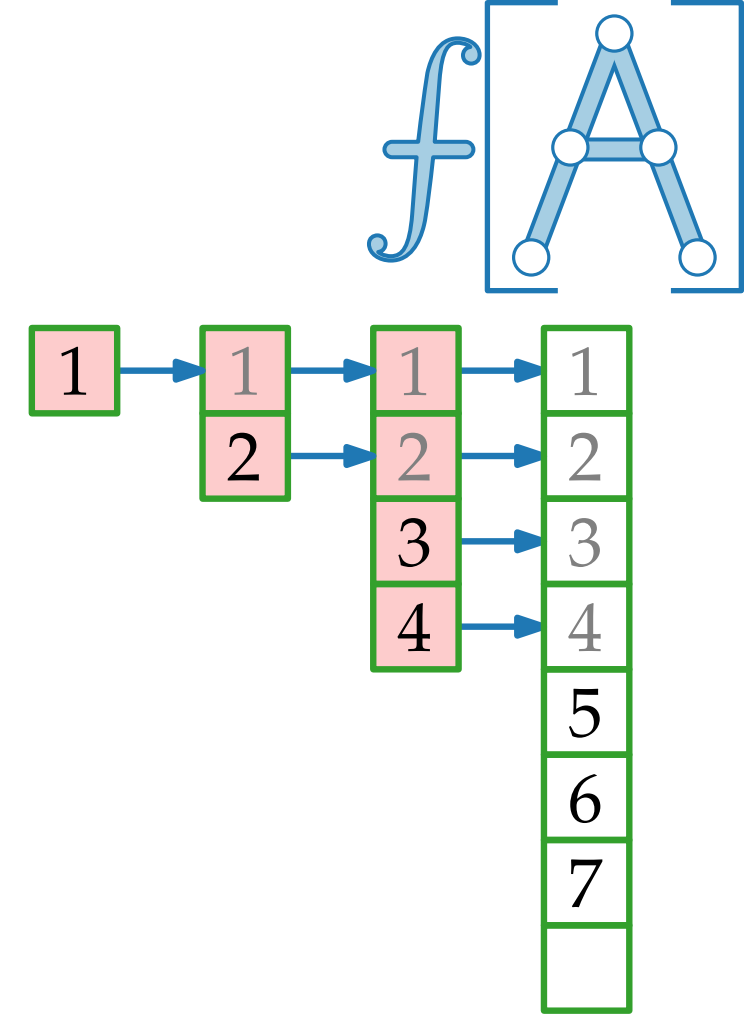
Antwort. ■ Tabelle wird genau $\lceil \log_2 n \rceil$ mal kopiert.
■ Im schlimmsten (letzten!) Fall ist der Aufwand $\Theta(n)$.
Also ist der Gesamtaufwand

Dynamische Tabellen

Idee.

- Wenn Tabelle voll, fordere doppelt so große Tabelle an
- Kopiere alle Einträge von alter in neue Tabelle.
- Gib Speicher für alte Tabelle frei.

INSERT(1)
INSERT(2)
INSERT(3)
INSERT(4)
INSERT(5)
INSERT(6)
INSERT(7)
...



Analyse. Welche Laufzeit benötigen n Einfügeoperationen im schlimmsten Fall?

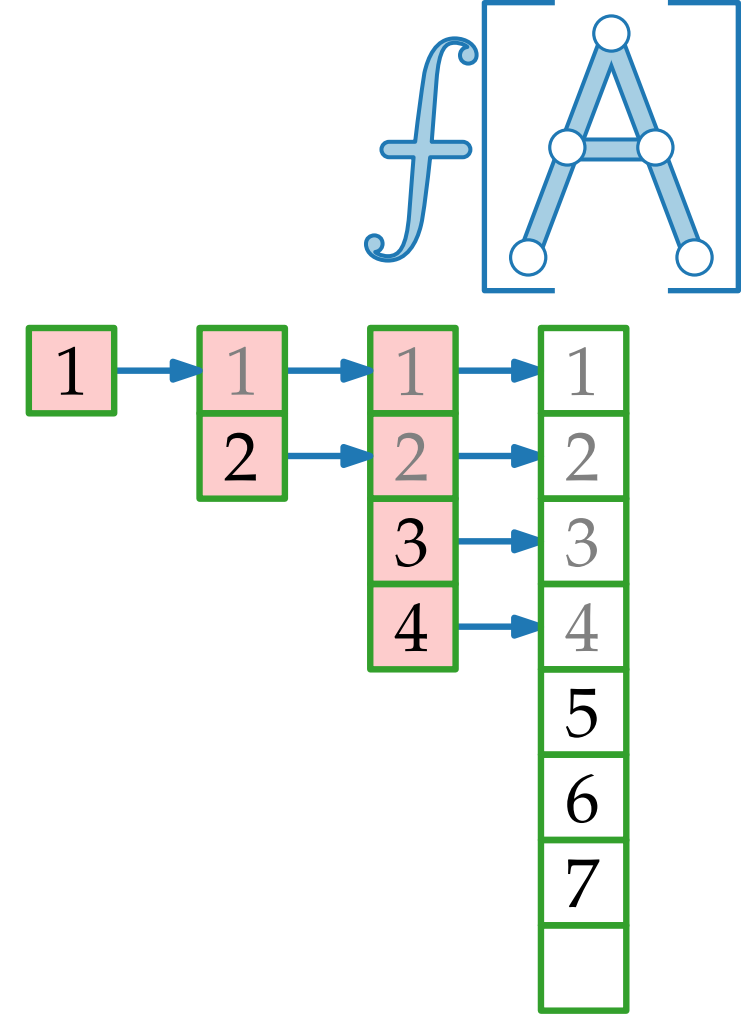
Antwort. ■ Tabelle wird genau $\lceil \log_2 n \rceil$ mal kopiert.
■ Im schlimmsten (letzten!) Fall ist der Aufwand $\Theta(n)$.
Also ist der Gesamtaufwand $\Theta(n \log n)$.

Dynamische Tabellen

Idee.

- Wenn Tabelle voll, fordere doppelt so große Tabelle an
- Kopiere alle Einträge von alter in neue Tabelle.
- Gib Speicher für alte Tabelle frei.

INSERT(1)
INSERT(2)
INSERT(3)
INSERT(4)
INSERT(5)
INSERT(6)
INSERT(7)
...



Analyse. Welche Laufzeit benötigen n Einfügeoperationen im schlimmsten Fall?

Antwort.

- Tabelle wird genau $\lceil \log_2 n \rceil$ mal kopiert.
- Im schlimmsten (letzten!) Fall ist der Aufwand $\Theta(n)$.

Also ist der Gesamtaufwand $\Theta(n \log n)$.

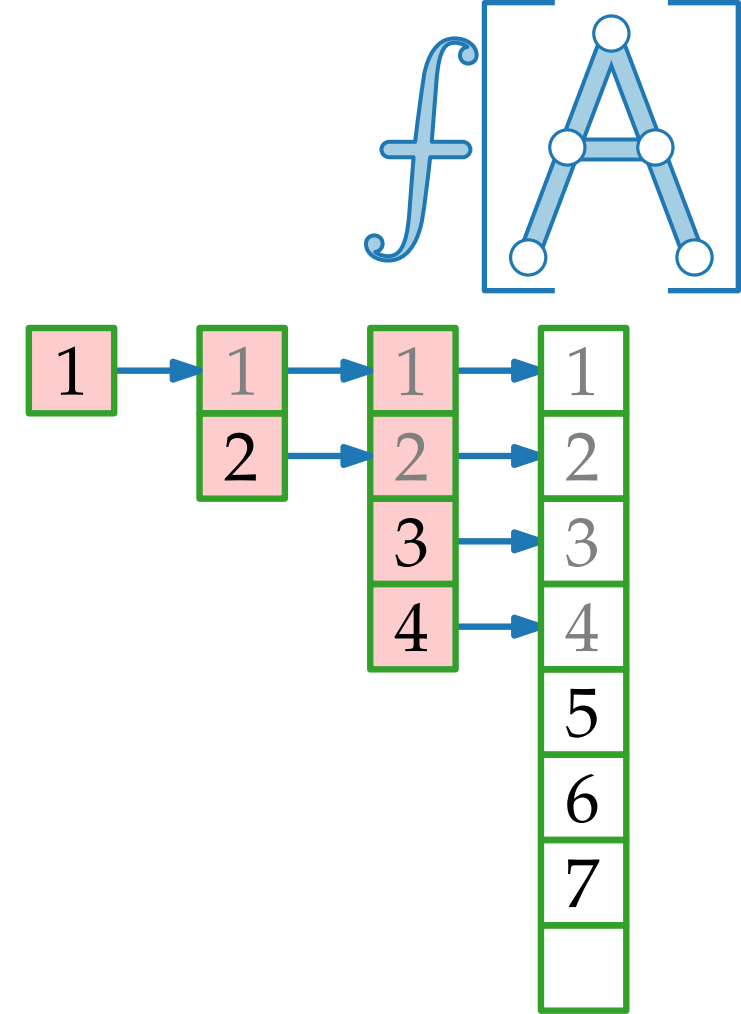
falsch!

Dynamische Tabellen

Idee.

- Wenn Tabelle voll, fordere doppelt so große Tabelle an
- Kopiere alle Einträge von alter in neue Tabelle.
- Gib Speicher für alte Tabelle frei.

INSERT(1)
INSERT(2)
INSERT(3)
INSERT(4)
INSERT(5)
INSERT(6)
INSERT(7)
...



Analyse. Welche Laufzeit benötigen n Einfügeoperationen im schlimmsten Fall?

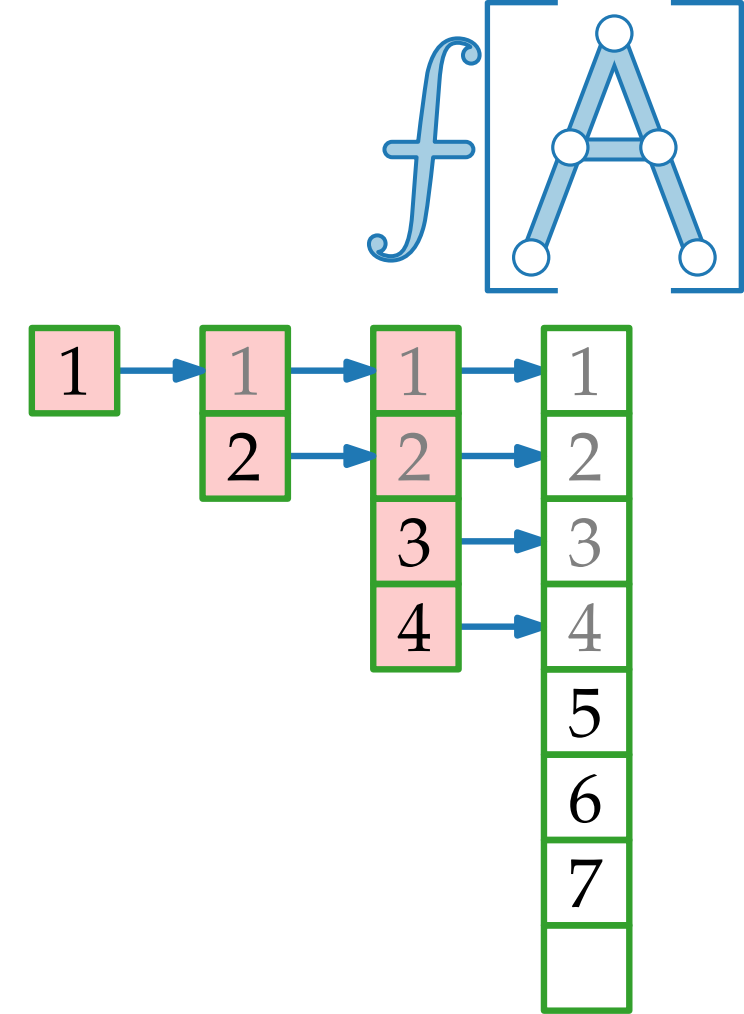
Antwort. ■ Tabelle wird genau $\lceil \log_2 n \rceil$ mal kopiert.
■ Im schlimmsten (letzten!) Fall ist der Aufwand $\Theta(n)$.
Also ist der Gesamtaufwand ~~$\Theta(n \log n)$~~ .
falsch! $\mathcal{O}$

Dynamische Tabellen

Idee.

- Wenn Tabelle voll, fordere doppelt so große Tabelle an
- Kopiere alle Einträge von alter in neue Tabelle.
- Gib Speicher für alte Tabelle frei.

INSERT(1)
INSERT(2)
INSERT(3)
INSERT(4)
INSERT(5)
INSERT(6)
INSERT(7)
...



Analyse. Welche Laufzeit benötigen n Einfügeoperationen im schlimmsten Fall?

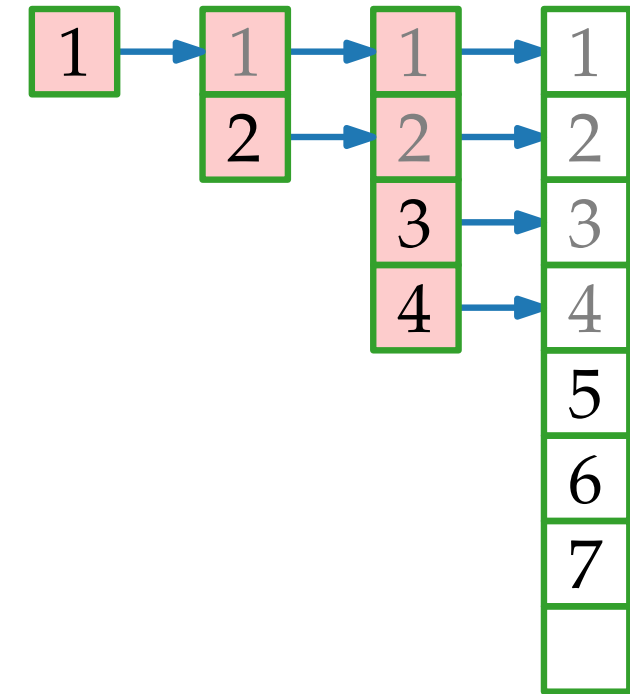
Antwort. ■ Tabelle wird genau $\lceil \log_2 n \rceil$ mal kopiert.
■ Im schlimmsten (letzten!) Fall ist der Aufwand $\Theta(n)$.
Also ist der Gesamtaufwand ~~$\Theta(n \log n)$~~ , **genauer falsch!** $\mathcal{O}$

Dynamische Tabellen

Idee.

- Wenn Tabelle voll, fordere doppelt so große Tabelle an
- Kopiere alle Einträge von alter in neue Tabelle.
- Gib Speicher für alte Tabelle frei.

INSERT(1)
INSERT(2)
INSERT(3)
INSERT(4)
INSERT(5)
INSERT(6)
INSERT(7)
...



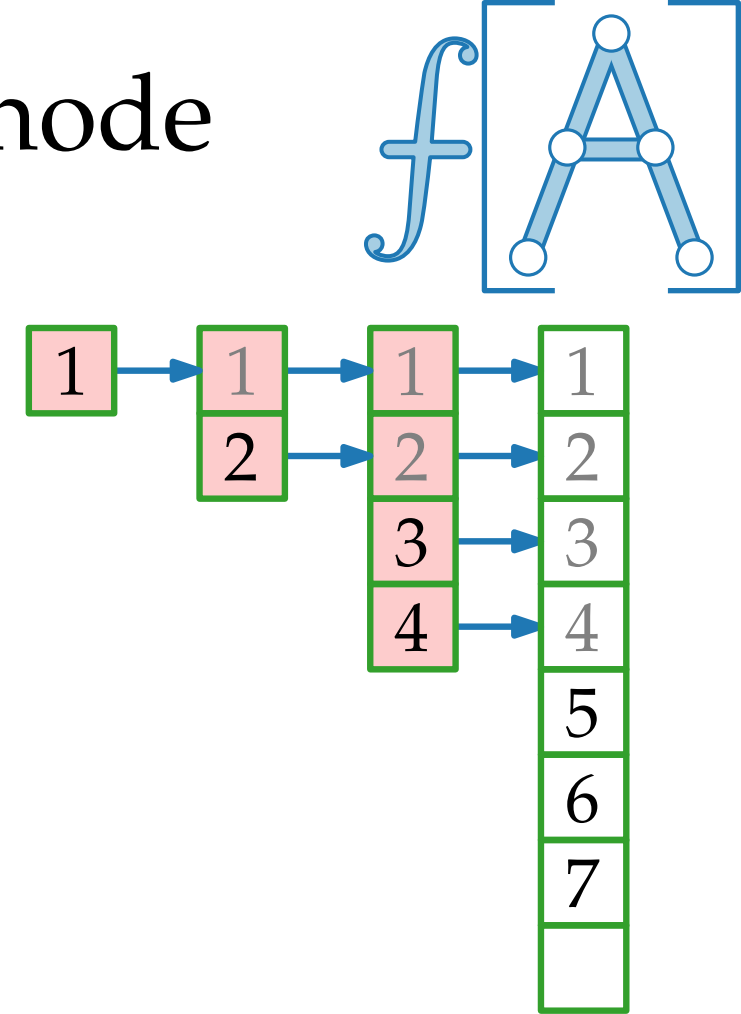
Analyse. Welche Laufzeit benötigen n Einfügeoperationen im schlimmsten Fall?

Antwort. ■ Tabelle wird genau $\lceil \log_2 n \rceil$ mal kopiert.
■ Im schlimmsten (letzten!) Fall ist der Aufwand $\Theta(n)$.
Also ist der Gesamtaufwand ~~$\Theta(n \log n)$~~ , **genauer** $\Theta(n)$.
falsch! $\mathcal{O}$

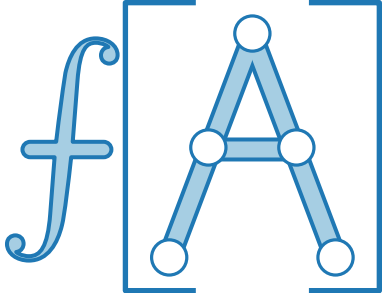
Genauere Abschätzung: Aggregationsmethode

Für $i = 1, \dots, n$ sei

c_i = Kosten fürs i -te Einfügen.

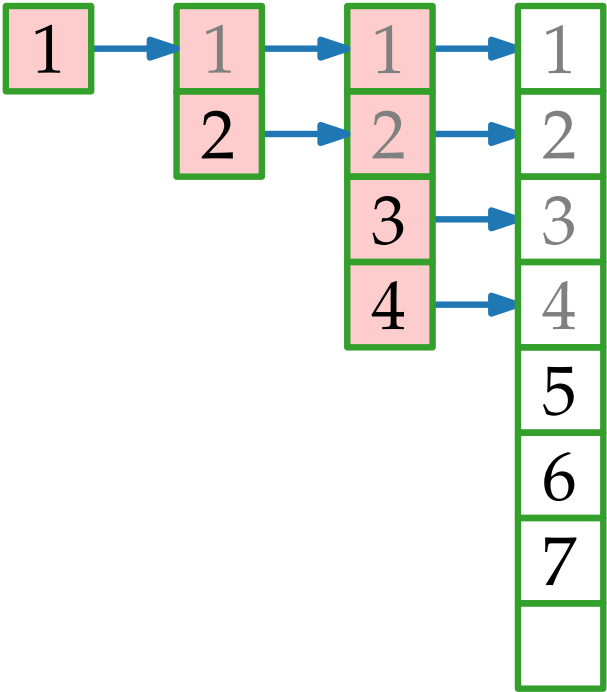


Genauere Abschätzung: Aggregationsmethode

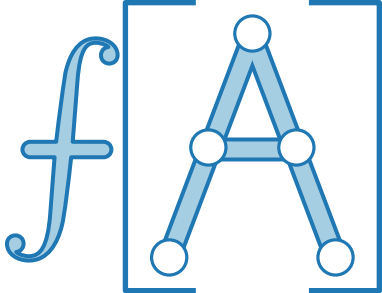


Für $i = 1, \dots, n$ sei
 c_i = Kosten fürs i -te Einfügen.

i									
$size_i$									

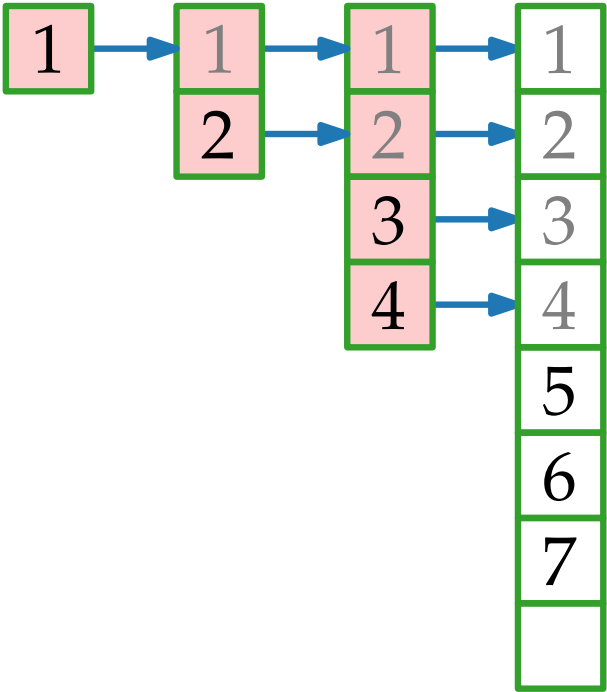


Genauere Abschätzung: Aggregationsmethode

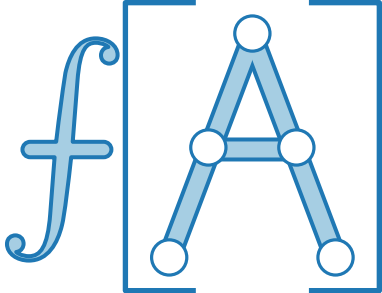


Für $i = 1, \dots, n$ sei
 c_i = Kosten fürs i -te Einfügen.

i	1								
$size_i$	1								

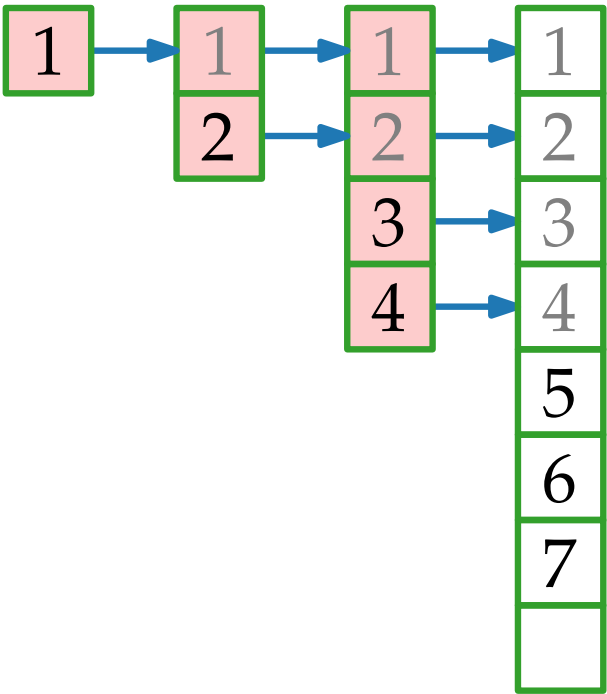


Genauere Abschätzung: Aggregationsmethode

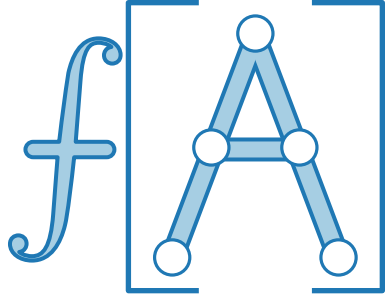


Für $i = 1, \dots, n$ sei
 c_i = Kosten fürs i -te Einfügen.

i	1	2							
$size_i$	1								

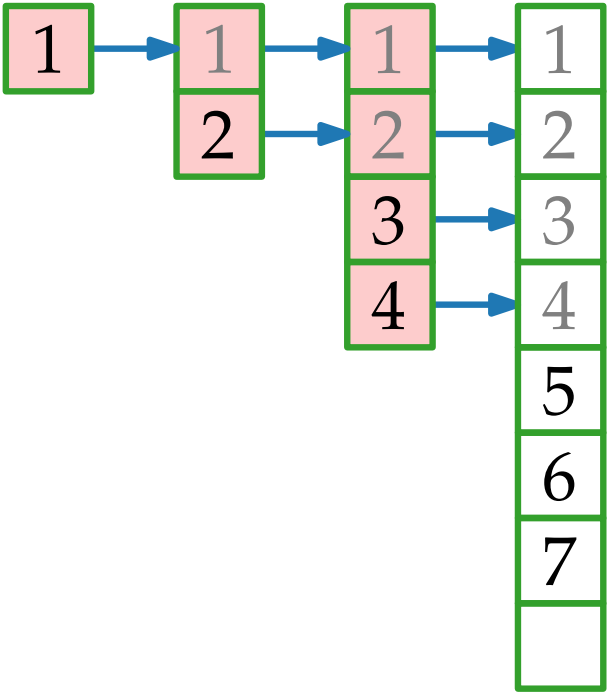


Genauere Abschätzung: Aggregationsmethode



Für $i = 1, \dots, n$ sei
 c_i = Kosten fürs i -te Einfügen.

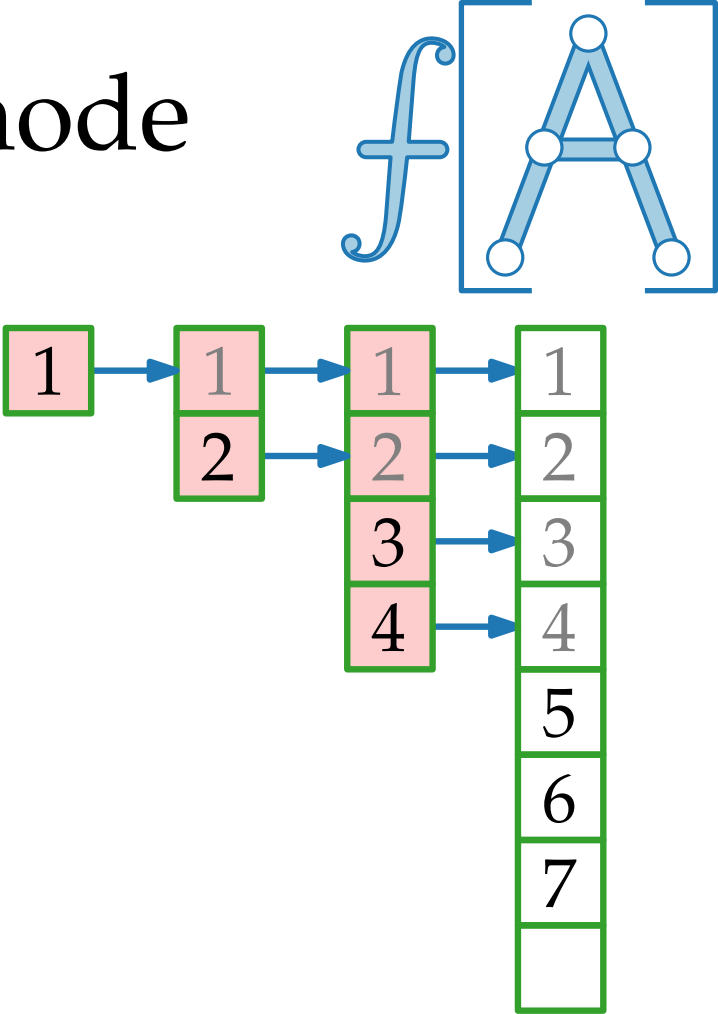
i	1	2							
$size_i$	1	2							



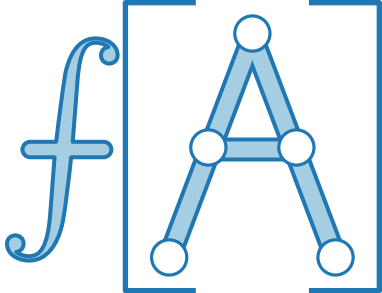
Genauere Abschätzung: Aggregationsmethode

Für $i = 1, \dots, n$ sei
 c_i = Kosten fürs i -te Einfügen.

i	1	2	3						
$size_i$	1	2							

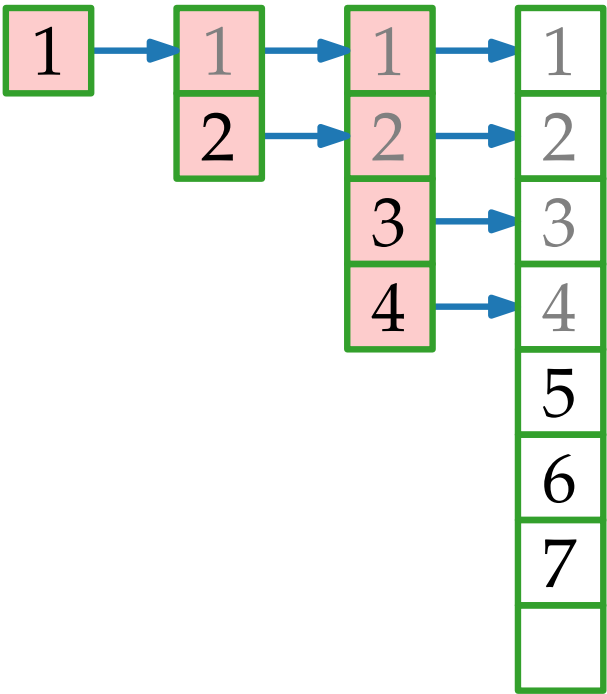


Genauere Abschätzung: Aggregationsmethode



Für $i = 1, \dots, n$ sei
 c_i = Kosten fürs i -te Einfügen.

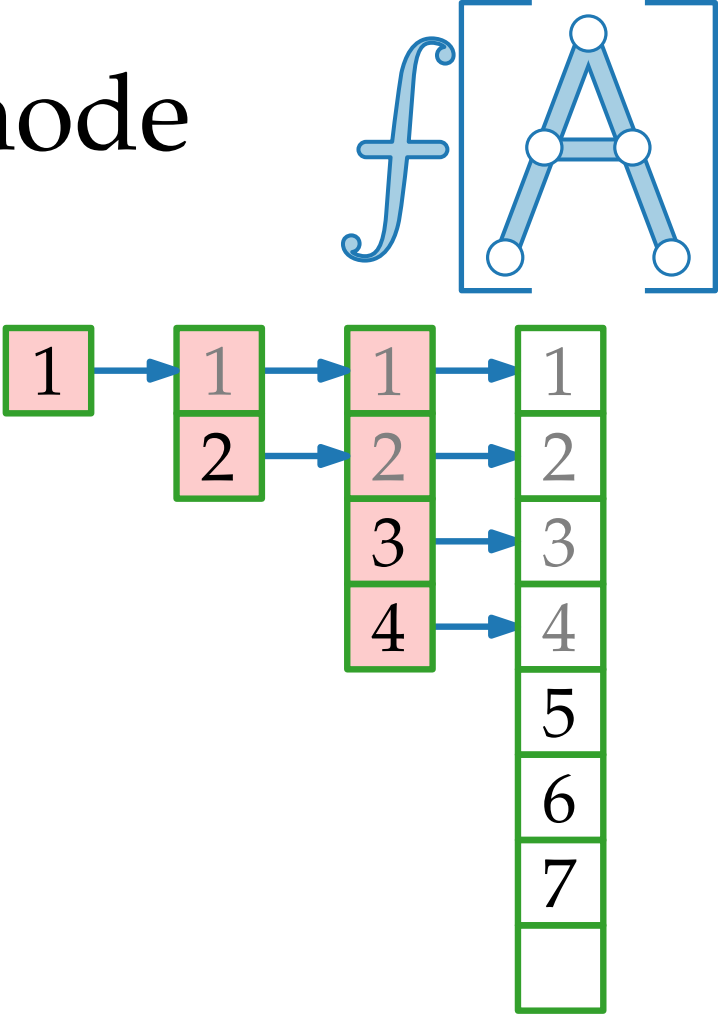
i	1	2	3						
$size_i$	1	2	4						



Genauere Abschätzung: Aggregationsmethode

Für $i = 1, \dots, n$ sei
 c_i = Kosten fürs i -te Einfügen.

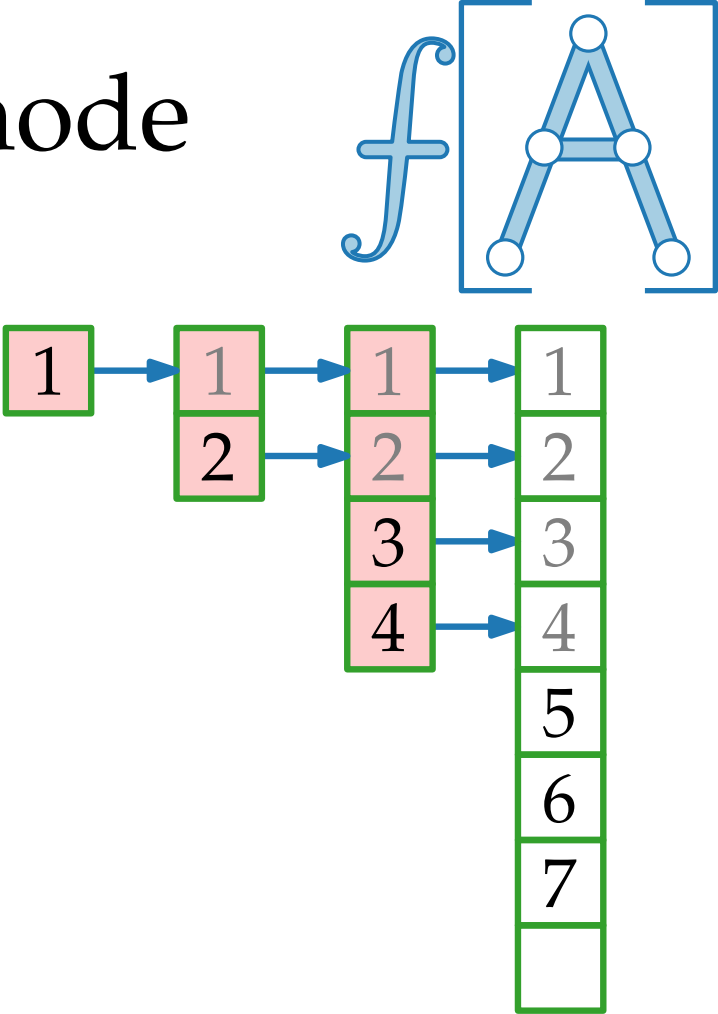
i	1	2	3	4					
$size_i$	1	2	4	4					



Genauere Abschätzung: Aggregationsmethode

Für $i = 1, \dots, n$ sei
 c_i = Kosten fürs i -te Einfügen.

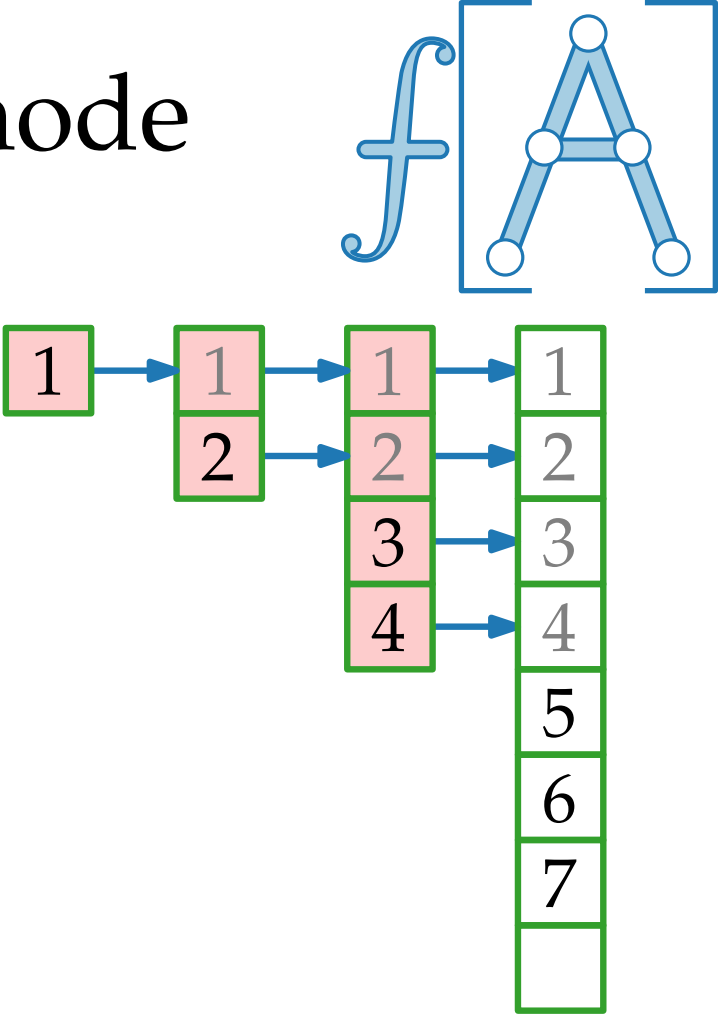
i	1	2	3	4	5	6	7	8	
$size_i$	1	2	4	4	8	8	8	8	



Genauere Abschätzung: Aggregationsmethode

Für $i = 1, \dots, n$ sei
 c_i = Kosten fürs i -te Einfügen.

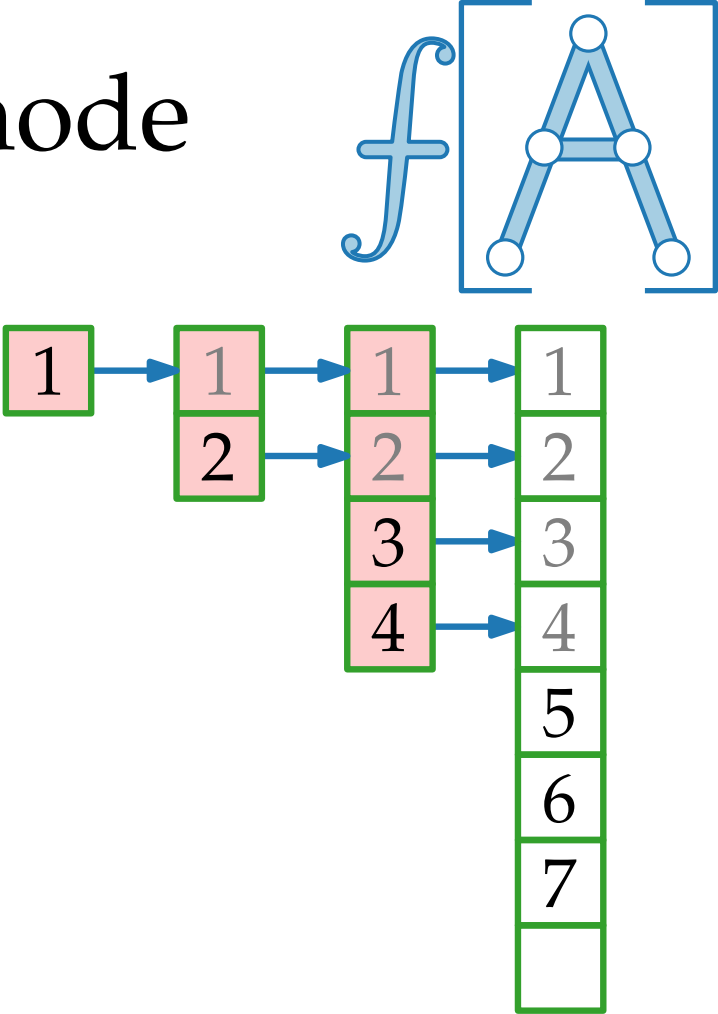
i	1	2	3	4	5	6	7	8	9
$size_i$	1	2	4	4	8	8	8	8	16



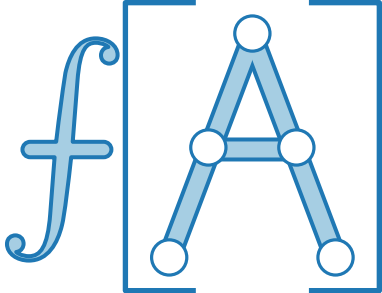
Genauere Abschätzung: Aggregationsmethode

Für $i = 1, \dots, n$ sei
 c_i = Kosten fürs i -te Einfügen.

i	1	2	3	4	5	6	7	8	9
$size_i$	1	2	4	4	8	8	8	8	16
c_i									



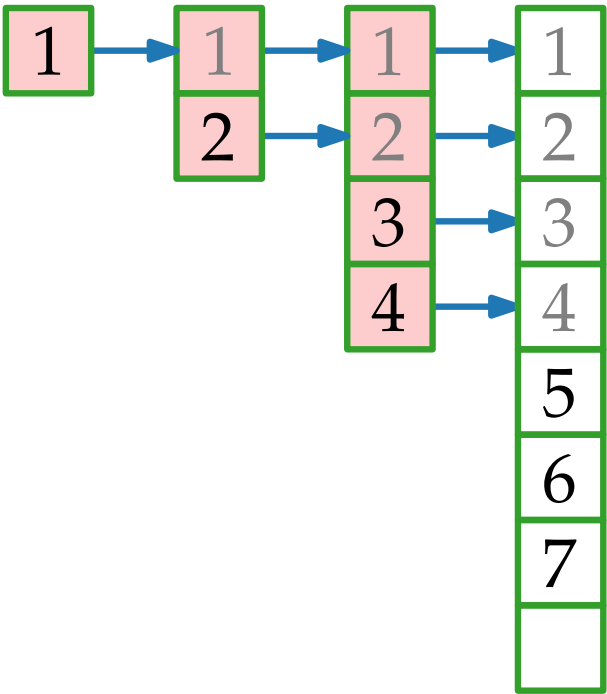
Genauere Abschätzung: Aggregationsmethode



Für $i = 1, \dots, n$ sei
 c_i = Kosten fürs i -te Einfügen.

i	1	2	3	4	5	6	7	8	9
$size_i$	1	2	4	4	8	8	8	8	16
c_i									

← Einfügen

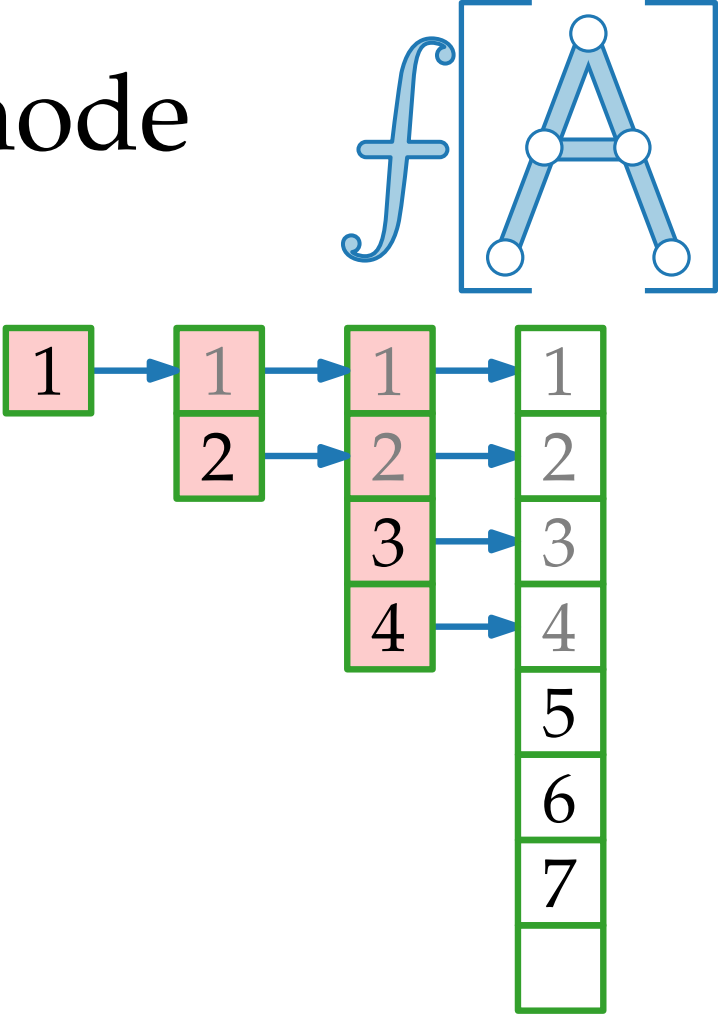


Genauere Abschätzung: Aggregationsmethode

Für $i = 1, \dots, n$ sei
 c_i = Kosten fürs i -te Einfügen.

i	1	2	3	4	5	6	7	8	9
$size_i$	1	2	4	4	8	8	8	8	16
c_i	€	€	€	€	€	€	€	€	€

← Einfügen

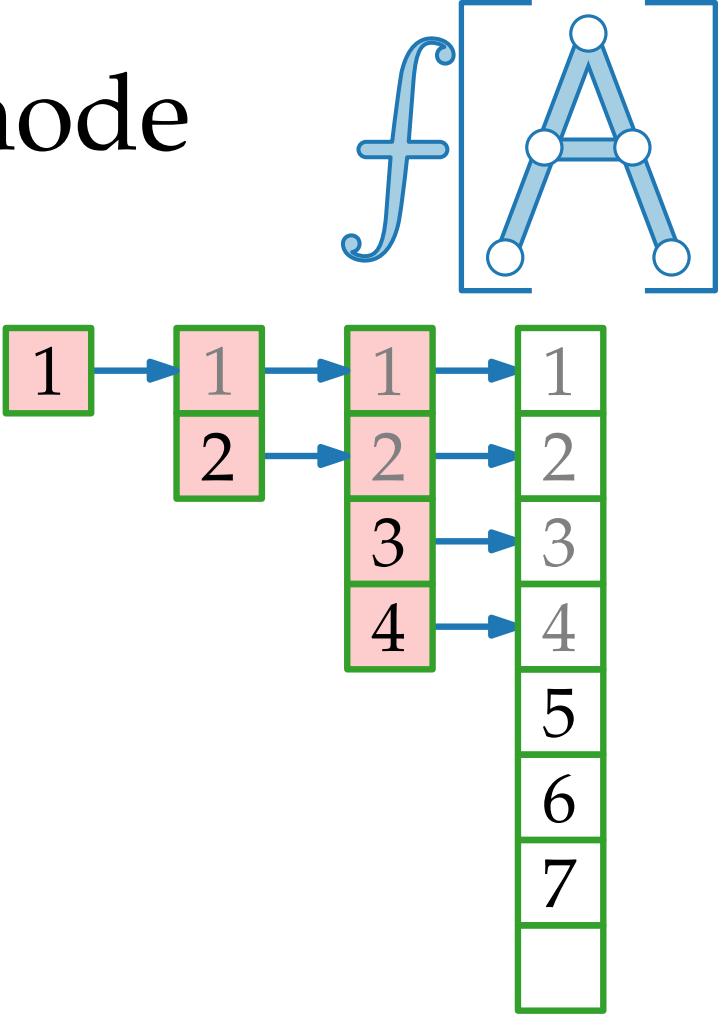


Genauere Abschätzung: Aggregationsmethode

Für $i = 1, \dots, n$ sei
 c_i = Kosten fürs i -te Einfügen.

i	1	2	3	4	5	6	7	8	9
$size_i$	1	2	4	4	8	8	8	8	16
c_i	€	€	€	€	€	€	€	€	€

← Einfügen
← Kopieren

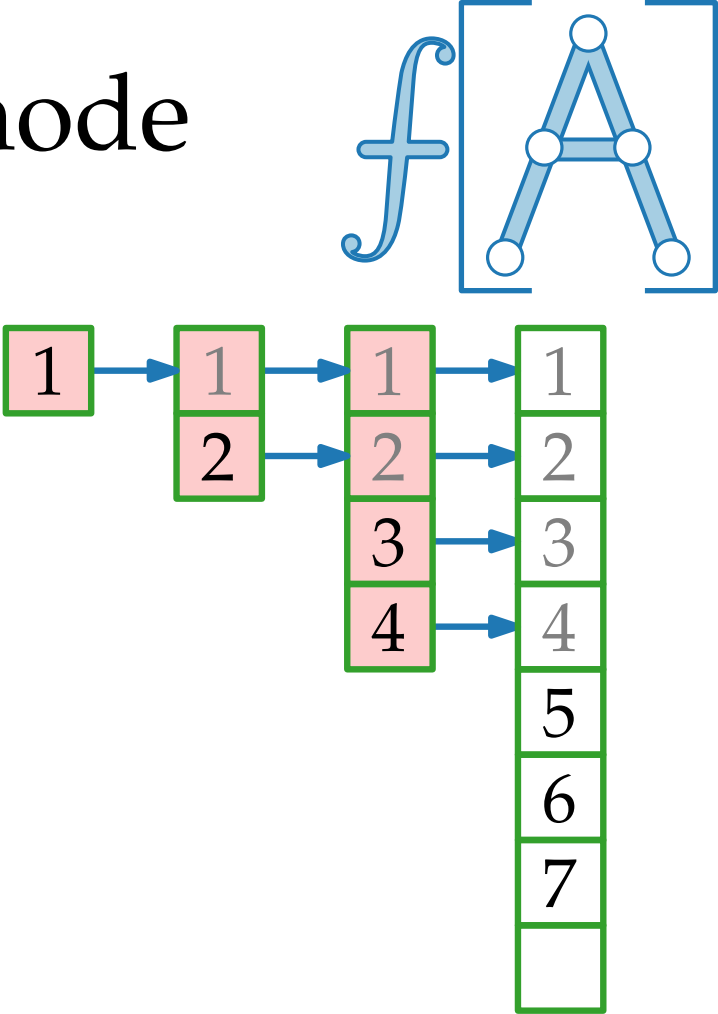


Genauere Abschätzung: Aggregationsmethode

Für $i = 1, \dots, n$ sei
 c_i = Kosten fürs i -te Einfügen.

i	1	2	3	4	5	6	7	8	9
$size_i$	1	2	4	4	8	8	8	8	16
c_i	€	€ €	€	€	€	€	€	€	€

← Einfügen
← Kopieren

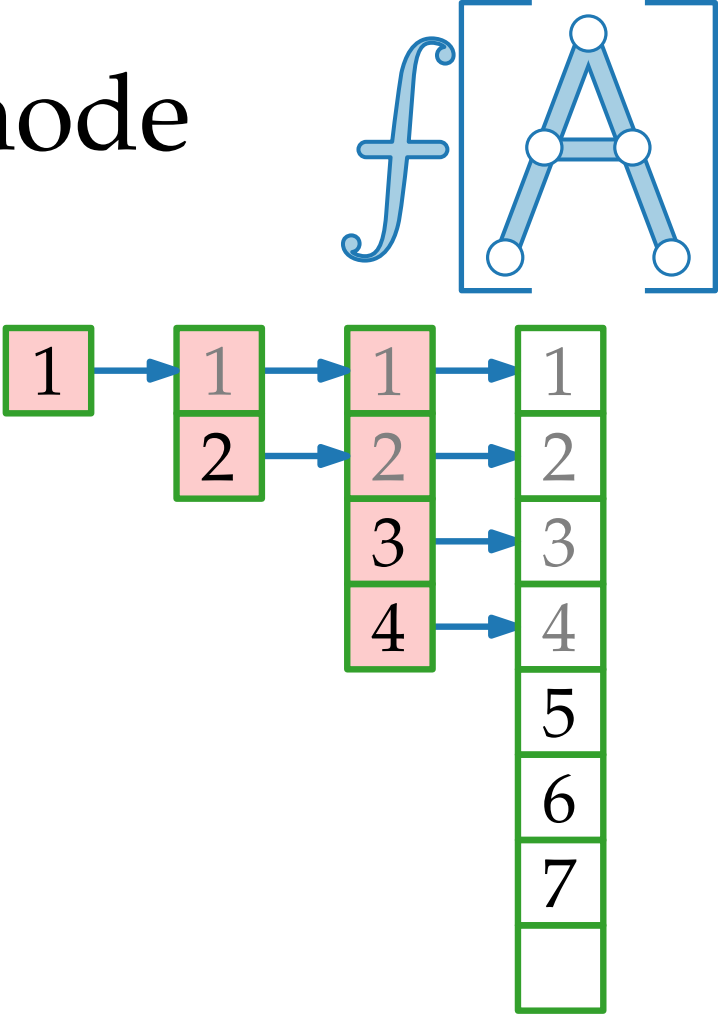


Genauere Abschätzung: Aggregationsmethode

Für $i = 1, \dots, n$ sei
 c_i = Kosten fürs i -te Einfügen.

i	1	2	3	4	5	6	7	8	9
$size_i$	1	2	4	4	8	8	8	8	16
c_i	€	€ €	€ €	€	€	€	€	€	€

← Einfügen
← Kopieren

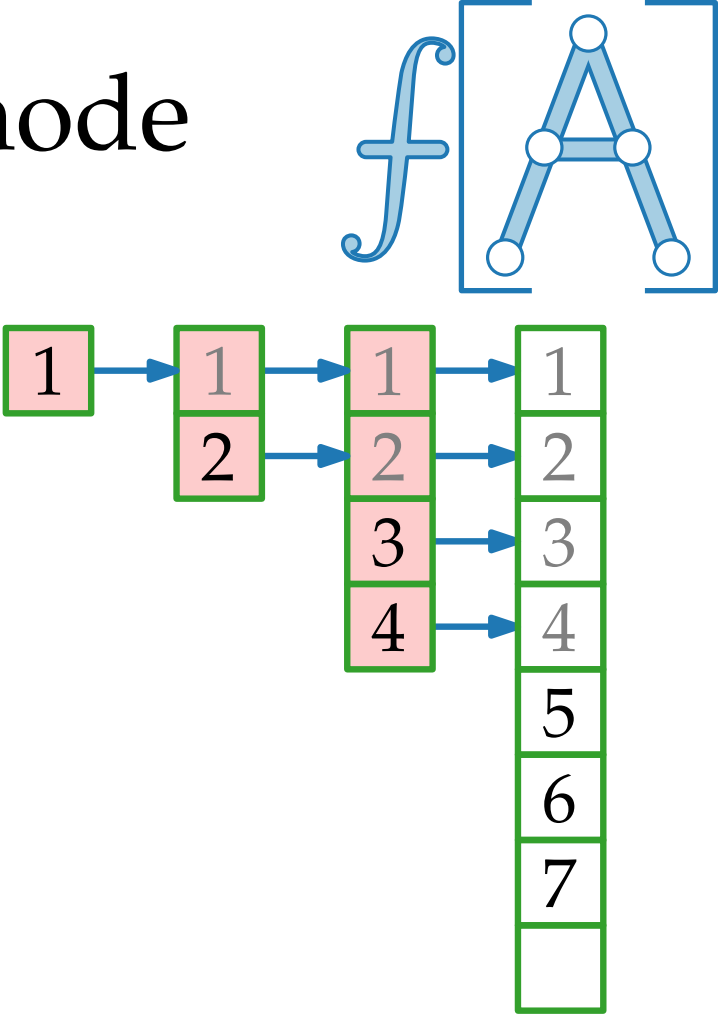


Genauere Abschätzung: Aggregationsmethode

Für $i = 1, \dots, n$ sei
 c_i = Kosten fürs i -te Einfügen.

i	1	2	3	4	5	6	7	8	9
$size_i$	1	2	4	4	8	8	8	8	16
c_i	€	€ €	€ €	€	€ € €	€	€	€	€

← Einfügen
← Kopieren

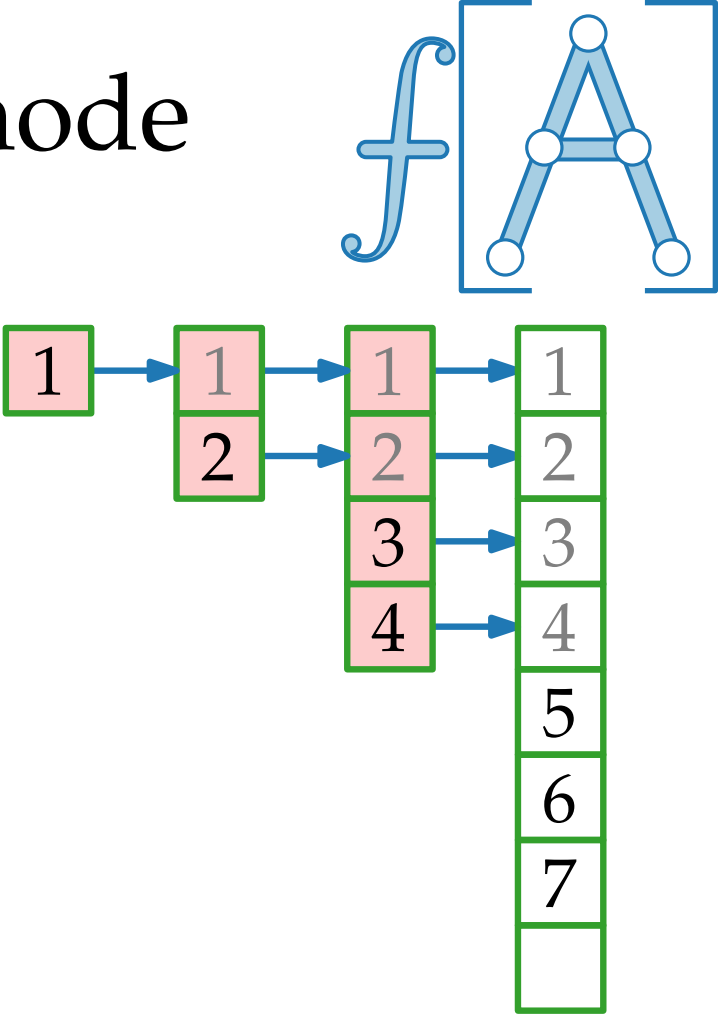


Genauere Abschätzung: Aggregationsmethode

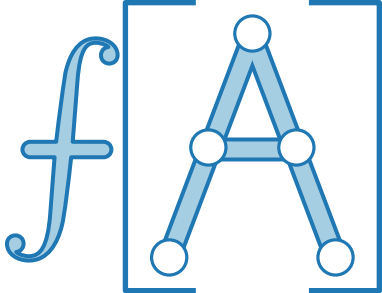
Für $i = 1, \dots, n$ sei
 c_i = Kosten fürs i -te Einfügen.

i	1	2	3	4	5	6	7	8	9
$size_i$	1	2	4	4	8	8	8	8	16
c_i	€	€ €	€ €	€	€ € €	€	€	€	€ •••

← Einfügen
← Kopieren



Genauere Abschätzung: Aggregationsmethode

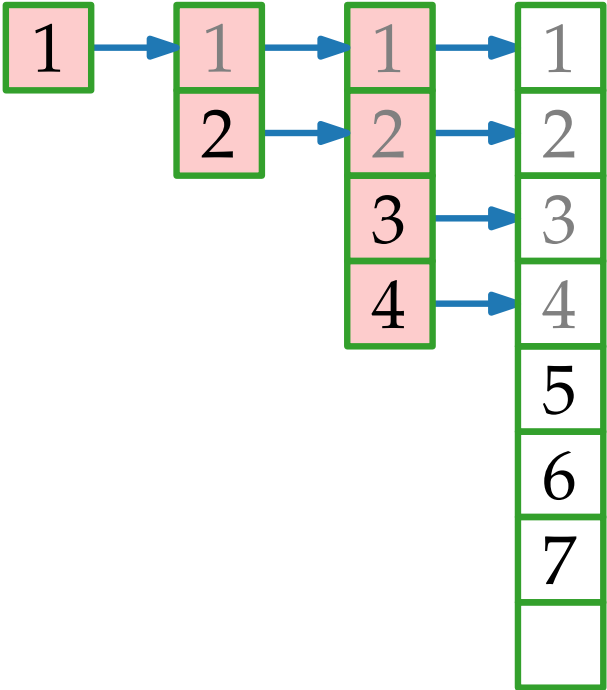


Für $i = 1, \dots, n$ sei
 c_i = Kosten fürs i -te Einfügen.

i	1	2	3	4	5	6	7	8	9
$size_i$	1	2	4	4	8	8	8	8	16
c_i	€	€ €	€ €	€	€ € €	€	€	€	€ € € €

← Einfügen
← Kopieren

Also betragen die Kosten für n Einfügeoperationen



Genauere Abschätzung: Aggregationsmethode

Für $i = 1, \dots, n$ sei

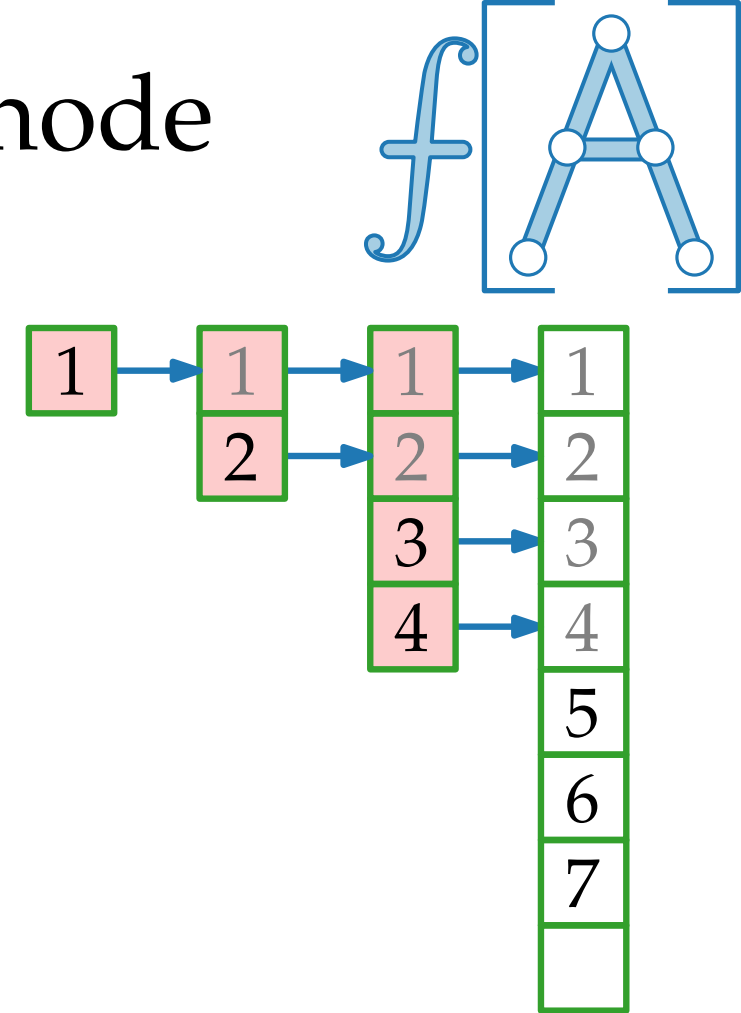
c_i = Kosten fürs i -te Einfügen.

i	1	2	3	4	5	6	7	8	9
$size_i$	1	2	4	4	8	8	8	8	16
c_i	€	€ €	€ €	€	€ € €	€	€	€	€ € € €

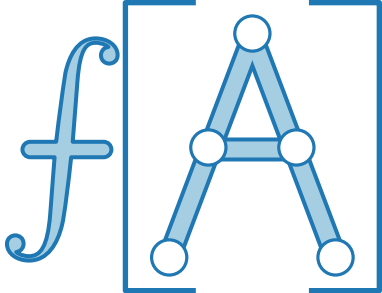
← Einfügen
← Kopieren

Also betragen die Kosten für n Einfügeoperationen

$$\sum_{i=1}^n c_i =$$



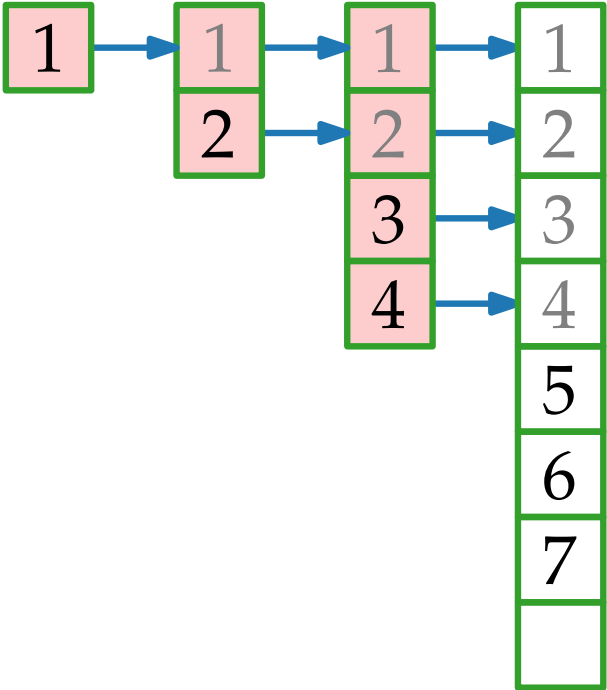
Genauere Abschätzung: Aggregationsmethode



Für $i = 1, \dots, n$ sei
 c_i = Kosten fürs i -te Einfügen.

i	1	2	3	4	5	6	7	8	9
$size_i$	1	2	4	4	8	8	8	8	16
c_i	€	€	€	€	€	€	€	€	€
		€	€		€ €				€ € €

← Einfügen
 ← Kopieren



Also betragen die Kosten für n Einfügeoperationen

$$\sum_1^n c_i = \text{orange square} + \sum_{j=0}$$

Genauere Abschätzung: Aggregationsmethode

Für $i = 1, \dots, n$ sei

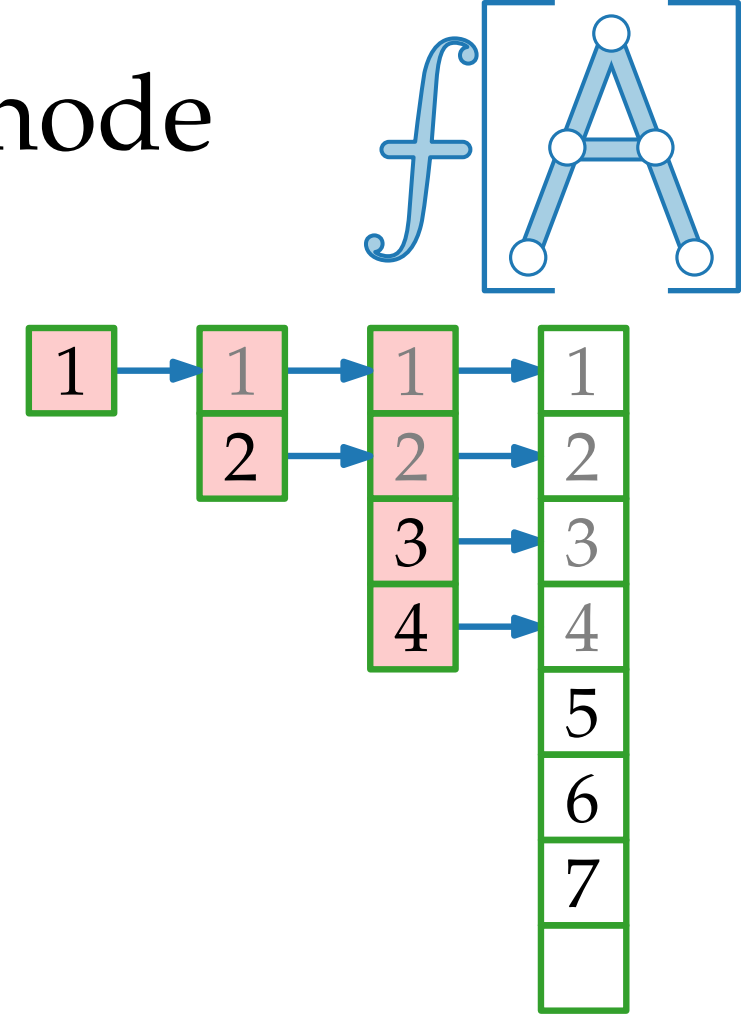
c_i = Kosten fürs i -te Einfügen.

i	1	2	3	4	5	6	7	8	9
$size_i$	1	2	4	4	8	8	8	8	16
c_i	€	€	€	€	€	€	€	€	€
		€	€		€ €				€ € €

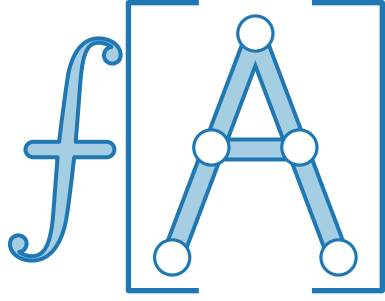
← Einfügen
← Kopieren

Also betragen die Kosten für n Einfügeoperationen

$$\sum_{i=1}^n c_i = n + \sum_{j=0}^n$$



Genauere Abschätzung: Aggregationsmethode

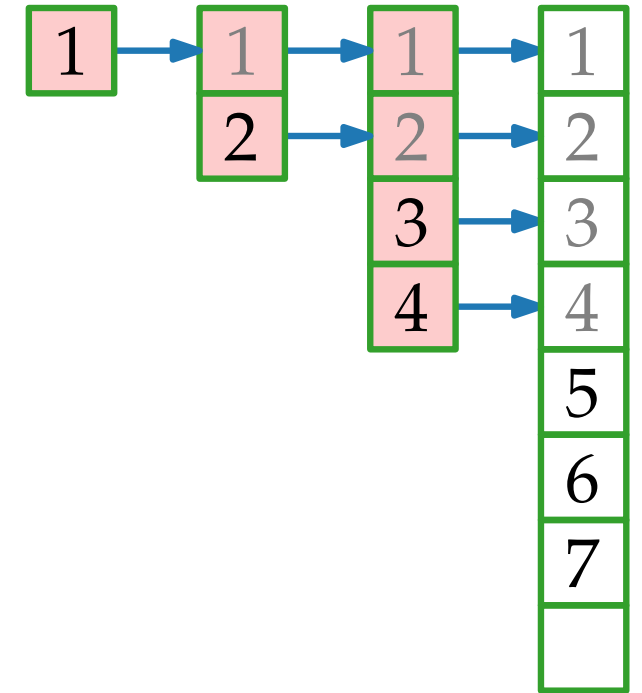


Für $i = 1, \dots, n$ sei

c_i = Kosten fürs i -te Einfügen.

i	1	2	3	4	5	6	7	8	9
$size_i$	1	2	4	4	8	8	8	8	16
c_i	€	€	€	€	€	€	€	€	€
		€	€		€ €				€ € €

← Einfügen
← Kopieren



Also betragen die Kosten für n Einfügeoperationen

$$\sum_{i=1}^n c_i = n + \sum_{j=0} 2^j$$

Genauere Abschätzung: Aggregationsmethode

Für $i = 1, \dots, n$ sei

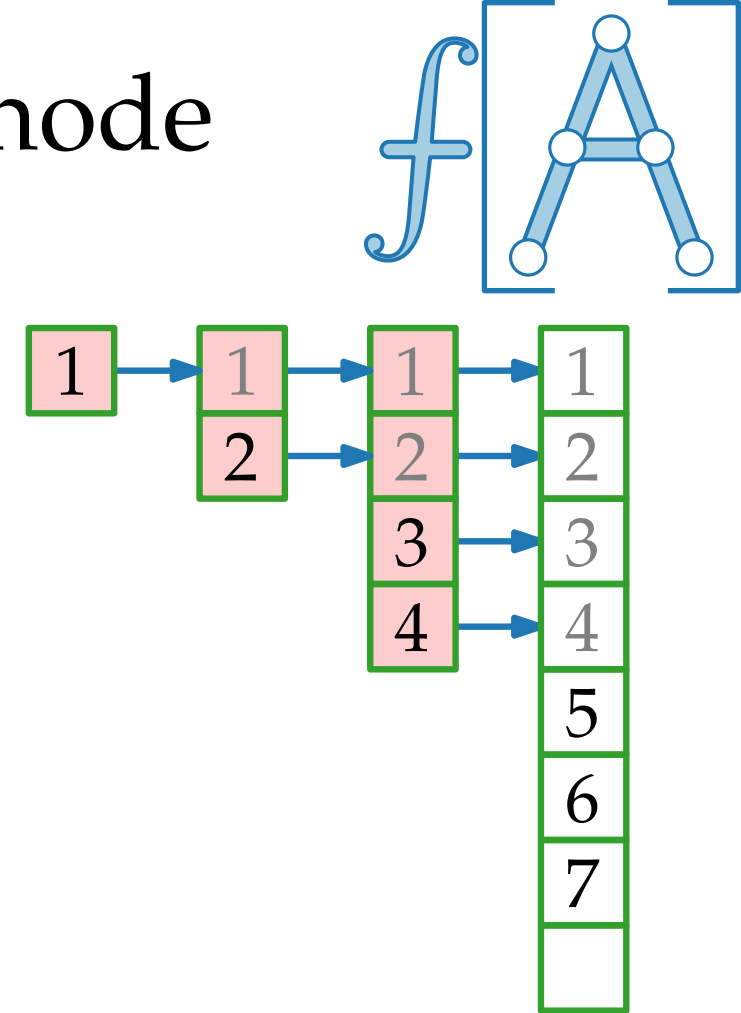
c_i = Kosten fürs i -te Einfügen.

i	1	2	3	4	5	6	7	8	9
$size_i$	1	2	4	4	8	8	8	8	16
c_i	€	€	€	€	€	€	€	€	€
		€	€		€ €				€ € €

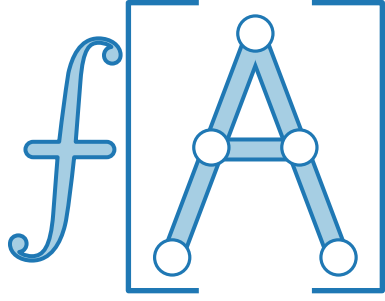
← Einfügen
← Kopieren

Also betragen die Kosten für n Einfügeoperationen

$$\sum_{i=1}^n c_i = n + \sum_{j=0}^{\lfloor \log_2(n-1) \rfloor} 2^j$$



Genauere Abschätzung: Aggregationsmethode

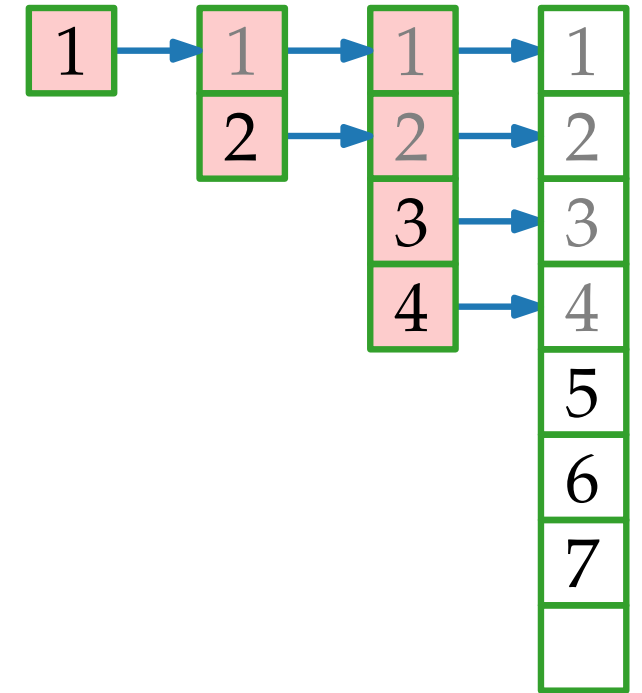


Für $i = 1, \dots, n$ sei

c_i = Kosten fürs i -te Einfügen.

i	1	2	3	4	5	6	7	8	9
$size_i$	1	2	4	4	8	8	8	8	16
c_i	€	€	€	€	€	€	€	€	€
		€	€		€ €				€ € €

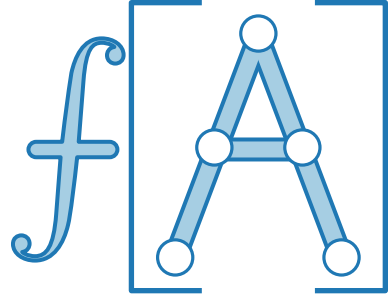
← Einfügen
← Kopieren



Also betragen die Kosten für n Einfügeoperationen

$$\sum_{i=1}^n c_i = n + \sum_{j=0}^{\lfloor \log_2(n-1) \rfloor} 2^j \leq$$

Genauere Abschätzung: Aggregationsmethode

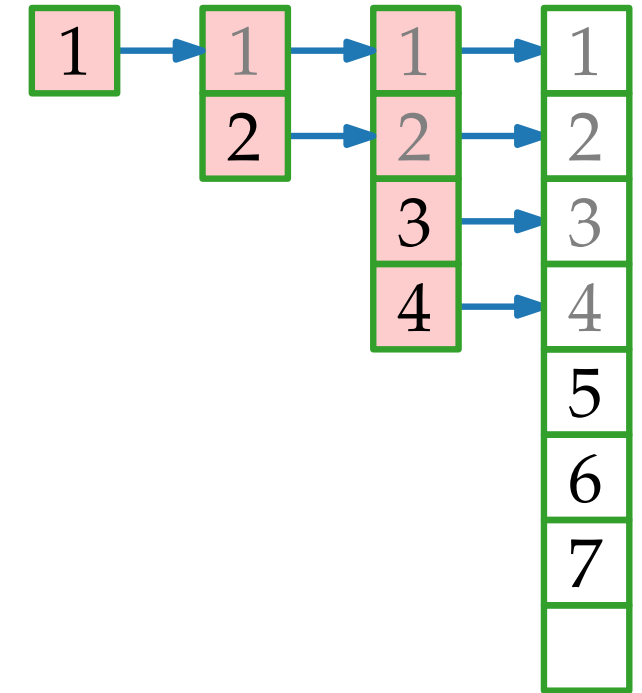


Für $i = 1, \dots, n$ sei

c_i = Kosten fürs i -te Einfügen.

i	1	2	3	4	5	6	7	8	9
$size_i$	1	2	4	4	8	8	8	8	16
c_i	€	€	€	€	€	€	€	€	€
		€	€		€ €				€ € €

← Einfügen
← Kopieren



Also betragen die Kosten für n Einfügeoperationen

$$\sum_{i=1}^n c_i = n + \sum_{j=0}^{\lfloor \log_2(n-1) \rfloor} 2^j \leq$$

$$\sum_{j=0}^n q^j = \frac{1-q^{n+1}}{1-q} \text{ geometrische Reihe}$$

Genauere Abschätzung: Aggregationsmethode

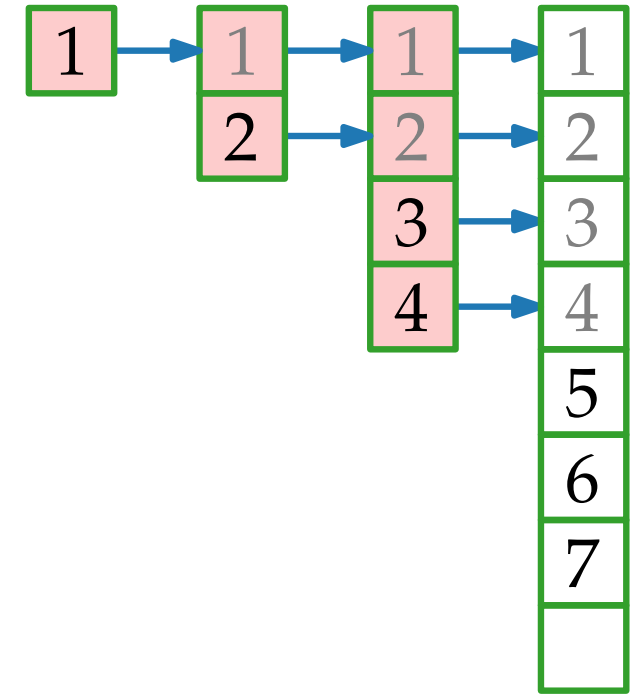


Für $i = 1, \dots, n$ sei

c_i = Kosten fürs i -te Einfügen.

i	1	2	3	4	5	6	7	8	9
$size_i$	1	2	4	4	8	8	8	8	16
c_i	€	€	€	€	€	€	€	€	€
		€	€		€ €				€ € €

← Einfügen
← Kopieren



Also betragen die Kosten für n Einfügeoperationen

$$\sum_{i=1}^n c_i = \boxed{n} + \sum_{j=0}^{\lfloor \log_2(n-1) \rfloor} 2^j \leq n + \frac{1 - 2^{\log_2(n-1)+1}}{1 - 2}$$

$$\sum_{j=0}^n q^j = \frac{1 - q^{n+1}}{1 - q} \text{ geometrische Reihe}$$

Genauere Abschätzung: Aggregationsmethode

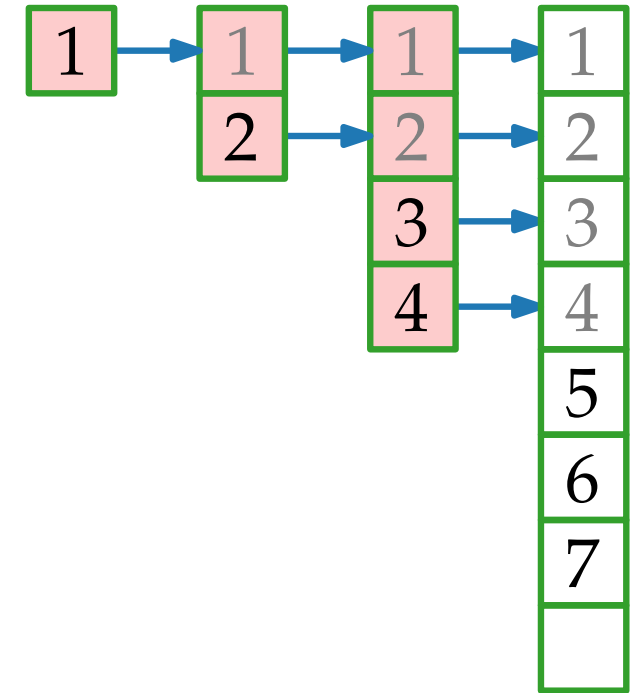


Für $i = 1, \dots, n$ sei

c_i = Kosten fürs i -te Einfügen.

i	1	2	3	4	5	6	7	8	9
$size_i$	1	2	4	4	8	8	8	8	16
c_i	€	€	€	€	€	€	€	€	€
		€	€		€ €				€ € €

← Einfügen
← Kopieren



Also betragen die Kosten für n Einfügeoperationen

$$\sum_{i=1}^n c_i = n + \sum_{j=0}^{\lfloor \log_2(n-1) \rfloor} 2^j \leq n + \frac{1 - 2^{\log_2(n-1)+1}}{1 - 2} = n + 2^{\log_2(n-1)+1} - 1$$

$$\sum_{j=0}^n q^j = \frac{1 - q^{n+1}}{1 - q} \text{ geometrische Reihe}$$

Genauere Abschätzung: Aggregationsmethode

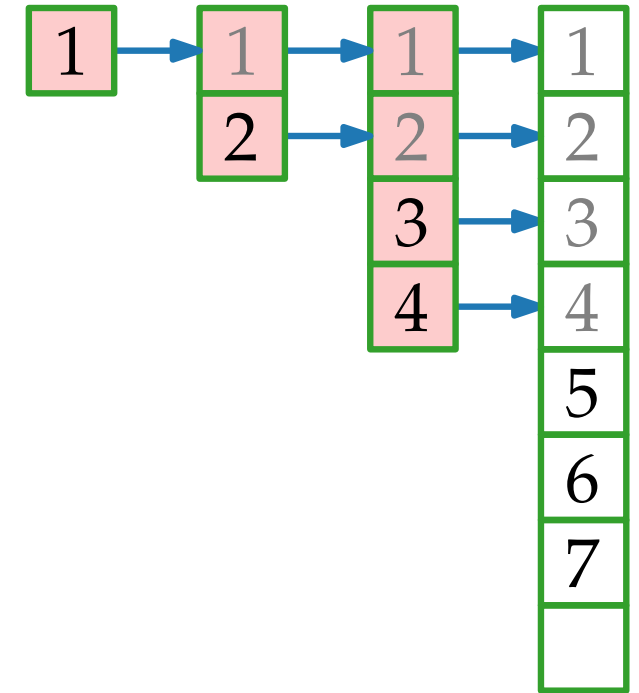


Für $i = 1, \dots, n$ sei

c_i = Kosten fürs i -te Einfügen.

i	1	2	3	4	5	6	7	8	9
$size_i$	1	2	4	4	8	8	8	8	16
c_i	€	€	€	€	€	€	€	€	€
		€	€		€ €				€ € €

← Einfügen
← Kopieren



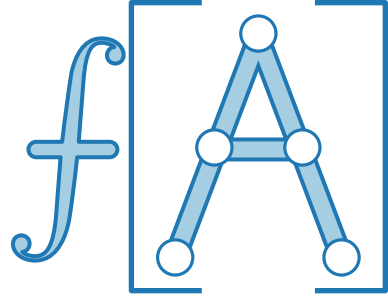
Also betragen die Kosten für n Einfügeoperationen

$$\sum_{i=1}^n c_i = n + \sum_{j=0}^{\lfloor \log_2(n-1) \rfloor} 2^j \leq n + \frac{1 - 2^{\log_2(n-1)+1}}{1 - 2} = n + 2^{\log_2(n-1)+1} - 1$$

$$= n + 2 \cdot 2^{\log_2(n-1)} - 1$$

$\sum_{j=0}^n q^j = \frac{1-q^{n+1}}{1-q}$ geometrische Reihe

Genauere Abschätzung: Aggregationsmethode

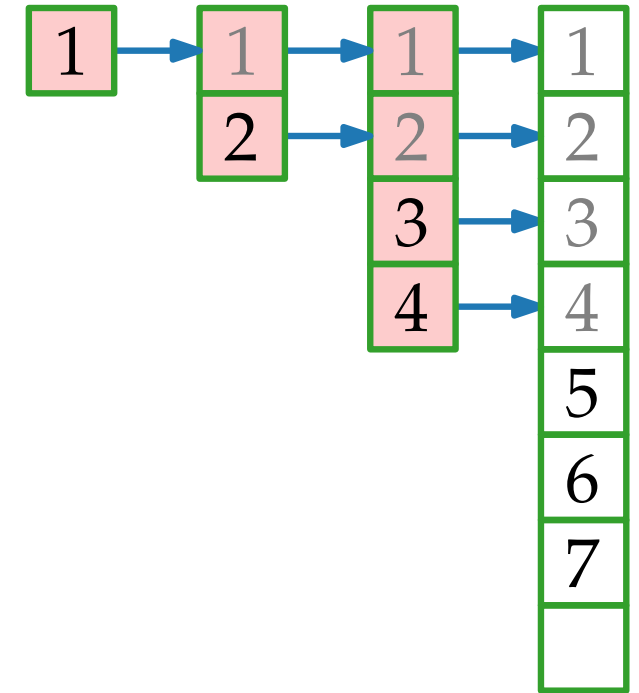


Für $i = 1, \dots, n$ sei

c_i = Kosten fürs i -te Einfügen.

i	1	2	3	4	5	6	7	8	9
$size_i$	1	2	4	4	8	8	8	8	16
c_i	€	€	€	€	€	€	€	€	€
		€	€		€ €				€ € €

← Einfügen
← Kopieren



Also betragen die Kosten für n Einfügeoperationen

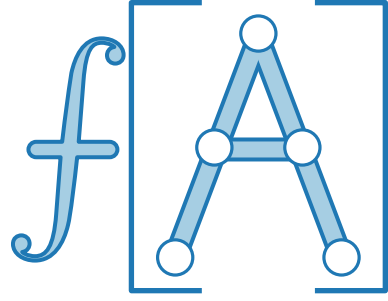
$$\sum_{i=1}^n c_i = n + \sum_{j=0}^{\lfloor \log_2(n-1) \rfloor} 2^j \leq n + \frac{1 - 2^{\log_2(n-1)+1}}{1 - 2} = n + 2^{\log_2(n-1)+1} - 1$$

$$= n + 2 \cdot 2^{\log_2(n-1)} - 1$$

$$= n + 2(n - 1) - 1$$

$$\sum_{j=0}^n q^j = \frac{1 - q^{n+1}}{1 - q} \text{ geometrische Reihe}$$

Genauere Abschätzung: Aggregationsmethode

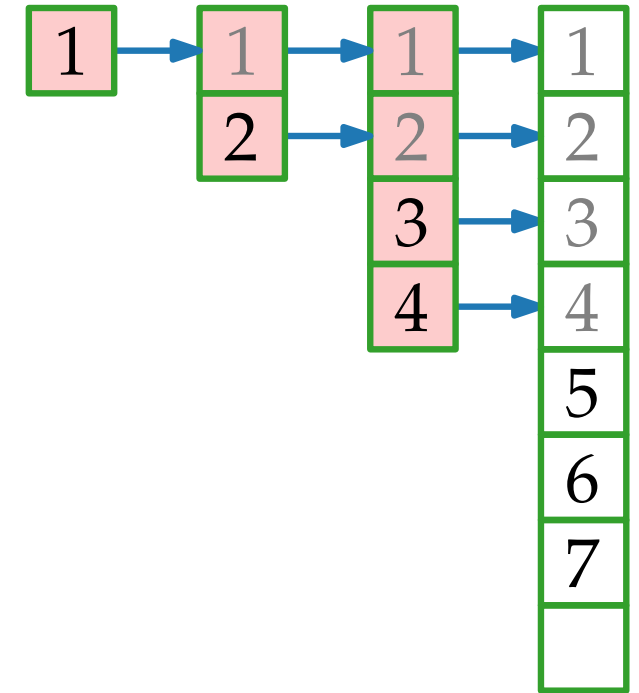


Für $i = 1, \dots, n$ sei

c_i = Kosten fürs i -te Einfügen.

i	1	2	3	4	5	6	7	8	9
$size_i$	1	2	4	4	8	8	8	8	16
c_i	€	€	€	€	€	€	€	€	€
		€	€		€ €				€ € €

← Einfügen
← Kopieren



Also betragen die Kosten für n Einfügeoperationen

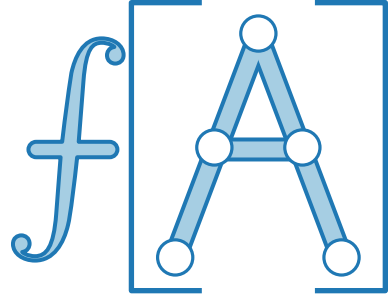
$$\sum_{i=1}^n c_i = n + \sum_{j=0}^{\lfloor \log_2(n-1) \rfloor} 2^j \leq n + \frac{1 - 2^{\log_2(n-1)+1}}{1 - 2} = n + 2^{\log_2(n-1)+1} - 1$$

$$= n + 2 \cdot 2^{\log_2(n-1)} - 1$$

$$= n + 2(n - 1) - 1 = 3n - 3$$

$$\sum_{j=0}^n q^j = \frac{1 - q^{n+1}}{1 - q} \text{ geometrische Reihe}$$

Genauere Abschätzung: Aggregationsmethode

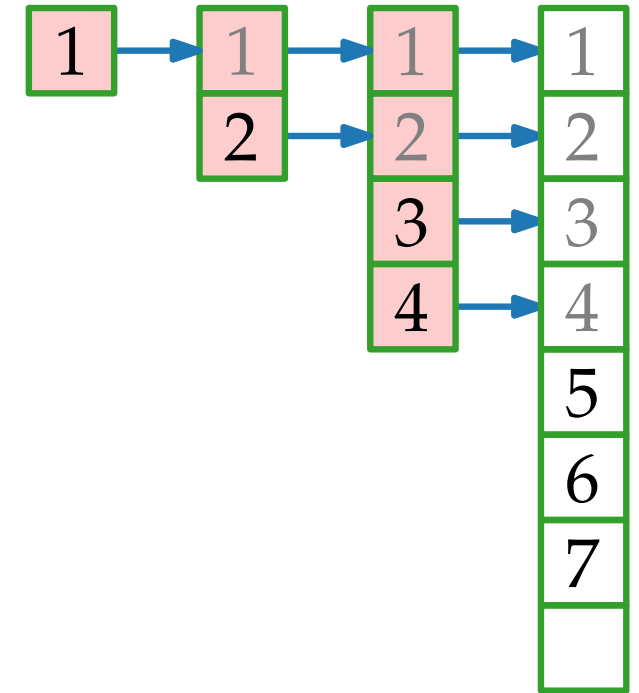


Für $i = 1, \dots, n$ sei

c_i = Kosten fürs i -te Einfügen.

i	1	2	3	4	5	6	7	8	9
$size_i$	1	2	4	4	8	8	8	8	16
c_i	€	€	€	€	€	€	€	€	€
		€	€		€ €				€ € €

← Einfügen
← Kopieren



Also betragen die Kosten für n Einfügeoperationen

$$\sum_{i=1}^n c_i = n + \sum_{j=0}^{\lfloor \log_2(n-1) \rfloor} 2^j \leq n + \frac{1 - 2^{\log_2(n-1)+1}}{1 - 2} = n + 2^{\log_2(n-1)+1} - 1$$

$$= n + 2 \cdot 2^{\log_2(n-1)} - 1$$

$$= n + 2(n - 1) - 1 = 3n - 3 \in \Theta(n)$$

$$\sum_{j=0}^n q^j = \frac{1 - q^{n+1}}{1 - q} \text{ geometrische Reihe}$$

Genauere Abschätzung: Aggregationsmethode

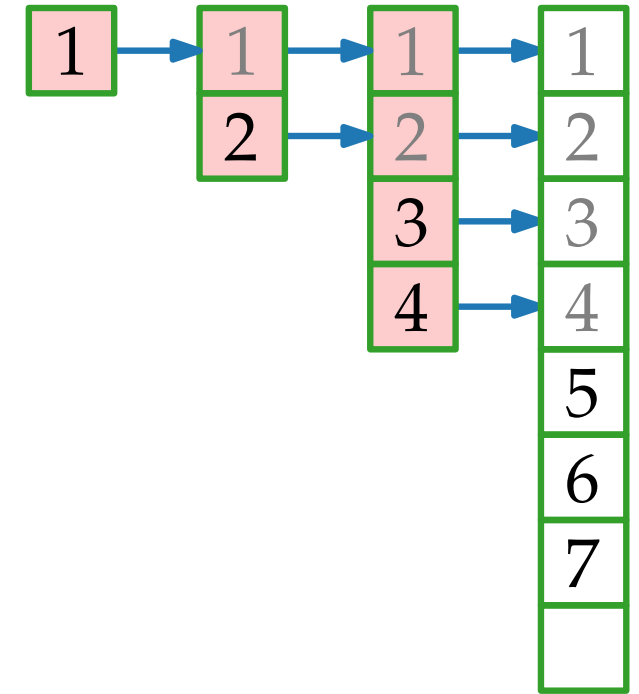


Für $i = 1, \dots, n$ sei

c_i = Kosten fürs i -te Einfügen.

i	1	2	3	4	5	6	7	8	9
$size_i$	1	2	4	4	8	8	8	8	16
c_i	€	€	€	€	€	€	€	€	€
		€	€		€				€

← Einfügen
← Kopieren



Also betragen die Kosten für n Einfügeoperationen

$$\sum_{i=1}^n c_i = n + \sum_{j=0}^{\lfloor \log_2(n-1) \rfloor} 2^j \leq n + \frac{1 - 2^{\log_2(n-1)+1}}{1 - 2} = n + 2^{\log_2(n-1)+1} - 1$$

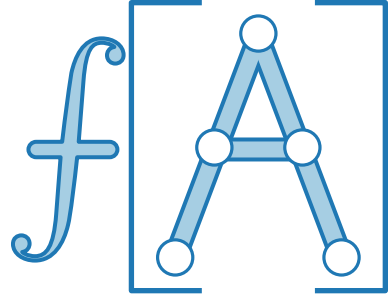
$$= n + 2 \cdot 2^{\log_2(n-1)} - 1$$

$$= n + 2(n - 1) - 1 = 3n - 3 \in \Theta(n)$$

$$\sum_{j=0}^n q^j = \frac{1 - q^{n+1}}{1 - q} \text{ geometrische Reihe}$$

D.h. die durchschnittlichen (**amortisierten**) Kosten sind $\Theta(1)$.

Genauere Abschätzung: Aggregationsmethode



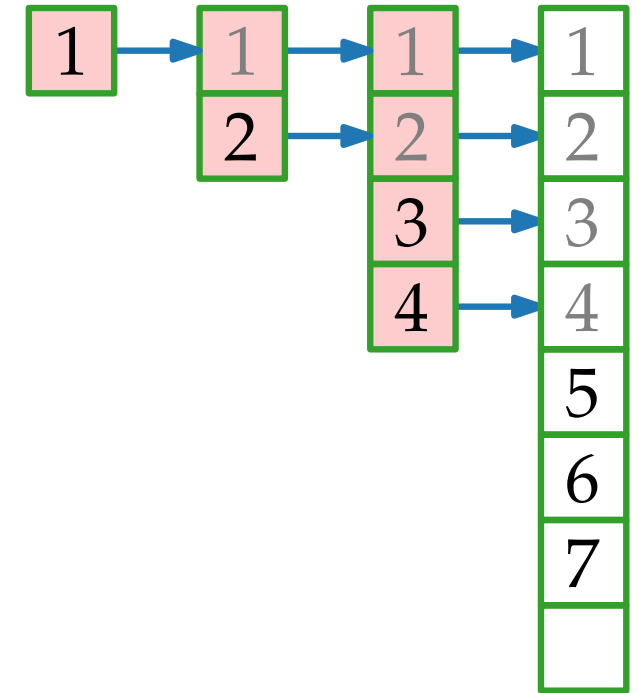
Für $i = 1, \dots, n$ sei

c_i = Kosten fürs i -te Einfügen.

i	1	2	3	4	5	6	7	8	9
$size_i$	1	2	4	4	8	8	8	8	16
c_i	€	€	€	€	€	€	€	€	€
		€	€		€				€

← Einfügen

← Kopieren



Also betragen die Kosten für n Einfügeoperationen

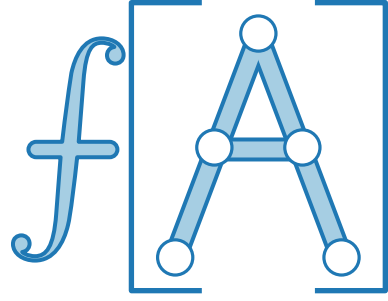
$$\begin{aligned}
 \sum_{i=1}^n c_i &= n + \sum_{j=0}^{\lfloor \log_2(n-1) \rfloor} 2^j \leq n + \frac{1 - 2^{\log_2(n-1)+1}}{1 - 2} = n + 2^{\log_2(n-1)+1} - 1 \\
 &= n + 2 \cdot 2^{\log_2(n-1)} - 1 \\
 &= n + 2(n - 1) - 1 = 3n - 3 \in \Theta(n)
 \end{aligned}$$

$\sum_{j=0}^n q^j = \frac{1 - q^{n+1}}{1 - q}$ geometrische Reihe

D.h. die durchschnittlichen (**amortisierten**) Kosten sind $\Theta(1)$.

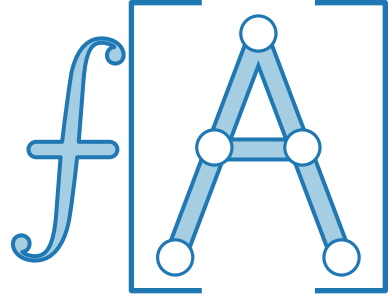
Amortisierte Analyse...

...bedeutet zu zeigen, dass die Operationen einer gegebenen Folge kleine durchschnittliche Kosten haben –



Amortisierte Analyse...

...bedeutet zu zeigen, dass die Operationen einer gegebenen Folge kleine durchschnittliche Kosten haben –
auch wenn einzelne Operationen in der Folge teuer sind!



Amortisierte Analyse...



...bedeutet zu zeigen, dass die Operationen einer gegebenen Folge kleine durchschnittliche Kosten haben –
auch wenn einzelne Operationen in der Folge teuer sind!

Auch *randomisierte Analyse* kann man als Durchschnittsbildung (über alle mögl. Ergebnisse, gewichtet nach Wahrscheinlichkeit) betrachten.

Amortisierte Analyse...



...bedeutet zu zeigen, dass die Operationen einer gegebenen Folge kleine durchschnittliche Kosten haben –
auch wenn einzelne Operationen in der Folge teuer sind!

Auch *randomisierte Analyse* kann man als Durchschnittsbildung (über alle mögl. Ergebnisse, gewichtet nach Wahrscheinlichkeit) betrachten.

Bei amortisierter Analyse geht es jedoch um die durchschnittliche Laufzeit *im schlechtesten Fall* – nicht im Erwartungswert!

Amortisierte Analyse...



...bedeutet zu zeigen, dass die Operationen einer gegebenen Folge kleine durchschnittliche Kosten haben –
auch wenn einzelne Operationen in der Folge teuer sind!

Auch *randomisierte Analyse* kann man als Durchschnittsbildung (über alle mögl. Ergebnisse, gewichtet nach Wahrscheinlichkeit) betrachten.

Bei amortisierter Analyse geht es jedoch um die durchschnittliche Laufzeit *im schlechtesten Fall* – nicht im Erwartungswert!

Die **amortisierte** Laufzeit einer Methode ist f , wenn jede Sequenz von k Aufrufen der Methode Laufzeit $\leq k \cdot f$ hat (wenn k groß genug ist).

Amortisierte Analyse...



...bedeutet zu zeigen, dass die Operationen einer gegebenen Folge kleine durchschnittliche Kosten haben –
auch wenn einzelne Operationen in der Folge teuer sind!

Auch *randomisierte Analyse* kann man als Durchschnittsbildung (über alle mögl. Ergebnisse, gewichtet nach Wahrscheinlichkeit) betrachten.

Bei amortisierter Analyse geht es jedoch um die durchschnittliche Laufzeit *im schlechtesten Fall* – nicht im Erwartungswert!

Die **amortisierte** Laufzeit einer Methode ist f , wenn jede Sequenz von k Aufrufen der Methode Laufzeit $\leq k \cdot f$ hat (wenn k groß genug ist).

Wir betrachten 3 verschiedene Typen von amortisierter Analyse:

Amortisierte Analyse...



...bedeutet zu zeigen, dass die Operationen einer gegebenen Folge kleine durchschnittliche Kosten haben –
auch wenn einzelne Operationen in der Folge teuer sind!

Auch *randomisierte Analyse* kann man als Durchschnittsbildung (über alle mögl. Ergebnisse, gewichtet nach Wahrscheinlichkeit) betrachten.

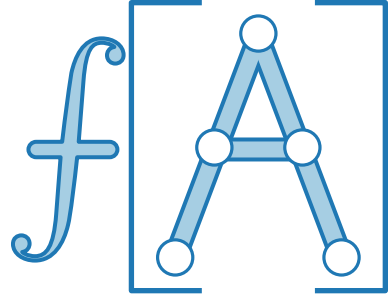
Bei amortisierter Analyse geht es jedoch um die durchschnittliche Laufzeit *im schlechtesten Fall* – nicht im Erwartungswert!

Die **amortisierte** Laufzeit einer Methode ist f , wenn jede Sequenz von k Aufrufen der Methode Laufzeit $\leq k \cdot f$ hat (wenn k groß genug ist).

Wir betrachten 3 verschiedene Typen von amortisierter Analyse:

- Aggregationsmethode
- Buchhaltermethode
- Potentialmethode

Amortisierte Analyse...



...bedeutet zu zeigen, dass die Operationen einer gegebenen Folge kleine durchschnittliche Kosten haben –
auch wenn einzelne Operationen in der Folge teuer sind!

Auch *randomisierte Analyse* kann man als Durchschnittsbildung (über alle mögl. Ergebnisse, gewichtet nach Wahrscheinlichkeit) betrachten.

Bei amortisierter Analyse geht es jedoch um die durchschnittliche Laufzeit *im schlechtesten Fall* – nicht im Erwartungswert!

Die **amortisierte** Laufzeit einer Methode ist f , wenn jede Sequenz von k Aufrufen der Methode Laufzeit $\leq k \cdot f$ hat (wenn k groß genug ist).

Wir betrachten 3 verschiedene Typen von amortisierter Analyse:

- Aggregationsmethode ✓
- Buchhaltermethode
- Potentialmethode

Amortisierte Analyse...



...bedeutet zu zeigen, dass die Operationen einer gegebenen Folge kleine durchschnittliche Kosten haben –
auch wenn einzelne Operationen in der Folge teuer sind!

Auch *randomisierte Analyse* kann man als Durchschnittsbildung (über alle mögl. Ergebnisse, gewichtet nach Wahrscheinlichkeit) betrachten.

Bei amortisierter Analyse geht es jedoch um die durchschnittliche Laufzeit *im schlechtesten Fall* – nicht im Erwartungswert!

Die **amortisierte** Laufzeit einer Methode ist f , wenn jede Sequenz von k Aufrufen der Methode Laufzeit $\leq k \cdot f$ hat (wenn k groß genug ist).

Wir betrachten 3 verschiedene Typen von amortisierter Analyse:

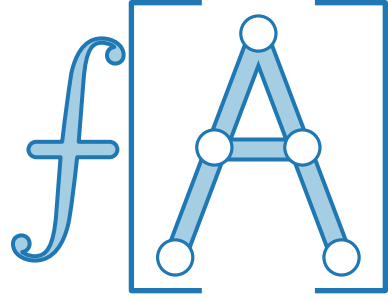
- Aggregationsmethode ✓
- Buchhaltermethode
- Potentialmethode

Buchhaltermethode



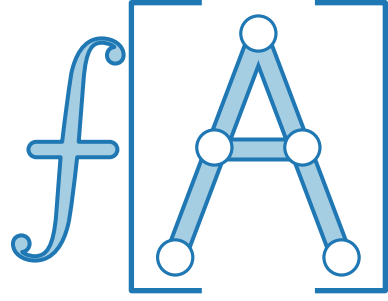
Buchhaltermethode

- Verbindet mit jeder Operation op_i **amortisierte** Kosten $\hat{c}_i$,



Buchhaltermethode

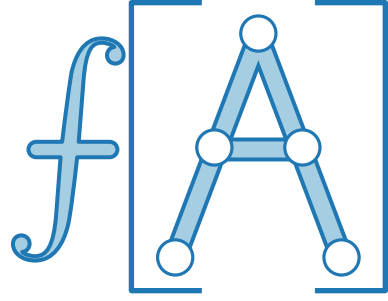
- Verbindet mit jeder Operation op_i **amortisierte** Kosten $\hat{c}_i$, die oft nicht mit den **tatsächlichen** Kosten c_i übereinstimmen.



Buchhaltermethode

- Verbindet mit jeder Operation op_i **amortisierte** Kosten $\hat{c}_i$, die oft nicht mit den **tatsächlichen** Kosten c_i übereinstimmen.

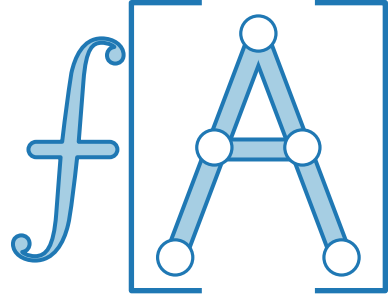
$$\hat{c}_i > c_i \Rightarrow$$



Buchhaltermethode

- Verbindet mit jeder Operation op_i **amortisierte** Kosten $\hat{c}_i$, die oft nicht mit den **tatsächlichen** Kosten c_i übereinstimmen.

$\hat{c}_i > c_i \Rightarrow$ Wir legen etwas beiseite. 🪙+



Buchhaltermethode

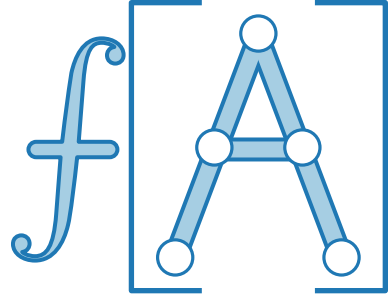


- Verbindet mit jeder Operation op_i **amortisierte** Kosten $\hat{c}_i$, die oft nicht mit den **tatsächlichen** Kosten c_i übereinstimmen.

$\hat{c}_i > c_i \Rightarrow$ Wir legen etwas beiseite. 🪙+

$\hat{c}_i < c_i \Rightarrow$

Buchhaltermethode



- Verbindet mit jeder Operation op_i **amortisierte** Kosten $\hat{c}_i$, die oft nicht mit den **tatsächlichen** Kosten c_i übereinstimmen.

$\hat{c}_i > c_i \Rightarrow$ Wir legen etwas beiseite. 🪙+

$\hat{c}_i < c_i \Rightarrow$ Wir bezahlen teure Operationen mit vorher Beiseitegelegtem. 🪙-

Buchhaltermethode



- Verbindet mit jeder Operation op_i **amortisierte** Kosten $\hat{c}_i$, die oft nicht mit den **tatsächlichen** Kosten c_i übereinstimmen.

$\hat{c}_i > c_i \Rightarrow$ Wir legen etwas beiseite. 🪙+

$\hat{c}_i < c_i \Rightarrow$ Wir bezahlen teure Operationen mit vorher Beiseitegelegtem. 🪙-

- Damit's klappt: wir dürfen nie in die Miesen kommen –



Buchhaltermethode



- Verbindet mit jeder Operation op_i **amortisierte** Kosten $\hat{c}_i$, die oft nicht mit den **tatsächlichen** Kosten c_i übereinstimmen.

$\hat{c}_i > c_i \Rightarrow$ Wir legen etwas beiseite. 🪙+

$\hat{c}_i < c_i \Rightarrow$ Wir bezahlen teure Operationen mit vorher Beiseitegelegtem. 🪙-

- Damit's klappt: wir dürfen nie in die Miesen kommen –

Guthaben $\sum_{i=1}^n \hat{c}_i - \sum_{i=1}^n c_i$ darf nicht negativ werden!



Buchhaltermethode



- Verbindet mit jeder Operation op_i **amortisierte** Kosten $\hat{c}_i$, die oft nicht mit den **tatsächlichen** Kosten c_i übereinstimmen.

$\hat{c}_i > c_i \Rightarrow$ Wir legen etwas beiseite. 🪙+

$\hat{c}_i < c_i \Rightarrow$ Wir bezahlen teure Operationen mit vorher Beiseitegelegtem. 🪙-

- Damit's klappt: wir dürfen nie in die Miesen kommen –

Guthaben $\sum_{i=1}^n \hat{c}_i - \sum_{i=1}^n c_i$ darf nicht negativ werden!



Dann gilt $\sum_{i=1}^n \hat{c}_i \geq \sum_{i=1}^n c_i$.

Buchhaltermethode



- Verbindet mit jeder Operation op_i **amortisierte** Kosten $\hat{c}_i$, die oft nicht mit den **tatsächlichen** Kosten c_i übereinstimmen.

$\hat{c}_i > c_i \Rightarrow$ Wir legen etwas beiseite. 🪙+

$\hat{c}_i < c_i \Rightarrow$ Wir bezahlen teure Operationen mit vorher Beiseitegelegtem. 🪙-

- Damit's klappt: wir dürfen nie in die Miesen kommen –

Guthaben $\sum_{i=1}^n \hat{c}_i - \sum_{i=1}^n c_i$ darf nicht negativ werden!

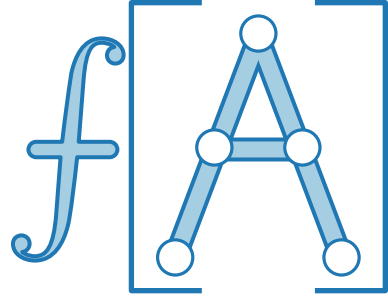


Dann gilt $\sum_{i=1}^n \hat{c}_i \geq \sum_{i=1}^n c_i$.

D.h. **amortisierte** Kosten sind obere Schranke für **tatsächliche** Kosten!

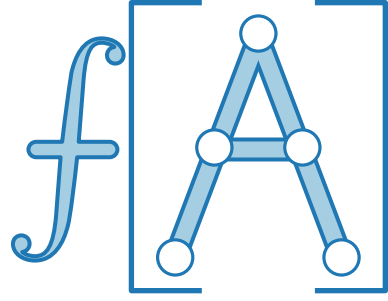
Buchhaltermethode für dynamische Tabellen

Jede Einfügeoperation op_i bezahlt $\hat{c}_i =$:



Buchhaltermethode für dynamische Tabellen

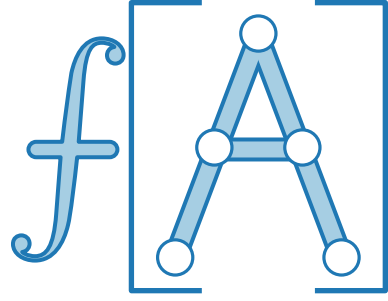
Jede Einfügeoperation op_i bezahlt $\hat{c}_i = \text{€€€}$:



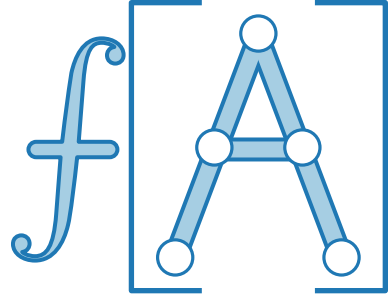
Buchhaltermethode für dynamische Tabellen

Jede Einfügeoperation op_i bezahlt $\hat{c}_i = \text{€€€}$:

■ € fürs tatsächliche Einfügen



Buchhaltermethode für dynamische Tabellen



Jede Einfügeoperation op_i bezahlt $\hat{c}_i = \text{€€€}$:

- € fürs tatsächliche Einfügen
- €€ für die nächste Tabellenvergrößerung

Buchhaltermethode für dynamische Tabellen



Jede Einfügeoperation op_i bezahlt $\hat{c}_i = \text{€€€}$:



- € fürs tatsächliche Einfügen
- €€ für die nächste Tabellenvergrößerung

Buchhaltermethode für dynamische Tabellen



Jede Einfügeoperation op_i bezahlt $\hat{c}_i = \text{€€€}$:

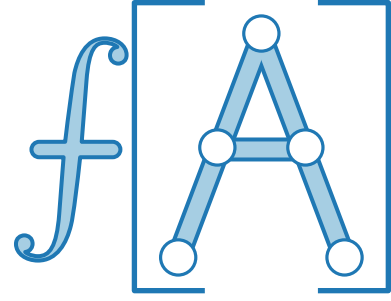
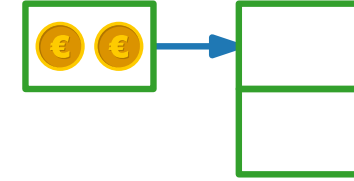


- € fürs tatsächliche Einfügen
- €€ für die nächste Tabellenvergrößerung

Buchhaltermethode für dynamische Tabellen

Jede Einfügeoperation op_i bezahlt $\hat{c}_i = \text{€€€}$:

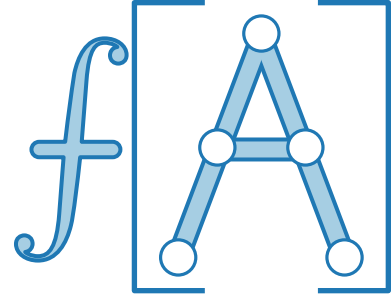
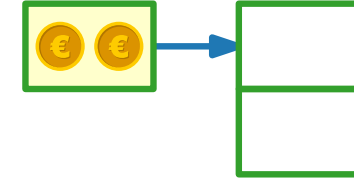
- € fürs tatsächliche Einfügen
- €€ für die nächste Tabellenvergrößerung



Buchhaltermethode für dynamische Tabellen

Jede Einfügeoperation op_i bezahlt $\hat{c}_i = \text{€€€}$:

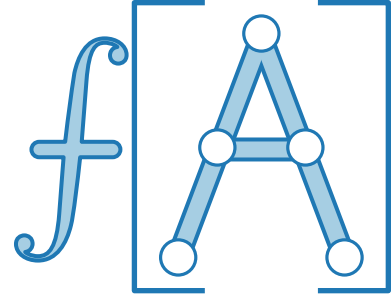
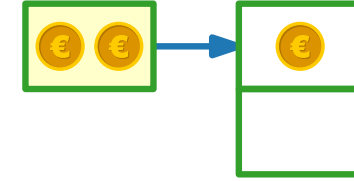
- € fürs tatsächliche Einfügen
- €€ für die nächste Tabellenvergrößerung



Buchhaltermethode für dynamische Tabellen

Jede Einfügeoperation op_i bezahlt $\hat{c}_i = \text{€€€}$:

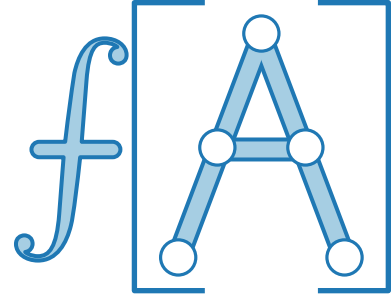
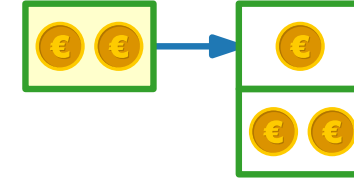
- € fürs tatsächliche Einfügen
- €€ für die nächste Tabellenvergrößerung



Buchhaltermethode für dynamische Tabellen

Jede Einfügeoperation op_i bezahlt $\hat{c}_i = \text{€€€}$:

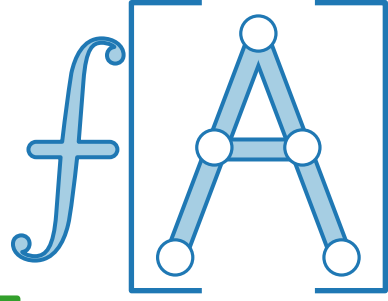
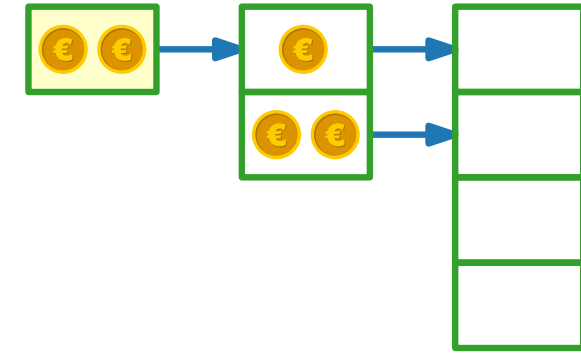
- € fürs tatsächliche Einfügen
- €€ für die nächste Tabellenvergrößerung



Buchhaltermethode für dynamische Tabellen

Jede Einfügeoperation op_i bezahlt $\hat{c}_i = \text{€€€}$:

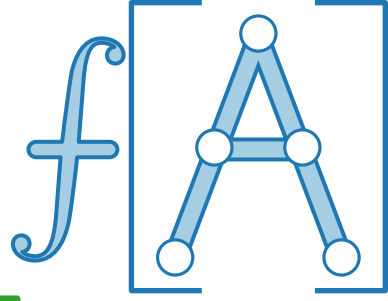
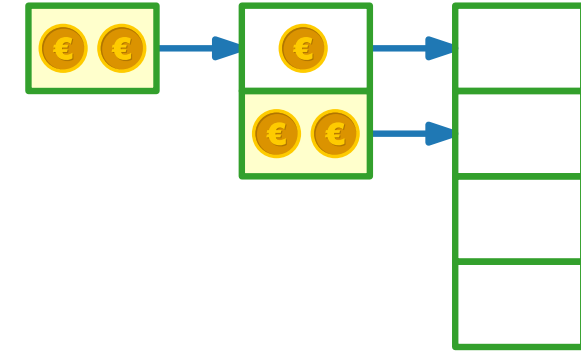
- € fürs tatsächliche Einfügen
- €€ für die nächste Tabellenvergrößerung



Buchhaltermethode für dynamische Tabellen

Jede Einfügeoperation op_i bezahlt $\hat{c}_i = \text{€€€}$:

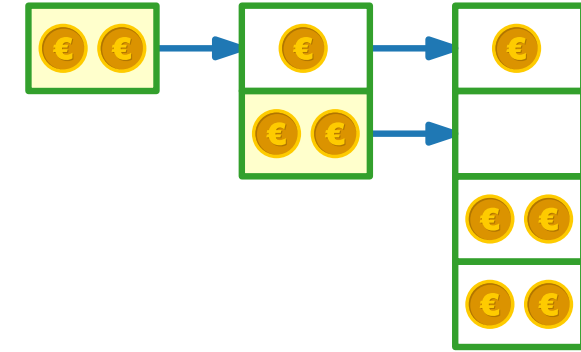
- € fürs tatsächliche Einfügen
- €€ für die nächste Tabellenvergrößerung



Buchhaltermethode für dynamische Tabellen

Jede Einfügeoperation op_i bezahlt $\hat{c}_i = \text{€€€}$:

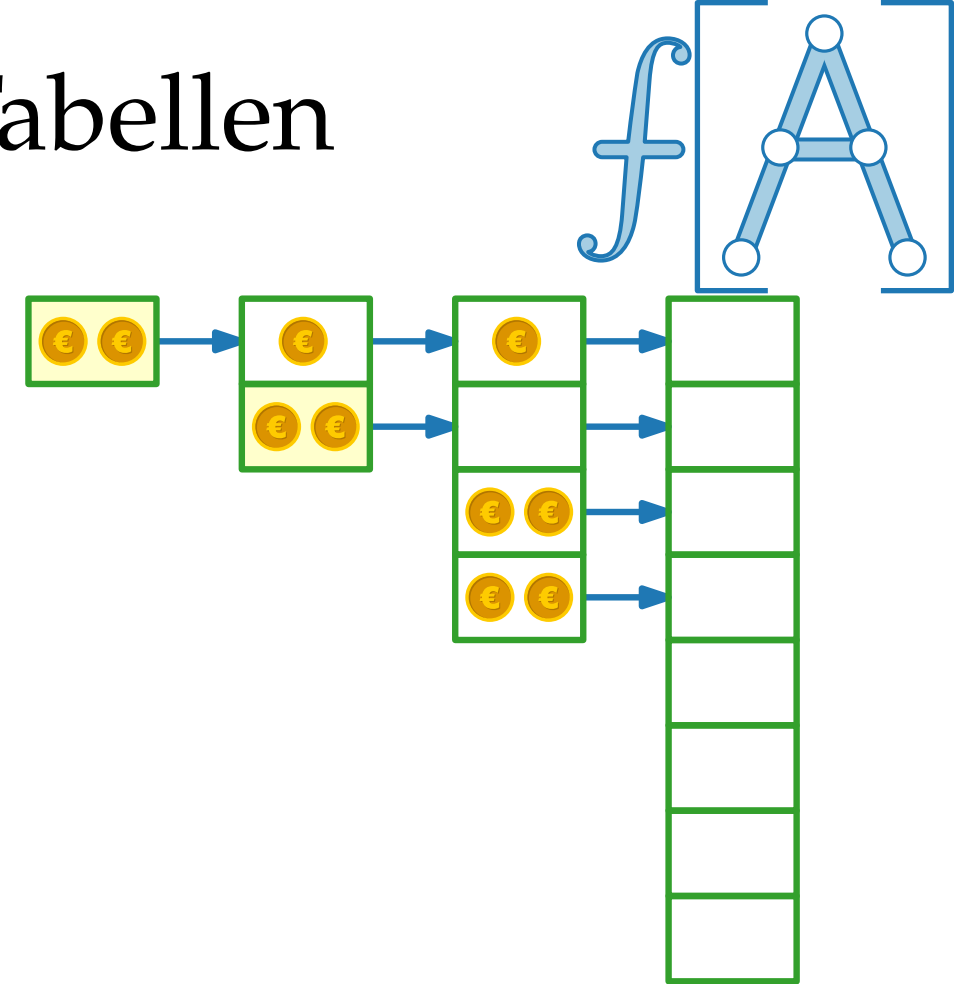
- € fürs tatsächliche Einfügen
- €€ für die nächste Tabellenvergrößerung



Buchhaltermethode für dynamische Tabellen

Jede Einfügeoperation op_i bezahlt $\hat{c}_i = \text{€€€}$:

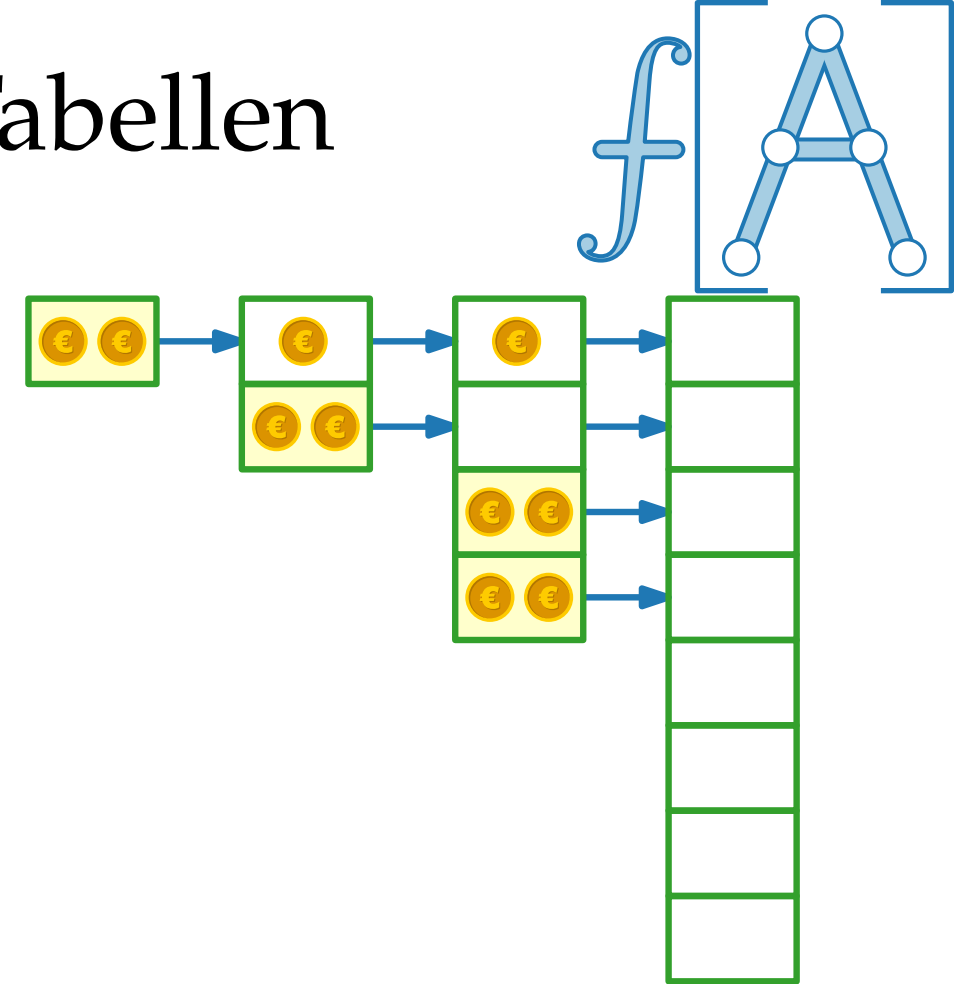
- € fürs tatsächliche Einfügen
- €€ für die nächste Tabellenvergrößerung



Buchhaltermethode für dynamische Tabellen

Jede Einfügeoperation op_i bezahlt $\hat{c}_i = \text{€€€}$:

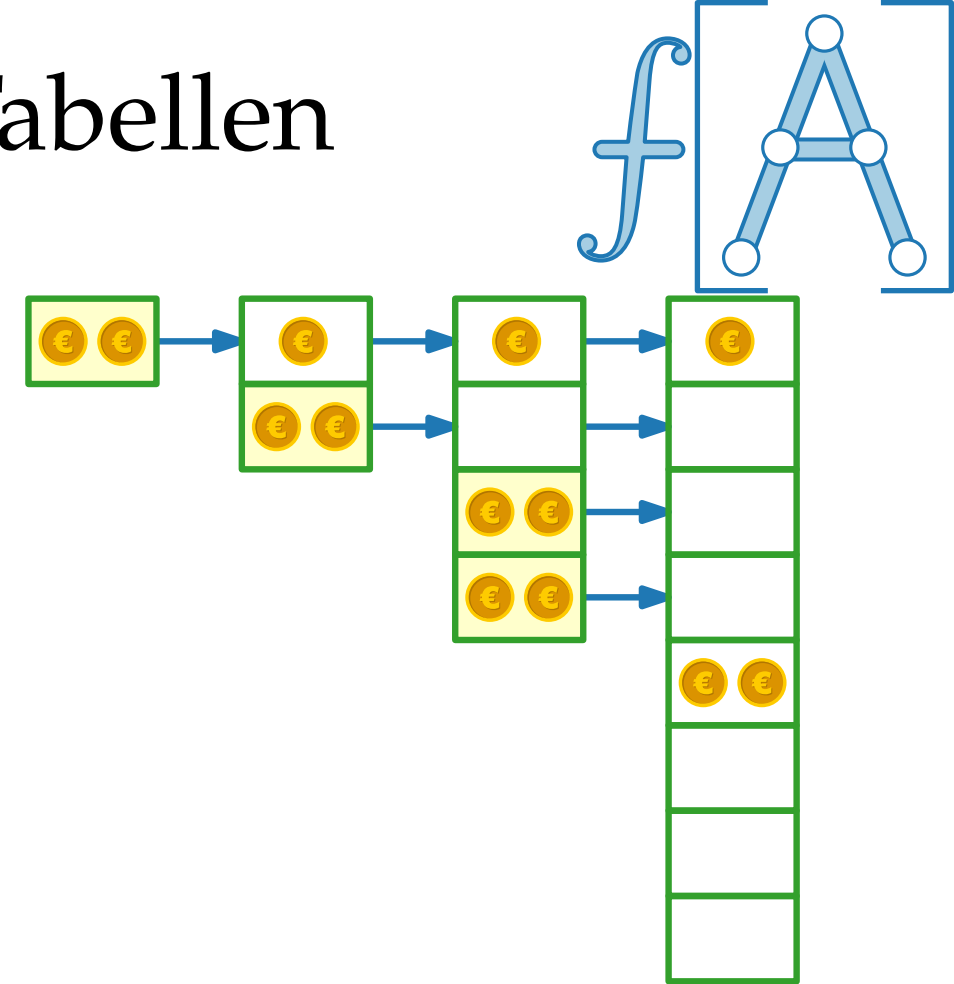
- € fürs tatsächliche Einfügen
- €€ für die nächste Tabellenvergrößerung



Buchhaltermethode für dynamische Tabellen

Jede Einfügeoperation op_i bezahlt $\hat{c}_i = \text{€€€}$:

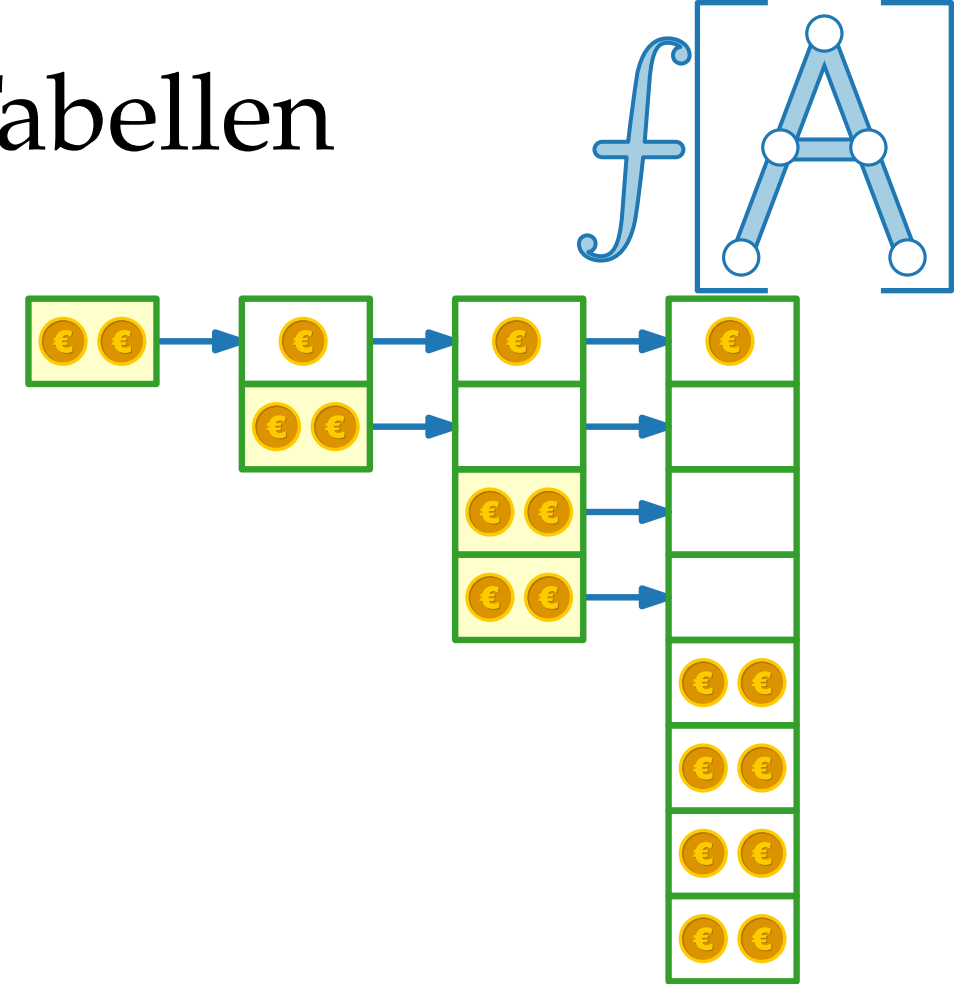
- € fürs tatsächliche Einfügen
- €€ für die nächste Tabellenvergrößerung



Buchhaltermethode für dynamische Tabellen

Jede Einfügeoperation op_i bezahlt $\hat{c}_i = \text{€€€}$:

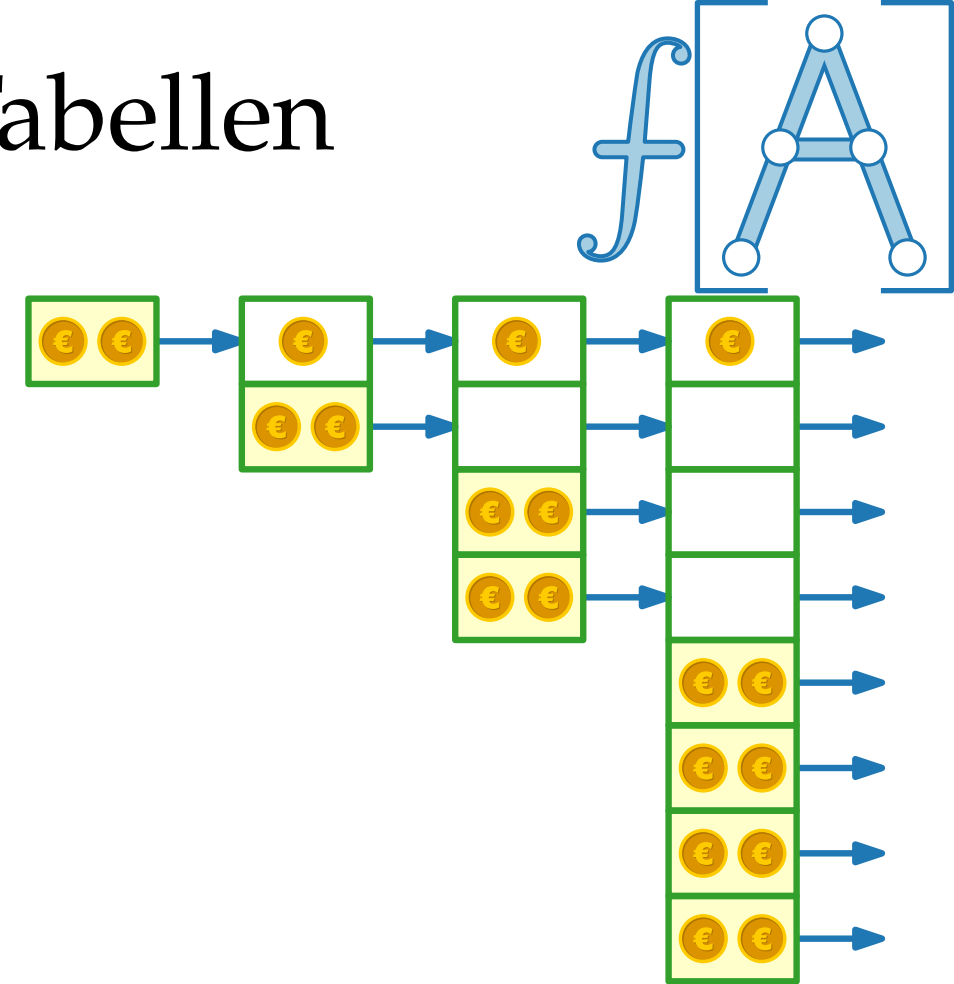
- € fürs tatsächliche Einfügen
- €€ für die nächste Tabellenvergrößerung



Buchhaltermethode für dynamische Tabellen

Jede Einfügeoperation op_i bezahlt $\hat{c}_i = \text{€€€}$:

- € fürs tatsächliche Einfügen
- €€ für die nächste Tabellenvergrößerung

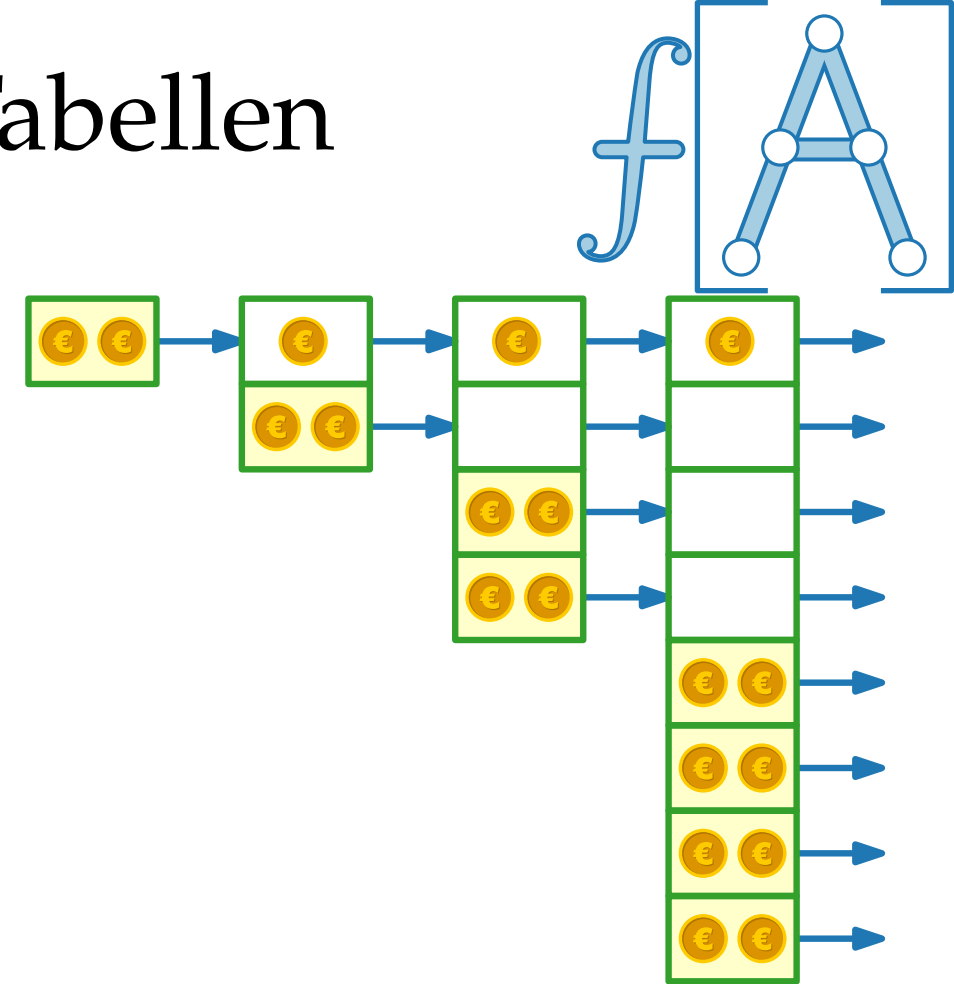


Buchhaltermethode für dynamische Tabellen

Jede Einfügeoperation op_i bezahlt $\hat{c}_i = \text{€€€}$:

- € fürs tatsächliche Einfügen
- €€ für die nächste Tabellenvergrößerung

Wir verknüpfen die Teilguthaben mit konkreten Objekten der Datenstruktur.



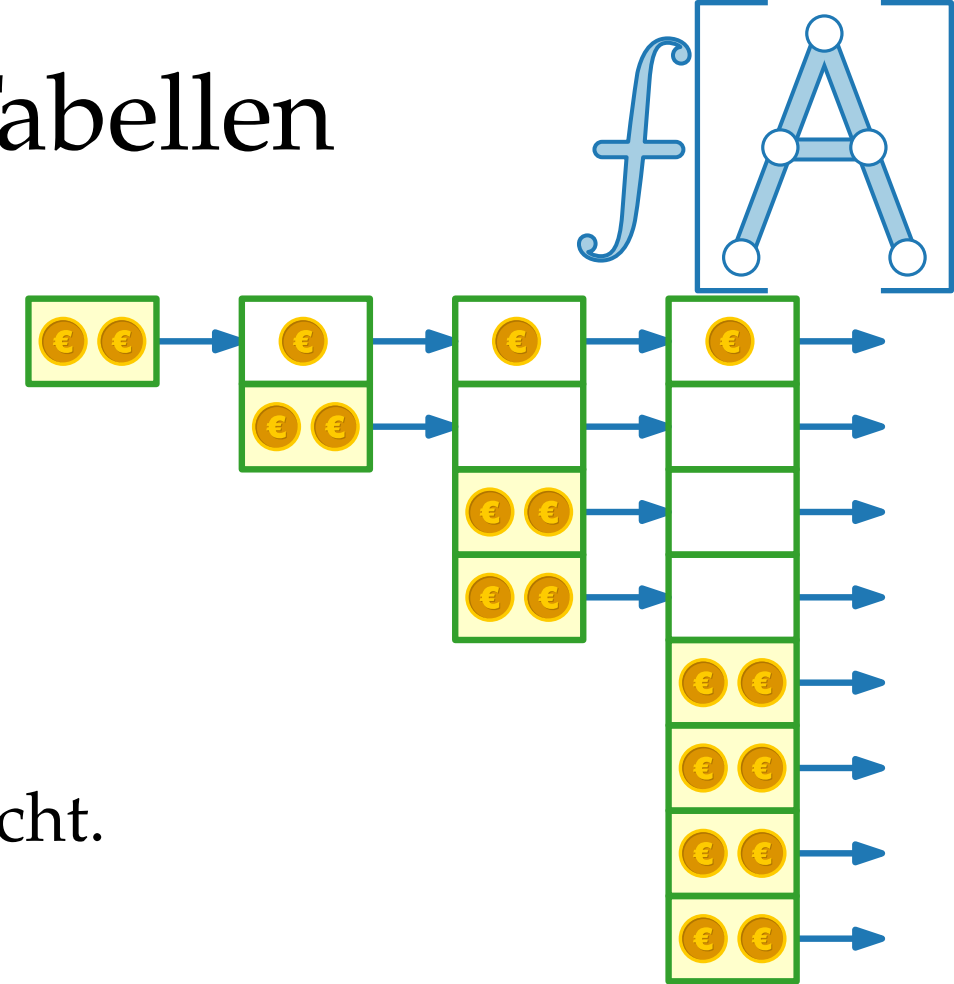
Buchhaltermethode für dynamische Tabellen

Jede Einfügeoperation op_i bezahlt $\hat{c}_i = \text{€€€}$:

- € fürs tatsächliche Einfügen
- €€ für die nächste Tabellenvergrößerung

Wir verknüpfen die Teilguthaben mit konkreten Objekten der Datenstruktur.

Damit wird deutlich, dass die Datenstruktur nie Miese macht.



Buchhaltermethode für dynamische Tabellen

Jede Einfügeoperation op_i bezahlt $\hat{c}_i = \text{€€€}$:

- € fürs tatsächliche Einfügen
- €€ für die nächste Tabellenvergrößerung

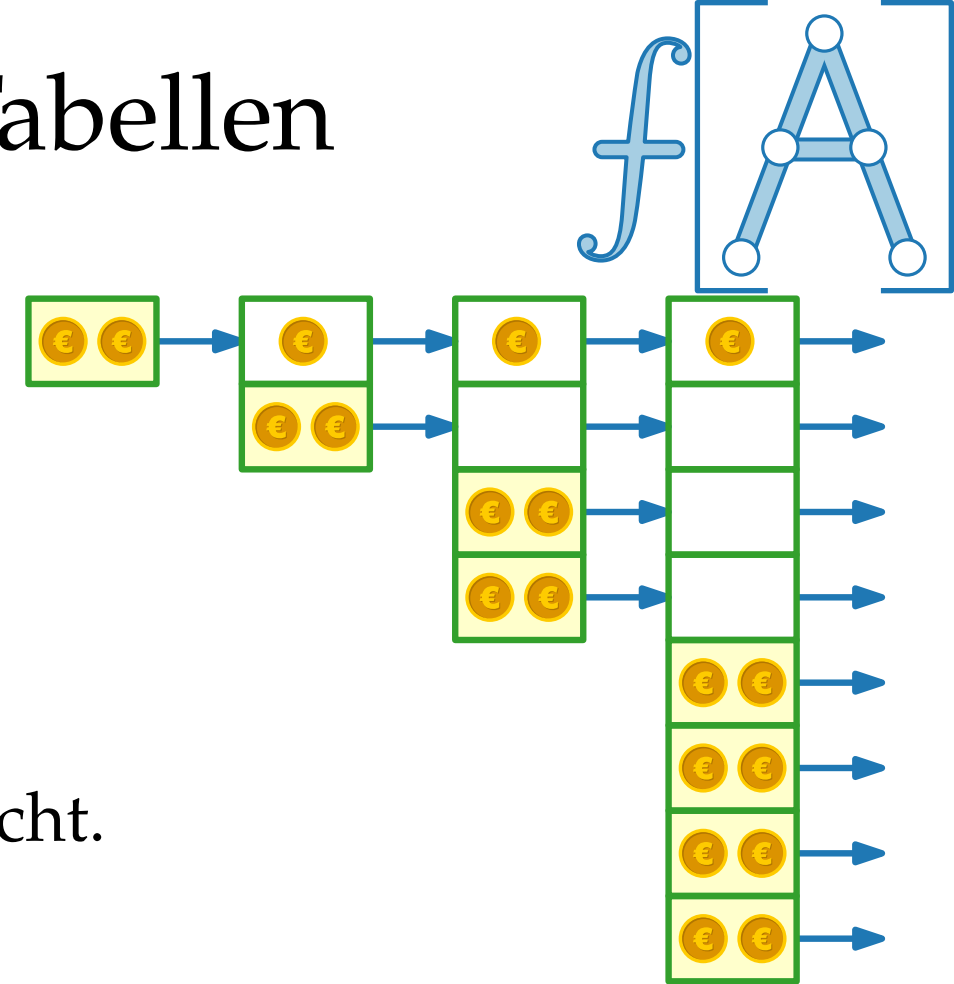
Wir verknüpfen die Teilguthaben mit konkreten Objekten der Datenstruktur.

Damit wird deutlich, dass die Datenstruktur nie Miese macht.



D.h. **amortisierte** Kosten
sind obere Schranke für
tatsächliche Kosten!

$$\sum_{i=1}^n c_i \leq \sum_{i=1}^n \hat{c}_i$$



Buchhaltermethode für dynamische Tabellen

Jede Einfügeoperation op_i bezahlt $\hat{c}_i = \text{€€€}$:

- € fürs tatsächliche Einfügen
- €€ für die nächste Tabellenvergrößerung

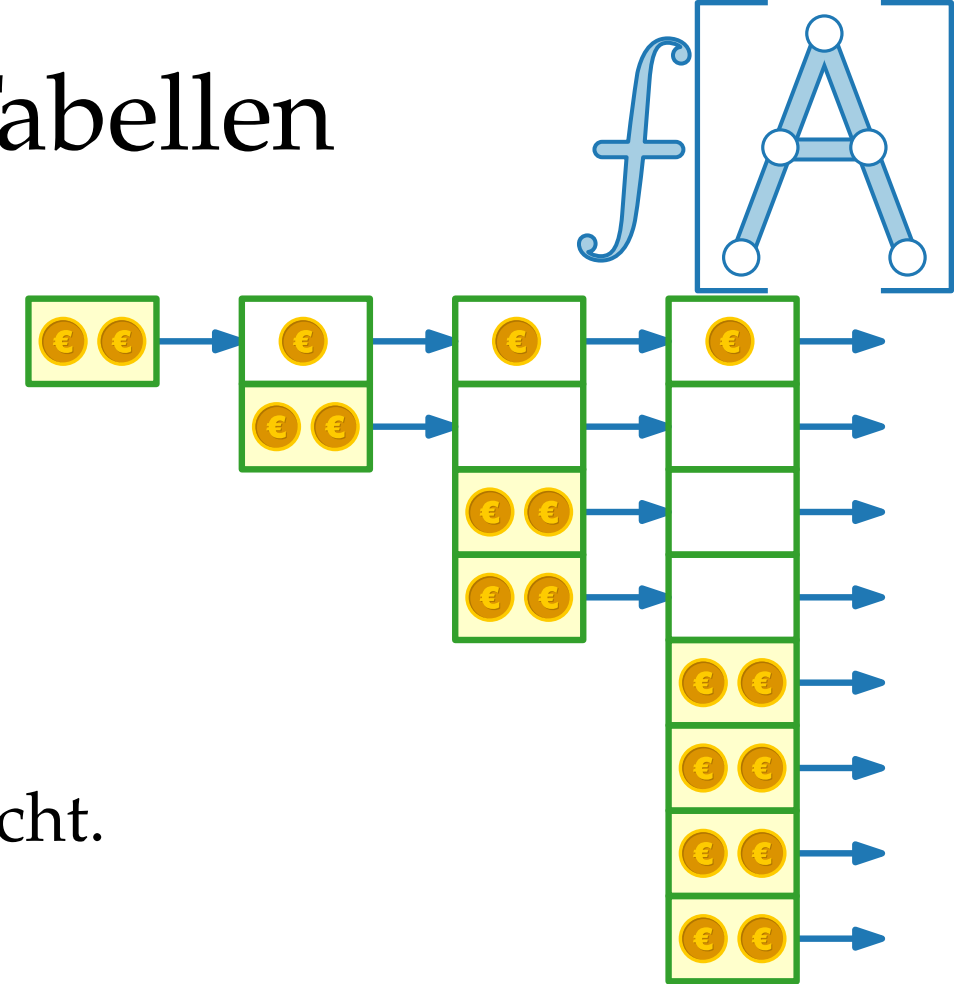
Wir verknüpfen die Teilguthaben mit konkreten Objekten der Datenstruktur.

Damit wird deutlich, dass die Datenstruktur nie Miese macht.



D.h. **amortisierte** Kosten
sind obere Schranke für
tatsächliche Kosten!

$$\sum_{i=1}^n c_i \leq \sum_{i=1}^n \hat{c}_i = 3n$$



Buchhaltermethode für dynamische Tabellen

Jede Einfügeoperation op_i bezahlt $\hat{c}_i = \text{€€€}$:

- € fürs tatsächliche Einfügen
- €€ für die nächste Tabellenvergrößerung

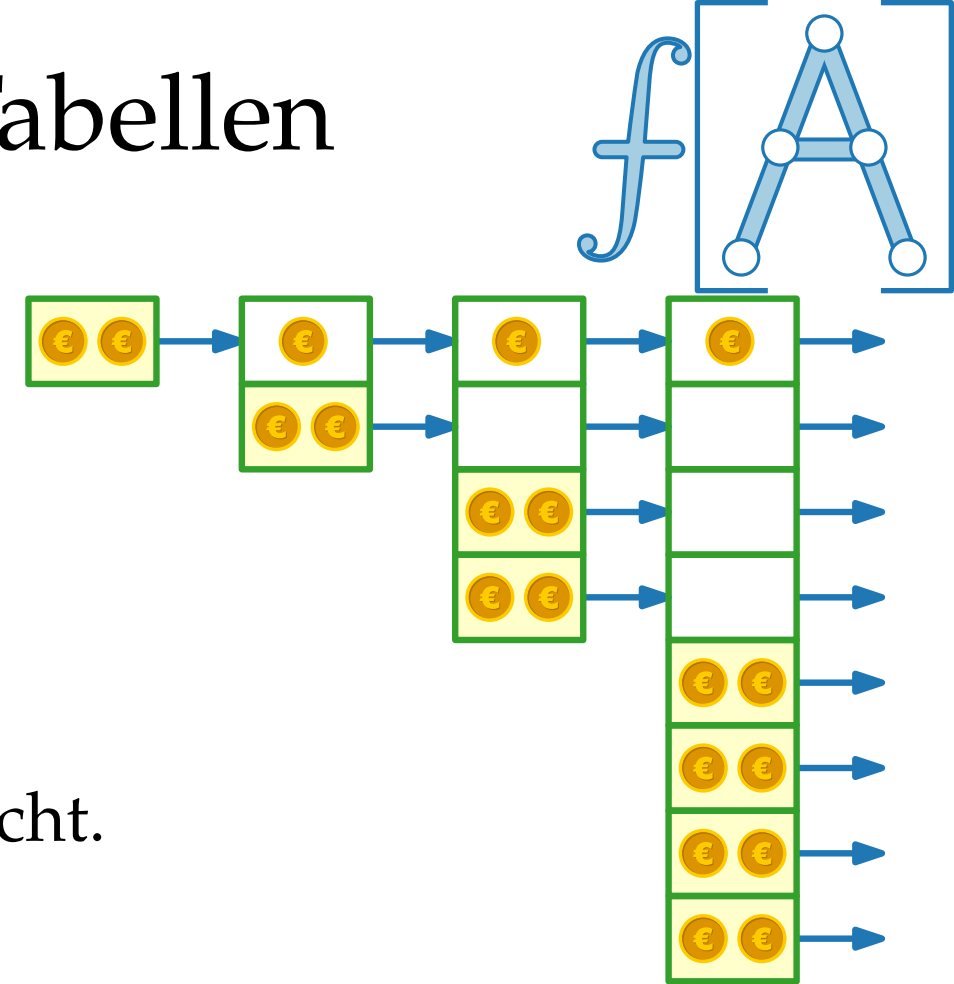
Wir verknüpfen die Teilguthaben mit konkreten Objekten der Datenstruktur.

Damit wird deutlich, dass die Datenstruktur nie Miese macht.

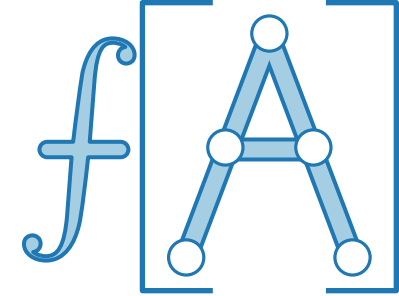


D.h. **amortisierte** Kosten
sind obere Schranke für
tatsächliche Kosten!

$$\sum_{i=1}^n c_i \leq \sum_{i=1}^n \hat{c}_i = 3n = \Theta(n)$$

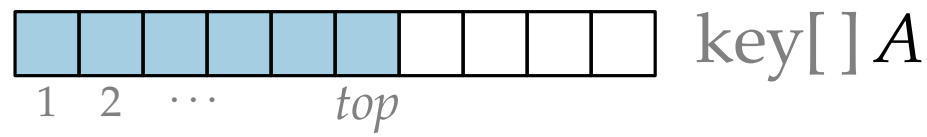


Buchhaltermethode für Stapel mit MULTIPOP

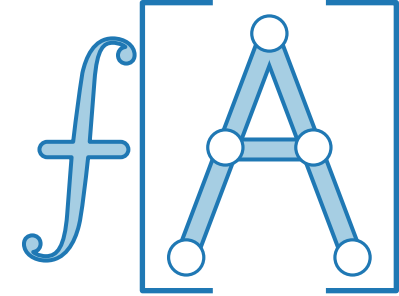


verwaltet sich ändernde Menge nach *LIFO-Prinzip*

Operation	Implementierung
Stack(int <i>n</i>)	
boolean EMPTY()	
PUSH(key <i>k</i>)	
key POP()	
key TOP()	



Buchhaltermethode für Stapel mit MULTIPOP

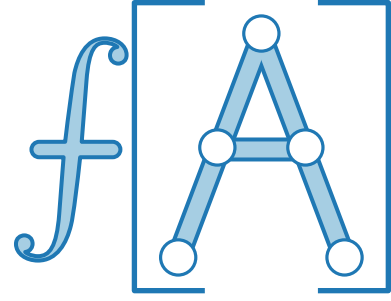


verwaltet sich ändernde Menge nach *LIFO-Prinzip*

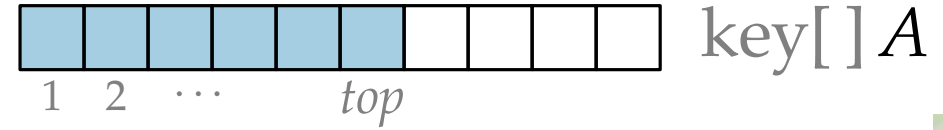
Operation	Implementierung
Stack(int <i>n</i>) boolean EMPTY() PUSH(key <i>k</i>) key POP() key TOP()	12...top key[] A
key[] MULTIPOP(int <i>k</i>) neu!	



Buchhaltermethode für Stapel mit MULTIPOP



verwaltet sich ändernde Menge nach *LIFO-Prinzip*



Operation

Implementierung

Stack(int n)
boolean EMPTY()
PUSH(key k)
key POP()
key TOP()

key[] MULTIPOP(int k)

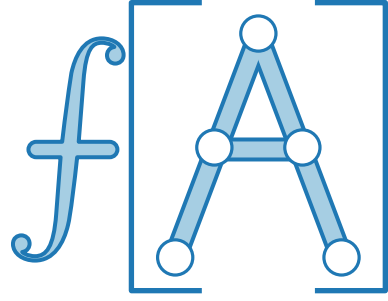
neu!

```
 $B = \text{new key}[k]$   
while  $k > 0$  and not EMPTY() do  
     $B[k] = \text{POP}()$   
     $k = k - 1$   
return  $B$ 
```



Buchhaltermethode für Stapel mit MULTIPOP

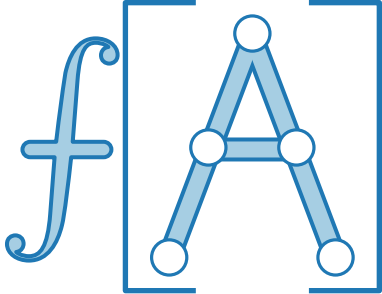
Betrachte Folge von PUSH-, POP- und MULTIPOP-Operationen.



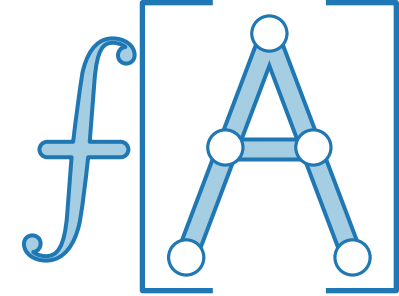
Buchhaltermethode für Stapel mit MULTIPOP

Betrachte Folge von PUSH-, POP- und MULTIPOP-Operationen.

Operation i	tatsächliche Kosten c_i	amortisierte Kosten $\hat{c}_i$
PUSH		
POP		
MULTIPOP(k_i)		



Buchhaltermethode für Stapel mit MULTIPOP

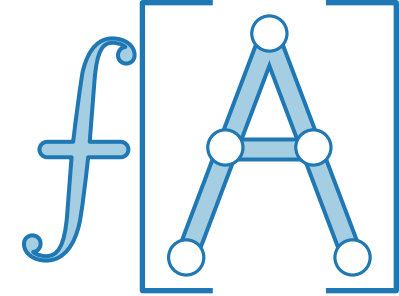


Betrachte Folge von PUSH-, POP- und MULTIPOP-Operationen.

Operation i	tatsächliche Kosten c_i	amortisierte Kosten $\hat{c}_i$
PUSH		
POP		
MULTIPOP(k_i)		



Buchhaltermethode für Stapel mit MULTIPOP

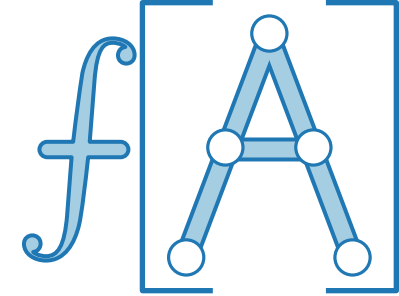


Betrachte Folge von PUSH-, POP- und MULTIPOP-Operationen.

Operation i	tatsächliche Kosten c_i	amortisierte Kosten $\hat{c}_i$
PUSH	€	
POP	€	
MULTIPOP(k_i)		



Buchhaltermethode für Stapel mit MULTIPOP

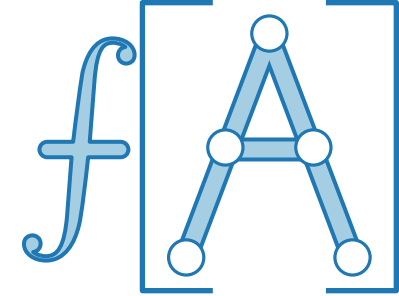


Betrachte Folge von PUSH-, POP- und MULTIPOP-Operationen.

Operation i	tatsächliche Kosten c_i	amortisierte Kosten $\hat{c}_i$
PUSH	€	
POP	€	
MULTIPOP(k_i)	k_i	

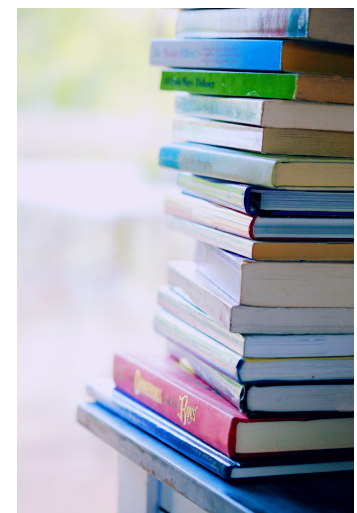


Buchhaltermethode für Stapel mit MULTIPOP

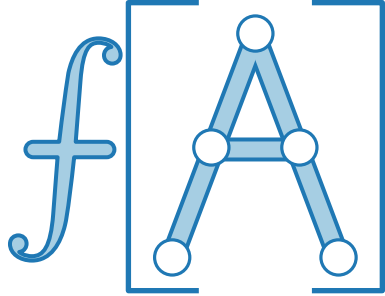


Betrachte Folge von PUSH-, POP- und MULTIPOP-Operationen.

Operation i	tatsächliche Kosten c_i	amortisierte Kosten $\hat{c}_i$
PUSH	€	
POP	€	
MULTIPOP(k_i)	$\min\{k_i, size_i\}$	



Buchhaltermethode für Stapel mit MULTIPOP

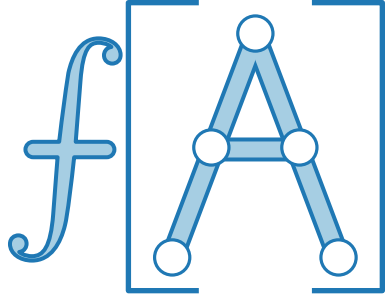


Betrachte Folge von PUSH-, POP- und MULTIPOP-Operationen.

Operation i	tatsächliche Kosten c_i	amortisierte Kosten $\hat{c}_i$
PUSH	€	€ €
POP	€	0
MULTIPOP(k_i)	$\min\{k_i, size_i\}$	0



Buchhaltermethode für Stapel mit MULTIPOP



Betrachte Folge von PUSH-, POP- und MULTIPOP-Operationen.

Operation i	tatsächliche Kosten c_i	amortisierte Kosten $\hat{c}_i$
PUSH	€	€ €
POP	€	0
MULTIPOP(k_i)	$\min\{k_i, size_i\}$	0

Geht das gut?



Buchhaltermethode für Stapel mit MULTIPOP



Betrachte Folge von PUSH-, POP- und MULTIPOP-Operationen.

Operation i	tatsächliche Kosten c_i	amortisierte Kosten $\hat{c}_i$
PUSH	€	€ €
POP	€	0
MULTIPOP(k_i)	$\min\{k_i, size_i\}$	0

Geht das gut?

Zeige: Amortisierte Kosten „bezahlen“ immer für die **tatsächlichen**!



Buchhaltermethode für Stapel mit MULTIPOP



Betrachte Folge von PUSH-, POP- und MULTIPOP-Operationen.

Operation i	tatsächliche Kosten c_i	amortisierte Kosten $\hat{c}_i$
PUSH	€	€ €
POP	€	0
MULTIPOP(k_i)	$\min\{k_i, size_i\}$	0

Geht das gut?

Zeige: Amortisierte Kosten „bezahlen“ immer für die **tatsächlichen**!

- Jede PUSH-Operation legt ein Buch auf den Stapel.



Buchhaltermethode für Stapel mit MULTIPOP



Betrachte Folge von PUSH-, POP- und MULTIPOP-Operationen.

Operation i	tatsächliche Kosten c_i	amortisierte Kosten $\hat{c}_i$
PUSH	€	€ €
POP	€	0
MULTIPOP(k_i)	$\min\{k_i, \text{size}_i\}$	0

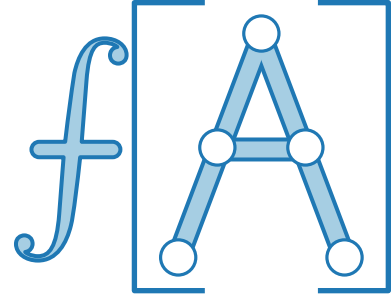
Geht das gut?



Zeige: Amortisierte Kosten „bezahlen“ immer für die **tatsächlichen**!

- Jede PUSH-Operation legt ein Buch auf den Stapel.
Dafür bezahlt sie € und legt noch € in das Buch.

Buchhaltermethode für Stapel mit MULTIPOP



Betrachte Folge von PUSH-, POP- und MULTIPOP-Operationen.

Operation i	tatsächliche Kosten c_i	amortisierte Kosten $\hat{c}_i$
PUSH	€	€ €
POP	€	0
MULTIPOP(k_i)	$\min\{k_i, \text{size}_i\}$	0

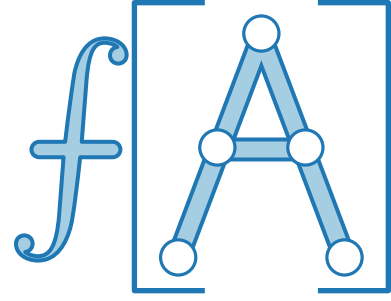


Geht das gut?

Zeige: Amortisierte Kosten „bezahlen“ immer für die **tatsächlichen**!

- Jede PUSH-Operation legt ein Buch auf den Stapel.
Dafür bezahlt sie € und legt noch € in das Buch.
- Jede (MULTI-)POP-Operation wird mit den Euros in den Büchern, die sie wegnimmt, komplett bezahlt.

Buchhaltermethode für Stapel mit MULTIPOP



Betrachte Folge von PUSH-, POP- und MULTIPOP-Operationen.

Operation i	tatsächliche Kosten c_i	amortisierte Kosten $\hat{c}_i$
PUSH	€	€ €
POP	€	0
MULTIPOP(k_i)	$\min\{k_i, \text{size}_i\}$	0

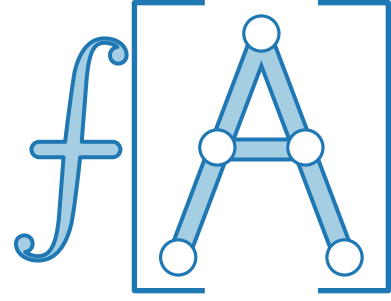
Geht das gut? – Ja!



Zeige: Amortisierte Kosten „bezahlen“ immer für die **tatsächlichen**!

- Jede PUSH-Operation legt ein Buch auf den Stapel.
Dafür bezahlt sie € und legt noch € in das Buch.
- Jede (MULTI-)POP-Operation wird mit den Euros in den Büchern, die sie wegnimmt, komplett bezahlt.

Buchhaltermethode für Stapel mit MULTIPOP



Betrachte Folge von PUSH-, POP- und MULTIPOP-Operationen.

Operation i	tatsächliche Kosten c_i	amortisierte Kosten $\hat{c}_i$
PUSH	€	€ €
POP	€	0
MULTIPOP(k_i)	$\min\{k_i, \text{size}_i\}$	0

Geht das gut? – Ja! D.h. Folge von n Operationen dauert $\Theta(n)$ Zeit.



Zeige: Amortisierte Kosten „bezahlen“ immer für die **tatsächlichen**!

- Jede PUSH-Operation legt ein Buch auf den Stapel.
Dafür bezahlt sie € und legt noch € in das Buch.
- Jede (MULTI-)POP-Operation wird mit den Euros in den Büchern, die sie wegnimmt, komplett bezahlt.

Amortisierte Analyse



...bedeutet zu zeigen, dass die Operationen einer gegebenen Folge kleine durchschnittliche Kosten haben –
auch wenn einzelne Operationen in der Folge teuer sind!

Auch *randomisierte Analyse* kann man als Durchschnittsbildung (über alle mögl. Ergebnisse, gewichtet nach Wahrscheinlichkeit) betrachten.

Bei amortisierter Analyse geht es jedoch um die durchschnittliche Laufzeit *im schlechtesten Fall* – nicht im Erwartungswert!

Die **amortisierte** Laufzeit einer Methode ist f , wenn jede Sequenz von k Aufrufen der Methode Laufzeit $\leq k \cdot f$ hat (wenn k groß genug ist).

Wir betrachten 3 verschiedene Typen von amortisierter Analyse:

- Aggregationsmethode ✓
- Buchhaltermethode ✓
- Potentialmethode

Amortisierte Analyse



...bedeutet zu zeigen, dass die Operationen einer gegebenen Folge kleine durchschnittliche Kosten haben –
auch wenn einzelne Operationen in der Folge teuer sind!

Auch *randomisierte Analyse* kann man als Durchschnittsbildung (über alle mögl. Ergebnisse, gewichtet nach Wahrscheinlichkeit) betrachten.

Bei amortisierter Analyse geht es jedoch um die durchschnittliche Laufzeit *im schlechtesten Fall* – nicht im Erwartungswert!

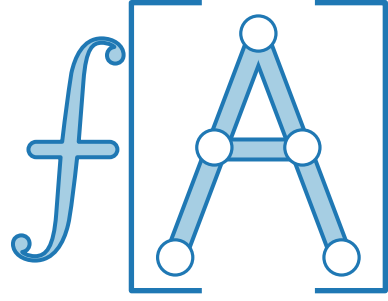
Die **amortisierte** Laufzeit einer Methode ist f , wenn jede Sequenz von k Aufrufen der Methode Laufzeit $\leq k \cdot f$ hat (wenn k groß genug ist).

Wir betrachten 3 verschiedene Typen von amortisierter Analyse:

- Aggregationsmethode ✓
- Buchhaltermethode ✓
- Potentialmethode

Potentialmethode

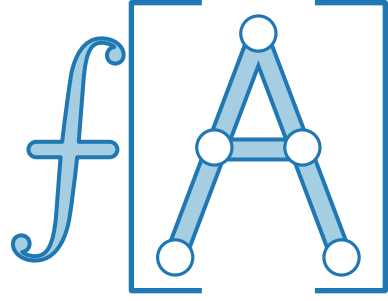
Idee. Betrachte Bankguthaben (siehe Buchhaltermethode)
als physikalische Größe,
die den augenblicklichen Zustand der DS beschreibt.



Potentialmethode

Idee. Betrachte Bankguthaben (siehe Buchhaltermethode)
als physikalische Größe,
die den augenblicklichen Zustand der DS beschreibt.

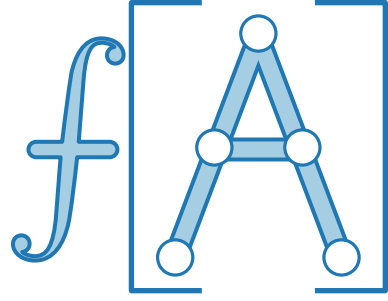
Datenstruktur $D_0 \longrightarrow D_1 \longrightarrow \dots \longrightarrow D_n$



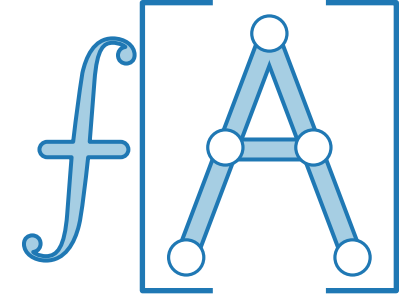
Potentialmethode

Idee. Betrachte Bankguthaben (siehe Buchhaltermethode)
als physikalische Größe,
die den augenblicklichen Zustand der DS beschreibt.

Datenstruktur $D_0 \xrightarrow{op_1} D_1 \xrightarrow{op_2} \dots \xrightarrow{op_n} D_n$



Potentialmethode

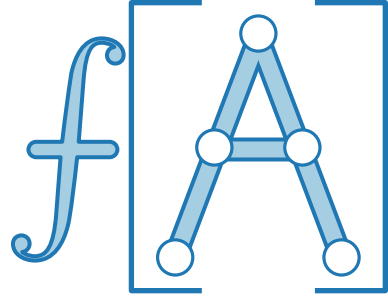


Idee. Betrachte Bankguthaben (siehe Buchhaltermethode)
als physikalische Größe,
die den augenblicklichen Zustand der DS beschreibt.

Datenstruktur $D_0 \xrightarrow{op_1} D_1 \xrightarrow{op_2} \dots \xrightarrow{op_n} D_n$

Wähle Potential $\Phi: D_i \rightarrow \mathbb{R}$.

Potentialmethode

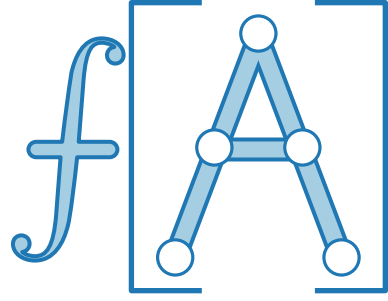


Idee. Betrachte Bankguthaben (siehe Buchhaltermethode)
als physikalische Größe,
die den augenblicklichen Zustand der DS beschreibt.

Datenstruktur $D_0 \xrightarrow{op_1} D_1 \xrightarrow{op_2} \dots \xrightarrow{op_n} D_n$

Wähle Potential $\Phi: D_i \rightarrow \mathbb{R}$. O.B.d.A. $\Phi(D_0) = 0$

Potentialmethode



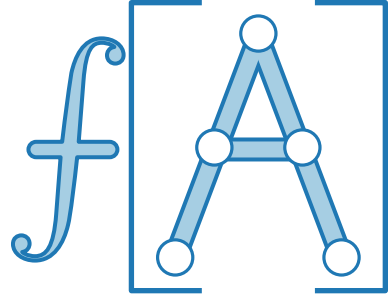
Idee. Betrachte Bankguthaben (siehe Buchhaltermethode)
als physikalische Größe,
die den augenblicklichen Zustand der DS beschreibt.

Datenstruktur $D_0 \xrightarrow{op_1} D_1 \xrightarrow{op_2} \dots \xrightarrow{op_n} D_n$

Wähle Potential $\Phi: D_i \rightarrow \mathbb{R}$. O.B.d.A. $\Phi(D_0) = 0$

Ziel. Bank macht keine Miesen.

Potentialmethode



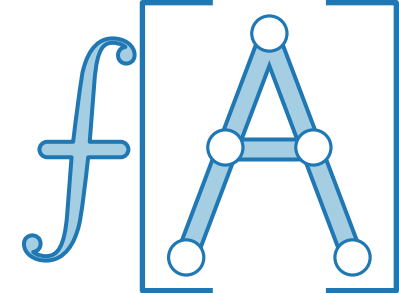
Idee. Betrachte Bankguthaben (siehe Buchhaltermethode)
als physikalische Größe,
die den augenblicklichen Zustand der DS beschreibt.

Datenstruktur $D_0 \xrightarrow{op_1} D_1 \xrightarrow{op_2} \dots \xrightarrow{op_n} D_n$

Wähle Potential $\Phi: D_i \rightarrow \mathbb{R}$. O.B.d.A. $\Phi(D_0) = 0$

Ziel. Bank macht keine Miesen.

Also fordern wir $\Phi(D_i) \geq 0$ für $i = 1, \dots, n$.



Potentialmethode

Idee. Betrachte Bankguthaben (siehe Buchhaltermethode)
als physikalische Größe,
die den augenblicklichen Zustand der DS beschreibt.

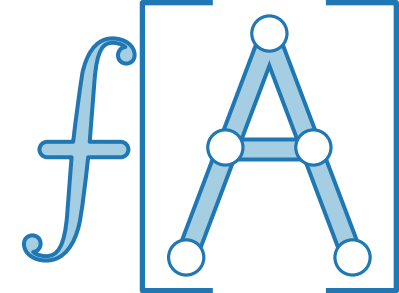
Datenstruktur $D_0 \xrightarrow{op_1} D_1 \xrightarrow{op_2} \dots \xrightarrow{op_n} D_n$

Wähle Potential $\Phi: D_i \rightarrow \mathbb{R}$. O.B.d.A. $\Phi(D_0) = 0$

Ziel. Bank macht keine Miesen.

Also fordern wir $\Phi(D_i) \geq 0$ für $i = 1, \dots, n$.

Def. $\hat{c}_i = c_i + \Delta\Phi(D_i)$



Potentialmethode

Idee. Betrachte Bankguthaben (siehe Buchhaltermethode)
als physikalische Größe,
die den augenblicklichen Zustand der DS beschreibt.

Datenstruktur $D_0 \xrightarrow{op_1} D_1 \xrightarrow{op_2} \dots \xrightarrow{op_n} D_n$

Wähle Potential $\Phi: D_i \rightarrow \mathbb{R}$. O.B.d.A. $\Phi(D_0) = 0$

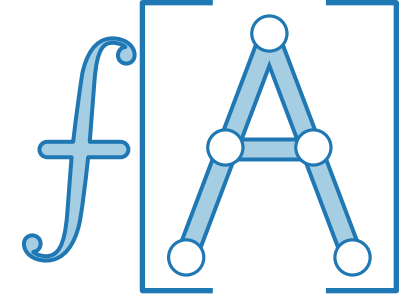
Ziel. Bank macht keine Miesen.

Also fordern wir $\Phi(D_i) \geq 0$ für $i = 1, \dots, n$.

amortisierte
Kosten

echte Kosten

Def. $\hat{c}_i = c_i + \Delta\Phi(D_i)$



Potentialmethode

Idee. Betrachte Bankguthaben (siehe Buchhaltermethode)
als physikalische Größe,
die den augenblicklichen Zustand der DS beschreibt.

Datenstruktur $D_0 \xrightarrow{op_1} D_1 \xrightarrow{op_2} \dots \xrightarrow{op_n} D_n$

Wähle Potential $\Phi: D_i \rightarrow \mathbb{R}$. O.B.d.A. $\Phi(D_0) = 0$

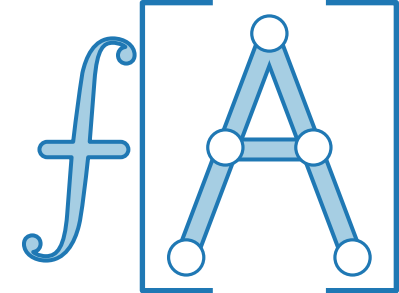
Ziel. Bank macht keine Miesen.

Also fordern wir $\Phi(D_i) \geq 0$ für $i = 1, \dots, n$.

amortisierte
Kosten

echte Kosten

Def. $\hat{c}_i = c_i + \Delta\Phi(D_i)$, wobei $\Delta\Phi(D_i) = \Phi(D_i) - \Phi(D_{i-1})$



Potentialmethode

Idee. Betrachte Bankguthaben (siehe Buchhaltermethode)
als physikalische Größe,
die den augenblicklichen Zustand der DS beschreibt.

Datenstruktur $D_0 \xrightarrow{op_1} D_1 \xrightarrow{op_2} \dots \xrightarrow{op_n} D_n$

Wähle Potential $\Phi: D_i \rightarrow \mathbb{R}$. O.B.d.A. $\Phi(D_0) = 0$

Ziel. Bank macht keine Miesen.

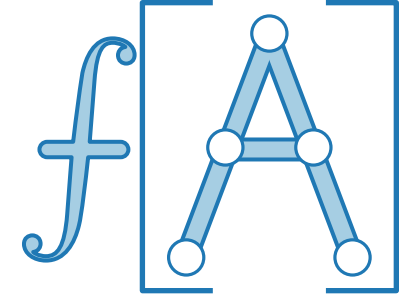
Also fordern wir $\Phi(D_i) \geq 0$ für $i = 1, \dots, n$.

amortisierte
Kosten

echte Kosten

Potentialdifferenz

Def. $\hat{c}_i = c_i + \Delta\Phi(D_i)$, wobei $\Delta\Phi(D_i) = \Phi(D_i) - \Phi(D_{i-1})$



Potentialmethode

Idee. Betrachte Bankguthaben (siehe Buchhaltermethode)
als physikalische Größe,
die den augenblicklichen Zustand der DS beschreibt.

Datenstruktur $D_0 \xrightarrow{op_1} D_1 \xrightarrow{op_2} \dots \xrightarrow{op_n} D_n$

Wähle Potential $\Phi: D_i \rightarrow \mathbb{R}$. O.B.d.A. $\Phi(D_0) = 0$

Ziel. Bank macht keine Miesen.

Also fordern wir $\Phi(D_i) \geq 0$ für $i = 1, \dots, n$.

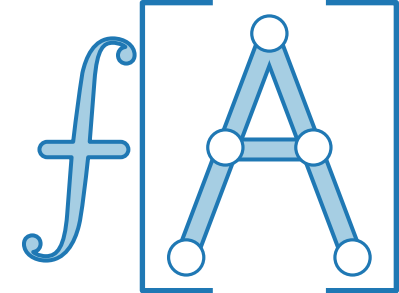
amortisierte
Kosten

echte Kosten

Potentialdifferenz

Def. $\hat{c}_i = c_i + \Delta\Phi(D_i)$, wobei $\Delta\Phi(D_i) = \Phi(D_i) - \Phi(D_{i-1})$

Folge: $\sum_{i=1}^n \hat{c}_i =$



Potentialmethode

Idee. Betrachte Bankguthaben (siehe Buchhaltermethode)
als physikalische Größe,
 die den augenblicklichen Zustand der DS beschreibt.

Datenstruktur $D_0 \xrightarrow{op_1} D_1 \xrightarrow{op_2} \dots \xrightarrow{op_n} D_n$

Wähle Potential $\Phi: D_i \rightarrow \mathbb{R}$. O.B.d.A. $\Phi(D_0) = 0$

Ziel. Bank macht keine Miesen.

Also fordern wir $\Phi(D_i) \geq 0$ für $i = 1, \dots, n$.

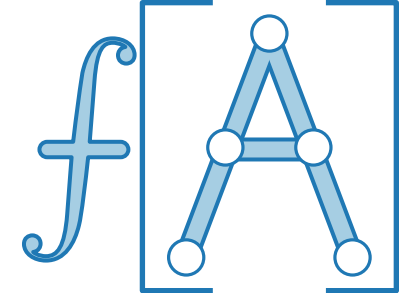
amortisierte
Kosten

echte Kosten

Potentialdifferenz

Def. $\hat{c}_i = c_i + \Delta\Phi(D_i)$, wobei $\Delta\Phi(D_i) = \Phi(D_i) - \Phi(D_{i-1})$

Folge: $\sum_{i=1}^n \hat{c}_i = \sum_{i=1}^n ($ [redacted] $)$



Potentialmethode

Idee. Betrachte Bankguthaben (siehe Buchhaltermethode)
als physikalische Größe,
die den augenblicklichen Zustand der DS beschreibt.

Datenstruktur $D_0 \xrightarrow{op_1} D_1 \xrightarrow{op_2} \dots \xrightarrow{op_n} D_n$

Wähle Potential $\Phi: D_i \rightarrow \mathbb{R}$. O.B.d.A. $\Phi(D_0) = 0$

Ziel. Bank macht keine Miesen.

Also fordern wir $\Phi(D_i) \geq 0$ für $i = 1, \dots, n$.

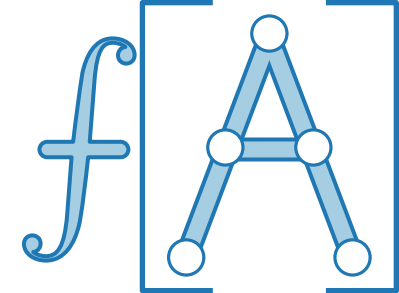
amortisierte
Kosten

echte Kosten

Potentialdifferenz

Def. $\hat{c}_i = c_i + \Delta\Phi(D_i)$, wobei $\Delta\Phi(D_i) = \Phi(D_i) - \Phi(D_{i-1})$

Folge: $\sum_{i=1}^n \hat{c}_i = \sum_{i=1}^n (c_i + \Phi(D_i) - \Phi(D_{i-1}))$



Potentialmethode

Idee. Betrachte Bankguthaben (siehe Buchhaltermethode)
als physikalische Größe,
die den augenblicklichen Zustand der DS beschreibt.

Datenstruktur $D_0 \xrightarrow{op_1} D_1 \xrightarrow{op_2} \dots \xrightarrow{op_n} D_n$

Wähle Potential $\Phi: D_i \rightarrow \mathbb{R}$. O.B.d.A. $\Phi(D_0) = 0$

Ziel. Bank macht keine Miesen.

Also fordern wir $\Phi(D_i) \geq 0$ für $i = 1, \dots, n$.

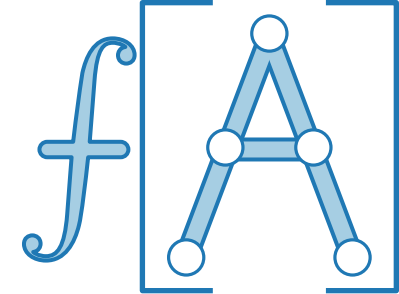
amortisierte
Kosten

echte Kosten

Potentialdifferenz

Def. $\hat{c}_i = c_i + \Delta\Phi(D_i)$, wobei $\Delta\Phi(D_i) = \Phi(D_i) - \Phi(D_{i-1})$

Folge: $\sum_{i=1}^n \hat{c}_i = \sum_{i=1}^n (c_i + \Phi(D_i) - \Phi(D_{i-1}))$ Teleskopsumme



Potentialmethode

Idee. Betrachte Bankguthaben (siehe Buchhaltermethode)
als physikalische Größe,
 die den augenblicklichen Zustand der DS beschreibt.

Datenstruktur $D_0 \xrightarrow{op_1} D_1 \xrightarrow{op_2} \dots \xrightarrow{op_n} D_n$

Wähle Potential $\Phi: D_i \rightarrow \mathbb{R}$. O.B.d.A. $\Phi(D_0) = 0$

Ziel. Bank macht keine Miesen.

Also fordern wir $\Phi(D_i) \geq 0$ für $i = 1, \dots, n$.

amortisierte
Kosten

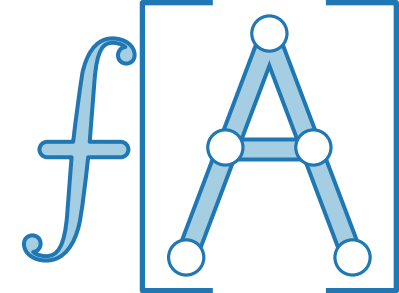
echte Kosten

Potentialdifferenz

Def. $\hat{c}_i = c_i + \Delta\Phi(D_i)$, wobei $\Delta\Phi(D_i) = \Phi(D_i) - \Phi(D_{i-1})$

Folge: $\sum_{i=1}^n \hat{c}_i = \sum_{i=1}^n (c_i + \Phi(D_i) - \Phi(D_{i-1}))$ Teleskopsumme

$$\stackrel{\text{Teleskopsumme}}{=} \sum_{i=1}^n c_i + \Phi(D_n) - \Phi(D_0)$$



Potentialmethode

Idee. Betrachte Bankguthaben (siehe Buchhaltermethode)
als physikalische Größe,
 die den augenblicklichen Zustand der DS beschreibt.

Datenstruktur $D_0 \xrightarrow{op_1} D_1 \xrightarrow{op_2} \dots \xrightarrow{op_n} D_n$

Wähle Potential $\Phi: D_i \rightarrow \mathbb{R}$. O.B.d.A. $\Phi(D_0) = 0$

Ziel. Bank macht keine Miesen.

Also fordern wir $\Phi(D_i) \geq 0$ für $i = 1, \dots, n$.

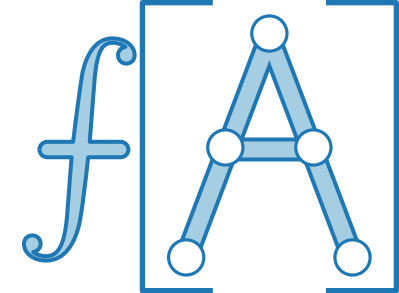
amortisierte
Kosten

echte Kosten

Potentialdifferenz

Def. $\hat{c}_i = c_i + \Delta\Phi(D_i)$, wobei $\Delta\Phi(D_i) = \Phi(D_i) - \Phi(D_{i-1})$

Folge: $\sum_{i=1}^n \hat{c}_i = \sum_{i=1}^n (c_i + \Phi(D_i) - \Phi(D_{i-1}))$ Teleskopsumme
 $\stackrel{\text{🔭}}{=} \sum_{i=1}^n c_i + \Phi(D_n) - \Phi(D_0)$



Potentialmethode

Idee. Betrachte Bankguthaben (siehe Buchhaltermethode)
als physikalische Größe,
 die den augenblicklichen Zustand der DS beschreibt.

Datenstruktur $D_0 \xrightarrow{op_1} D_1 \xrightarrow{op_2} \dots \xrightarrow{op_n} D_n$

Wähle Potential $\Phi: D_i \rightarrow \mathbb{R}$. O.B.d.A. $\Phi(D_0) = 0$

Ziel. Bank macht keine Miesen.

Also fordern wir $\Phi(D_i) \geq 0$ für $i = 1, \dots, n$.

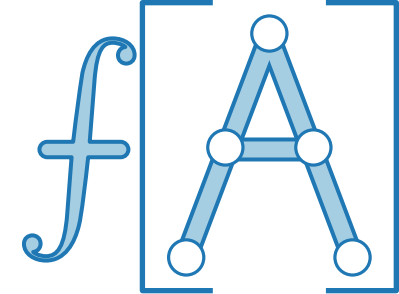
amortisierte
Kosten

echte Kosten

Potentialdifferenz

Def. $\hat{c}_i = c_i + \Delta\Phi(D_i)$, wobei $\Delta\Phi(D_i) = \Phi(D_i) - \Phi(D_{i-1})$

Folge: $\sum_{i=1}^n \hat{c}_i = \sum_{i=1}^n (c_i + \Phi(D_i) - \Phi(D_{i-1}))$ Teleskopsumme
 $\stackrel{\text{🔭}}{=} \sum_{i=1}^n c_i + \Phi(D_n) - \Phi(D_0)$



Potentialmethode

Idee. Betrachte Bankguthaben (siehe Buchhaltermethode)
als physikalische Größe,
 die den augenblicklichen Zustand der DS beschreibt.

Datenstruktur $D_0 \xrightarrow{op_1} D_1 \xrightarrow{op_2} \dots \xrightarrow{op_n} D_n$

Wähle Potential $\Phi: D_i \rightarrow \mathbb{R}$. O.B.d.A. $\Phi(D_0) = 0$

Ziel. Bank macht keine Miesen.

Also fordern wir $\Phi(D_i) \geq 0$ für $i = 1, \dots, n$.

amortisierte
Kosten

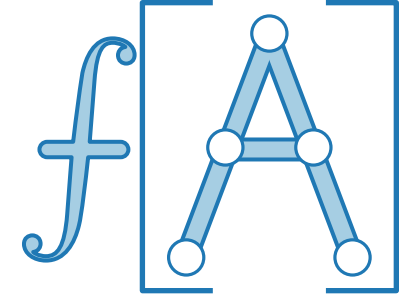
echte Kosten

Potentialdifferenz

Def. $\hat{c}_i = c_i + \Delta\Phi(D_i)$, wobei $\Delta\Phi(D_i) = \Phi(D_i) - \Phi(D_{i-1})$

Folge: $\sum_{i=1}^n \hat{c}_i = \sum_{i=1}^n (c_i + \Phi(D_i) - \Phi(D_{i-1}))$ Teleskopsumme

$$\stackrel{\text{Teleskopsumme}}{=} \sum_{i=1}^n c_i + \Phi(D_n) - \Phi(D_0) \geq \sum_{i=1}^n c_i$$



Potentialmethode

Idee. Betrachte Bankguthaben (siehe Buchhaltermethode)
als physikalische Größe,
 die den augenblicklichen Zustand der DS beschreibt.

Datenstruktur $D_0 \xrightarrow{op_1} D_1 \xrightarrow{op_2} \dots \xrightarrow{op_n} D_n$

Wähle Potential $\Phi: D_i \rightarrow \mathbb{R}$. O.B.d.A. $\Phi(D_0) = 0$

Ziel. Bank macht keine Miesen.

Also fordern wir $\Phi(D_i) \geq 0$ für $i = 1, \dots, n$.

amortisierte
Kosten

echte Kosten

Potentialdifferenz

Def. $\hat{c}_i = c_i + \Delta\Phi(D_i)$, wobei $\Delta\Phi(D_i) = \Phi(D_i) - \Phi(D_{i-1})$

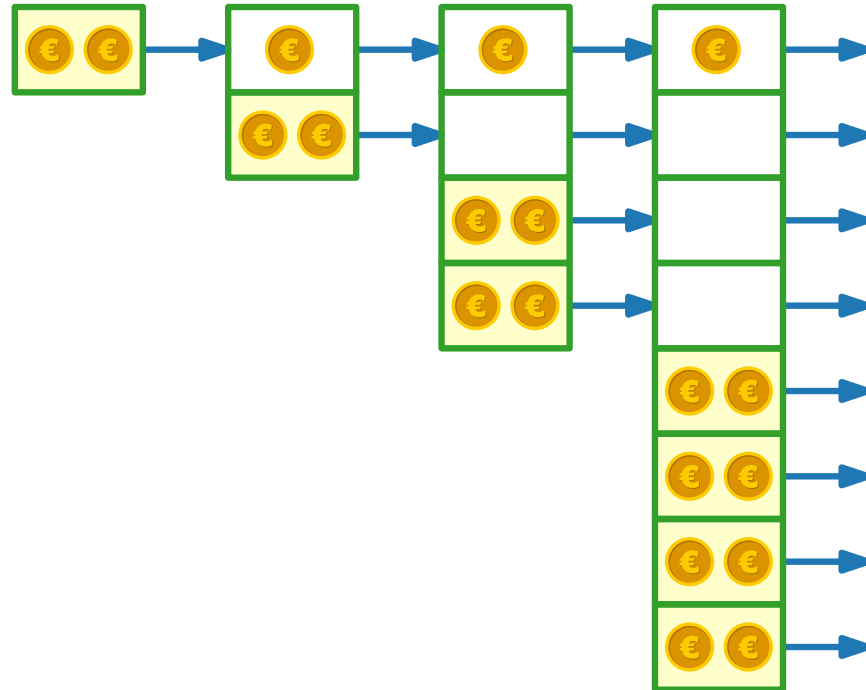
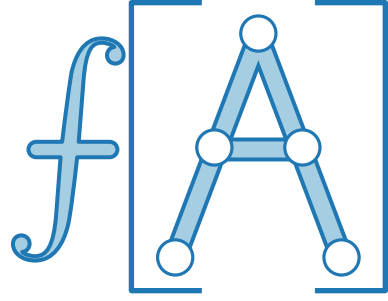
Folge: $\sum_{i=1}^n \hat{c}_i = \sum_{i=1}^n (c_i + \Phi(D_i) - \Phi(D_{i-1}))$ Teleskopsumme

$$\stackrel{\text{Teleskop}}{=} \sum_{i=1}^n c_i + \Phi(D_n) - \Phi(D_0) \geq \sum_{i=1}^n c_i$$

D.h. **amortisierte** Kosten „bezahlen“ für **tatsächliche** Kosten.

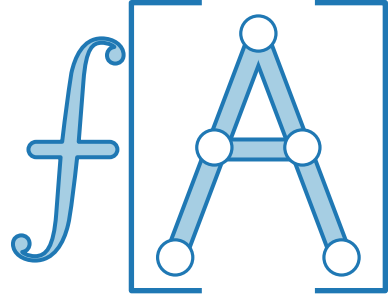
Potentialmethode für dynamische Tabellen

Idee.



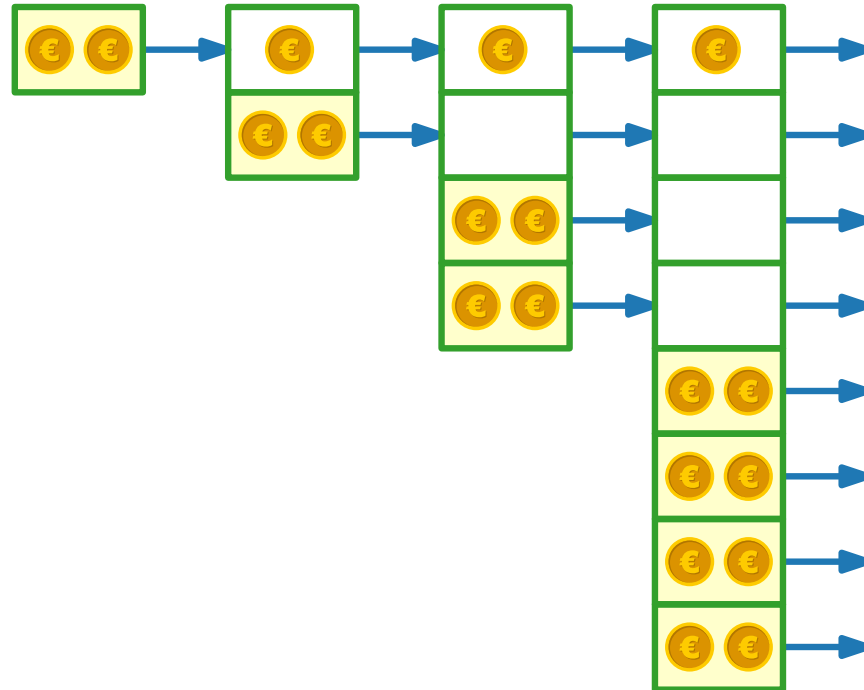
Potentialmethode für dynamische Tabellen

Idee.

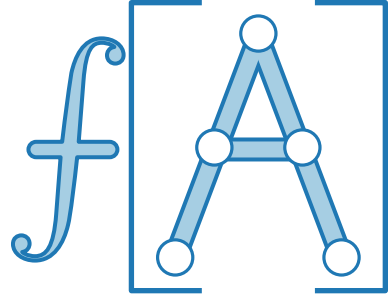


$$\hat{c}_i = c_i + \Delta\Phi(D_i), \text{ wobei } \Delta\Phi(D_i) = \Phi(D_i) - \Phi(D_{i-1})$$

$$\sum_{i=1}^n \hat{c}_i = \sum_{i=1}^n c_i + \Phi(D_n) - \Phi(D_0) \geq \sum_{i=1}^n c_i$$



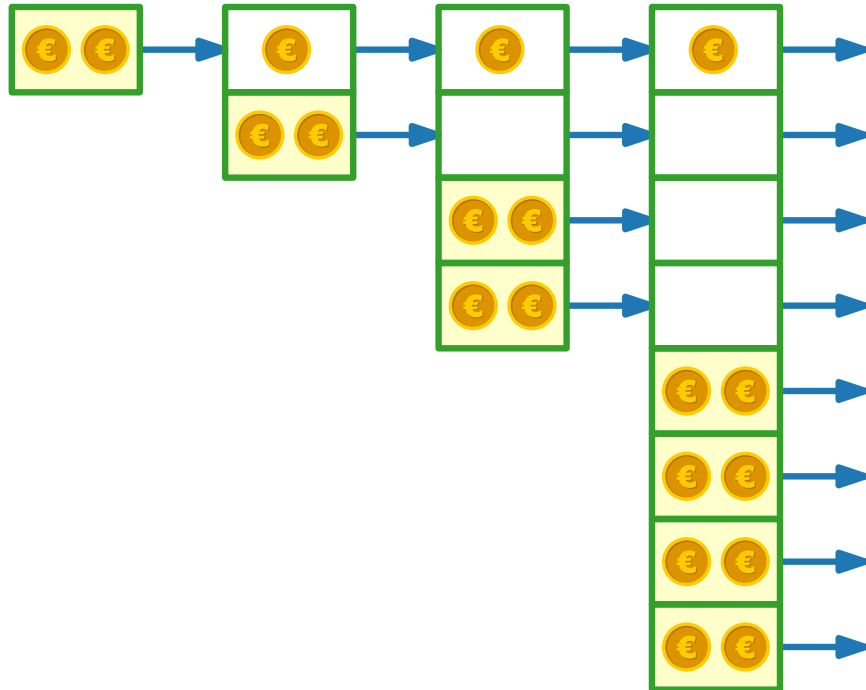
Potentialmethode für dynamische Tabellen



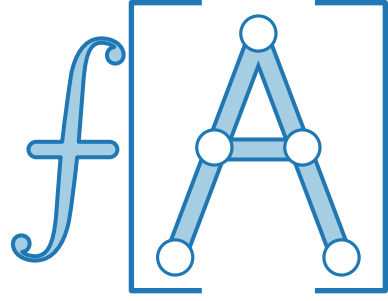
Idee. ■ Kein kopieren $\Rightarrow \Delta\Phi(D_i) = 2$

$$\hat{c}_i = c_i + \Delta\Phi(D_i), \text{ wobei } \Delta\Phi(D_i) = \Phi(D_i) - \Phi(D_{i-1})$$

$$\sum_{i=1}^n \hat{c}_i = \sum_{i=1}^n c_i + \Phi(D_n) - \Phi(D_0) \geq \sum_{i=1}^n c_i$$



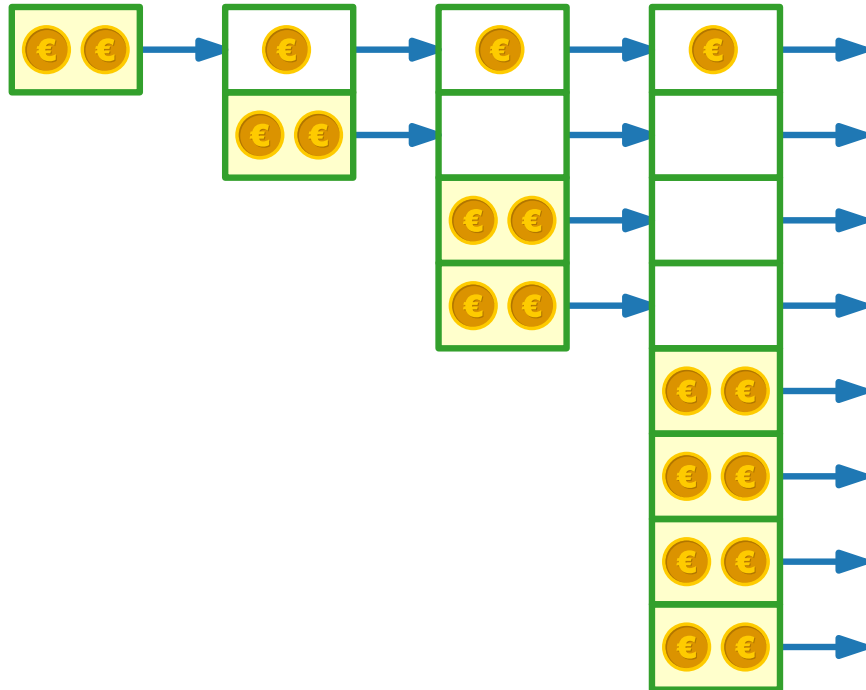
Potentialmethode für dynamische Tabellen



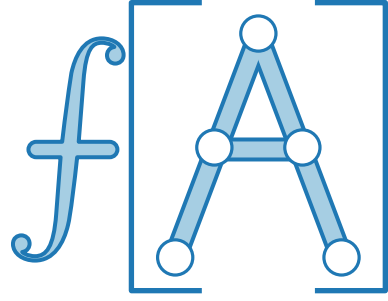
- Idee.**
- Kein kopieren $\Rightarrow \Delta\Phi(D_i) = 2$
 - kopieren $\Rightarrow \Delta\Phi(D_i) = 2 - (i - 1)$

$$\hat{c}_i = c_i + \Delta\Phi(D_i), \text{ wobei } \Delta\Phi(D_i) = \Phi(D_i) - \Phi(D_{i-1})$$

$$\sum_{i=1}^n \hat{c}_i = \sum_{i=1}^n c_i + \Phi(D_n) - \Phi(D_0) \geq \sum_{i=1}^n c_i$$



Potentialmethode für dynamische Tabellen

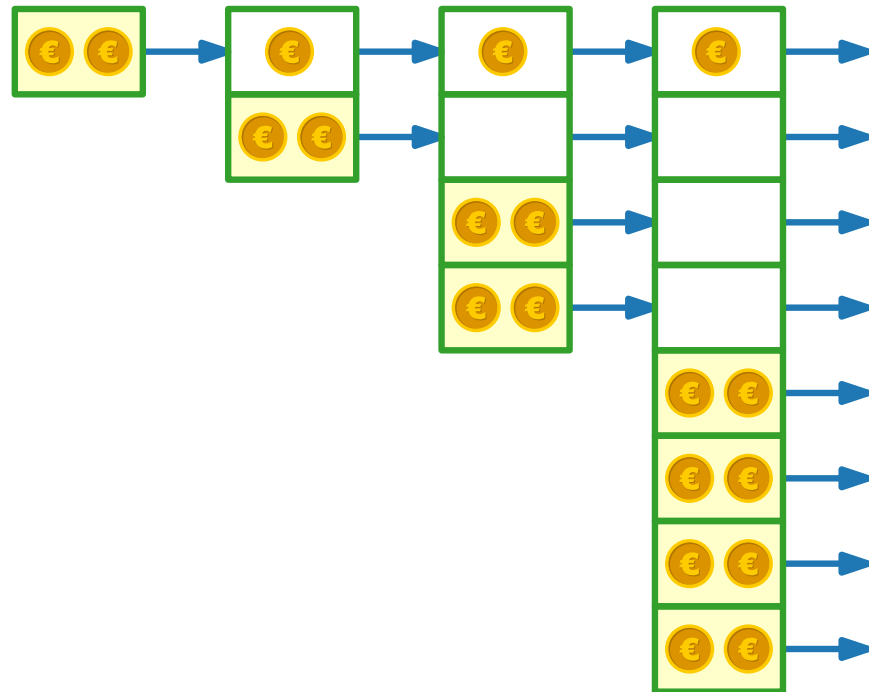


- Idee.**
- Kein kopieren $\Rightarrow \Delta\Phi(D_i) = 2$
 - kopieren $\Rightarrow \Delta\Phi(D_i) = 2 - (i - 1)$

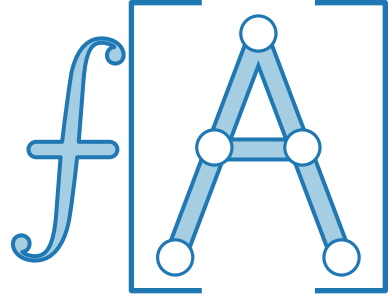
$i - 1$ Elemente werden kopiert

$$\hat{c}_i = c_i + \Delta\Phi(D_i), \text{ wobei } \Delta\Phi(D_i) = \Phi(D_i) - \Phi(D_{i-1})$$

$$\sum_{i=1}^n \hat{c}_i = \sum_{i=1}^n c_i + \Phi(D_n) - \Phi(D_0) \geq \sum_{i=1}^n c_i$$



Potentialmethode für dynamische Tabellen

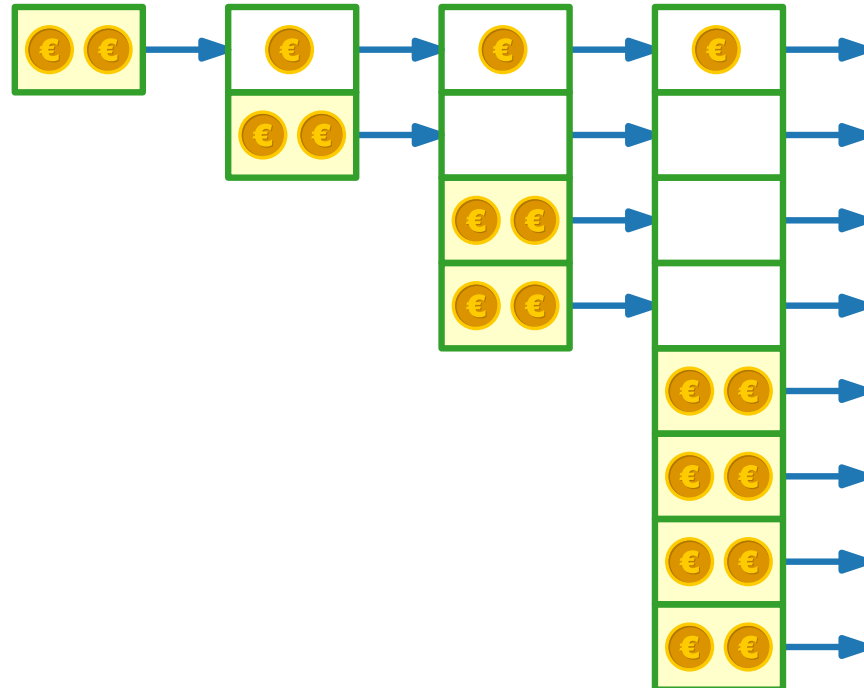


- Idee.**
- Kein kopieren $\Rightarrow \Delta\Phi(D_i) = 2$
 - kopieren $\Rightarrow \Delta\Phi(D_i) = 2 - (i - 1)$
 - $\Rightarrow \Phi(D_i) = 1 + 2 \cdot \text{size}_i - \text{table-size}_i$

$i - 1$ Elemente werden kopiert

$$\hat{c}_i = c_i + \Delta\Phi(D_i), \text{ wobei } \Delta\Phi(D_i) = \Phi(D_i) - \Phi(D_{i-1})$$

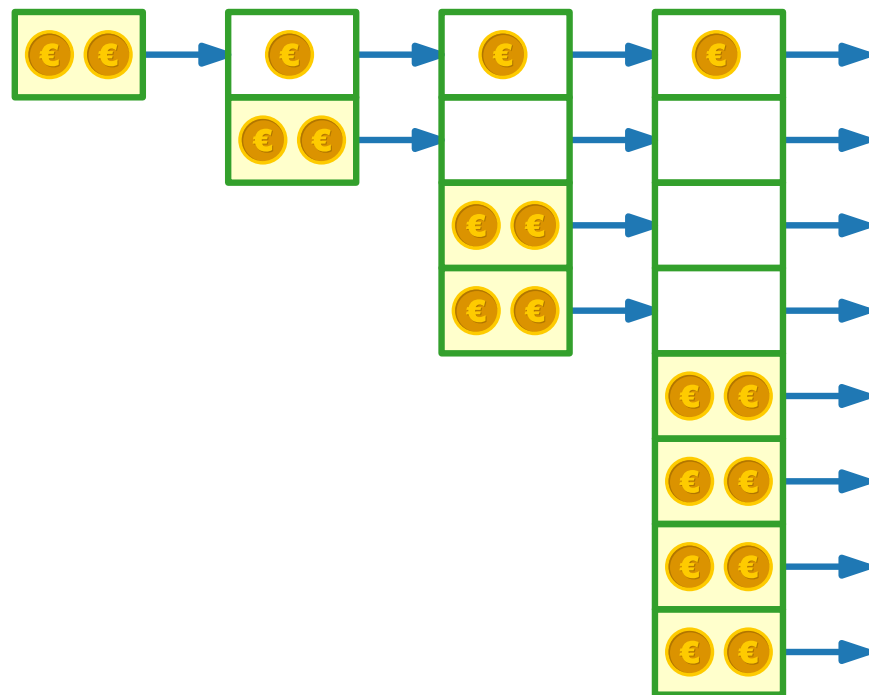
$$\sum_{i=1}^n \hat{c}_i = \sum_{i=1}^n c_i + \Phi(D_n) - \Phi(D_0) \geq \sum_{i=1}^n c_i$$



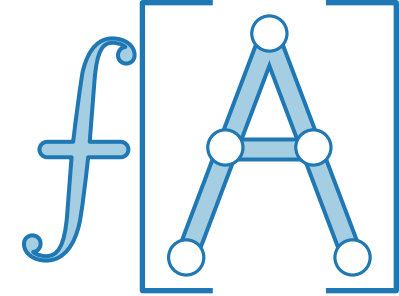
Idee.

- Kein kopieren $\Rightarrow \Delta\Phi(D_i) = 2$ i - 1 Elemente werden kopiert
- kopieren $\Rightarrow \Delta\Phi(D_i) = 2 - (i - 1)$
- $\Rightarrow \Phi(D_i) = 1 + 2 \cdot \text{size}_i - \text{table-size}_i$

$$\sum_{i=1}^n \hat{c}_i = \sum_{i=1}^n c_i + \Phi(D_n) - \Phi(D_0) \geq \sum_{i=1}^n c_i$$



i	$\hat{c}_i$	c_i	$\Delta\Phi(D_i)$	$\Phi(D_i)$



Potentialmethode für dynamische Tabellen

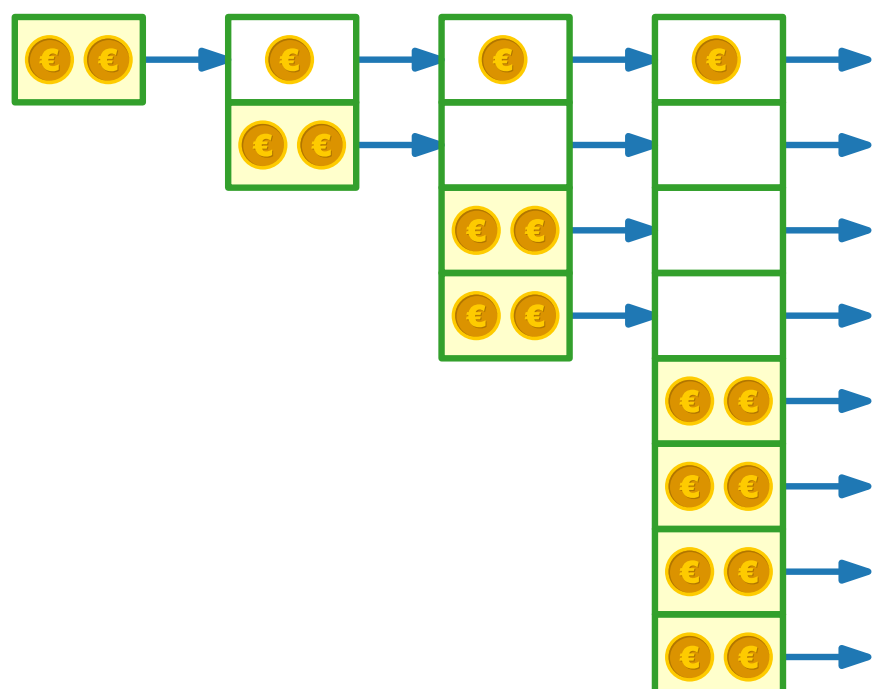
- Idee.**
- Kein kopieren $\Rightarrow \Delta\Phi(D_i) = 2$
 - kopieren $\Rightarrow \Delta\Phi(D_i) = 2 - (i - 1)$
 - $\Rightarrow \Phi(D_i) = 1 + 2 \cdot \text{size}_i - \text{table-size}_i$

$i - 1$ Elemente werden kopiert

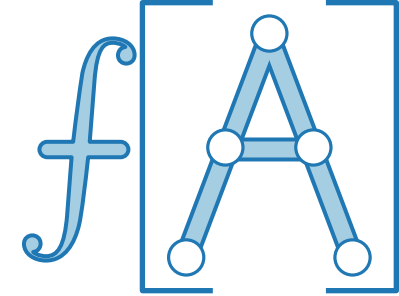


$\hat{c}_i = c_i + \Delta\Phi(D_i)$, wobei $\Delta\Phi(D_i) = \Phi(D_i) - \Phi(D_{i-1})$

$$\sum_{i=1}^n \hat{c}_i = \sum_{i=1}^n c_i + \Phi(D_n) - \Phi(D_0) \geq \sum_{i=1}^n c_i$$



i	$\hat{c}_i$	c_i	$\Delta\Phi(D_i)$	$\Phi(D_i)$
0				0



Potentialmethode für dynamische Tabellen

- Idee.**
- Kein kopieren $\Rightarrow \Delta\Phi(D_i) = 2$
 - kopieren $\Rightarrow \Delta\Phi(D_i) = 2 - (i - 1)$
 - $\Rightarrow \Phi(D_i) = 1 + 2 \cdot \text{size}_i - \text{table-size}_i$

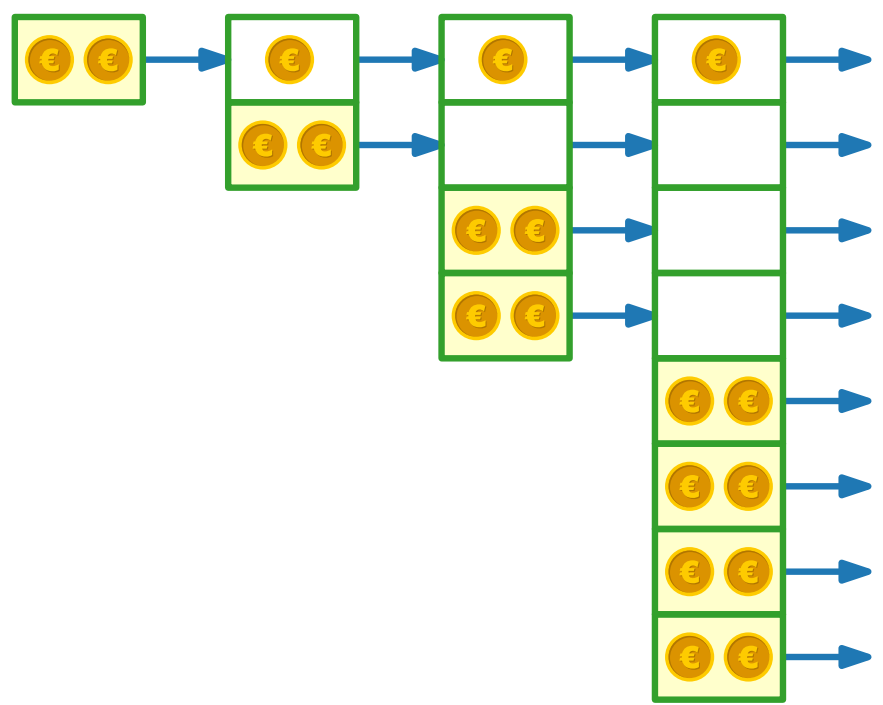
$i - 1$ Elemente werden kopiert

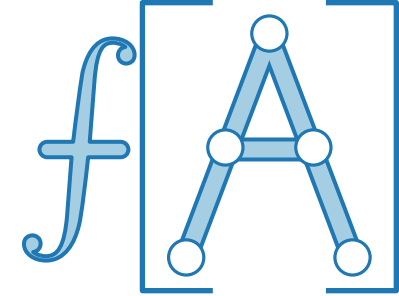
INSERT(1)

$\hat{c}_i = c_i + \Delta\Phi(D_i)$, wobei $\Delta\Phi(D_i) = \Phi(D_i) - \Phi(D_{i-1})$

$\sum_{i=1}^n \hat{c}_i = \sum_{i=1}^n c_i + \Phi(D_n) - \Phi(D_0) \geq \sum_{i=1}^n c_i$

i	$\hat{c}_i$	c_i	$\Delta\Phi(D_i)$	$\Phi(D_i)$
0				0
1				





Potentialmethode für dynamische Tabellen

- Idee.**
- Kein kopieren $\Rightarrow \Delta\Phi(D_i) = 2$
 - kopieren $\Rightarrow \Delta\Phi(D_i) = 2 - (i - 1)$
 - $\Rightarrow \Phi(D_i) = 1 + 2 \cdot \text{size}_i - \text{table-size}_i$

$i - 1$ Elemente werden kopiert

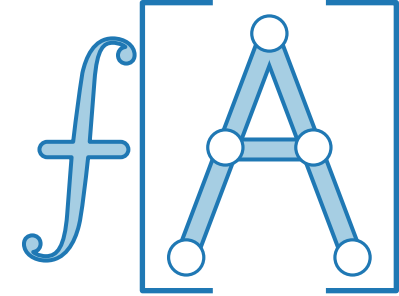
INSERT(1)

1

$\hat{c}_i = c_i + \Delta\Phi(D_i)$, wobei $\Delta\Phi(D_i) = \Phi(D_i) - \Phi(D_{i-1})$

$$\sum_{i=1}^n \hat{c}_i = \sum_{i=1}^n c_i + \Phi(D_n) - \Phi(D_0) \geq \sum_{i=1}^n c_i$$

i	$\hat{c}_i$	c_i	$\Delta\Phi(D_i)$	$\Phi(D_i)$
0				0
1		1		



Potentialmethode für dynamische Tabellen

- Idee.**
- Kein kopieren $\Rightarrow \Delta\Phi(D_i) = 2$
 - kopieren $\Rightarrow \Delta\Phi(D_i) = 2 - (i - 1)$
 - $\Rightarrow \Phi(D_i) = 1 + 2 \cdot \text{size}_i - \text{table-size}_i$

$i - 1$ Elemente werden kopiert

INSERT(1)

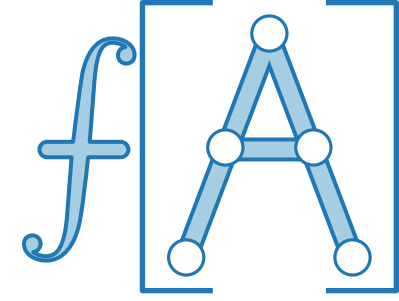
1

$\hat{c}_i = c_i + \Delta\Phi(D_i)$, wobei $\Delta\Phi(D_i) = \Phi(D_i) - \Phi(D_{i-1})$

$$\sum_{i=1}^n \hat{c}_i = \sum_{i=1}^n c_i + \Phi(D_n) - \Phi(D_0) \geq \sum_{i=1}^n c_i$$

i	$\hat{c}_i$	c_i	$\Delta\Phi(D_i)$	$\Phi(D_i)$
0				0
1		1	2	

Potentialmethode für dynamische Tabellen



- Idee.**
- Kein kopieren $\Rightarrow \Delta\Phi(D_i) = 2$
 - kopieren $\Rightarrow \Delta\Phi(D_i) = 2 - (i - 1)$
 - $\Rightarrow \Phi(D_i) = 1 + 2 \cdot \text{size}_i - \text{table-size}_i$

$i - 1$ Elemente werden kopiert

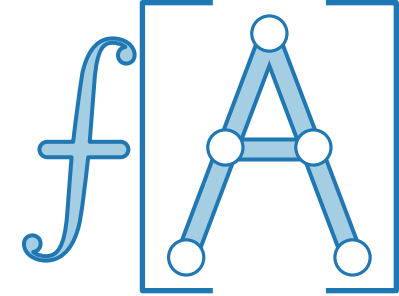
INSERT(1)

1

$$\hat{c}_i = c_i + \Delta\Phi(D_i), \text{ wobei } \Delta\Phi(D_i) = \Phi(D_i) - \Phi(D_{i-1})$$

$$\sum_{i=1}^n \hat{c}_i = \sum_{i=1}^n c_i + \Phi(D_n) - \Phi(D_0) \geq \sum_{i=1}^n c_i$$

i	$\hat{c}_i$	c_i	$\Delta\Phi(D_i)$	$\Phi(D_i)$
0				0
1		1	2	2



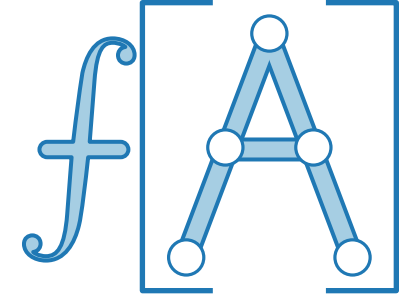
Potentialmethode für dynamische Tabellen

- Idee.**
- Kein kopieren $\Rightarrow \Delta\Phi(D_i) = 2$ $i - 1$ Elemente werden kopiert
 - kopieren $\Rightarrow \Delta\Phi(D_i) = 2 - (i - 1)$ INSERT(1) 1
 - $\Rightarrow \Phi(D_i) = 1 + 2 \cdot \text{size}_i - \text{table-size}_i$

$\hat{c}_i = c_i + \Delta\Phi(D_i)$, wobei $\Delta\Phi(D_i) = \Phi(D_i) - \Phi(D_{i-1})$

$\sum_{i=1}^n \hat{c}_i = \sum_{i=1}^n c_i + \Phi(D_n) - \Phi(D_0) \geq \sum_{i=1}^n c_i$

i	$\hat{c}_i$	c_i	$\Delta\Phi(D_i)$	$\Phi(D_i)$
0				0
1	3	1	2	2



Potentialmethode für dynamische Tabellen

- Idee.**
- Kein kopieren $\Rightarrow \Delta\Phi(D_i) = 2$
 - kopieren $\Rightarrow \Delta\Phi(D_i) = 2 - (i - 1)$
 - $\Rightarrow \Phi(D_i) = 1 + 2 \cdot \text{size}_i - \text{table-size}_i$

$i - 1$ Elemente werden kopiert

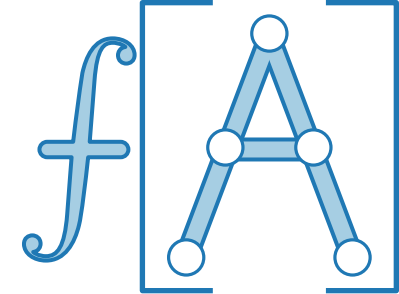
INSERT(1) 1
INSERT(2)

$\hat{c}_i = c_i + \Delta\Phi(D_i)$, wobei $\Delta\Phi(D_i) = \Phi(D_i) - \Phi(D_{i-1})$

$\sum_{i=1}^n \hat{c}_i = \sum_{i=1}^n c_i + \Phi(D_n) - \Phi(D_0) \geq \sum_{i=1}^n c_i$

i	$\hat{c}_i$	c_i	$\Delta\Phi(D_i)$	$\Phi(D_i)$
0				0
1	3	1	2	2
2				

Potentialmethode für dynamische Tabellen

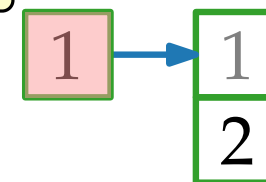


- Idee.**
- Kein kopieren $\Rightarrow \Delta\Phi(D_i) = 2$
 - kopieren $\Rightarrow \Delta\Phi(D_i) = 2 - (i - 1)$
 - $\Rightarrow \Phi(D_i) = 1 + 2 \cdot \text{size}_i - \text{table-size}_i$

$i - 1$ Elemente werden kopiert

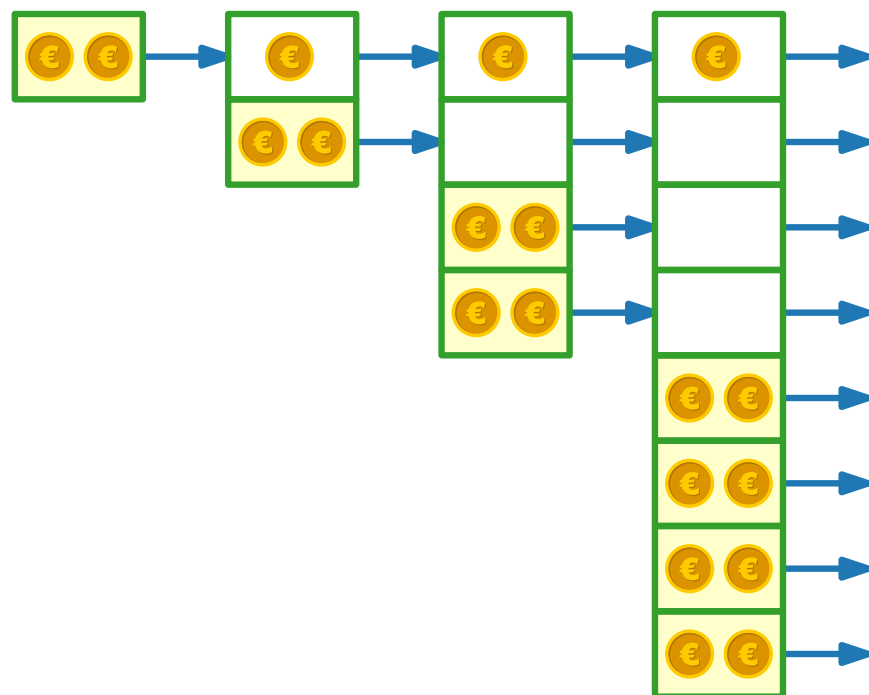
INSERT(1)

INSERT(2)



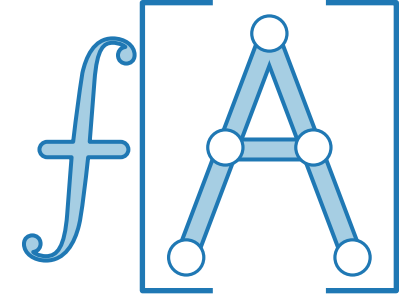
$\hat{c}_i = c_i + \Delta\Phi(D_i)$, wobei $\Delta\Phi(D_i) = \Phi(D_i) - \Phi(D_{i-1})$

$$\sum_{i=1}^n \hat{c}_i = \sum_{i=1}^n c_i + \Phi(D_n) - \Phi(D_0) \geq \sum_{i=1}^n c_i$$



i	$\hat{c}_i$	c_i	$\Delta\Phi(D_i)$	$\Phi(D_i)$
0				0
1	3	1	2	2
2				

Potentialmethode für dynamische Tabellen

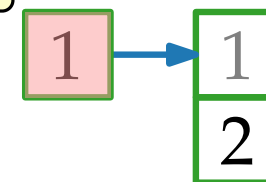


- Idee.**
- Kein kopieren $\Rightarrow \Delta\Phi(D_i) = 2$
 - kopieren $\Rightarrow \Delta\Phi(D_i) = 2 - (i - 1)$
 - $\Rightarrow \Phi(D_i) = 1 + 2 \cdot \text{size}_i - \text{table-size}_i$

$i - 1$ Elemente werden kopiert

INSERT(1)

INSERT(2)

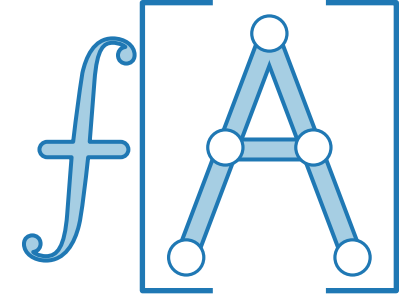


$$\hat{c}_i = c_i + \Delta\Phi(D_i), \text{ wobei } \Delta\Phi(D_i) = \Phi(D_i) - \Phi(D_{i-1})$$

$$\sum_{i=1}^n \hat{c}_i = \sum_{i=1}^n c_i + \Phi(D_n) - \Phi(D_0) \geq \sum_{i=1}^n c_i$$

i	$\hat{c}_i$	c_i	$\Delta\Phi(D_i)$	$\Phi(D_i)$
0				0
1	3	1	2	2
2		2		

Potentialmethode für dynamische Tabellen

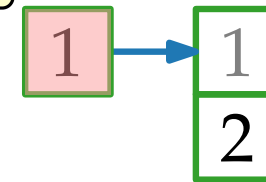


- Idee.**
- Kein kopieren $\Rightarrow \Delta\Phi(D_i) = 2$
 - kopieren $\Rightarrow \Delta\Phi(D_i) = 2 - (i - 1)$
 - $\Rightarrow \Phi(D_i) = 1 + 2 \cdot \text{size}_i - \text{table-size}_i$

$i - 1$ Elemente werden kopiert

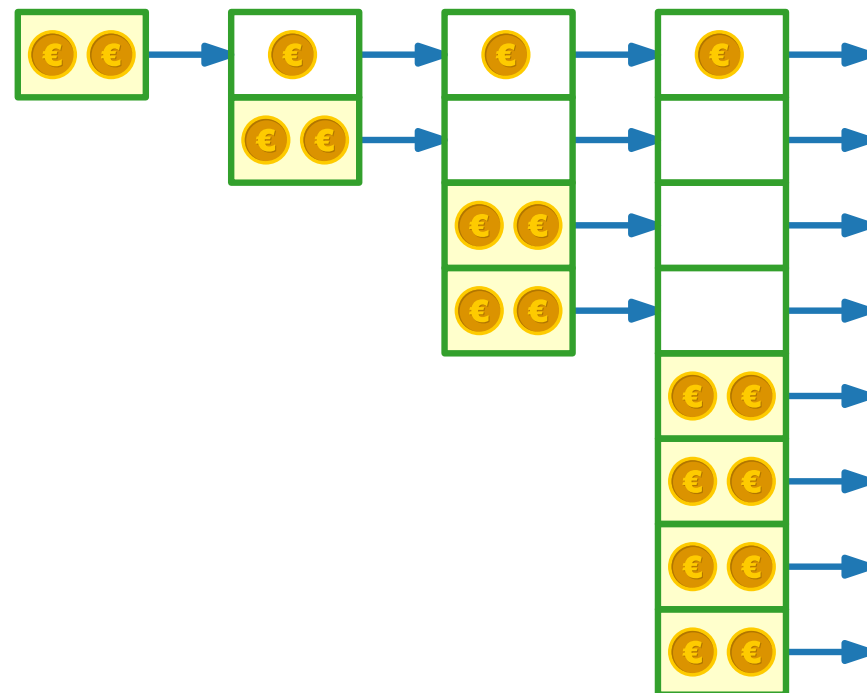
INSERT(1)

INSERT(2)



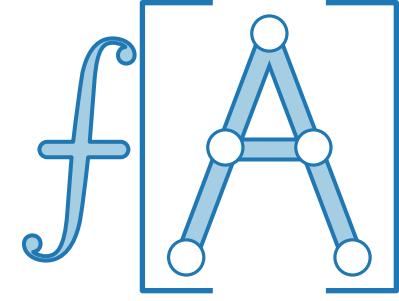
$\hat{c}_i = c_i + \Delta\Phi(D_i)$, wobei $\Delta\Phi(D_i) = \Phi(D_i) - \Phi(D_{i-1})$

$$\sum_{i=1}^n \hat{c}_i = \sum_{i=1}^n c_i + \Phi(D_n) - \Phi(D_0) \geq \sum_{i=1}^n c_i$$



i	$\hat{c}_i$	c_i	$\Delta\Phi(D_i)$	$\Phi(D_i)$
0				0
1	3	1	2	2
2		2	1	

Potentialmethode für dynamische Tabellen

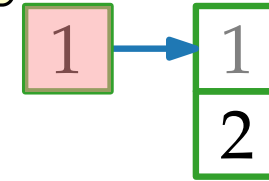


- Idee.**
- Kein kopieren $\Rightarrow \Delta\Phi(D_i) = 2$
 - kopieren $\Rightarrow \Delta\Phi(D_i) = 2 - (i - 1)$
 - $\Rightarrow \Phi(D_i) = 1 + 2 \cdot \text{size}_i - \text{table-size}_i$

$i - 1$ Elemente werden kopiert

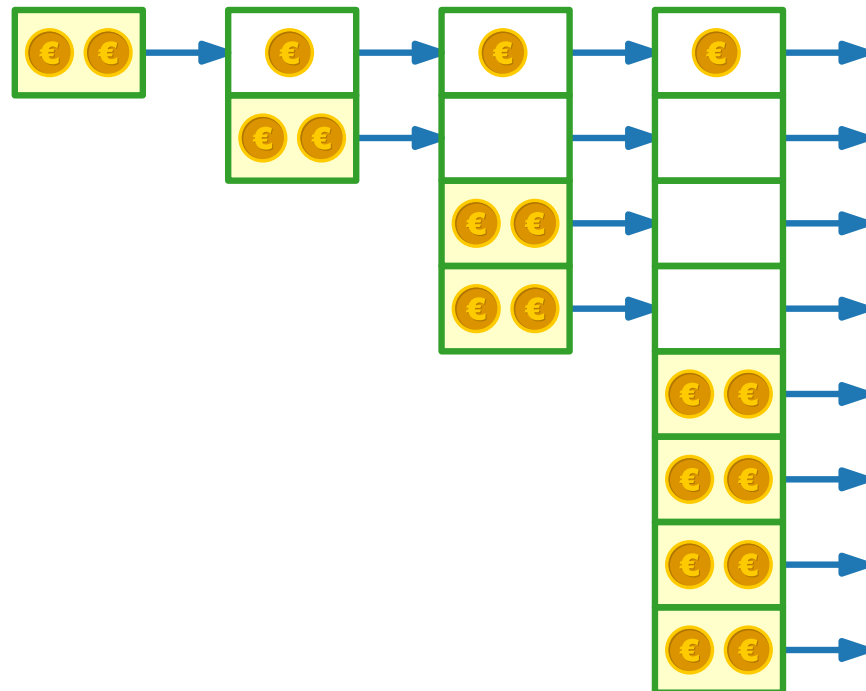
INSERT(1)

INSERT(2)



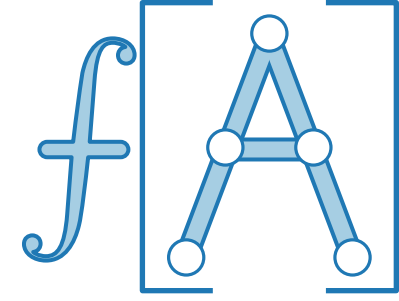
$$\hat{c}_i = c_i + \Delta\Phi(D_i), \text{ wobei } \Delta\Phi(D_i) = \Phi(D_i) - \Phi(D_{i-1})$$

$$\sum_{i=1}^n \hat{c}_i = \sum_{i=1}^n c_i + \Phi(D_n) - \Phi(D_0) \geq \sum_{i=1}^n c_i$$



i	$\hat{c}_i$	c_i	$\Delta\Phi(D_i)$	$\Phi(D_i)$
0				0
1	3	1	2	2
2		2	1	3

Potentialmethode für dynamische Tabellen

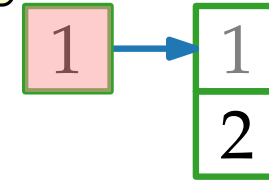


- Idee.**
- Kein kopieren $\Rightarrow \Delta\Phi(D_i) = 2$
 - kopieren $\Rightarrow \Delta\Phi(D_i) = 2 - (i - 1)$
 - $\Rightarrow \Phi(D_i) = 1 + 2 \cdot \text{size}_i - \text{table-size}_i$

$i - 1$ Elemente werden kopiert

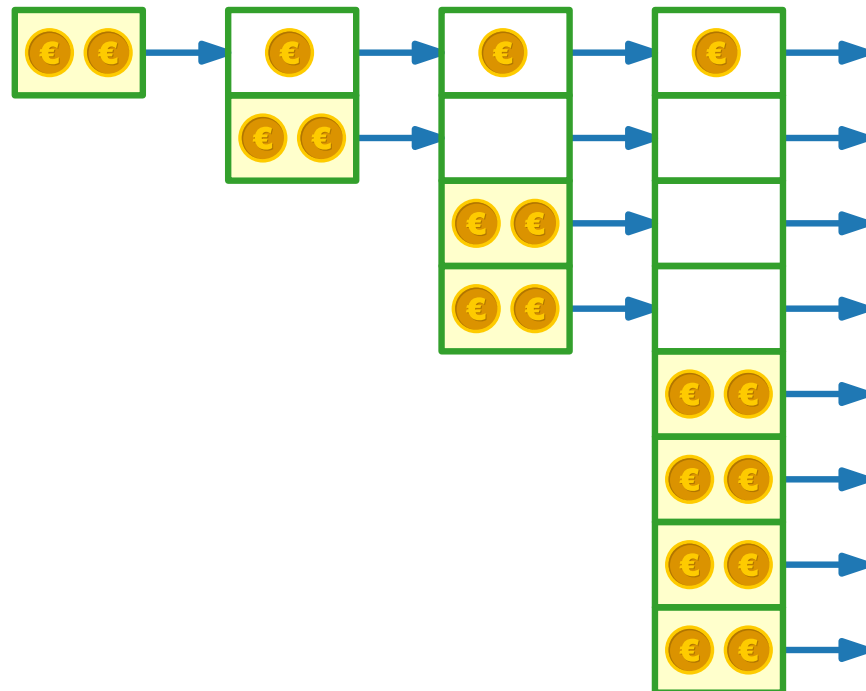
INSERT(1)

INSERT(2)



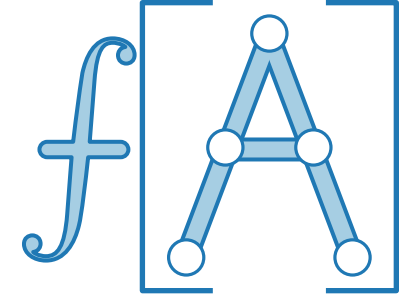
$$\hat{c}_i = c_i + \Delta\Phi(D_i), \text{ wobei } \Delta\Phi(D_i) = \Phi(D_i) - \Phi(D_{i-1})$$

$$\sum_{i=1}^n \hat{c}_i = \sum_{i=1}^n c_i + \Phi(D_n) - \Phi(D_0) \geq \sum_{i=1}^n c_i$$



i	$\hat{c}_i$	c_i	$\Delta\Phi(D_i)$	$\Phi(D_i)$
0				0
1	3	1	2	2
2	3	2	1	3

Potentialmethode für dynamische Tabellen



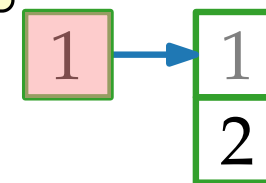
- Idee.**
- Kein kopieren $\Rightarrow \Delta\Phi(D_i) = 2$
 - kopieren $\Rightarrow \Delta\Phi(D_i) = 2 - (i - 1)$
 - $\Rightarrow \Phi(D_i) = 1 + 2 \cdot \text{size}_i - \text{table-size}_i$

$i - 1$ Elemente werden kopiert

INSERT(1)

INSERT(2)

INSERT(3)



$$\hat{c}_i = c_i + \Delta\Phi(D_i), \text{ wobei } \Delta\Phi(D_i) = \Phi(D_i) - \Phi(D_{i-1})$$

$$\sum_{i=1}^n \hat{c}_i = \sum_{i=1}^n c_i + \Phi(D_n) - \Phi(D_0) \geq \sum_{i=1}^n c_i$$

i	$\hat{c}_i$	c_i	$\Delta\Phi(D_i)$	$\Phi(D_i)$
0				0
1	3	1	2	2
2	3	2	1	3
3				

Potentialmethode für dynamische Tabellen

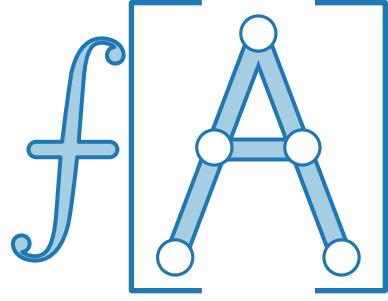
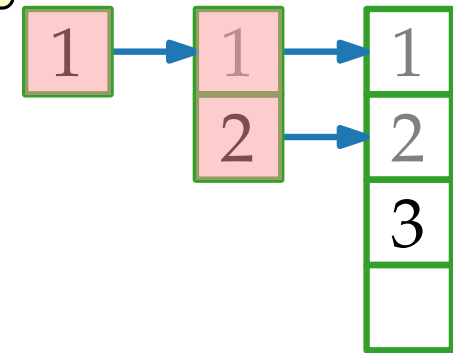
- Idee.**
- Kein kopieren $\Rightarrow \Delta\Phi(D_i) = 2$
 - kopieren $\Rightarrow \Delta\Phi(D_i) = 2 - (i - 1)$
 - $\Rightarrow \Phi(D_i) = 1 + 2 \cdot \text{size}_i - \text{table-size}_i$

$i - 1$ Elemente werden kopiert

INSERT(1)

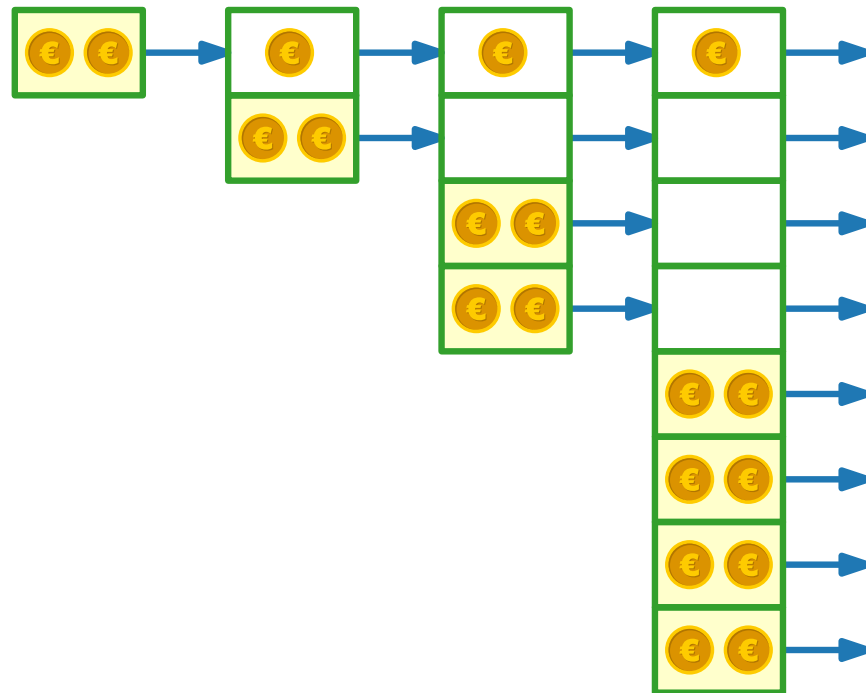
INSERT(2)

INSERT(3)



$$\hat{c}_i = c_i + \Delta\Phi(D_i), \text{ wobei } \Delta\Phi(D_i) = \Phi(D_i) - \Phi(D_{i-1})$$

$$\sum_{i=1}^n \hat{c}_i = \sum_{i=1}^n c_i + \Phi(D_n) - \Phi(D_0) \geq \sum_{i=1}^n c_i$$



i	$\hat{c}_i$	c_i	$\Delta\Phi(D_i)$	$\Phi(D_i)$
0				0
1	3	1	2	2
2	3	2	1	3
3				

Potentialmethode für dynamische Tabellen

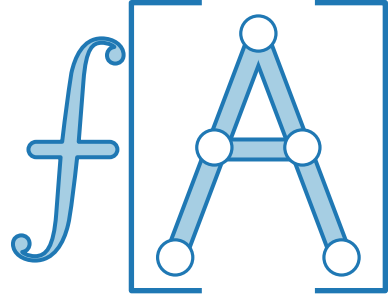
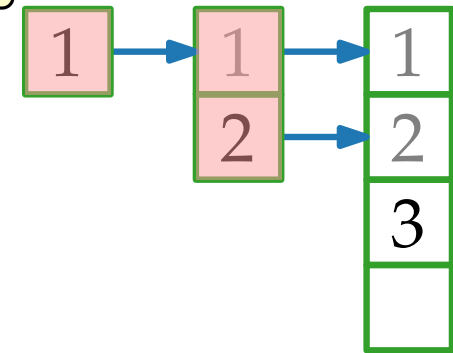
- Idee.**
- Kein kopieren $\Rightarrow \Delta\Phi(D_i) = 2$
 - kopieren $\Rightarrow \Delta\Phi(D_i) = 2 - (i - 1)$
 - $\Rightarrow \Phi(D_i) = 1 + 2 \cdot \text{size}_i - \text{table-size}_i$

$i - 1$ Elemente werden kopiert

INSERT(1)

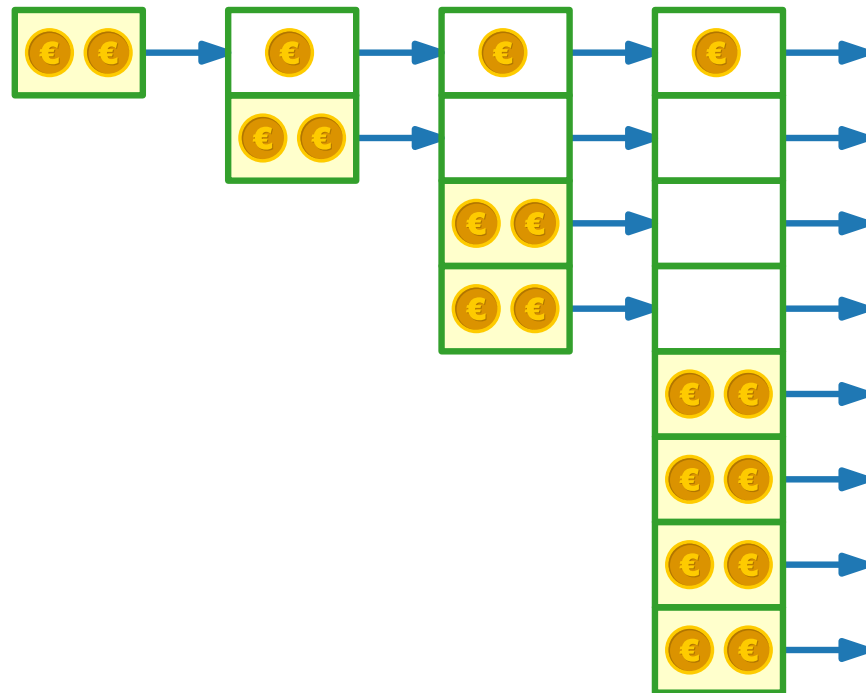
INSERT(2)

INSERT(3)



$$\hat{c}_i = c_i + \Delta\Phi(D_i), \text{ wobei } \Delta\Phi(D_i) = \Phi(D_i) - \Phi(D_{i-1})$$

$$\sum_{i=1}^n \hat{c}_i = \sum_{i=1}^n c_i + \Phi(D_n) - \Phi(D_0) \geq \sum_{i=1}^n c_i$$



i	$\hat{c}_i$	c_i	$\Delta\Phi(D_i)$	$\Phi(D_i)$
0				0
1	3	1	2	2
2	3	2	1	3
3		3		

Potentialmethode für dynamische Tabellen

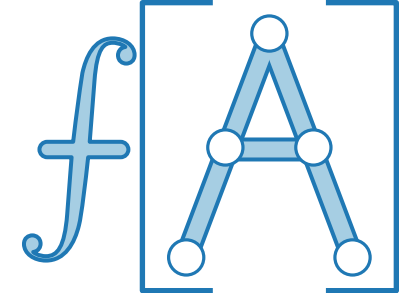
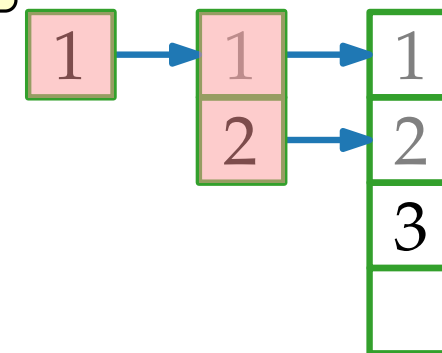
- Idee.**
- Kein kopieren $\Rightarrow \Delta\Phi(D_i) = 2$
 - kopieren $\Rightarrow \Delta\Phi(D_i) = 2 - (i - 1)$
 - $\Rightarrow \Phi(D_i) = 1 + 2 \cdot \text{size}_i - \text{table-size}_i$

$i - 1$ Elemente werden kopiert

INSERT(1)

INSERT(2)

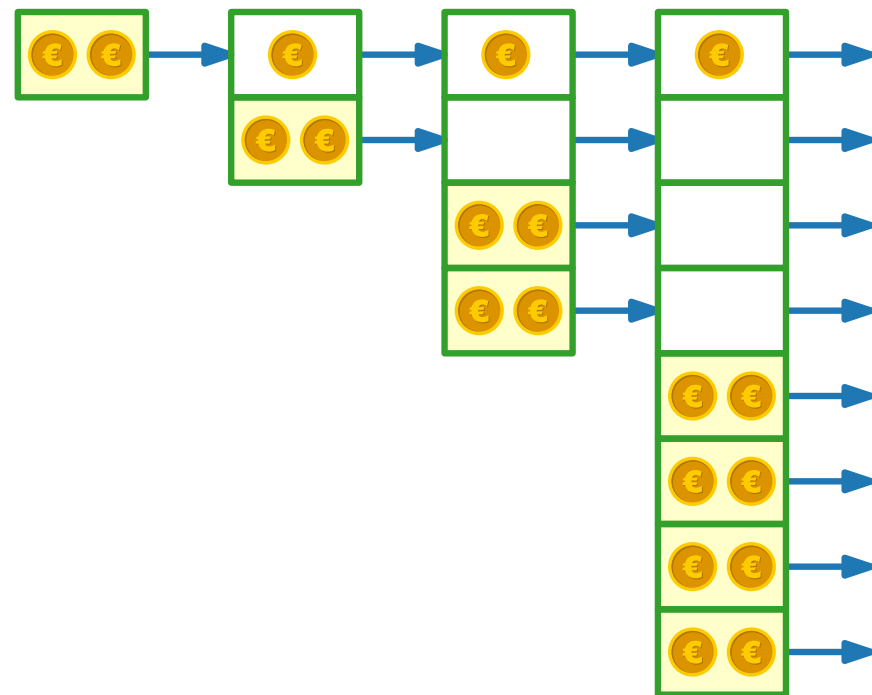
INSERT(3)



$$\hat{c}_i = c_i + \Delta\Phi(D_i), \text{ wobei } \Delta\Phi(D_i) = \Phi(D_i) - \Phi(D_{i-1})$$

$$\sum_{i=1}^n \hat{c}_i = \sum_{i=1}^n c_i + \Phi(D_n) - \Phi(D_0) \geq \sum_{i=1}^n c_i$$

i	$\hat{c}_i$	c_i	$\Delta\Phi(D_i)$	$\Phi(D_i)$
0				0
1	3	1	2	2
2	3	2	1	3
3		3	0	



Potentialmethode für dynamische Tabellen

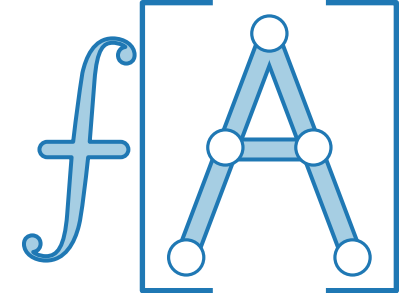
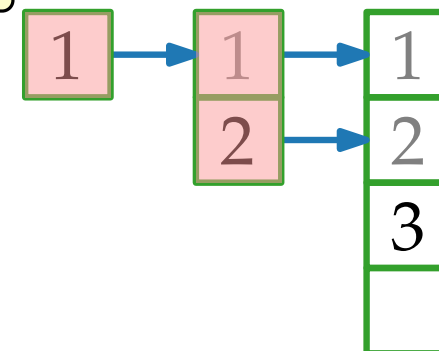
- Idee.**
- Kein kopieren $\Rightarrow \Delta\Phi(D_i) = 2$
 - kopieren $\Rightarrow \Delta\Phi(D_i) = 2 - (i - 1)$
 - $\Rightarrow \Phi(D_i) = 1 + 2 \cdot \text{size}_i - \text{table-size}_i$

$i - 1$ Elemente werden kopiert

INSERT(1)

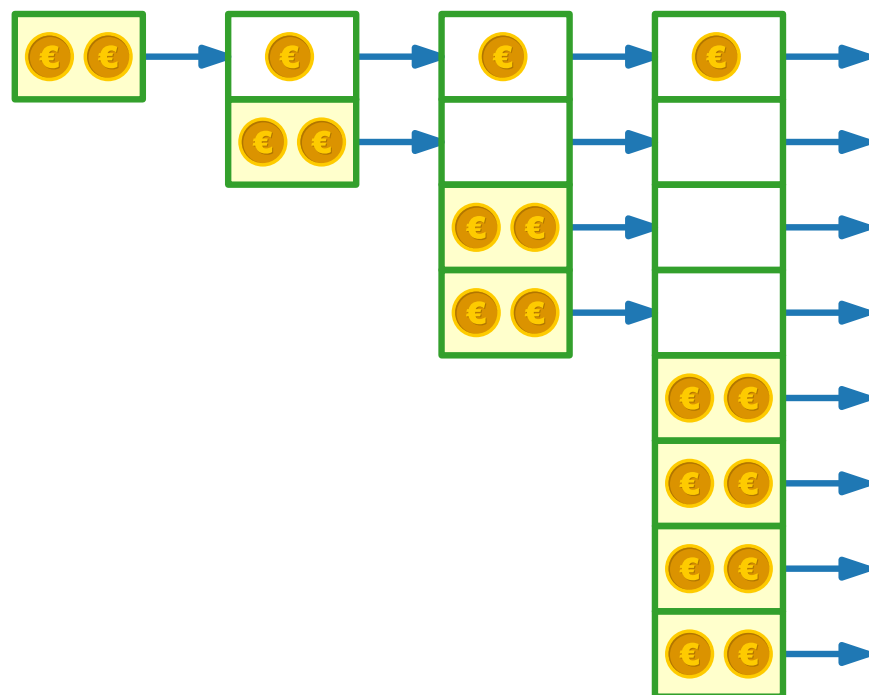
INSERT(2)

INSERT(3)



$$\hat{c}_i = c_i + \Delta\Phi(D_i), \text{ wobei } \Delta\Phi(D_i) = \Phi(D_i) - \Phi(D_{i-1})$$

$$\sum_{i=1}^n \hat{c}_i = \sum_{i=1}^n c_i + \Phi(D_n) - \Phi(D_0) \geq \sum_{i=1}^n c_i$$



i	$\hat{c}_i$	c_i	$\Delta\Phi(D_i)$	$\Phi(D_i)$
0				0
1	3	1	2	2
2	3	2	1	3
3		3	0	3

Potentialmethode für dynamische Tabellen

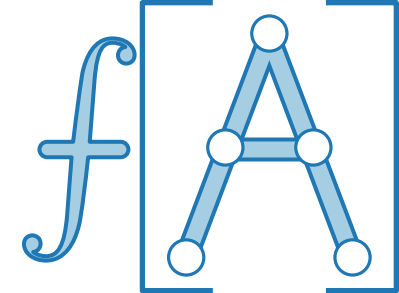
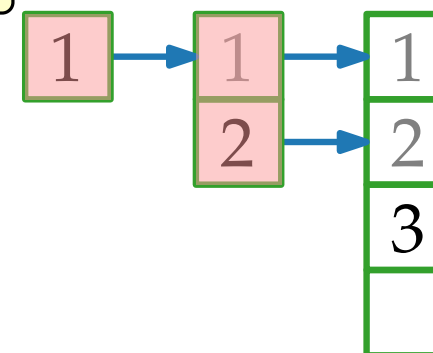
- Idee.**
- Kein kopieren $\Rightarrow \Delta\Phi(D_i) = 2$
 - kopieren $\Rightarrow \Delta\Phi(D_i) = 2 - (i - 1)$
 - $\Rightarrow \Phi(D_i) = 1 + 2 \cdot \text{size}_i - \text{table-size}_i$

$i - 1$ Elemente werden kopiert

INSERT(1)

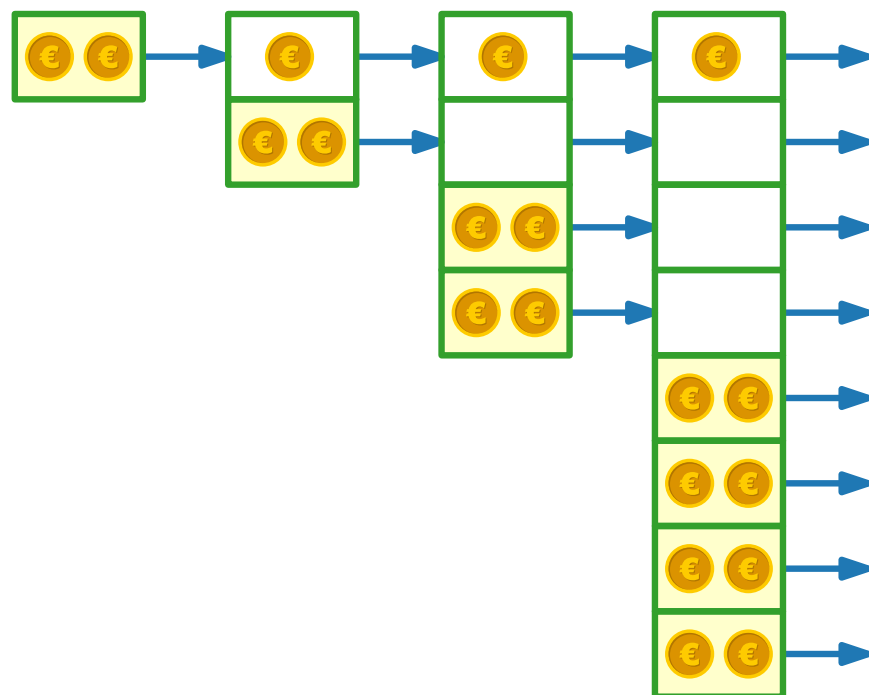
INSERT(2)

INSERT(3)



$$\hat{c}_i = c_i + \Delta\Phi(D_i), \text{ wobei } \Delta\Phi(D_i) = \Phi(D_i) - \Phi(D_{i-1})$$

$$\sum_{i=1}^n \hat{c}_i = \sum_{i=1}^n c_i + \Phi(D_n) - \Phi(D_0) \geq \sum_{i=1}^n c_i$$



i	$\hat{c}_i$	c_i	$\Delta\Phi(D_i)$	$\Phi(D_i)$
0				0
1	3	1	2	2
2	3	2	1	3
3	3	3	0	3

Potentialmethode für dynamische Tabellen

- Idee.**
- Kein kopieren $\Rightarrow \Delta\Phi(D_i) = 2$
 - kopieren $\Rightarrow \Delta\Phi(D_i) = 2 - (i - 1)$
 - $\Rightarrow \Phi(D_i) = 1 + 2 \cdot \text{size}_i - \text{table-size}_i$

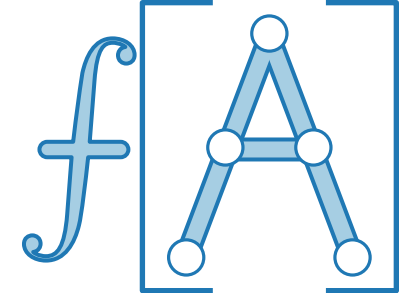
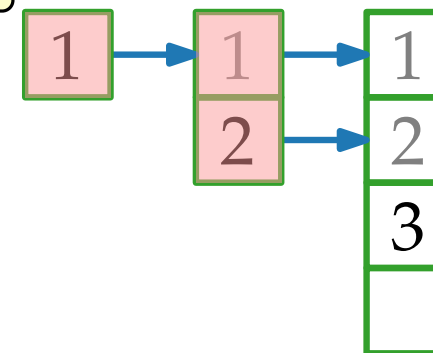
$i - 1$ Elemente werden kopiert

INSERT(1)

INSERT(2)

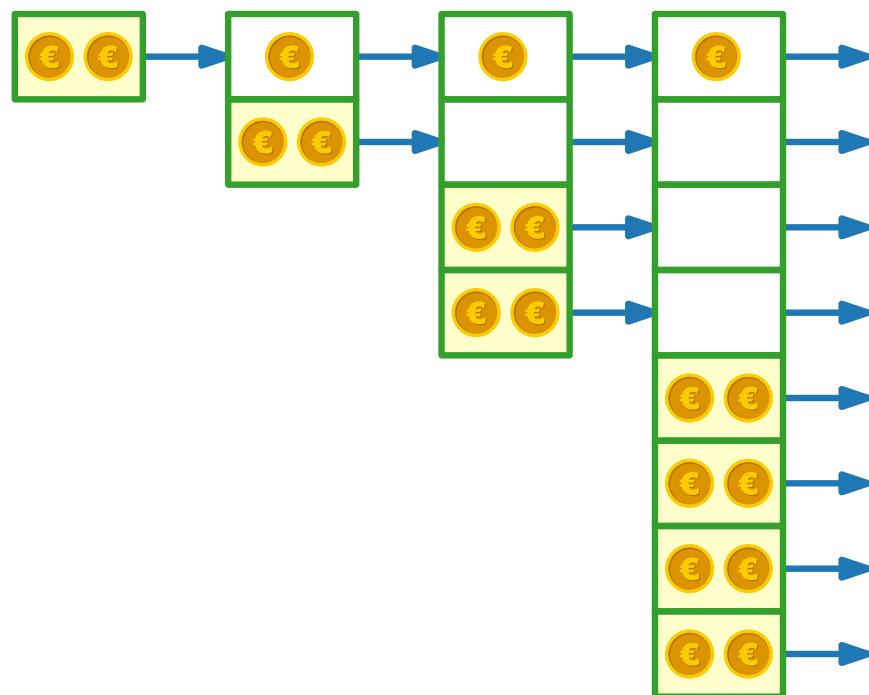
INSERT(3)

INSERT(4)



$$\hat{c}_i = c_i + \Delta\Phi(D_i), \text{ wobei } \Delta\Phi(D_i) = \Phi(D_i) - \Phi(D_{i-1})$$

$$\sum_{i=1}^n \hat{c}_i = \sum_{i=1}^n c_i + \Phi(D_n) - \Phi(D_0) \geq \sum_{i=1}^n c_i$$



i	$\hat{c}_i$	c_i	$\Delta\Phi(D_i)$	$\Phi(D_i)$
0				0
1	3	1	2	2
2	3	2	1	3
3	3	3	0	3
4				

Potentialmethode für dynamische Tabellen

- Idee.**
- Kein kopieren $\Rightarrow \Delta\Phi(D_i) = 2$
 - kopieren $\Rightarrow \Delta\Phi(D_i) = 2 - (i - 1)$
 - $\Rightarrow \Phi(D_i) = 1 + 2 \cdot \text{size}_i - \text{table-size}_i$

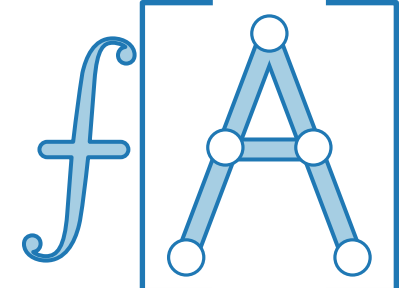
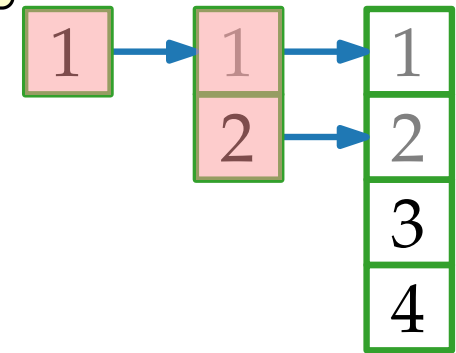
$i - 1$ Elemente werden kopiert

INSERT(1)

INSERT(2)

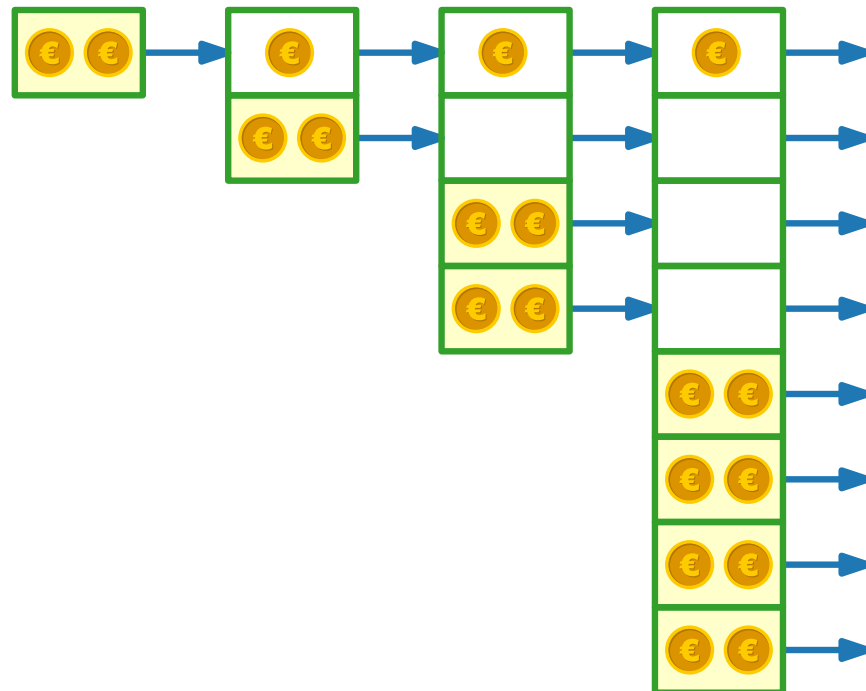
INSERT(3)

INSERT(4)



$$\hat{c}_i = c_i + \Delta\Phi(D_i), \text{ wobei } \Delta\Phi(D_i) = \Phi(D_i) - \Phi(D_{i-1})$$

$$\sum_{i=1}^n \hat{c}_i = \sum_{i=1}^n c_i + \Phi(D_n) - \Phi(D_0) \geq \sum_{i=1}^n c_i$$



i	$\hat{c}_i$	c_i	$\Delta\Phi(D_i)$	$\Phi(D_i)$
0				0
1	3	1	2	2
2	3	2	1	3
3	3	3	0	3
4				

Potentialmethode für dynamische Tabellen

- Idee.**
- Kein kopieren $\Rightarrow \Delta\Phi(D_i) = 2$
 - kopieren $\Rightarrow \Delta\Phi(D_i) = 2 - (i - 1)$
 - $\Rightarrow \Phi(D_i) = 1 + 2 \cdot \text{size}_i - \text{table-size}_i$

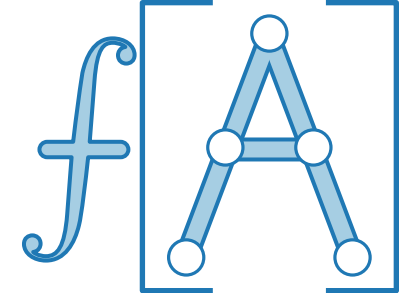
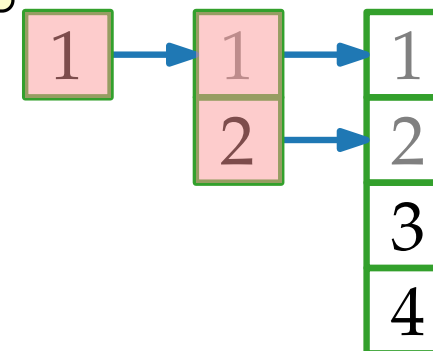
$i - 1$ Elemente werden kopiert

INSERT(1)

INSERT(2)

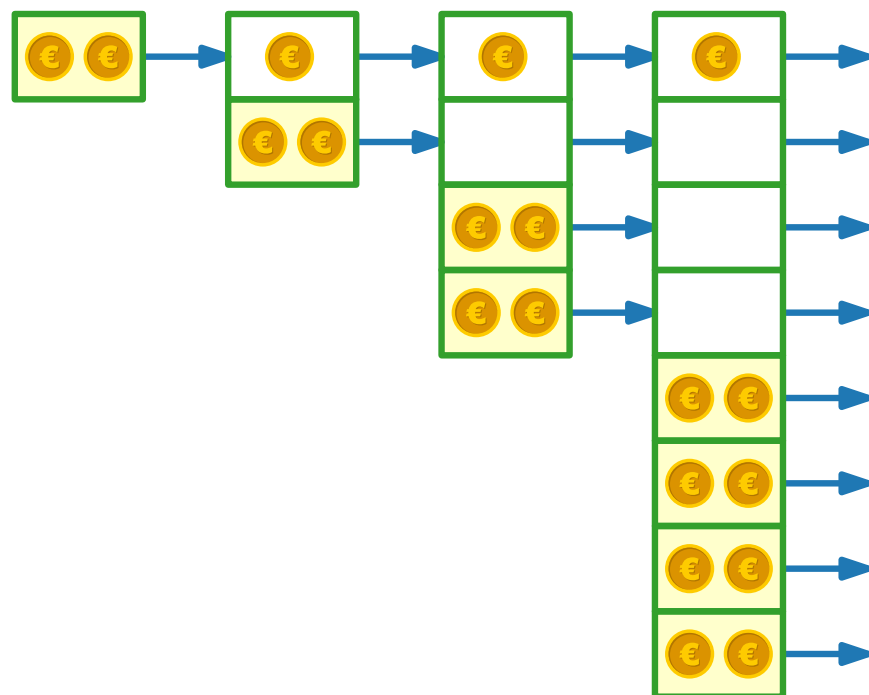
INSERT(3)

INSERT(4)



$$\hat{c}_i = c_i + \Delta\Phi(D_i), \text{ wobei } \Delta\Phi(D_i) = \Phi(D_i) - \Phi(D_{i-1})$$

$$\sum_{i=1}^n \hat{c}_i = \sum_{i=1}^n c_i + \Phi(D_n) - \Phi(D_0) \geq \sum_{i=1}^n c_i$$



i	$\hat{c}_i$	c_i	$\Delta\Phi(D_i)$	$\Phi(D_i)$
0				0
1	3	1	2	2
2	3	2	1	3
3	3	3	0	3
4		1		

Potentialmethode für dynamische Tabellen

- Idee.**
- Kein kopieren $\Rightarrow \Delta\Phi(D_i) = 2$
 - kopieren $\Rightarrow \Delta\Phi(D_i) = 2 - (i - 1)$
 - $\Rightarrow \Phi(D_i) = 1 + 2 \cdot \text{size}_i - \text{table-size}_i$

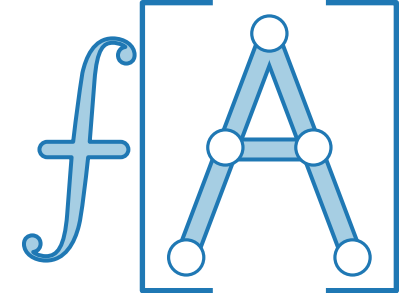
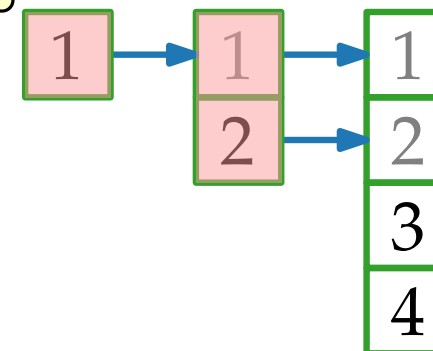
$i - 1$ Elemente werden kopiert

INSERT(1)

INSERT(2)

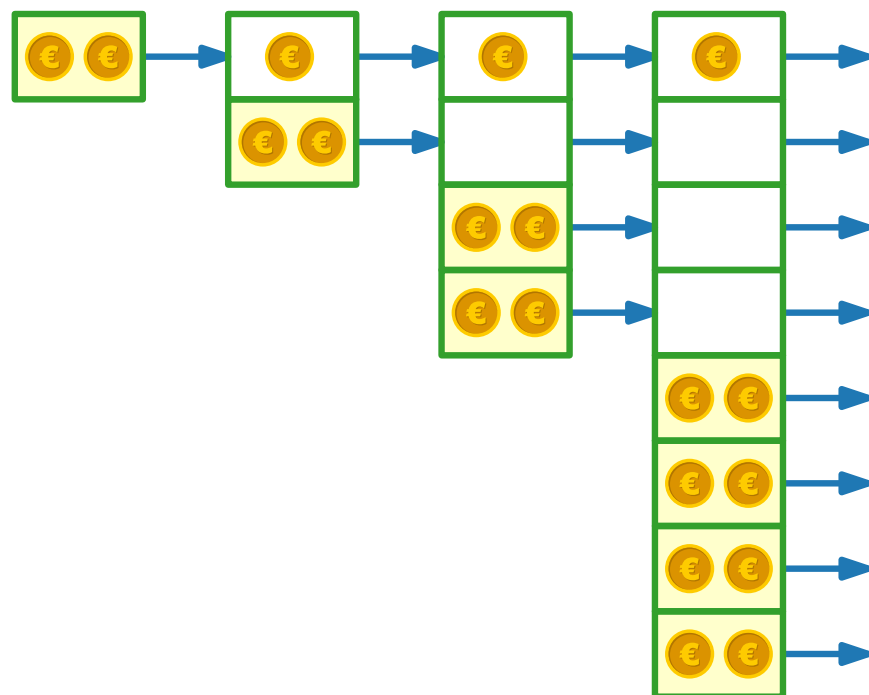
INSERT(3)

INSERT(4)



$$\hat{c}_i = c_i + \Delta\Phi(D_i), \text{ wobei } \Delta\Phi(D_i) = \Phi(D_i) - \Phi(D_{i-1})$$

$$\sum_{i=1}^n \hat{c}_i = \sum_{i=1}^n c_i + \Phi(D_n) - \Phi(D_0) \geq \sum_{i=1}^n c_i$$



i	$\hat{c}_i$	c_i	$\Delta\Phi(D_i)$	$\Phi(D_i)$
0				0
1	3	1	2	2
2	3	2	1	3
3	3	3	0	3
4		1	2	

Potentialmethode für dynamische Tabellen

- Idee.**
- Kein kopieren $\Rightarrow \Delta\Phi(D_i) = 2$
 - kopieren $\Rightarrow \Delta\Phi(D_i) = 2 - (i - 1)$
 - $\Rightarrow \Phi(D_i) = 1 + 2 \cdot \text{size}_i - \text{table-size}_i$

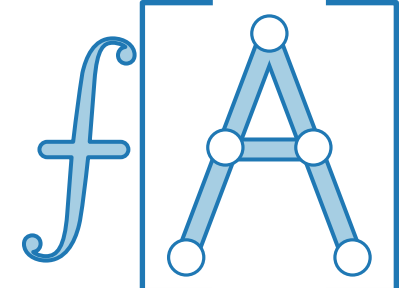
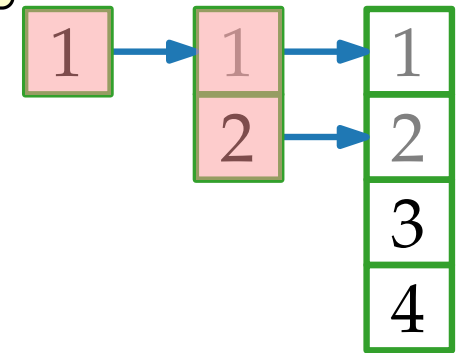
$i - 1$ Elemente werden kopiert

INSERT(1)

INSERT(2)

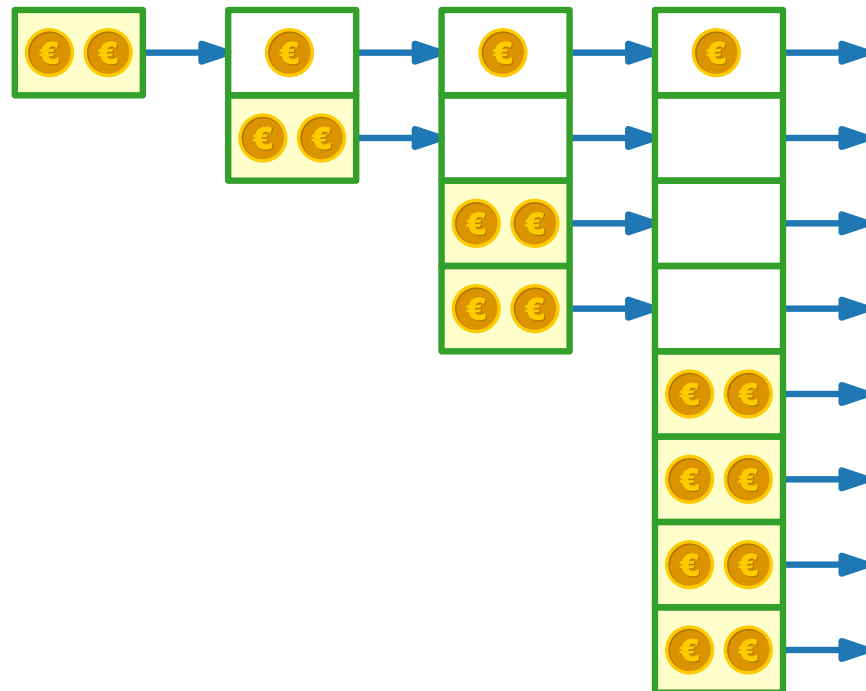
INSERT(3)

INSERT(4)



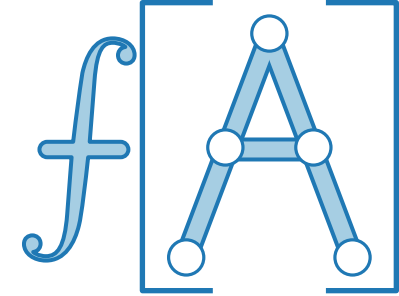
$$\hat{c}_i = c_i + \Delta\Phi(D_i), \text{ wobei } \Delta\Phi(D_i) = \Phi(D_i) - \Phi(D_{i-1})$$

$$\sum_{i=1}^n \hat{c}_i = \sum_{i=1}^n c_i + \Phi(D_n) - \Phi(D_0) \geq \sum_{i=1}^n c_i$$



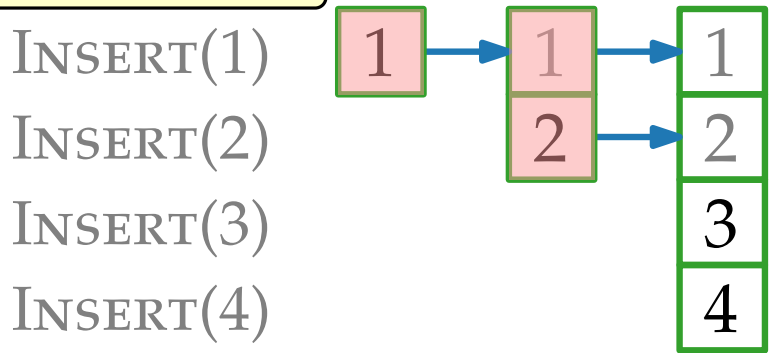
i	$\hat{c}_i$	c_i	$\Delta\Phi(D_i)$	$\Phi(D_i)$
0				0
1	3	1	2	2
2	3	2	1	3
3	3	3	0	3
4		1	2	5

Potentialmethode für dynamische Tabellen



- Idee.**
- Kein kopieren $\Rightarrow \Delta\Phi(D_i) = 2$
 - kopieren $\Rightarrow \Delta\Phi(D_i) = 2 - (i - 1)$
 - $\Rightarrow \Phi(D_i) = 1 + 2 \cdot \text{size}_i - \text{table-size}_i$

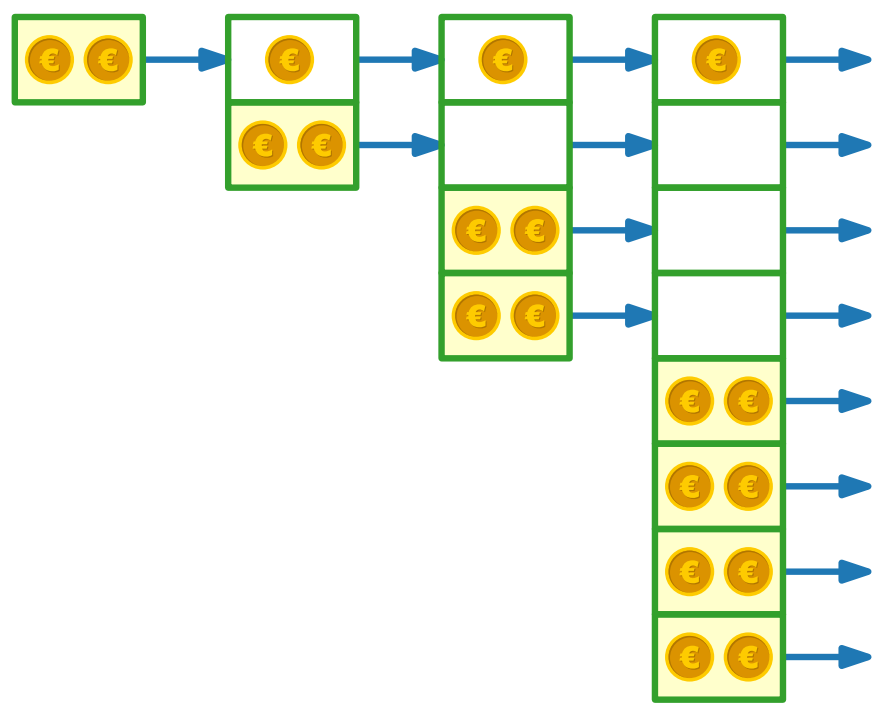
$i - 1$ Elemente werden kopiert



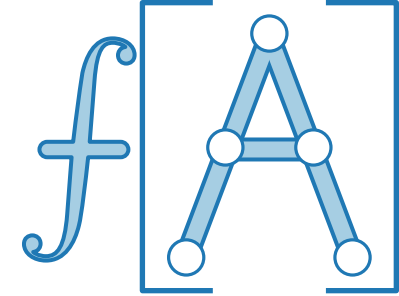
$\hat{c}_i = c_i + \Delta\Phi(D_i)$, wobei $\Delta\Phi(D_i) = \Phi(D_i) - \Phi(D_{i-1})$

$\sum_{i=1}^n \hat{c}_i = \sum_{i=1}^n c_i + \Phi(D_n) - \Phi(D_0) \geq \sum_{i=1}^n c_i$

i	$\hat{c}_i$	c_i	$\Delta\Phi(D_i)$	$\Phi(D_i)$
0				0
1	3	1	2	2
2	3	2	1	3
3	3	3	0	3
4	3	1	2	5

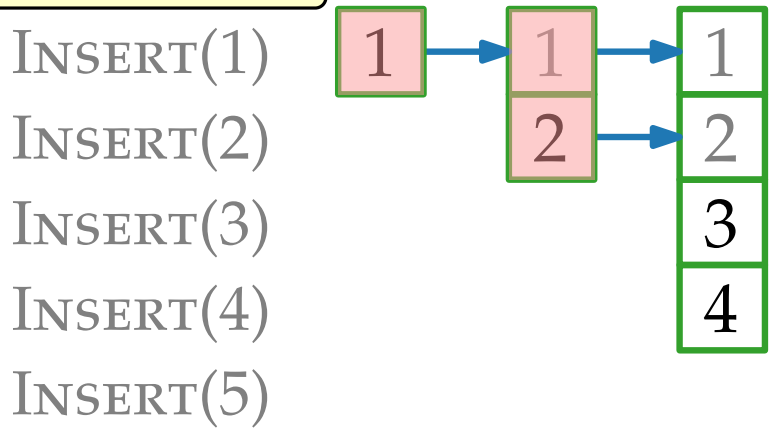


Potentialmethode für dynamische Tabellen



- Idee.**
- Kein kopieren $\Rightarrow \Delta\Phi(D_i) = 2$
 - kopieren $\Rightarrow \Delta\Phi(D_i) = 2 - (i - 1)$
 - $\Rightarrow \Phi(D_i) = 1 + 2 \cdot \text{size}_i - \text{table-size}_i$

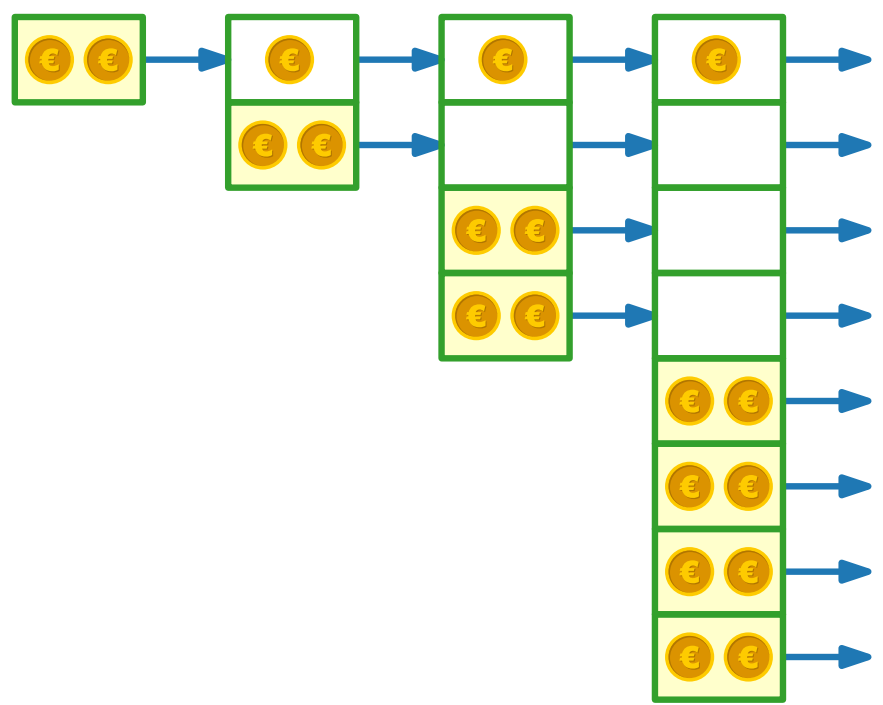
$i - 1$ Elemente werden kopiert



$$\hat{c}_i = c_i + \Delta\Phi(D_i), \text{ wobei } \Delta\Phi(D_i) = \Phi(D_i) - \Phi(D_{i-1})$$

$$\sum_{i=1}^n \hat{c}_i = \sum_{i=1}^n c_i + \Phi(D_n) - \Phi(D_0) \geq \sum_{i=1}^n c_i$$

i	$\hat{c}_i$	c_i	$\Delta\Phi(D_i)$	$\Phi(D_i)$
0				0
1	3	1	2	2
2	3	2	1	3
3	3	3	0	3
4	3	1	2	5
5				

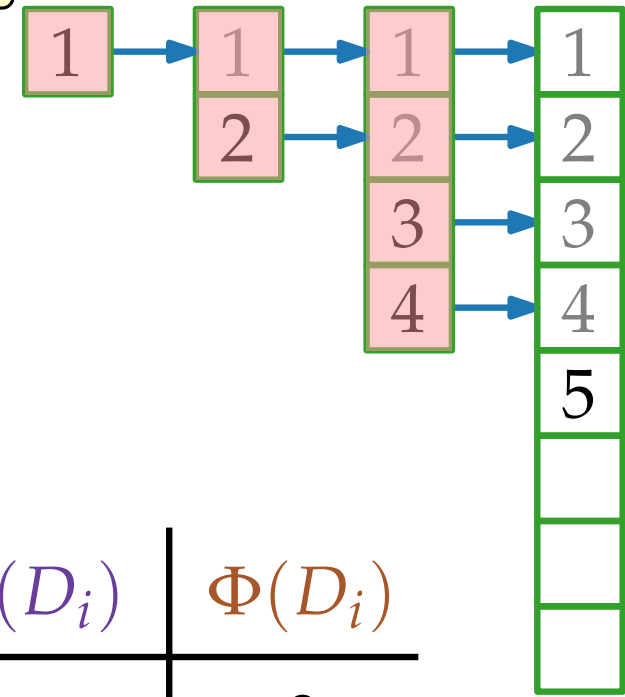


Potentialmethode für dynamische Tabellen

- Idee.**
- Kein kopieren $\Rightarrow \Delta\Phi(D_i) = 2$
 - kopieren $\Rightarrow \Delta\Phi(D_i) = 2 - (i - 1)$
 - $\Rightarrow \Phi(D_i) = 1 + 2 \cdot \text{size}_i - \text{table-size}_i$

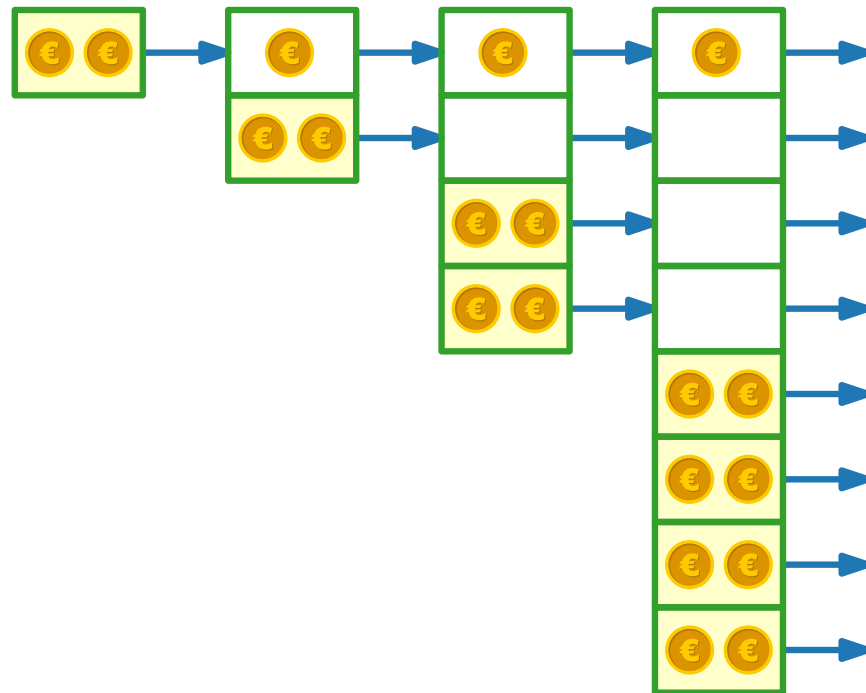
$i - 1$ Elemente werden kopiert

INSERT(1)
INSERT(2)
INSERT(3)
INSERT(4)
INSERT(5)



$$\hat{c}_i = c_i + \Delta\Phi(D_i), \text{ wobei } \Delta\Phi(D_i) = \Phi(D_i) - \Phi(D_{i-1})$$

$$\sum_{i=1}^n \hat{c}_i = \sum_{i=1}^n c_i + \Phi(D_n) - \Phi(D_0) \geq \sum_{i=1}^n c_i$$



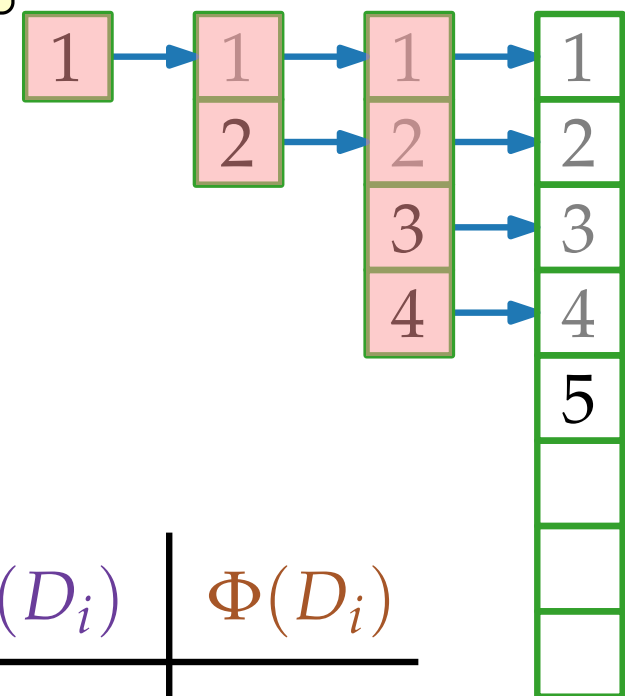
i	$\hat{c}_i$	c_i	$\Delta\Phi(D_i)$	$\Phi(D_i)$
0				0
1	3	1	2	2
2	3	2	1	3
3	3	3	0	3
4	3	1	2	5
5				

Potentialmethode für dynamische Tabellen

- Idee.**
- Kein kopieren $\Rightarrow \Delta\Phi(D_i) = 2$
 - kopieren $\Rightarrow \Delta\Phi(D_i) = 2 - (i - 1)$
 - $\Rightarrow \Phi(D_i) = 1 + 2 \cdot \text{size}_i - \text{table-size}_i$

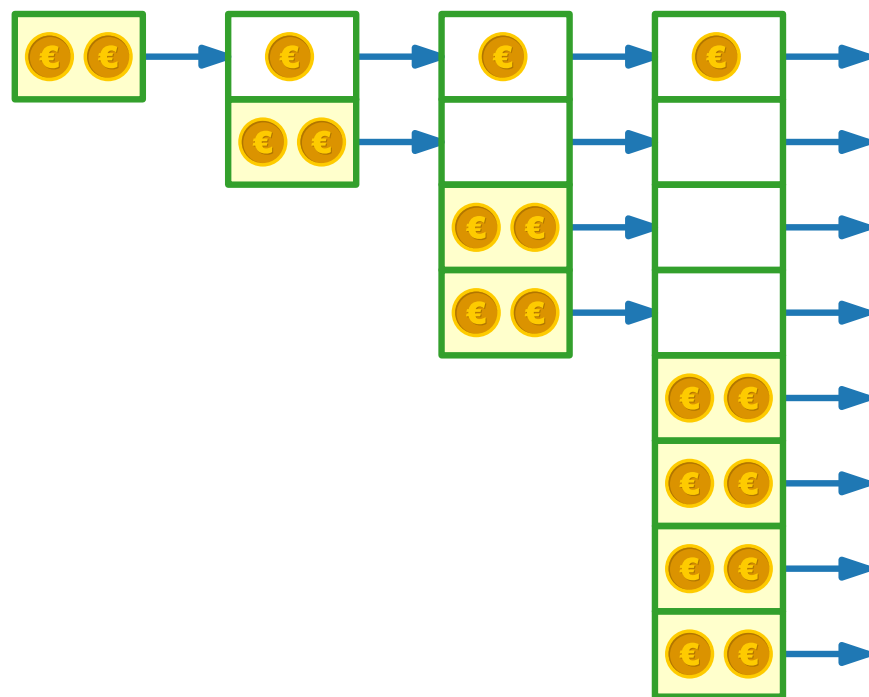
$i - 1$ Elemente werden kopiert

INSERT(1)
INSERT(2)
INSERT(3)
INSERT(4)
INSERT(5)



$$\hat{c}_i = c_i + \Delta\Phi(D_i), \text{ wobei } \Delta\Phi(D_i) = \Phi(D_i) - \Phi(D_{i-1})$$

$$\sum_{i=1}^n \hat{c}_i = \sum_{i=1}^n c_i + \Phi(D_n) - \Phi(D_0) \geq \sum_{i=1}^n c_i$$



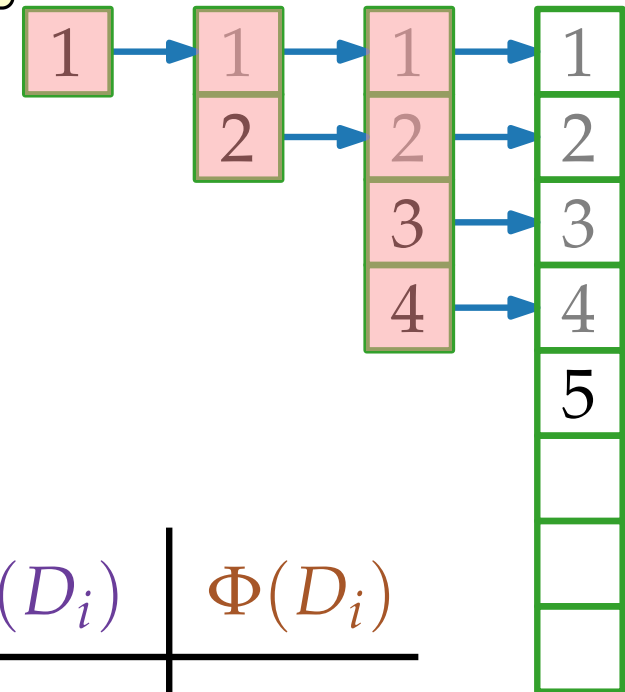
i	$\hat{c}_i$	c_i	$\Delta\Phi(D_i)$	$\Phi(D_i)$
0				0
1	3	1	2	2
2	3	2	1	3
3	3	3	0	3
4	3	1	2	5
5		5		

Potentialmethode für dynamische Tabellen

- Idee.**
- Kein kopieren $\Rightarrow \Delta\Phi(D_i) = 2$
 - kopieren $\Rightarrow \Delta\Phi(D_i) = 2 - (i - 1)$
 - $\Rightarrow \Phi(D_i) = 1 + 2 \cdot \text{size}_i - \text{table-size}_i$

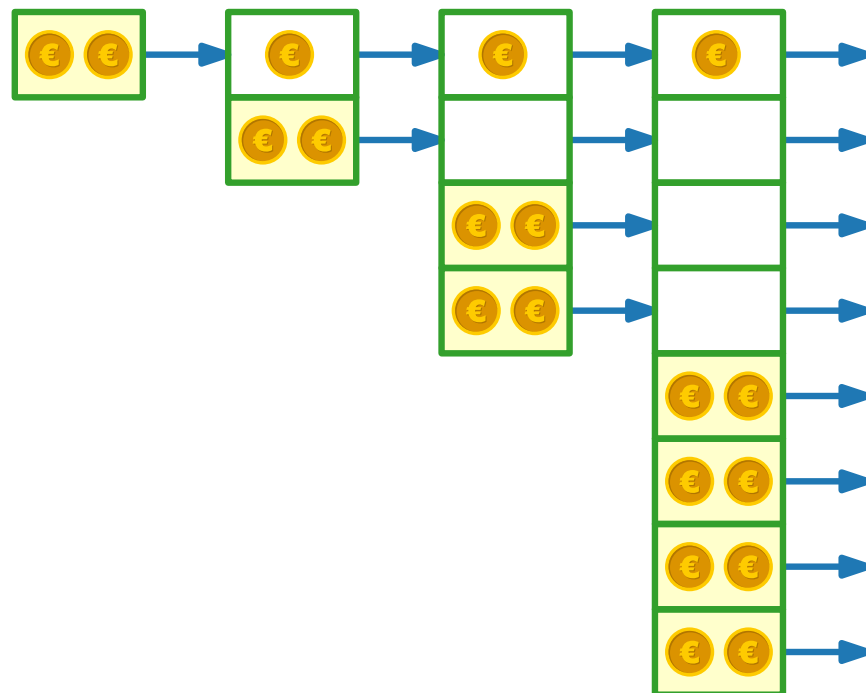
$i - 1$ Elemente werden kopiert

INSERT(1)
INSERT(2)
INSERT(3)
INSERT(4)
INSERT(5)



$$\hat{c}_i = c_i + \Delta\Phi(D_i), \text{ wobei } \Delta\Phi(D_i) = \Phi(D_i) - \Phi(D_{i-1})$$

$$\sum_{i=1}^n \hat{c}_i = \sum_{i=1}^n c_i + \Phi(D_n) - \Phi(D_0) \geq \sum_{i=1}^n c_i$$



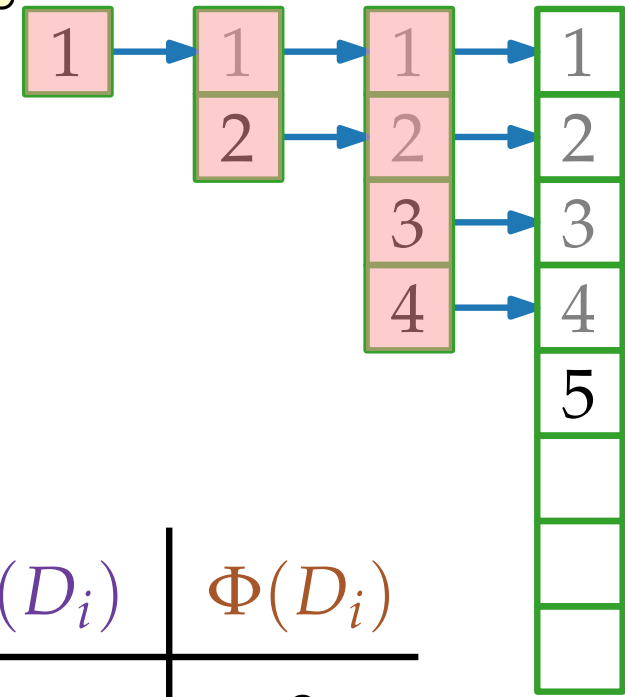
i	$\hat{c}_i$	c_i	$\Delta\Phi(D_i)$	$\Phi(D_i)$
0				0
1	3	1	2	2
2	3	2	1	3
3	3	3	0	3
4	3	1	2	5
5		5	-2	

Potentialmethode für dynamische Tabellen

- Idee.**
- Kein kopieren $\Rightarrow \Delta\Phi(D_i) = 2$
 - kopieren $\Rightarrow \Delta\Phi(D_i) = 2 - (i - 1)$
 - $\Rightarrow \Phi(D_i) = 1 + 2 \cdot \text{size}_i - \text{table-size}_i$

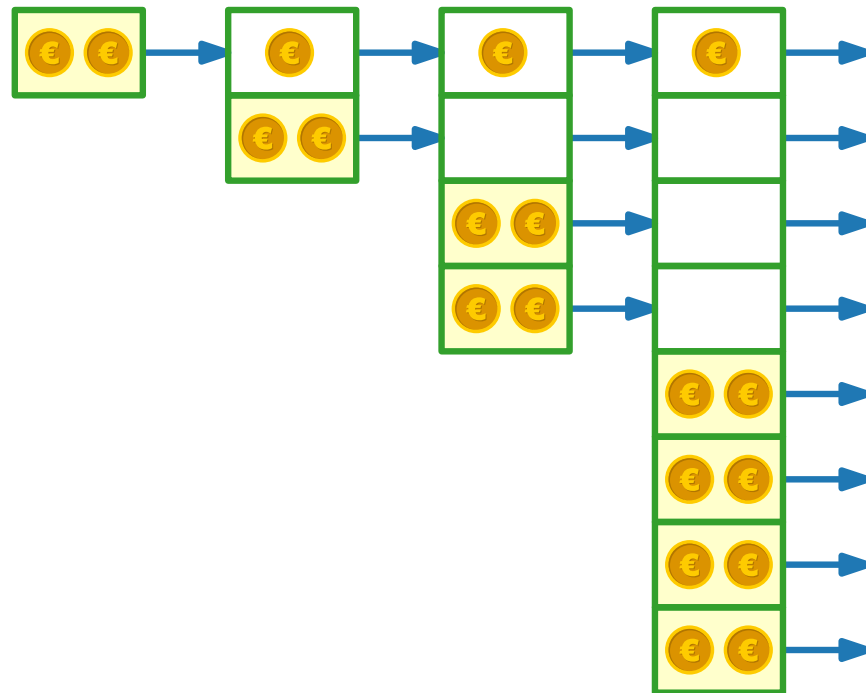
$i - 1$ Elemente werden kopiert

INSERT(1)
INSERT(2)
INSERT(3)
INSERT(4)
INSERT(5)



$$\hat{c}_i = c_i + \Delta\Phi(D_i), \text{ wobei } \Delta\Phi(D_i) = \Phi(D_i) - \Phi(D_{i-1})$$

$$\sum_{i=1}^n \hat{c}_i = \sum_{i=1}^n c_i + \Phi(D_n) - \Phi(D_0) \geq \sum_{i=1}^n c_i$$



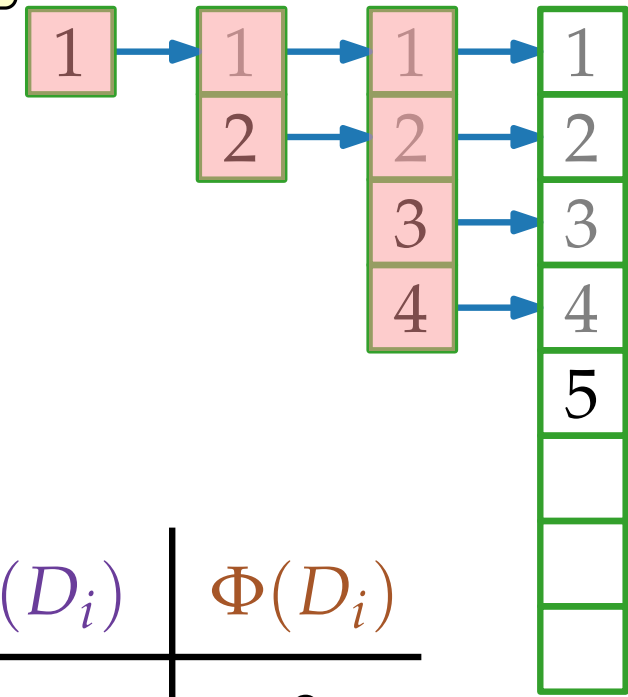
i	$\hat{c}_i$	c_i	$\Delta\Phi(D_i)$	$\Phi(D_i)$
0				0
1	3	1	2	2
2	3	2	1	3
3	3	3	0	3
4	3	1	2	5
5		5	-2	3

Potentialmethode für dynamische Tabellen

- Idee.**
- Kein kopieren $\Rightarrow \Delta\Phi(D_i) = 2$
 - kopieren $\Rightarrow \Delta\Phi(D_i) = 2 - (i - 1)$
 - $\Rightarrow \Phi(D_i) = 1 + 2 \cdot \text{size}_i - \text{table-size}_i$

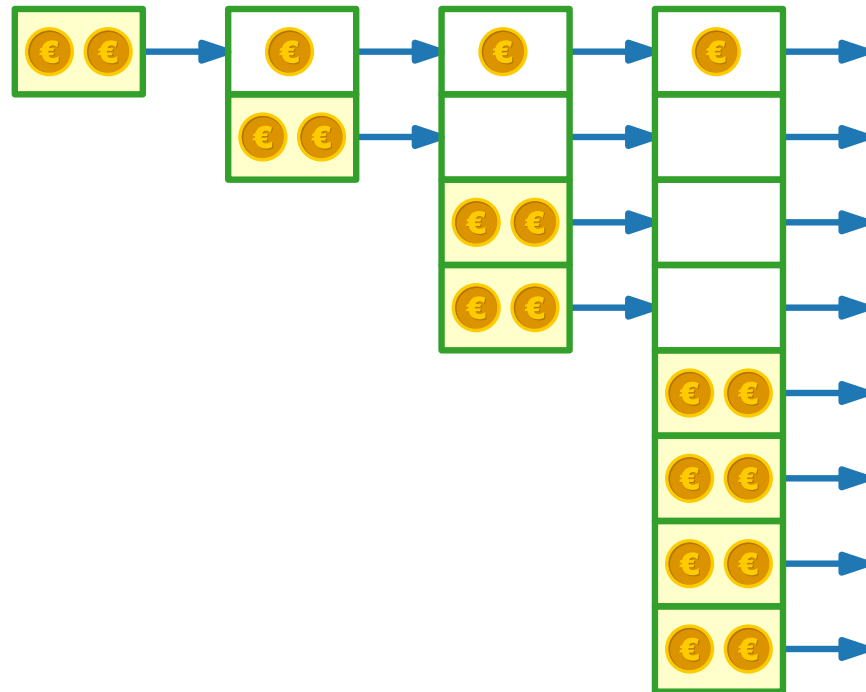
$i - 1$ Elemente werden kopiert

INSERT(1)
INSERT(2)
INSERT(3)
INSERT(4)
INSERT(5)



$$\hat{c}_i = c_i + \Delta\Phi(D_i), \text{ wobei } \Delta\Phi(D_i) = \Phi(D_i) - \Phi(D_{i-1})$$

$$\sum_{i=1}^n \hat{c}_i = \sum_{i=1}^n c_i + \Phi(D_n) - \Phi(D_0) \geq \sum_{i=1}^n c_i$$



i	$\hat{c}_i$	c_i	$\Delta\Phi(D_i)$	$\Phi(D_i)$
0				0
1	3	1	2	2
2	3	2	1	3
3	3	3	0	3
4	3	1	2	5
5	3	5	-2	3

Potentialmethode für dynamische Tabellen

- Idee.**
- Kein kopieren $\Rightarrow \Delta\Phi(D_i) = 2$
 - kopieren $\Rightarrow \Delta\Phi(D_i) = 2 - (i - 1)$
 - $\Rightarrow \Phi(D_i) = 1 + 2 \cdot \text{size}_i - \text{table-size}_i$

$i - 1$ Elemente werden kopiert

INSERT(1)

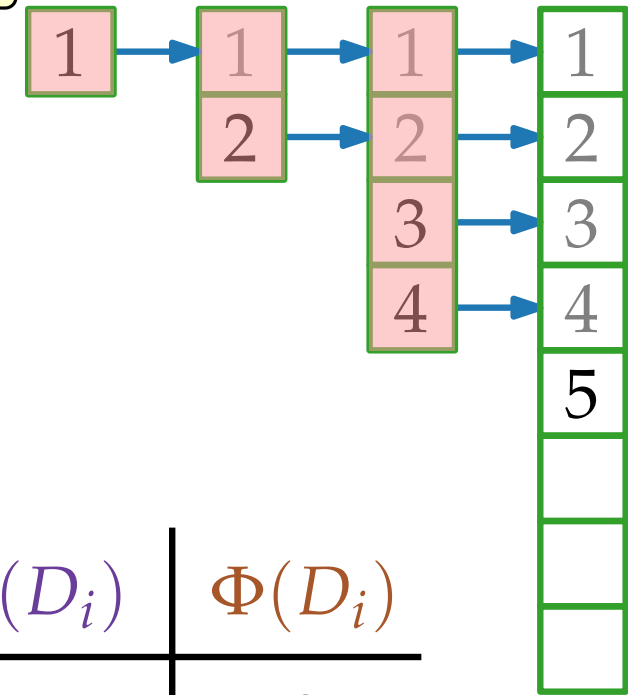
INSERT(2)

INSERT(3)

INSERT(4)

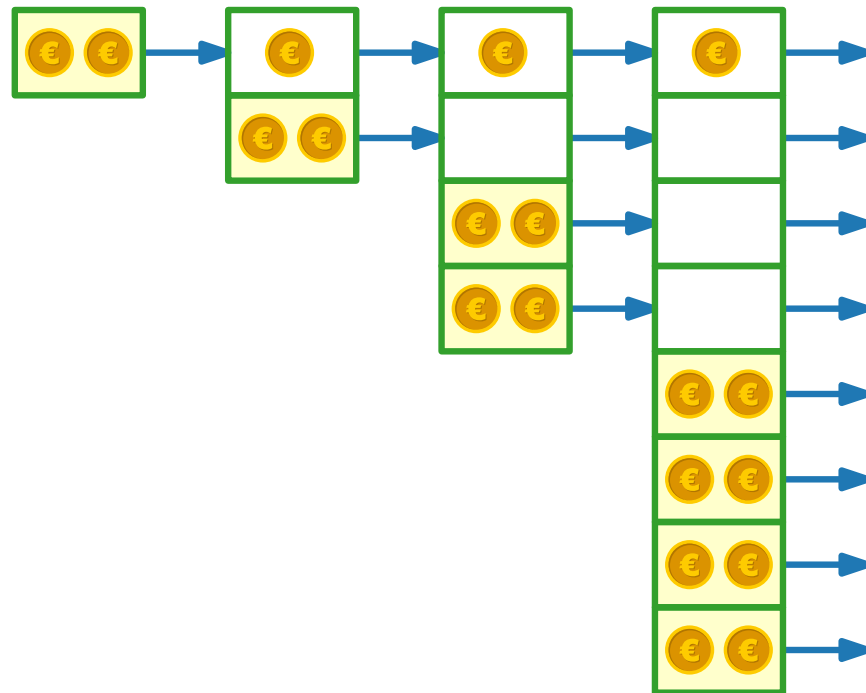
INSERT(5)

INSERT(6)



$$\hat{c}_i = c_i + \Delta\Phi(D_i), \text{ wobei } \Delta\Phi(D_i) = \Phi(D_i) - \Phi(D_{i-1})$$

$$\sum_{i=1}^n \hat{c}_i = \sum_{i=1}^n c_i + \Phi(D_n) - \Phi(D_0) \geq \sum_{i=1}^n c_i$$



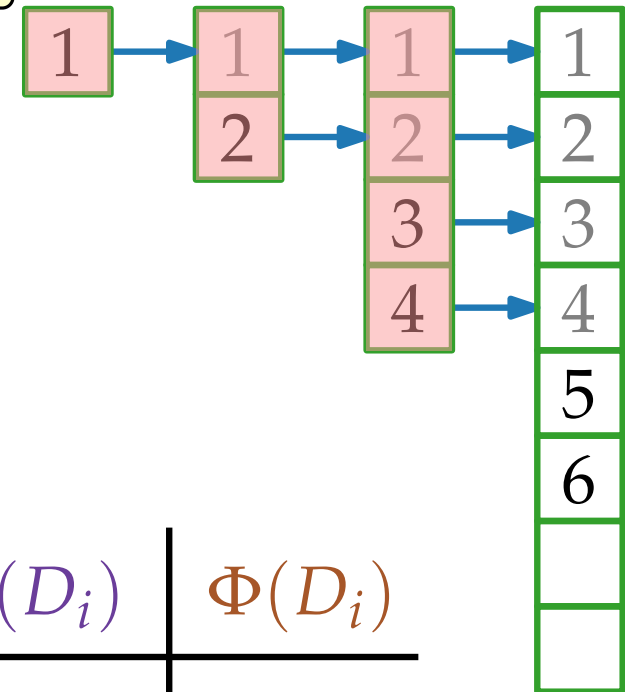
i	$\hat{c}_i$	c_i	$\Delta\Phi(D_i)$	$\Phi(D_i)$
0				0
1	3	1	2	2
2	3	2	1	3
3	3	3	0	3
4	3	1	2	5
5	3	5	-2	3
6				

Potentialmethode für dynamische Tabellen

- Idee.**
- Kein kopieren $\Rightarrow \Delta\Phi(D_i) = 2$
 - kopieren $\Rightarrow \Delta\Phi(D_i) = 2 - (i - 1)$
 - $\Rightarrow \Phi(D_i) = 1 + 2 \cdot \text{size}_i - \text{table-size}_i$

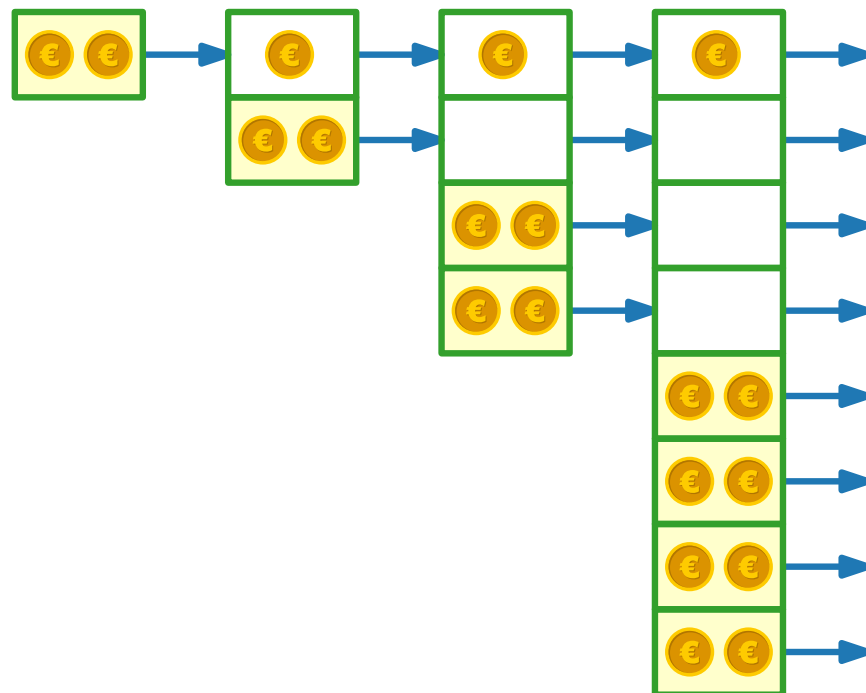
$i - 1$ Elemente werden kopiert

INSERT(1)
INSERT(2)
INSERT(3)
INSERT(4)
INSERT(5)
INSERT(6)



$$\hat{c}_i = c_i + \Delta\Phi(D_i), \text{ wobei } \Delta\Phi(D_i) = \Phi(D_i) - \Phi(D_{i-1})$$

$$\sum_{i=1}^n \hat{c}_i = \sum_{i=1}^n c_i + \Phi(D_n) - \Phi(D_0) \geq \sum_{i=1}^n c_i$$



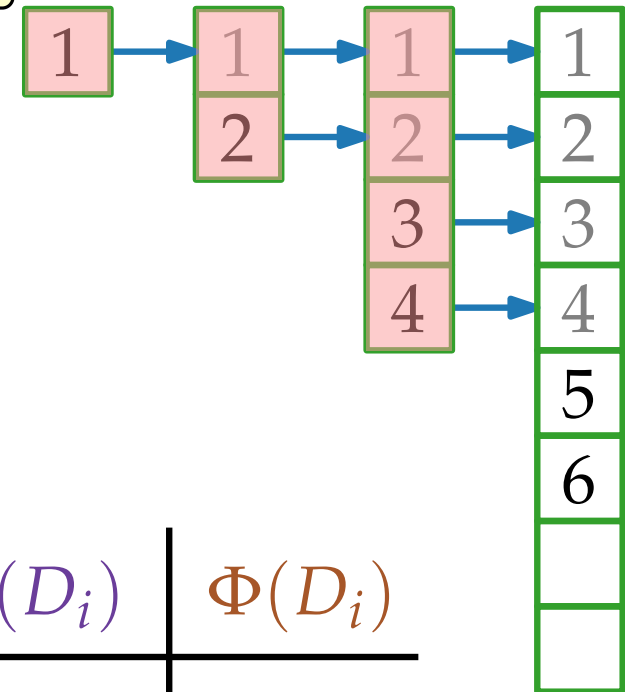
i	$\hat{c}_i$	c_i	$\Delta\Phi(D_i)$	$\Phi(D_i)$
0				0
1	3	1	2	2
2	3	2	1	3
3	3	3	0	3
4	3	1	2	5
5	3	5	-2	3
6				

Potentialmethode für dynamische Tabellen

- Idee.**
- Kein kopieren $\Rightarrow \Delta\Phi(D_i) = 2$
 - kopieren $\Rightarrow \Delta\Phi(D_i) = 2 - (i - 1)$
 - $\Rightarrow \Phi(D_i) = 1 + 2 \cdot \text{size}_i - \text{table-size}_i$

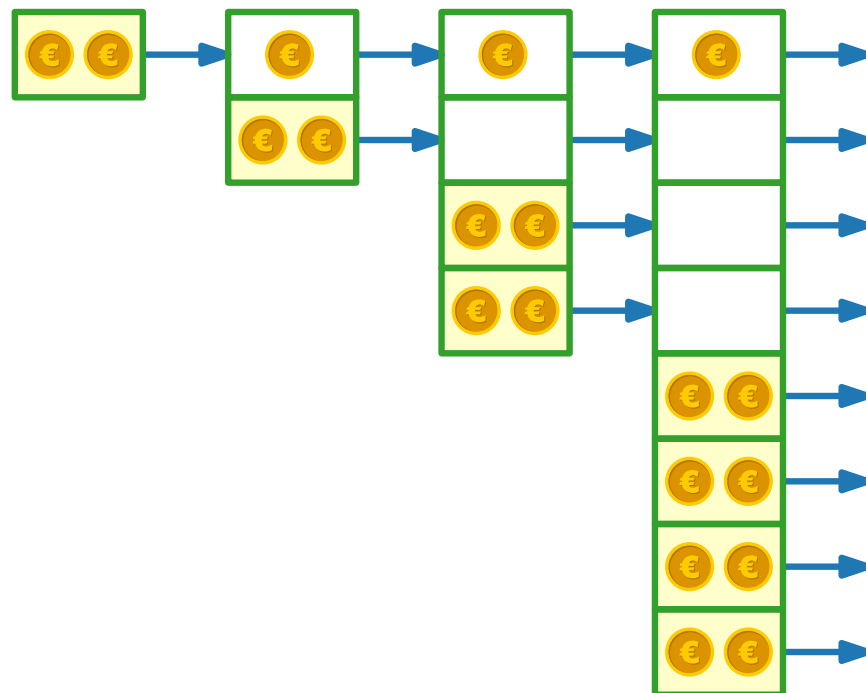
$i - 1$ Elemente werden kopiert

INSERT(1)
INSERT(2)
INSERT(3)
INSERT(4)
INSERT(5)
INSERT(6)



$$\hat{c}_i = c_i + \Delta\Phi(D_i), \text{ wobei } \Delta\Phi(D_i) = \Phi(D_i) - \Phi(D_{i-1})$$

$$\sum_{i=1}^n \hat{c}_i = \sum_{i=1}^n c_i + \Phi(D_n) - \Phi(D_0) \geq \sum_{i=1}^n c_i$$



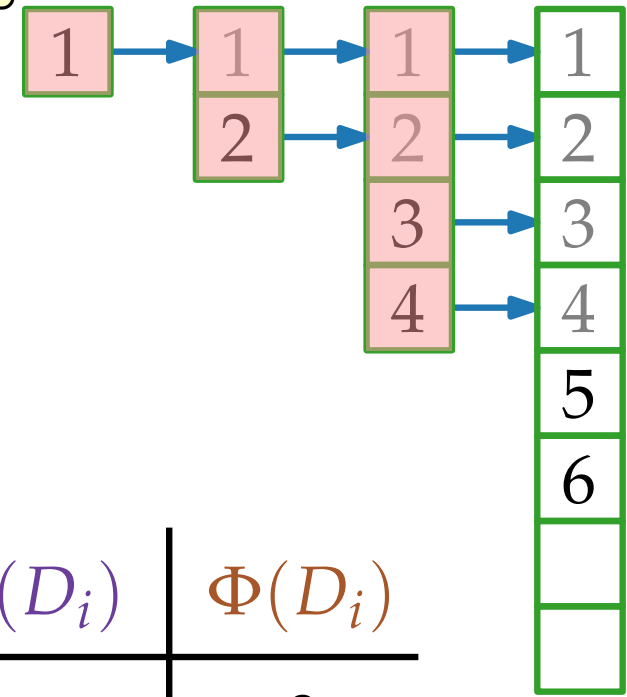
i	$\hat{c}_i$	c_i	$\Delta\Phi(D_i)$	$\Phi(D_i)$
0				0
1	3	1	2	2
2	3	2	1	3
3	3	3	0	3
4	3	1	2	5
5	3	5	-2	3
6	3	1	2	5

Potentialmethode für dynamische Tabellen

- Idee.**
- Kein kopieren $\Rightarrow \Delta\Phi(D_i) = 2$
 - kopieren $\Rightarrow \Delta\Phi(D_i) = 2 - (i - 1)$
 - $\Rightarrow \Phi(D_i) = 1 + 2 \cdot \text{size}_i - \text{table-size}_i$

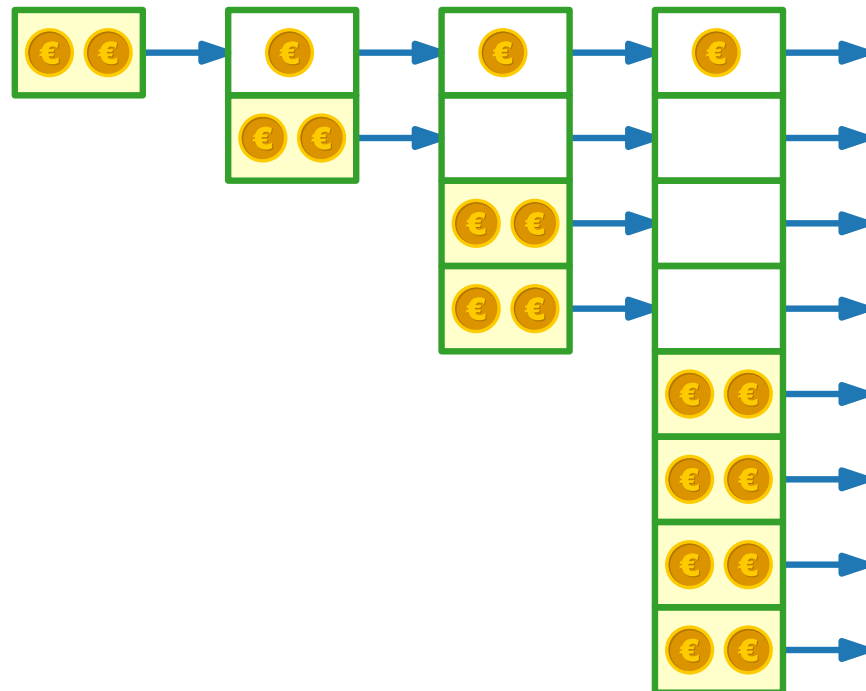
$i - 1$ Elemente werden kopiert

INSERT(1)
INSERT(2)
INSERT(3)
INSERT(4)
INSERT(5)
INSERT(6)



$$\hat{c}_i = c_i + \Delta\Phi(D_i), \text{ wobei } \Delta\Phi(D_i) = \Phi(D_i) - \Phi(D_{i-1})$$

$$\sum_{i=1}^n \hat{c}_i = \sum_{i=1}^n c_i + \Phi(D_n) - \Phi(D_0) \geq \sum_{i=1}^n c_i$$



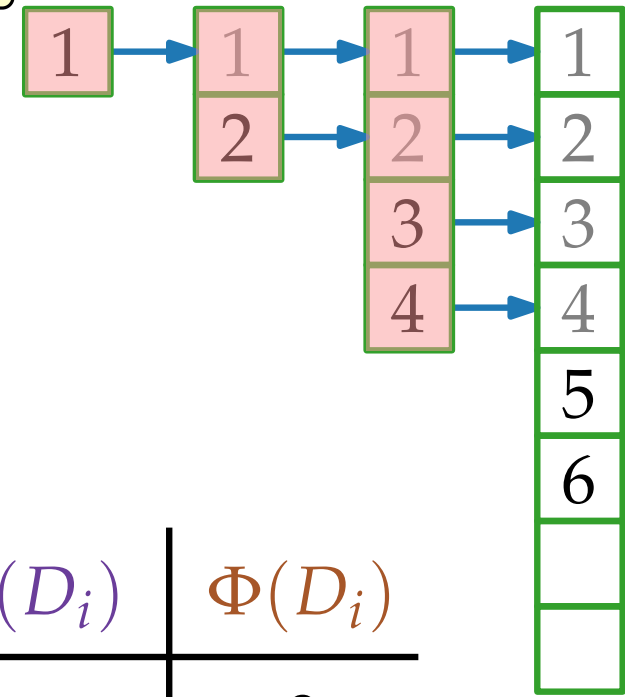
i	$\hat{c}_i$	c_i	$\Delta\Phi(D_i)$	$\Phi(D_i)$
0				0
1	3	1	2	2
2	3	2	1	3
3	3	3	0	3
4	3	1	2	5
5	3	5	-2	3
6	3	1	2	5

Potentialmethode für dynamische Tabellen

- Idee.**
- Kein kopieren $\Rightarrow \Delta\Phi(D_i) = 2$
 - kopieren $\Rightarrow \Delta\Phi(D_i) = 2 - (i - 1)$
 - $\Rightarrow \Phi(D_i) = 1 + 2 \cdot \text{size}_i - \text{table-size}_i$

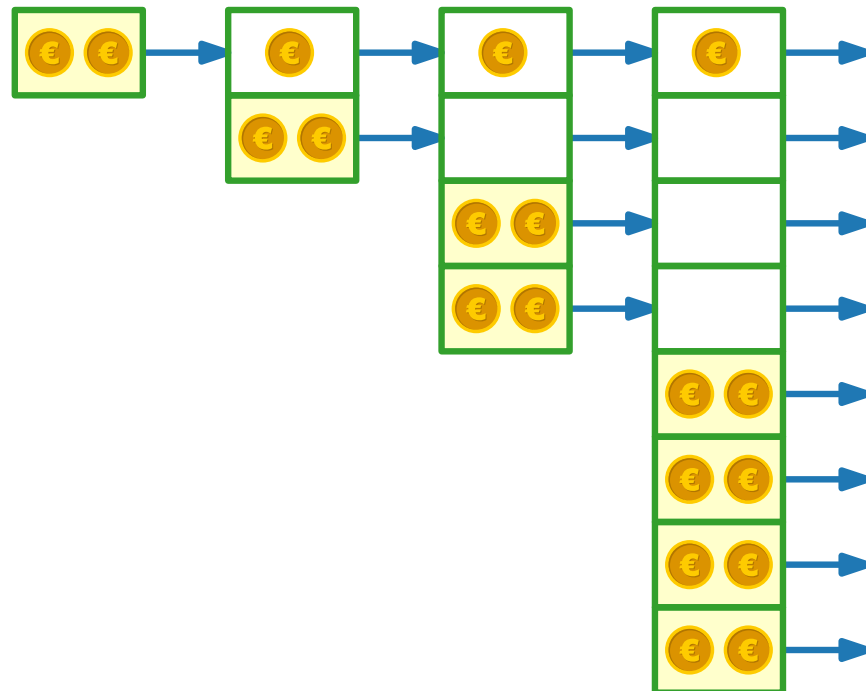
$i - 1$ Elemente werden kopiert

INSERT(1)
INSERT(2)
INSERT(3)
INSERT(4)
INSERT(5)
INSERT(6)



$$\hat{c}_i = c_i + \Delta\Phi(D_i), \text{ wobei } \Delta\Phi(D_i) = \Phi(D_i) - \Phi(D_{i-1})$$

$$\sum_{i=1}^n \hat{c}_i = \sum_{i=1}^n c_i + \Phi(D_n) - \Phi(D_0) \geq \sum_{i=1}^n c_i$$

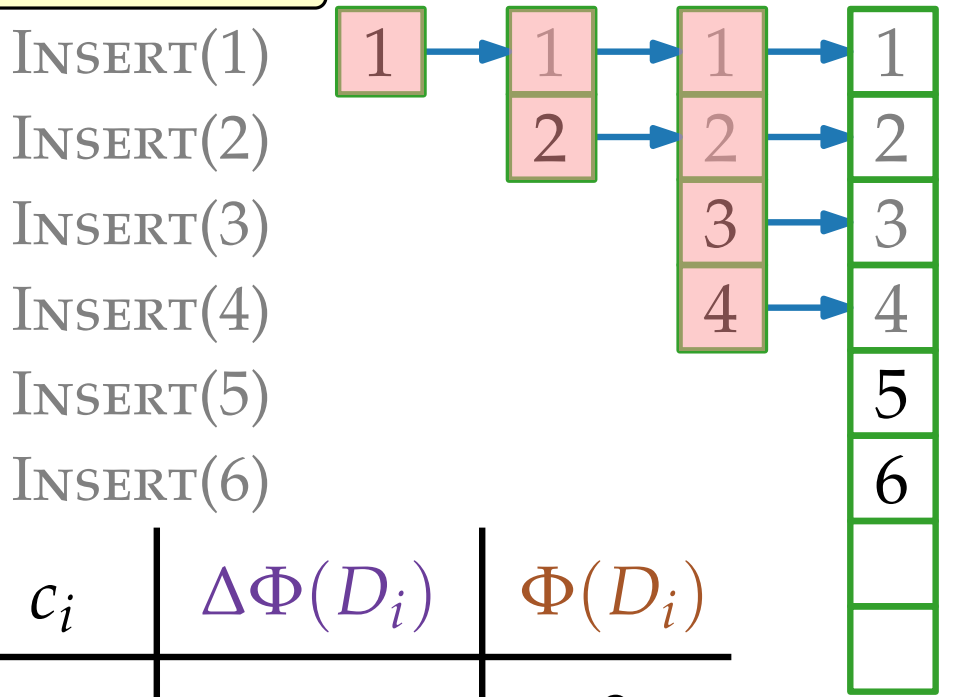


i	$\hat{c}_i$	c_i	$\Delta\Phi(D_i)$	$\Phi(D_i)$
0				0
1	3	1	2	2
2	3	2	1	3
3	3	3	0	3
4	3	1	2	5
5	3	5	-2	3
6	3	1	2	5

Potentialmethode für dynamische Tabellen

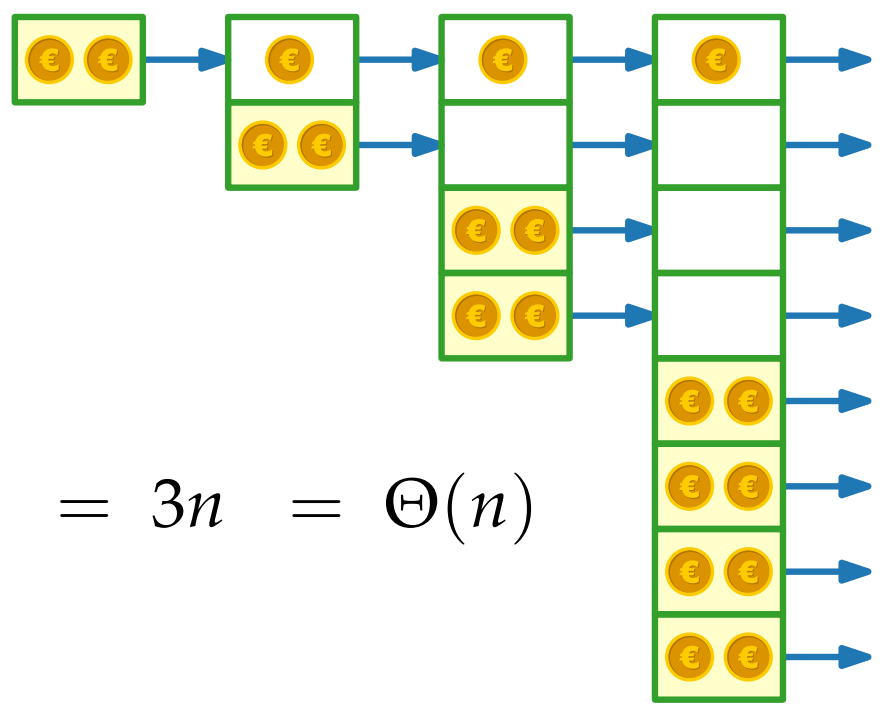
- Idee.**
- Kein kopieren $\Rightarrow \Delta\Phi(D_i) = 2$
 - kopieren $\Rightarrow \Delta\Phi(D_i) = 2 - (i - 1)$
 - $\Rightarrow \Phi(D_i) = 1 + 2 \cdot \text{size}_i - \text{table-size}_i$

$i - 1$ Elemente werden kopiert



$\hat{c}_i = c_i + \Delta\Phi(D_i)$, wobei $\Delta\Phi(D_i) = \Phi(D_i) - \Phi(D_{i-1})$

$\sum_{i=1}^n \hat{c}_i = \sum_{i=1}^n c_i + \Phi(D_n) - \Phi(D_0) \geq \sum_{i=1}^n c_i$

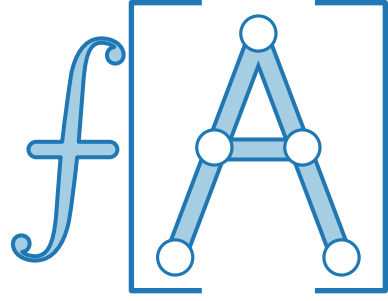


i	$\hat{c}_i$	c_i	$\Delta\Phi(D_i)$	$\Phi(D_i)$
0				0
1	3	1	2	2
2	3	2	1	3
3	3	3	0	3
4	3	1	2	5
5	3	5	-2	3
6	3	1	2	5

$\sum_{i=1}^n c_i \leq \sum_{i=1}^n \hat{c}_i = 3n = \Theta(n)$

Zusammenfassung

Zeige mit **amortisierter Analyse**, dass die Operationen einer gegebenen Folge kleine durchschnittliche Kosten haben –
auch wenn einzelne Operationen in der Folge teuer sind!



Zusammenfassung



Zeige mit **amortisierter Analyse**, dass die Operationen einer gegebenen Folge kleine durchschnittliche Kosten haben –
auch wenn einzelne Operationen in der Folge teuer sind!

Drei Typen von amortisierter Analyse:

- Aggregationsmethode
- Buchhaltermethode
- Potentialmethode

Zusammenfassung



Zeige mit **amortisierter Analyse**, dass die Operationen einer gegebenen Folge kleine durchschnittliche Kosten haben –
auch wenn einzelne Operationen in der Folge teuer sind!

Drei Typen von amortisierter Analyse:

- Aggregationsmethode ✓
Summiere tatsächliche Kosten (oder obere Schranken dafür) auf.
- Buchhaltermethode
- Potentialmethode

Zusammenfassung



Zeige mit **amortisierter Analyse**, dass die Operationen einer gegebenen Folge kleine durchschnittliche Kosten haben –
auch wenn einzelne Operationen in der Folge teuer sind!

Drei Typen von amortisierter Analyse:

- Aggregationsmethode ✓
Summiere tatsächliche Kosten (oder obere Schranken dafür) auf.
- Buchhaltermethode ✓
Verbinde Extrakosten mit konkreten Objekten der Datenstruktur und bezahle damit teure Operationen.
- Potentialmethode

Zusammenfassung



Zeige mit **amortisierter Analyse**, dass die Operationen einer gegebenen Folge kleine durchschnittliche Kosten haben – *auch wenn einzelne Operationen in der Folge teuer sind!*

Drei Typen von amortisierter Analyse:

- Aggregationsmethode ✓
Summiere tatsächliche Kosten (oder obere Schranken dafür) auf.
- Buchhaltermethode ✓
Verbinde Extrakosten mit konkreten Objekten der Datenstruktur und bezahle damit teure Operationen.
- Potentialmethode ✓
Definiere Potential der gesamten Datenstruktur, so dass mit der Potentialdifferenz teure Operationen bezahlt werden können.